



AI 辅助数理建模与应用实践

写给本科生的 vibe coding 实践讲义

金磊

版权所有 © 2026 金磊

同济大学物理科学与工程学院，上海市 200092

邮箱: jinlei_8765@hotmail.com

本讲义基于 The Legrand Orange Book 模板 (CC BY-NC-SA 3.0) 排版，正文内容遵循同名许可协议发布。你可以自由复制、分发、改编本讲义用于非商业的教学用途，但须署名并以相同方式共享。

本讲义的写作得到了国家自然科学基金（批准号：12475132, 12535009）和中央高校基本科研业务费的资助。

第一版，2026 年

目录

前言	11
----------	----

I

像物理学家一样指挥 AI

1	像物理学家一样指挥 AI	15
1.1	什么是 vibe coding	16
1.2	这门课加的一条硬约束: pause and ask why	16
1.3	director 与抄代码实习生的区别	17
1.4	五条反射动作总览	18
1.5	cognitive ownership: 为什么不让 AI 全做	19
1.6	这门课在研究生命周期里的位置	20
2	量纲分析与数量级估算	23
2.1	量纲: 物理量自带的身份证	23
2.2	量纲齐次性: 一秒钟否决错误公式	24
2.3	靠量纲猜出公式的形状: 单摆与终端速度	25
2.4	Fermi 估算: 把估不出的大问题拆成估得出的小问题	26
2.4.1	第一题: 人在月球上能跳多高	26
2.4.2	第二题: 一杯咖啡冷却到能喝要多久	27
2.4.3	第三题: 雨滴落地的终端速度	27

2.5	AI 协作实录：用量纲在六十秒内抓出错误公式	28
3	建模：从模糊想法到可计算任务	31
3.1	什么是物理建模	32
3.2	完整案例：一杯咖啡多久能喝	32
3.3	模型的适用范围与失效	34
3.4	AI 协作实录：让 AI 帮忙建模	35
3.5	建模与 Spec-first	36
4	Spec-first：需求文档的写法	39
4.1	为什么先写规格，而不是先写代码	40
4.2	五类必答问题	40
4.3	原则优先，格式自己长出来	42
4.4	好需求文档与差需求文档的对照	43
4.5	AI 协作实录：完整规格换来精准实现	44
5	架构先行：先画系统块图，再让 AI 写代码	47
5.1	为什么一大坨代码会让你失去对项目的控制	48
5.2	把阻尼振子交互页拆成五块以内	48
5.3	每一块背后的真实取舍	50
5.4	让 AI 按块实现，并用块图当对账单	51
6	写好提示词：Anthropic 六维框架	55
6.1	六个维度：一份完整工作说明书的骨架	56
6.1.1	D1 任务情境与角色：先告诉 AI 它是谁	56
6.1.2	D2 语气与信心设定：让 AI 学会说“我不确定”	56
6.1.3	D3 背景细节与资料：把它需要的全喂给它，不要让它猜	57
6.1.4	D4 详细执行指令：不是“做 X”，而是“先 X1，再 X2，然后 X3”	57
6.1.5	D5 示例与输出格式：贴一个范例，胜过描述十句	58
6.1.6	D6 最终提醒与强化：在结尾再钉一次最硬的约束	58
6.2	对比演示：同一个拟合任务，两份提示词，两种命运	58

II

数值方法

7	读懂并指挥 AI 写 Python	65
7.1	为什么“读和改”比“从零写”更重要	66

7.2	用一段完整脚本学最小的 Python	66
7.3	手算对账：本章的 no-AI 纪律	69
7.4	改参数重跑：让代码听你的，也看它怎么翻车	70
7.5	AI 协作实录：让 AI 改脚本，再亲手揪出它的错	71
8	一阶常微分方程与 Euler 法	73
8.1	从三个物理问题到同一个数学骨架	74
8.2	Euler 显式法：把连续演化切成小步	74
8.3	第一个完整例子：RC 电路放电	75
8.4	截断误差与步长：为什么 dt 不能太大	77
8.5	AI 协作实录：让 AI 写放电代码并审计它	77
8.6	用验证清单确认结果	78
9	二阶常微分方程与守恒律法庭	81
9.1	从弹簧振子到阻尼振子	82
9.2	状态向量约化：把一个二阶拆成两个一阶	82
9.3	用显式 Euler 推进阻尼振子	83
9.4	用能量审计数值解	85
9.5	显式 Euler 的能量漂移：代码没错，能量却在涨	86
9.6	半隐式 Euler：一行改动止住漂移（黑箱披露）	87
9.7	AI 协作实录：状态向量配错，能量三联图当庭抓获	87
10	升维：二维就是两个一维 ODE	91
10.1	核心思想：二维就是两个一维并列	92
10.2	主例：带空气阻力的抛体	93
10.3	用显式 Euler 解二维抛体	94
10.4	极限验证：本章的灵魂	95
10.5	量纲检查终端速度：一分钟的审计反射	97
10.6	AI 协作实录：让 AI 写抛体代码并审计它	97
10.7	选读：二维行星轨道	98
11	数据拟合：最小二乘与残差	101
11.1	从数据反推参数：为什么需要拟合	102
11.2	最小二乘原理	102
11.2.1	线性情形的闭式解	103

11.3	非线性拟合：把算法当黑箱，但要能验证	104
11.4	残差图比 R 方诚实	106
11.5	参数不确定性：用 $\pm 10\%$ 扫描代替统计推断	106
11.6	no-AI 基线：用纸笔和 Excel 对账线性拟合	107
11.7	AI 协作实录：用残差图推翻一个高 R 方的拟合	108
11.8	用验证清单确认拟合结果	109

12 蒙特卡罗初步：用随机数做物理 111

12.1	核心思想：用随机采样估计一个确定的量	112
12.2	第一个例子：撒点估计圆周率	112
12.3	收敛行为：精度随样本数怎么提高	114
12.4	第二个例子：用随机数估计定积分	115
12.5	第三个例子：一维随机行走与扩散	116
12.6	AI 协作实录：识破小样本的收敛假象	118

III

Trust but Verify: 验证与审计

13 五项验证清单 123

13.1	为什么需要一份固定的清单	124
13.2	第一项：量纲检查	124
13.3	第二项：解析极限	125
13.4	第三项：守恒律或单调性	125
13.5	第四项：参数扫描	126
13.6	第五项：与文献或参考值对比	126
13.7	把清单落成工件：checklist.md	127
13.8	综合演示：给一个阻尼振子工件跑完五项	127
13.9	AI 协作实录：表面通过却被第三项推翻	128

14 AI 输出审计：十类错误模式 131

14.1	三类速通：六十秒就能识别的基础错误	131
14.1.1	第一类，单位错：量纲不一致或者忘标单位	132
14.1.2	第二类，符号错：正负号反了或者方向搞歪了	132
14.1.3	第三类，数量级错：结果差了几个数量级却没察觉	132

14.2	七类进阶：症状、识别、修复	133
14.2.1	第四类，求解器误用：步长不当、用错积分器	133
14.2.2	第五类，收敛假象：把没收敛当成收敛了	133
14.2.3	第六类，虚假引用：编造不存在的文献	133
14.2.4	第七类，混淆模型：把两个物理模型的公式张冠李戴	133
14.2.5	第八类，过拟合掩盖：参数多到能拟合噪声	133
14.2.6	第九类，误用近似：在近似失效的区间继续用近似	134
14.2.7	第十类，丢边界条件或能量不守恒：初值边界设错、守恒律被破坏	134
14.3	完整审计流程演示：二十分钟拆一段错代码	134
15	文献核验：防止 AI 编造引用	137
15.1	AI 为什么会编造文献	138
15.2	三方核验：DOI、arXiv、学术搜索引擎	138
15.3	课程红线：未核验的引用不计入有效文献	139
15.4	AI 协作实录：逐条反查一份文献清单	140
15.5	一套负责任的文献 workflow	141

IV

Ship & Skill：交付与复用

16	把工件做成可分享的网页	145
16.1	为什么 ship 才算交付，PDF 报告不算	146
16.2	Streamlit 是什么	146
16.3	把脚本变成交互页	147
16.4	工程契约：入口文件、依赖清单、一键启动	149
16.5	默认交付是一键启动 zip，云端部署是可选加分	150
16.6	AI 协作实录：包装时被悄悄换掉的数据	150
17	AGENTS.md，给项目立规矩	153
17.1	规矩与任务：AGENTS.md 不是需求文档	154
17.2	一份物理项目的 AGENTS.md 该立哪几类规矩	154
17.3	为什么这一页纸如此强大：规矩随项目走，不随人走	155
17.4	一份完整的物理项目 AGENTS.md 长什么样	156
17.5	老师不发模板：规矩要你自己立	157
17.6	AI 协作实录：有没有这一页纸，差别有多大	158

18	构建你自己的 Skill	161
18.1	从重复的动作到一个可复用的文件	161
18.2	SKILL.md 的结构：工程契约与你的自由	162
18.3	Skill 是抽象，不是某个工具的语法	163
18.4	一份完整的 SKILL.md 示例	164
18.5	AI 协作实录：激活 Skill，喂一段陌生结果	165
19	AI 使用规范与学术诚信	169
19.1	诚信不是约束，是科学的底座	169
19.2	四栏 AI 使用说明：报告里必须有的一段	170
19.3	三条红线：越过即按学术不端处理	171
19.4	主动报告灰色地带：失报本身即违规	172
19.5	来源链：让每一个结论都可以回溯	172
19.6	认知所有权：为什么不让 AI 全做	173
19.7	工具的现实选择：国内栈是平替，不是降级	174

V

项目实战与案例

20	十个项目案例	179
20.1	P1 大角度单摆：精确周期与小角度近似的决裂	180
20.1.1	物理背景与核心问题	180
20.1.2	最低交付要求	181
20.1.3	扩展方向	181
20.2	P2 抛体含空气阻力：线性与二次阻力的两种世界	182
20.2.1	物理背景与核心问题	182
20.2.2	最低交付要求	182
20.2.3	扩展方向	182
20.3	P3 RC、RL、RLC 电路充放电：从一阶到二阶的电路演化	183
20.3.1	物理背景与核心问题	183
20.3.2	最低交付要求	183
20.3.3	扩展方向	184
20.4	P4 二维行星轨道：开普勒问题与守恒量审判	185
20.4.1	物理背景与核心问题	185
20.4.2	最低交付要求	185
20.4.3	扩展方向	185

20.5	P5 杯水冷却：牛顿冷却律与一把温度计	186
20.5.1	物理背景与核心问题	186
20.5.2	最低交付要求	186
20.5.3	扩展方向	187
20.6	P6 弹簧振子链：多自由度耦合与简正模式	187
20.6.1	物理背景与核心问题	187
20.6.2	最低交付要求	188
20.6.3	扩展方向	188
20.7	P7 蒙特卡洛估算 π 与定积分：让随机数替你算几何	189
20.7.1	物理背景与核心问题	189
20.7.2	最低交付要求	189
20.7.3	扩展方向	189
20.8	P8 自由落体录像：从手机视频到重力加速度	190
20.8.1	物理背景与核心问题	190
20.8.2	最低交付要求	190
20.8.3	扩展方向	190
20.9	P9 一维随机行走：从醉汉散步到扩散定律	191
20.9.1	物理背景与核心问题	191
20.9.2	最低交付要求	191
20.9.3	扩展方向	191
20.10	P10 光学衍射图案：从激光斑点到 sinc 平方拟合	192
20.10.1	物理背景与核心问题	192
20.10.2	最低交付要求	192
20.10.3	扩展方向	193
20.11	十个项目共享的同一套骨架	193
A	环境搭建与装机	195
A.1	装什么：三样东西，一个都不能少	195
A.2	在 Windows 和 macOS 上的安装要点	196
A.3	装好后的三件套自测	197
A.4	常见装机问题与对策	198
A.5	为什么要用虚拟环境	199
B	工具栈选择指南	201
B.1	先按你的处境分流	202
B.2	各类工具简介	202
B.3	关于境外工具	203

B.4	一句务实的建议	203
C	Fermi 估算题库	205
C.1	第一组：日常生活	206
C.2	第二组：地球与天空	207
C.3	第三组：物理与工程	207
C.4	第四组：生物与人体	208
C.5	练这身本领到底是为了什么	209
D	项目报告与展示写作	211
D.1	摆正位置：工件是主角，报告是它的说明	211
D.2	一份短报告里该有什么	212
D.3	答辩 slides：七页以内，一页一个要点	213
D.4	Demo 怎么讲：让观众亲手把图调动	213
	参考文献	217
	索引	219

前言

如果你刚念完一年物理，高等数学、线性代数、普通物理还热乎着，编程方面或许在 C 语言课上写过几个能编译通过的小程序，或许连那也没怎么碰过，这本书是写给你的。我不假设你会 Python，不假设你听说过 Git，更不假设你知道什么叫部署一个网页。如果你恰好都会，这本书一样能读，只是有些铺垫你可以略过；如果你都不会，请放心，这正是我设想的读者。

先把这本书要做什么、不做什么讲清楚，免得你抱着错的期待翻下去。它不是一本 Python 语法手册。市面上那种书已经够多了，而且在一个 AI 能替你把这层活干掉的时代，再花一个学期背语法，性价比正在迅速下降。这本书要教你的，是在 AI 时代不会过时的另一样东西：以“项目主理人”的身份去指挥 AI，在大约两周里，亲手造出一个能跑、能让别人点开就玩的物理交互工件。项目主理人，英文叫 *director*，意思是：你定方向、做判断、担责任，把从零敲代码这种最机械的活交给 AI。中间的分寸，第一章会细说，这里先点一句，你要学的不是怎么用 AI，而是怎么在用它的时候，始终不把判断权交出去。

全书围绕五条“反射动作”展开。说是反射，是希望它们将来不需要你刻意去想，遇到对应情境就自动发生。第一条 *Spec-first*，拿到一个模糊想法，先别急着让 AI 写代码，先用一页纸把它压成一份可验证的需求文档。第二条 *Architecture-before-code*，动手之前先画出系统的架构，想清楚数据从哪进、计算在哪发生、结果从哪出。第三条 *Trust but Verify*，AI 给的任何能跑的结果都先验一验，用量纲、符号、数量级这些物理判据去考它，而不是看它出了图就当真。第四条 *Ship something real*，把成果做成一个别人能真正用起来的工件，而不是一份躺在硬盘里的 PDF。第五条 *Skill-building*，把你反复做过的动作写成一份可复用的能力文档，让自己下一次省力。这五条从第一章一直贯穿到最后，是这本书真正的主线。

至于怎么用这本书，我说得直白些：它不会发给你一套模板让你照着格子填。我一直

觉得，给你一份现成的五栏需求文档表，你学到的不是“怎样想清楚一个问题的边界”，而是“怎样把模糊想法塞进五个格子”，后者恰恰是 AI 时代最危险的退化。所以这本书讲的是原则，给的是案例，剩下的格式留给你自己在动手的过程里慢慢长出来。每一章里都有一个叫“AI 协作实录”的盒子，把我真实地对 AI 说了什么、它回了什么、我又怎么一条条审它的过程摊开给你看，让你知道指挥与审计具体长什么样子。每章末尾还有思考题，那不是用来检查你记没记住名词的，是逼你停下来想一想。这本书里的代码你都可以原样拷下来真跑，我强烈建议你边读边动手，光看不跑，这本书的价值会折掉一大半。

还有一件事，是这本书所有设计的灵魂，我管它叫认知所有权，英文是 *cognitive ownership*。诚实地说，让 AI 把每一步都替你做完，短期里确实更快。但那些真正定义你不能成为一名科学家的能力，提出一个好假设、判断一份证据的分量、在论证被挑战时重新把它搭起来，全都得靠你亲手在一个个具体问题上反复磨出来。每一次你把这样的判断让渡出去，就少了一次磨它的机会，而且这种萎缩没有即时反馈。等你某天真正需要独立判断时，才会发现那块肌肉早就没了。这本书坚持让你自己写需求文档、自己画架构、自己做验证，全部用意就在这里：我们守的不是效率，是你的认知所有权。AI 是你的工具和放大器，不该是你的替身。

最后说句掏心的话。你的第一版需求文档，多半写得很糙；你第一次做验证，多半会漏掉最该查的那一项。这太正常了，我自己当年也一样。一份东西，糙过、被人指出过、又被你自己改过，它才真正变成你的。所以别怕交出一份不完美的初稿，怕的是从头到尾都没真正经手过。两周以后，当你看着自己亲手造出来、还能分享给别人的那个小工件时，希望你心里清楚的是：这里面的每一个判断，都是你自己做的。

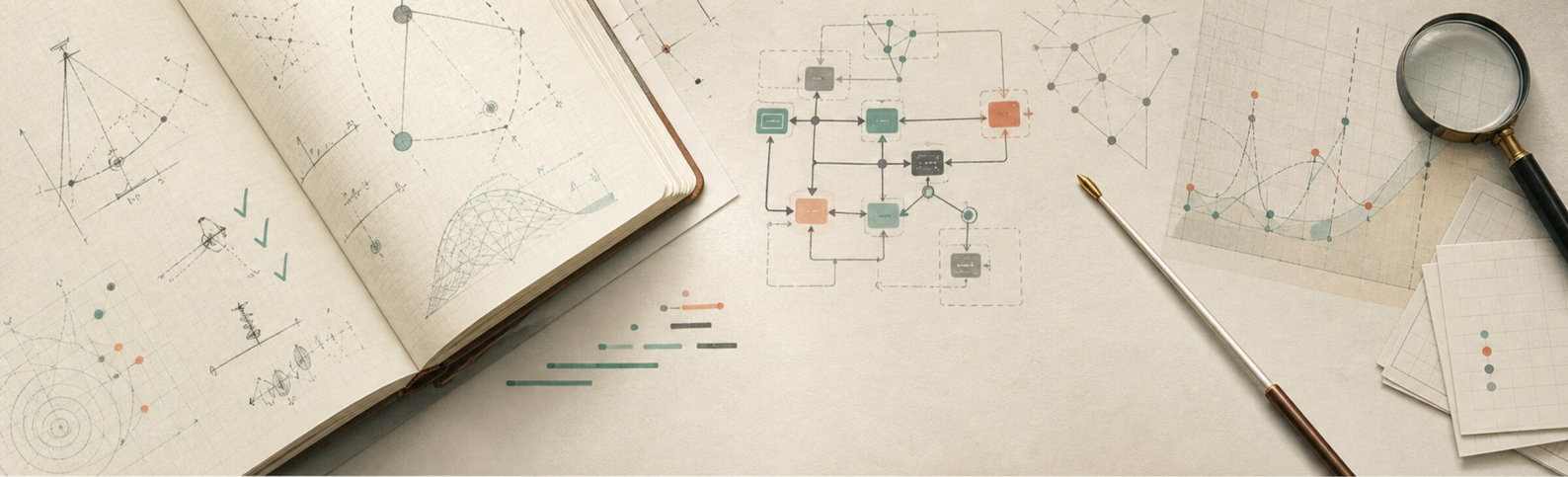
金磊

同济大学物理科学与工程学院

2026 年

像物理学家一样指挥 AI

1	像物理学家一样指挥 AI	15
1.1	什么是 vibe coding	
1.2	这门课加的一条硬约束：pause and ask why	
1.3	director 与抄代码实习生的区别	
1.4	五条反射动作总览	
1.5	cognitive ownership：为什么不让 AI 全做	
1.6	这门课在研究生命周期里的位置	
2	量纲分析与数量级估算	23
2.1	量纲：物理量自带的身份证	
2.2	量纲齐次性：一秒钟否决错误公式	
2.3	靠量纲猜出公式的形状：单摆与终端速度	
2.4	Fermi 估算：把估不出的大问题拆成估得出的小问题	
2.5	AI 协作实录：用量纲在六十秒内抓出错误公式	
3	建模：从模糊想法到可计算任务	31
3.1	什么是物理建模	
3.2	完整案例：一杯咖啡多久能喝	
3.3	模型的适用范围与失效	
3.4	AI 协作实录：让 AI 帮忙建模	
3.5	建模与 Spec-first	
4	Spec-first：需求文档的写法	39
4.1	为什么先写规格，而不是先写代码	
4.2	五类必答问题	
4.3	原则优先，格式自己长出来	
4.4	好需求文档与差需求文档的对照	
4.5	AI 协作实录：完整规格换来精准实现	
5	架构先行：先画系统块图，再让 AI 写代码	47
5.1	为什么一大坨代码会让你失去对项目的控制	
5.2	把阻尼振子交互页拆成五块以内	
5.3	每一块背后的真实取舍	
5.4	让 AI 按块实现，并用块图当对账单	
6	写好提示词：Anthropic 六维框架	55
6.1	六个维度：一份完整工作说明书的骨架	
6.2	对比演示：同一个拟合任务，两份提示词，两种命运	



1. 像物理学家一样指挥 AI

翻开这本书的人，绝大多数都让 AI 替自己写过代码。你可能问过它怎么用 Python 画一条正弦曲线，怎么读 CSV 文件，怎么解二次方程。AI 给了你一段能跑的代码，你拷进编辑器，运行，出图，搞定，翻篇。这一章想让你在合上书的时候，对刚才那串动作感到一点不安：代码能跑，不等于你懂了；问题解决，不等于你成长了。这门课要把你从一个“会让 AI 写代码的人”，扭转成一个“懂得指挥 AI、对 AI 的产出负责的人”。前者是 AI 时代随处可见的消费者，后者才是物理学院想培养的那种人，未来十年真正稀缺的那种。我们给这种人一个名字：，英文叫 **director**。

项目主理人

本章地图

小节	这一节解决什么	关键方法 / 产物
1.1	认识 vibe coding 这种新型编程方式	vibe coding 定义与双重性
1.2	建立 pause and ask why 这条硬约束	反射动作的本体、审计的起点
1.3	区分 director 与抄代码实习生的角色层级	定规格、画架构、做取舍、审产出、担责任
1.4	逐条讲清五条反射动作及其流水线	Spec-first、Architecture-before-code、Trust but Verify、Ship something real、Skill-building
1.5	论证为什么认知所有权不能让渡给 AI	cognitive ownership、判断力的不可外包性
1.6	定位这门课在科研全生命周期中的位置	idea-experiment-figure-share 缩影、反射复用

1.1 什么是 vibe coding

要理解这门课为什么这样设计，得先认识一个 2025 年才正式得名的现象。那年，Andrej Karpathy 在社交平台上写了一段话，描述了一种他称为“vibe coding”的新型编程方式。他说，这种编程“我只是看东西、说东西、运行东西、复制粘贴东西，大部分时候都能正常工作”[2]。话听起来随意，却精确捕捉到了一件大事：自然语言，你平时说的中文或英文，正在取代 Python、C++ 这类形式化编程语言，成为人和计算机的主要接口。

过去几十年，让计算机做事的唯一办法是学会它的语言。一个分号写错、一个缩进没对齐，程序就罢工。现在不一样了。你用人话描述想要什么，模型翻译成可运行的代码，你看结果，不满意就再用人话调整。编程的门槛被极大地拉低了。过去需要一个学期才能写出来的小程序，现在一句话就能让 AI 端上来。Karpathy 那句“大部分时候都能正常工作”，恰好说出了这种方式的诱人和危险：大部分时候能 work，意味着总有些时候不 work，而且你很可能看不出它什么时候在骗你。

对一个大一物理学生，这有两重意义。好的一面：你不必先花两年啃完数据结构和算法，才有资格去模拟单摆、拟合实验数据、画相图。建模，这件以往属于高年级和研究生的事，大一就能上手。坏的一面：正因为门槛低了，一种新的失败模式变得极其普遍。你拿到一段能跑、出图也漂亮的代码，却说不清它为什么对。甚至它根本就是错的，只是错得不显眼。物理不像聊天，一段措辞优雅但物理上错的代码，照样能用一条平滑曲线骗过你的眼睛。

Definition 1.1.1 : vibe coding. 一种以自然语言为主要编程接口的工作方式：人用日常语言描述意图，AI 生成可运行代码，人观察结果并继续用自然语言迭代，逐步逼近目标。它把编程的门槛从“掌握语法”降低到“能清楚表达想要什么、并能判断结果对不对”。

读懂这个定义的后半句很关键。门槛降下来的地方，正是“能判断结果对不对”。语法不再是你的负担，判断力却成了你唯一的、无法外包的本钱。这门课接下来两周要反复锤炼的，就是这个判断力。

本节要点

vibe coding 把编程的门槛从语法降到了判断力，这恰好翻转了学习的重心。能跑只是开始，能判断才是你不可外包的本钱。

1.2 这门课加的一条硬约束：pause and ask why

Karpathy 描述了一种事实上的工作方式，他没这样说就一定对。这门课在他的描述之上，钉了一条硬约束。请从现在起把它当身体反应来训练：AI 给了你一段能跑的代码，先停一下，问一句“它为什么 work”，再继续。英文叫 pause and ask why。

这话听起来像在给效率踩刹车，正好相反，它是你将来还敢信任 AI 的前提。想想看，你凭什么信一个工具？信的不是它永远不出错，是你有能力在它出错时揪出来。一个你完全没法检验的工具，它对的时候你不知道为什么对，错的时候也不知道为什么错，你对它的所谓“信任”其实就是赌博。停下来问“为什么 work”，是在你和 AI 之间建一笔交易：你

今天花六十秒搞清楚它这次为什么对，换来下次有底气交给它更难的任务。一句话：先停下来问为什么，是我们为了下次能信任 AI 而和它做的约定。

更重要的是，这一停不是表面上装个样子。它是整门课最核心的那条神经反射的本体。我们会反复讲到“审计 AI 的输出”。所谓审计，本质就是这一停。你停下来的那一刻，物理学家的本能就该接管：这个量的量纲对吗？符号的正负号合理吗？数量级离谱吗？第八章 RC 电路那个例子里，AI 把时间常数写成了 R/C ，曲线照样画得出来，只是衰减快得离谱。揪出它靠的不是把循环读一百遍，而是停下来做一次六十秒的量纲检查，发现 R/C 的量纲根本不是秒。那一刻的停顿，就是 *pause and ask why* 落到实处的样子。

本能 · Trust but Verify

看到任何能跑的代码或漂亮的结果，先停一下，问“它为什么该长这样”。这一停不是浪费时间，是你和 AI 之间的信任交易：你今天有能力检验它，明天才敢把更重的任务交给它。停下来之后，让物理学家的本能接管，先查量纲、符号、数量级，这三样最快暴露错误。

1.3 director 与抄代码实习生的区别

现在把这门课要你演的角色说清楚。设想软件公司里一个初级实习生。他每天接到明确的任务，“写个函数把摄氏度转成华氏度”，然后埋头敲代码。他不关心这个函数为什么需要，不关心它跟系统其他部分怎么衔接，更不关心做完之后交付给谁、对方拿去做什么。他是执行末端。别人替他想好了一切，他只负责把想法翻译成代码。

AI 时代最尴尬的处境，就是把自己活成了这样一个实习生，而且连代码都让 AI 替你敲。你成了实习生的实习生，站在价值链的最末端，唯一做的事是把 AI 吐出来的东西复制粘贴。这正是这门课要你极力避免的位置。

项目主理人做的是完全不同层级的事。一个 director 拿到的不是“写个函数”这种现成任务，而是一张模糊的草图，比如“我想看看阻尼大小怎么影响振子的衰减”。他做的第一件事，是把模糊想法压成一份谁都能照着实现的规格：做什么，输入是什么，输出是什么，怎么算成功，用什么技术。这份规格叫需求文档。接着他画出系统架构，想清楚数据从哪进、计算在哪发生、结果从哪出，把整个项目拆成几个能各自验证的块。然后才让 AI 去填每块的代码，自己始终握着全局，不被任何一段代码的细节淹没。AI 写完了，他要审，像审下属交上来的报告那样审：查量纲、查符号、查数量级、查边界。审出问题，他会改 prompt 让 AI 自己修。最后，他把这一切 ship 出去，做成一个别人点开就能玩、一键就能启动的真东西，而不是躺在硬盘里的 PDF。

注意到了吗，从头到尾，director 做的全是定方向、做判断、担责任。唯独“从零敲代码”这件最机械的活，可以放心交给 AI。这恰好对调了传统编程课的重心。传统课花大量时间教你记语法、调分号；这门课假设语法这层 AI 替你干了，把你解放出来，专门练那些 AI 干不了、也不该替你干的事：提出问题、压成规格、画出架构、做出取舍、验证产出。这五件事连起来就是这门课的主线，五条反射动作。

本节要点

director 和抄代码实习生的分水岭不在代码能力，在责任归属。前者把规格、架构、取舍、审计、交付攥在手里，只把敲代码的机械劳动交给 AI；后者则反过来，连判断也一并外包。

1.4 五条反射动作总览

这门课要在两周里把练成本能。叫“反射”，是因为我们希望它们将来不需要你刻意想，遇到对应情境就自动发生，像手碰到烫的东西立刻缩回来一样。下面逐条说。

第一条是 Spec-first，先写规格再写代码。拿到一个模糊想法，物理学生最容易犯的错是立刻冲去让 AI 写代码，结果自己都没想清楚要什么，来回折腾好几轮还在原地。正确的反射是：先用一页纸把想法压成可验证的规格，逼自己回答五个问题，做什么、输入是什么、输出是什么、怎么算成功、用什么技术栈以及为什么。五个问题答清楚了，给 AI 的指令才精准，AI 才有可能一次到位。这一页纸就是需求文档。

本能 · Spec-first

拿到模糊想法，先别让 AI 写代码。花半小时把想法压成一页需求文档，逼自己回答清楚：做什么、输入输出是什么、怎么算成功、用什么技术。可验证的边界先定下来，再让 AI 动手。

第二条是 Architecture-before-code，先画架构再写代码。一个稍微像样的物理建模项目，通常包含好几个部分：参数从哪输入，物理状态怎么表示和更新，结果怎么算，图怎么画。如果不先把这些块和它们之间的数据流画清楚，就让 AI 一股脑写一大段代码，你很快会被细节淹没，出了问题都说不清是哪块崩的。正确的反射是先画一张系统块图，把项目拆成不到五块、每块都能单独验证的结构，然后让 AI 按块实现。这样你始终站在全局高度俯视整个系统，不是趴在某一行代码里出不来。

本能 · Architecture-before-code

让 AI 写代码之前，先画一张系统块图：数据从哪进、模型在哪算、结果从哪出。把项目拆成几个能各自验证的块，再让 AI 逐块实现。你掌握的是全局，不被任何一块的代码细节淹没。

第三条是 Trust but Verify，信任但要验证。这条前面用一整节铺垫过了，它就是 pause and ask why 的工件化版本。AI 给的结果你可以用，但必须验。验的方法不是读一百遍代码，得用物理判据去考它：量纲对不对、符号合不合理、数量级离不离谱、解析极限对不对、守恒量守不守。把这些检查收拢成一份验证清单，这门课最重要的工件之一。一个不做验证就把 AI 输出当结论的人，迟早会在答辩现场被一句“你这量纲对吗”问得哑口无言。

本能 · Trust but Verify

AI 的输出可以用，但必须验。不靠逐行读代码，而靠物理判据去考它：量纲、符号、数

量级、解析极限、守恒量。把这些检查写成一份验证清单，每一项背后都是一个能推翻结论的判据，不是形式主义的打勾。

第四条是 Ship something real，交付一个真东西。这门课的交付件不是 PDF 报告，是一个别人点开网址就能玩、一键就能启动的物理交互工件。报告可以附在工件旁边，但主角是那个能跑、能调参数、能看到图随之变化的真东西。这条看起来是工程要求，背后是一种科学态度：一个 ship 不出去、别人无法复现、没法亲手把玩的结果，在科学交流的意义约等于不存在。把东西做到能交付，会逼你把前面所有环节都做扎实。

本能 · Ship something real

交付的不是 PDF，是一个别人点开网址就能玩、一键就能启动的真工件。让没看过你代码的人也能调参数、看到图变。ship 不出去的结果，在科学交流里约等于不存在。

第五条是 Skill-building，把反复做的动作积累成可复用的能力。一个项目里，某个动作做过三遍以上，比如每次都要让 AI 按你的规矩验证数值解，或者每次都要让它按统一风格画图，你就该把它抽象出来，写成一份描述触发场景和工作流的文档，让 AI 下次自动照办。这种积累下来的能力，我们沿用业界的叫法称为 Skill。它的意义在于：你不是每次都从零和 AI 重新磨合，而是在不断积累属于自己的、可复用的工作方法，让你这个 director 越当越省力、越当越专业。

本能 · Skill-building

把项目里做过三遍以上的动作抽象成可复用的 Skill 文档，描述清楚它在什么场景触发、AI 该执行什么工作流。你积累的是一次次可复用的方法，不是一次性代码。

这五条不是孤立清单，它们是一条流水线：先 Spec-first 把想法压成规格，再 Architecture-before-code 画出结构，AI 填码的同时用 Trust but Verify 守住每块的物理正确性，最后 Ship something real 交付出去，过程中不断 Skill-building 积累方法。两周里你会在一个真实物理项目上把这条流水线走好几遍，走到它变成不假思索的反射为止。

本节要点

五条反射不是五条孤立的纪律，是从模糊想法到可交付工件的一条完整流水线。Spec-first 定边界，Architecture-before-code 画骨架，Trust but Verify 守底线，Ship something real 让成果可见，Skill-building 把经验固化为可复用方法。

1.5 cognitive ownership: 为什么不让 AI 全做

讲到这里，一个尖锐的问题大概已经冒出来了：既然 AI 这么强，需求文档它能写，架构它能画，代码它能敲，验证它大概也能帮做，那一个大一学生，何必费这个劲一步步来？让 AI 全包了，岂不又快又好？

老实说，如果只看短期效率，让 AI 全做确实更快。但这个问题的答案不在效率那一层。答案在一个叫认知所有权的概念上，英文是 cognitive ownership。它说的是：如果你

把每一步判断都委托给 AI，这学期的作业也许交得又快又漂亮，但那些真正定义你将来不能成为科学家的能力，会在你看不见的地方悄悄萎缩。哪些能力？提出一个好假设的能力，判断一份证据有多大分量的能力，论证被人挑战时重新把它搭起来的能力，回应批评、修正自己的能力。这些能力没一样是天生的，全都得靠你亲手在一个又一个具体问题上反复磨出来。每让渡一次判断给 AI，你就少磨一次。

更可怕的是，这个过程没有即时反馈。你不会在 AI 替你写完需求文档的那一刻觉得自己变笨了，恰恰相反，你会觉得自己很高效。萎缩是缓慢的、无声的，等到某天你需要独立提出原创想法、独立判断数据不可信，才会发现那块肌肉早就没了。这门课坚持让你自己写需求文档、自己画架构、自己做验证，哪怕第一版写得很糟、被老师当众撕开点评，也要你自己长出来。全部用意就在于此：我们守的不是效率，是你的认知所有权。AI 应该辅助你做信息转化、让过程透明、支持你反思修正，而不是取代认知投入本身。一句话：AI 是你的工具和放大器，不是你的替身。

还有一点值得说清楚。守不等于排斥 AI，更不是要你退回从零敲代码的老路。这门课从头到尾都在用 AI，而且用得很重。区别在于，你是带着判断去用它，还是放弃判断去依赖它。一个好的 director 会让 AI 做掉所有机械的、可被验证的活，而把提出问题、做出取舍、对结果负责，这些不可外包的核心，牢牢攥在自己手里。这条界线，就是这门课希望你两周后能本能把握的东西。

AI 陷阱预警 · 把判断整个让渡给 AI

最危险的失败模式不是 AI 写错了代码，是你不再判断对错。当你开始把“这个结果对不对”“这个方法合不合理”“这个假设站不站得住”这类判断也一并交给 AI，你就从一个项目主理人退化成了传声筒。AI 写错代码，你能审出来；可你一旦停止思考，连“它错了”这件事本身都将无人发现。短期里这让你显得高效，长期里它让你失去成为科学家的资格。守住判断权，是这门课对你唯一不可妥协的要求。

本节要点

cognitive ownership 的底线是：AI 替你做的必须是你能验证的，你做的必须是 AI 不能替的。提出问题、做取舍、担责任这三件事一旦外包，你就不再是项目的 owner，只是 AI 的传声筒。

1.6 这门课在研究生周期里的位置

最后，把视野放大一点，看看你现在练的这套东西在一个完整的科研生涯里处于什么位置，又会通向哪里。

一个科研项目从无到有，大致走过这几段路：萌生一个想法，做实验或计算检验它，把结果整理成图表，分享出去让同行看到。用英文串起来就是 idea, experiment, figure, share。这门两周的课覆盖的，正是从想法到实验、到图表、到分享这一段。你为一个物理小项目提出问题、建模计算、画图、最后 ship 成一个可交互的工件，走的的就是这条路的一

个完整缩影。

到了大三大四乃至研究生阶段，你会学到这条路的后续：把结果写成正式学术论文，回应同行评审的尖锐质疑，给审稿人写答辩信，再把工作传播给更广的受众。这些环节你今天还接触不到。但请记住这门课最有抱负的一句话：你今天练的这五条反射动作，到了那些后续阶段会一字不变地继续适用，变的只是工件本身。

这不是修辞。你今天为一个 Streamlit 物理交互页写需求文档，将来给一篇投往物理评论的论文定研究规格，本质是同一个 Spec-first 反射，只是把“做什么、怎么算成功”从小项目放大成了研究。你今天用量纲和守恒律审 AI 写的模拟代码，将来在同行评审里审别人的稿子、或回应审稿人对自己论文的质疑，本质是同一个 Trust but Verify 反射，只是被审的对象从代码变成了论证。你今天把项目 ship 成一个能玩的网页，将来把一项研究 ship 成论文、组会报告、学术演讲，本质是同一个 Ship something real 反射。换句话说，这门课不是教你一套两周后就过期的技巧，而是在帮你装上一组能用一辈子的神经反射。物理学院后面四年的专业课大多不会专门教这些，而它们恰恰是 AI 时代物理学生最稀缺的能力。这门课，就是来补这一块的。

本节要点

这门课练的五条反射，从 idea 到 experiment 到 figure 到 share，覆盖了科研生命周期的前半段。它们会在论文写作、同行评审、学术传播中一字不变地复用，是能用一辈子的神经反射。

本章小结

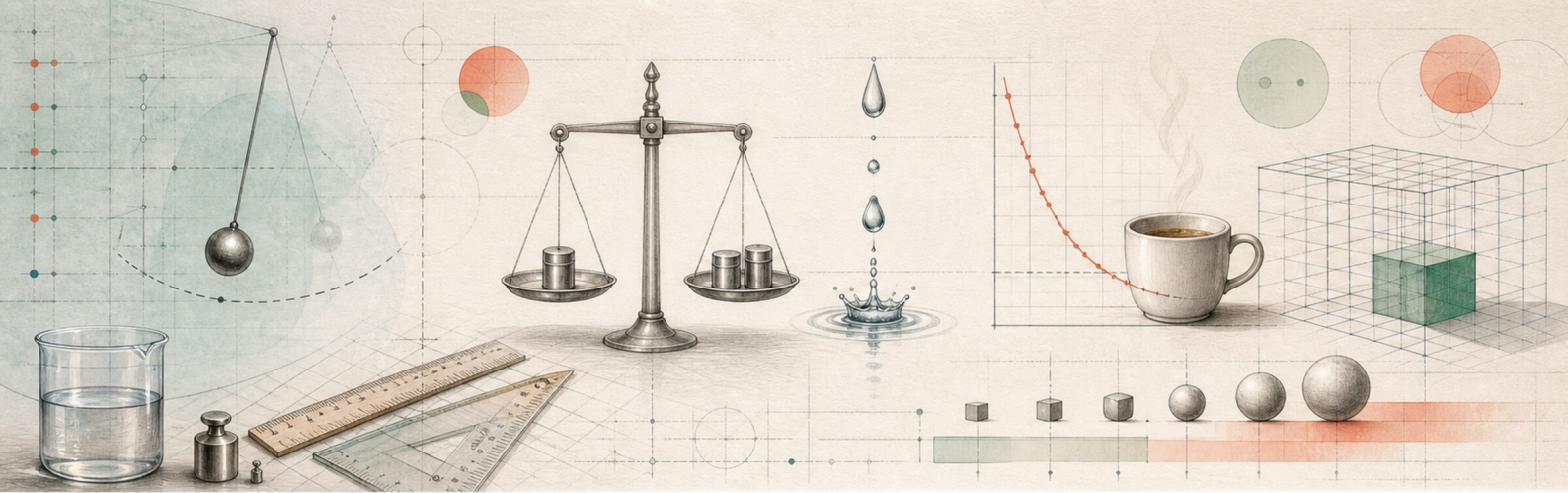
这一章做了一次身份的扭转。vibe coding 把编程的门槛从语法降到了判断力，而判断力无法外包。Karpathy 描述之上我们钉了一条硬约束 pause and ask why：它是信任 AI 的前提，也是后面所有审计动作的本体。director 定规格、画架构、做取舍、审产出、担责任，是项目主理人；抄代码实习生只负责复制粘贴，是我们要你极力避免的位置。五条反射动作 (Spec-first、Architecture-before-code、Trust but Verify、Ship something real、Skill-building) 连起来是一条从想法到交付的流水线。核心的底线是认知所有权：AI 替你做的必须是你能验证的，你做的必须是 AI 不能替的。这五条反射，放在 idea 到 experiment 到 figure 到 share 的研究生周期里，将来在论文写作和同行评审中会一字不变地复用。

从下一章开始，我们就要在一个个具体的物理问题上，把这五条反射动作一遍遍练成本能。你会发现，真正难的从来不是让 AI 写出代码，是写出代码之后你还敢不敢、能不能停下来问一句“它为什么对”。

思考题

1. 我们反复强调，看到 AI 给出能跑的代码要先停下来问“它为什么 work”。可这分明会拖慢你的速度。请你结合“信任”这个词，论证为什么 pause and ask why 不是浪费时间，而是你将来还敢把更难的任务交给 AI 的前提。试着举一个你自己用 AI 的真实经历，说明如果当时停下来问了，会有什么不同。

2. 假设这学期有一门别的课布置了一份编程作业，你完全可以让 AI 从头到尾全做，自己一行不写、一处不验，交上去也能得高分。短期看你赢了，省下大把时间。请你从认知所有权的角度，说清楚这样做长期会让你失去什么。这种损失为什么难以被你当下察觉？
3. 五条反射动作里，Trust but Verify 这一条将来会“一字不变”地用到你大三大四的论文写作和同行评审里。请你设想自己几年后要审一篇同学的论文稿，把“查大纲、查符号、查数量级、查解析极限、查守恒量”这套审代码的动作，翻译成你审一篇论文论证时会问的五个对应问题。



2. 量纲分析与数量级估算

在这门课里，你会一遍遍把物理问题交给 AI 去推公式、写代码、画图，而 AI 几乎总能返回一些看起来很专业的东西。问题在于，看起来专业和真的对，是两码事。一个合格的项目主理人，不能等到代码跑出难看的曲线、答辩现场被人当场质疑，才发现公式从一开始就错了。我们需要一道关卡，在任何计算之前、不用开电脑、几十秒就能完成，把大量错误挡在门外。这道关卡就是，配上一项孪生本领：估算。这一章全程用纸和笔，一行代码不写，先把”物理学家的第一反射”练出来。

量纲分析

本章地图

小节	这一节解决什么	关键方法 / 产物
2.1	认识量纲作为每个物理量的身份标识	基本量纲 L 、 M 、 T 、无量纲量
2.2	掌握量纲齐次性原理并用它一秒否决错误公式	量纲齐次性、必要条件非充分条件
2.3	用量纲分析反推公式的幂次结构	单摆周期 $\sqrt{L/g}$ 、终端速度
2.4	学会 Fermi 估算把大问题拆成可估的小问题	三道 Fermi 题、数量级判断
2.5	现场演示量纲检查在六十秒内抓出 AI 的错误	ailog 单摆周期公式中写错的指数

2.1 量纲：物理量自带的身份证

每一个物理量都不只是一个数字，它还带着一种身份，告诉你它衡量的是什么类别的东西。长度、时间、质量、速度、能量，彼此不可混淆。你不会说一段路有 5 公斤长，也不

会说一块石头重 3 秒钟，这些组合在物理上根本没有意义。这种“是什么类别”的信息，就叫量纲。

国际单位制选出了几个最基本、彼此独立的物理量作为基石，其余所有量纲都由它们组合而来。对大一普物涉及的力学与热学，最常用的三个基本量纲是长度、质量和时间，习惯上记作 L 、 M 、 T 。表示一个量的量纲时，用方括号 $[\cdot]$ 把它框起来。比如速度是单位时间走过的距离，所以 $[v] = LT^{-1}$ ；加速度是速度的变化率，再除一次时间， $[a] = LT^{-2}$ ；力等于质量乘加速度， $[F] = MLT^{-2}$ ；能量是力乘距离， $[E] = ML^2T^{-2}$ 。涉及电磁学和热学后，国际单位制还会添上电流 I 、温度 Θ 、物质的量、发光强度等基本量纲。不过在我们这门课里，把 L 、 M 、 T 三个用熟，已经足够对付绝大多数把关问题了。

Definition 2.1.1 : 物理量的量纲. 一个物理量的量纲，是用基本量纲（力学中通常取长度 L 、质量 M 、时间 T ）为底、各自取适当幂次相乘所得的表达式，记作 $[Q] = L^a M^b T^c$ 。量纲只关心物理量属于哪一类、由基本量如何组合而来，不关心它数值多大，也不关心用米还是厘米去量。纯数字、角度、比值这类不带任何基本量纲的量，称为无量纲量，量纲记作 1。

这里要特别留意无量纲量。像 π 这样的常数、两个同类量之比、弧度制下的角度，都是无量纲的。指数函数 e^x 、对数 $\ln x$ 、三角函数 $\sin x$ 的自变量也必须是无量纲的。道理不复杂：这些函数都能展成幂级数， $e^x = 1 + x + x^2/2 + \cdots$ ，要让 1 、 x 、 x^2 能相加， x 只能是纯数。所以，当你看到 AI 写出 e^{-t} 而 t 带着秒的单位时，警报就该响了。正确的写法是 $e^{-t/\tau}$ ，让分母上的某个时间常数 τ 约掉时间的量纲。

本节要点

量纲是物理量的身份，告诉你它衡量的是长度、时间、质量还是它们的某种组合。无量纲量是特殊的纯数，指数、对数、三角函数的自变量必须是无量纲的，看到 e^{-t} 就该拉警报。

2.2 量纲齐次性：一秒钟否决错误公式

掌握了量纲这套身份系统，就能引出本章最锋利的工具。它的出发点是一条几乎不证自明、却威力巨大的原理：任何有物理意义的等式，等号两边的量纲必须完全一致；用加号或减号连起来的各项，量纲也必须彼此相同。道理很朴素，不能把 3 米和 5 秒加出一个有意义的结果，就像不能把 3 个苹果和 5 块石头说成 8 个什么东西。

Definition 2.2.1 : 量纲齐次性原理. 一个物理方程在量纲上是齐次的，指的是等式两边的量纲完全相同，并且方程中用加减号相连的每一项也都具有相同的量纲。任何违反这一原理的等式，无论数字看起来多诱人，在物理上都必定是错的。量纲齐次只是方程正确的必要条件而非充分条件：齐次的方程未必对，但不齐次的方程一定错。

请把最后一句话刻进脑子，因为它精确界定了这件武器的用途。量纲检查不能帮你确认一个公式是对的，它能帮你的是迅速、低成本地确认一个公式是错的。在你和 AI 协作时，后者恰恰是更高频、更要命的需求：你不需要把 AI 给的每个公式都重新推导一遍，只

需要在它返回的一堆表达式里，几秒钟筛掉那些量纲对不上的，剩下的再细究。

本节要点

量纲齐次性是必要条件而不是充分条件：齐次的方程未必对，但不齐次的方程一定错。它不能帮你确认公式正确，却能帮你秒级否决错误，是你和 AI 协作时成本最低的防线。

举个最直白的例子。你想回忆自由落体下落距离和时间的关系，AI 或你自己写出了 $s = \frac{1}{2}gt$ 。检查一下：右边 g 的量纲是 LT^{-2} ，乘上时间 t 的量纲 T ，得到 LT^{-1} ，这是速度的量纲，而左边 s 是长度 L 。两边对不上，这个公式当场毙掉。正确的应该是 $s = \frac{1}{2}gt^2$ ，右边量纲 $LT^{-2} \cdot T^2 = L$ ，与左边一致，过关。注意那个 $\frac{1}{2}$ 是无量纲纯数，量纲检查既不会因为它存在而报错，也根本看不见它。这一点后面还要专门回来说。

2.3 靠量纲猜出公式的形状：单摆与终端速度

量纲齐次能否决错误，还能在你对某个现象一无所知、连方程都列不出来的时候，反过来帮你猜出公式该长什么样。这套做法叫量纲分析法：先列出你认为这个物理量可能依赖的所有因素，给每个因素配一个待定幂次，再要求整个组合的量纲和目标量一致，解出这些幂次。

我们先拿单摆来演示，这也是这门课最爱用的开场例子。一根长度为 L 的细绳吊一个小球，在重力下小幅摆动。想知道它来回一次的周期 $T_{\text{周期}}$ 由什么决定。凭直觉，周期可能跟绳长 L 、重力加速度 g 和小球质量 m 有关。于是假设

$$T_{\text{周期}} = C L^a g^b m^c,$$

其中 C 是一个待定无量纲常数， a 、 b 、 c 是待定幂次。现在写出两边的量纲。左边周期的量纲是 T 。右边： L 的量纲是 L ， g 的量纲是 LT^{-2} ， m 的量纲是 M ，于是右边的量纲是 $L^a (LT^{-2})^b M^c = L^{a+b} T^{-2b} M^c$ 。要让两边量纲齐次，对照 L 、 M 、 T 三种基本量纲的幂次，必须有

$$L: a + b = 0, \quad M: c = 0, \quad T: -2b = 1.$$

三个方程立刻解出 $b = -\frac{1}{2}$ ， $a = \frac{1}{2}$ ， $c = 0$ 。最妙的是 $c = 0$ ，它意味着质量根本不出现在周期里。这正是关于单摆最反直觉、也最深刻的结论之一。伽利略当年正是据此察觉单摆等时性的端倪，而我们仅凭量纲，没列任何运动方程，就预言了它。把幂次代回去，得到

$$T_{\text{周期}} = C \sqrt{\frac{L}{g}}.$$

量纲分析能告诉你的就到此为止：周期正比于 $\sqrt{L/g}$ 。但那个无量纲常数 C 它给不出来。要真正解一遍单摆的运动方程，才能知道在小角度下 $C = 2\pi$ ，也就是熟悉的

$T_{\text{周期}} = 2\pi\sqrt{L/g}$ 。这恰好画出了工具的边界：它能锁定公式里各物理量的幂次组合，却对前面那个纯数字无能为力。

本节要点

量纲分析法在你对现象一无所知时就能猜出公式的幂次结构，单摆周期正比于 $\sqrt{L/g}$ 、终端速度正比于 $\sqrt{mg/(\rho A)}$ ，但无量纲系数（如 2π ）永远给不出，必须靠解方程或实验来定。

再看一个稍微复杂、也更贴近本章后面 Fermi 题的例子：终端速度。一个物体在流体里下落，速度越快，阻力越大。当阻力增大到恰好抵消重力时，物体不再加速，此后匀速下落，这个速度就是终端速度 $v_{\text{终}}$ 。对在空气里下落、个头不太小、速度不太慢的物体（比如雨滴或跳伞者），阻力近似正比于截面积、空气密度和速度的平方，写成 $F_{\text{阻}} \sim \rho A v^2$ ，其中 ρ 是空气密度、 A 是迎风截面积。终端时阻力等于重力 mg ，于是

$$\rho A v_{\text{终}}^2 \sim mg \implies v_{\text{终}} \sim \sqrt{\frac{mg}{\rho A}}.$$

用量纲核对一下这个结果。右边根号里， mg 的量纲是 $M \cdot LT^{-2} = MLT^{-2}$ ，即力； ρ 是单位体积的质量，量纲 ML^{-3} ； A 是面积，量纲 L^2 。于是根号里的量纲是 $\frac{MLT^{-2}}{ML^{-3} \cdot L^2} = \frac{MLT^{-2}}{ML^{-1}} = L^2T^{-2}$ ，开方后是 LT^{-1} ，正好是速度。量纲过关，公式的形状可信。这个 $v_{\text{终}} \sim \sqrt{mg/(\rho A)}$ 一会儿估算雨滴时直接拿来用。

2.4 Fermi 估算：把估不出的大问题拆成估得出的小问题

费米（Enrico Fermi）有一项著名的本事：面对一个看似无从下手、数据残缺的问题，他能在信封背面、几分钟内估出一个数量级正确的答案。据说在第一颗原子弹试爆的现场，他撒下一把碎纸片，凭冲击波把纸片吹出多远，当场估出爆炸当量，事后与精密仪器测的结果同在一个数量级。这项本事的精髓不在记数据，在一种拆解的智慧：把一个你估不出的大问题，拆成若干个你能凭常识各报一个粗略数字的小问题，再把这些数字乘除起来。每一步的误差有大有小，但因为它们彼此独立、方向随机，相乘之后往往相互抵消，最终的数量级常常出奇地准。

这里要先说清楚我们追求的是什么。Fermi 估算不追求精确，它追求的是数量级，也就是答案大约是 10 的几次方那个量级。在物理建模里，数量级常常比小数点后几位重要得多：一个结果是几秒还是几小时、几米还是几千米、室温还是上万度，这类判断决定了你的模型有没有跑偏，也决定了你能不能一眼看穿 AI 给的数字是否离谱。下面详细做三道经典 Fermi 题，每道都把拆解链条完整摊开。

2.4.1 第一题：人在月球上能跳多高

想估一个普通人站在月球表面，原地纵跳能跳多高。直接想很难，先拆。第一步，问自己在地球上能跳多高。一个普通人原地摸高，重心大约能抬起 0.5 m，这是人人都凭经验

报得出的数字。第二步，问起跳和落地背后的物理机制。无论在地球还是月球，你蹬地时双腿做的功是差不多的，因为那取决于你的肌肉，不取决于你在哪颗星球。这份功 W 转化成重力势能 mgh ，于是 $W = mgh$ ，跳起高度 $h = W/(mg)$ 。第三步，注意到同一个人在两颗星球上 W 和 m 都不变，所以跳高 h 只反比于当地重力加速度 g 。

地球和月球一比： $\frac{h_{\text{月}}}{h_{\text{地}}} = \frac{g_{\text{地}}}{g_{\text{月}}}$ 。月球表面重力加速度大约是地球的六分之一（这个数值许多人记得，记不得也可以由月球质量与半径估），所以

$$h_{\text{月}} \approx 6 \times h_{\text{地}} \approx 6 \times 0.5 \text{ m} = 3 \text{ m}.$$

数量级答案是几米。这个结论本身就很有教益：人在月球上能轻松跳过自己两倍身高，但也别幻想能跳上几十米的楼，那要么是高估了肌肉，要么是把六分之一记成了更夸张的比例。一个值得做的旁证：穿宇航服会拖累跳高，所以阿波罗宇航员的实际弹跳要打个折扣，但数量级仍在一米尺度，与几米的估算相符。

2.4.2 第二题：一杯咖啡冷却到能喝要多久

一杯刚倒出的咖啡大约 90°C ，烫得没法喝。想估它降到大约 60°C 能入口的温度，需要多久。核心物理是热量从咖啡散失到环境，散热速率大致正比于咖啡与室温的温差，这正是牛顿冷却的精神。但 Fermi 估算不必解微分方程，只要抓住冷却的特征时间尺度。

先拆。需要散掉多少热量？一杯咖啡约 0.25 kg 的水，水的比热约 $4200 \text{ J}/(\text{kg} \cdot \text{K})$ 。从 90°C 降到 60°C 降了 30 K ，要散掉的热量 $Q \approx 0.25 \times 4200 \times 30 \approx 3 \times 10^4 \text{ J}$ 。再问散热有多快。热量主要经杯壁和液面通过对流和蒸发散失，散热功率 P 不易精确算，但可以从经验锚定：一杯咖啡放桌上，差不多十几分钟就从烫嘴凉到温吞。把这个经验翻译成功率， $P \approx Q/t$ 。若 t 取十分钟即 600 s ，则 $P \approx 3 \times 10^4 / 600 \approx 50 \text{ W}$ ，这个量级和一只裸露热水杯向室温空气散热的常识相符，自洽。

那么降这 30 K 所需时间就是 $t \approx Q/P \approx 3 \times 10^4 / 50 \approx 600 \text{ s} \approx 10 \text{ min}$ 。数量级答案：十分钟，几分钟到二十分钟这个区间。这里有个自洽的小圈：用十分钟的经验锚定了功率，又算回十分钟，看似循环。它的价值在于检验内部一致性：假如你拍脑袋猜要两小时，用上面 50 W 的功率一算，散掉的热量得有几十万焦耳，远超一杯水能存的热，矛盾立刻暴露。真正更稳的做法是独立估出散热功率（由杯子表面积、温差和对流换热系数），再算时间。两条路给出同一个数量级，才是 Fermi 估算最让人信服的样子。

2.4.3 第三题：雨滴落地的终端速度

最后估一颗雨滴落到地面大约有多快。前面用量纲已经推得终端速度 $v_{\text{终}} \sim \sqrt{mg/(\rho A)}$ ，现在只要把各量的数量级填进去。拆解如下：一颗中等雨滴半径约 $r \approx 1 \text{ mm} = 10^{-3} \text{ m}$ 。它的质量 $m = \rho_{\text{水}} \cdot \frac{4}{3}\pi r^3$ ，水密度 $\rho_{\text{水}} \approx 10^3 \text{ kg/m}^3$ ，于是 $m \approx 10^3 \times 4 \times (10^{-3})^3 \approx 4 \times 10^{-6} \text{ kg}$ ，即几毫克。迎风截面积 $A = \pi r^2 \approx 3 \times (10^{-3})^2 \approx 3 \times 10^{-6} \text{ m}^2$ 。空气密度 $\rho \approx 1.2 \text{ kg/m}^3$ ，取约 1。重力加速度 $g \approx 10 \text{ m/s}^2$ 。

代进去：

$$v_{\text{终}} \sim \sqrt{\frac{mg}{\rho A}} \approx \sqrt{\frac{4 \times 10^{-6} \times 10}{1 \times 3 \times 10^{-6}}} = \sqrt{\frac{4 \times 10^{-5}}{3 \times 10^{-6}}} \approx \sqrt{13} \approx 4 \text{ m/s}.$$

考虑到阻力公式只取了正比关系、丢掉了—个量级为 1 的无量纲阻力系数（实际约 0.5，会把答案再抬高点），真实中等雨滴的终端速度约为 6 到 9 m/s。我们估出的几米每秒落在正确的数量级上。这个结果也解释了为什么雨打在身上是温柔的拍打而不是子弹般的冲击：终端速度只有每秒几米，远不是从几千米高空自由落体该有的每秒上百米，空气阻力把雨滴的速度死死摁在了这个量级。

本节要点

Fermi 估算的精髓不在记住数据，在把大问题拆成小问题。每一步的数字可以粗，但因为误差方向随机、彼此独立，乘除之后往往相互抵消，最终数量级常常准得出奇。三道 Fermi 题（月球跳高、咖啡冷却、雨滴终端速度）都在正确的数量级上落了地。

2.5 AI 协作实录：用量纲在六十秒内抓出错误公式

现在把这一章的工作方式现场演示一遍。作为项目主理人，你会频繁让 AI 替你回忆或推导公式，而 AI 偶尔会返回一个量纲不齐次、却包装得相当专业的表达式。下面是真实风格的协作记录，展示量纲检查如何在一分钟内、不开电脑就把它揪出来。

AI 协作实录 · 单摆周期公式里被写错的指数

我对 AI 说：

你是一名熟悉大一普物的物理建模助手。我在估算一个长度为 L 的小角度单摆的周期，请直接给我周期 $T_{\text{周期}}$ 关于绳长 L 和重力加速度 g 的公式，并简单说明每个量的物理含义。

AI 给我：

AI 返回了一段条理清晰、措辞专业的说明，结论给出的公式是

$$T = 2 * \pi * \text{sqrt}(L / g^2)$$

也就是 $T_{\text{周期}} = 2\pi\sqrt{L/g^2}$ ，并煞有介事地解释说，分母用 g^2 是为了体现重力对回复力的“加倍”作用。

我的审计：

我没有去纠结它那段关于“加倍作用”的解释，那种听起来有道理的说辞最容易让人放松警惕。我直接做量纲检查，这是六十秒就能完成的第一道关。周期左边的量纲是时间 T 。看它给的右边： 2π 是无量纲纯数，略去；根号里 L 的量纲是 L ， g^2 的量纲是 $(LT^{-2})^2 = L^2T^{-4}$ ，于是 L/g^2 的量纲是 $\frac{L}{L^2T^{-4}} = L^{-1}T^4$ ，开方后是 $L^{-1/2}T^2$ 。这和左边的 T 根本对不上。结论立刻清楚：公式错了，错在分母的指数。正确的应当是 L/g ，量纲为 $\frac{L}{LT^{-2}} = T^2$ ，开方恰好得时间 T 。所以公式必须是 $T_{\text{周期}} = 2\pi\sqrt{L/g}$ 。我

把这个改正连同量纲推导一并回给 AI，要求它纠正。至于前面那个 2π ，量纲分析既给不出也否决不了，它的正确性要靠解运动方程，但本题的错误压根不在那里。

这段实录的要害不在于 AI 犯了错，而在于揪出错误靠的根本不是读它那段花言巧语的解释，而是一个物理学家的本能动作：先看量纲对不对。AI 极擅长写出语法漂亮、逻辑自洽的文字包装，但它对物理量纲的把关并不可靠，而这恰恰是你不会出错、也最该负责把关的地方。

本节要点

AI 写公式时最危险的不是语法报错，是量纲不齐次却被一大段专业措辞包装起来。对付它的唯一办法是绕开文字、直接核量纲。六十秒，纸笔，就能把这类错误拦在门外。

本能 · Trust but Verify

拿到任何一个公式，无论出自 AI、教材还是你自己的草稿，第一反射不是去算数值，而是先核两边量纲对不对。这是整套验证清单里最快、成本最低的一关：加减号连起来的各项量纲必须相同，等号两边量纲必须相同。这道关在几十秒内、完全不用开电脑，就能把一大批错误公式拦在门外。剩下量纲齐次的，再花力气去推敲细究。养成“先看量纲”的反射，你就为这门课后面所有的数值工作筑好了第一道防线。

AI 陷阱预警 · 量纲错误的公式，往往伪装得最专业

AI 写公式时最危险的失误不是语法报错（那种一跑就发现），而是写出一个量纲不齐次、却配着一大段头头是道解释的表达式：指数写错、漏乘常数项、该出现的物理量被丢掉、或两个量纲不同的东西用加号连起来。这些错误外表光鲜，读文字说明很容易被说服。对付它的唯一可靠办法，就是绕开文字、直接核量纲。请牢记：量纲齐次只是必要条件，AI 给的公式即使量纲对了也未必正确，但只要量纲不齐次，无论解释多动听，它一定是错的。而这恰恰是你最不该让步、也最容易守住的一关。

本章小结

这一章练了两样本领。用量纲 L 、 M 、 T 三个基本量纲去表示力学里的各种量，把无量纲量这个特殊角色单独拎清楚。量纲齐次性原理是全章最锋利的工具：方程两边、各加减项的量纲必须一致，不齐次的公式一定错。它在秒级否决了 $s = \frac{1}{2}gt$ 这类错误，反过来又能猜出单摆周期 $T_{\text{周期}} \sim \sqrt{L/g}$ 和终端速度 $v_{\text{终}} \sim \sqrt{mg/(\rho A)}$ 的形状，同时给不出 2π 这样的无量纲常数。费米的拆解智慧把估不出的大问题拆成估得出的小问题，三道 Fermi 题分别估出：人在月球能跳几米，咖啡冷却约十分钟，雨滴终端速度约几米每秒。AI 协作实录里，量纲检查在六十秒内、不开电脑就抓出了一个被包装得很专业的错误公式。

从下一章起，我们将开始真正的建模：把现实问题翻译成物理假设、变量、方程，再交给计算机求解。每当模型搭好、AI 给出第一版代码或公式，你的第一个动作仍是这一章练出的那个本能：先看量纲对不对。

思考题

1. 我们说量纲分析能锁定单摆周期里 L 和 g 的幂次，却给不出前面那个 2π 。请你解释，为什么量纲分析原则上无法定出任何无量纲常数？如果 $T_{\text{周期}} = 2\pi\sqrt{L/g}$ 里的系数不是 2π 而是 $5\sqrt{L/g}$ 或 $\sqrt{L/g}$ ，量纲检查能区分这几种写法吗？这告诉你量纲分析必须配合什么手段才能给出完整的公式？
2. 在推导单摆周期时，我们一开始把质量 m 也放进了候选因素，结果量纲方程逼出 $c = 0$ ，质量自动消失。假如有人坚持认为重的摆球摆得更慢，于是给周期公式人为塞进一个 $\sqrt{m}$ 因子，写成 $T_{\text{周期}} = 2\pi\sqrt{mL/g}$ 。请用量纲检查判断这个公式还能不能成立；如果不能，缺了什么量纲、需要再补进哪个物理量才能让它重新齐次？
3. 咖啡冷却那道题里，我们用十分钟的日常经验锚定了散热功率，又算回十分钟，看上去是个循环论证。请你设计一条不依赖“十分钟经验”的独立估算路径：从咖啡杯的表面积、咖啡与室温的温差出发，估出散热功率，再算冷却时间。如果你这条独立路径给出的数量级仍然是十分钟，它为什么比原来那条更让人信服？



3. 建模：从模糊想法到可计算任务

每一个能交给计算机算的物理问题，背后都站着一个曾经模糊不清的真实疑问。比如：一杯刚倒出来的热咖啡，多久能凉到能喝？这句话听起来根本不像物理题，没有数字，没有方程，连“能喝”是几度都没说清。可正是这种半成品的疑问，才是建模的真实起点。这一章要练习的就是：把一句含糊的人话，一步步逼成一个边界清楚、变量明确、方程写得出来的可计算任务。请记住一件事，它是这一章也是这门课最容易被新手忽略的真相：的产出是假设和方程，不是代码。代码是后面的事，把想清楚的东西翻译给机器听。如果在还没想清楚的时候就急着让 AI 写代码，那机器只会忠实地、飞快地帮你算出一个错的答案。

物理建模

本章地图

小节	这一节解决什么	关键方法 / 产物
3.1	讲清物理建模的五步动作与核心产出	五步法：问清楚、列假设、选变量、写方程、解问题
3.2	在咖啡冷却问题上走通完整五步建模流程	牛顿冷却律、冷却系数 k 、十二分钟
3.3	学会判断模型的适用边界与失效条件	辐射 vs 对流、抛体阻力量级取舍
3.4	演示 AI 协作建模时常见的假设遗漏	aiolog 恒温热库假设、温差符号反号
3.5	把建模连回 Spec-first：假设就是交给代码的规格	建模产出即验证标尺

3.1 什么是物理建模

物理建模不是神秘天赋，它是一套可以拆开来教、也可以拆开来检查的动作。面对一个真实问题，你要依次回答五个问题，每个的答案都是下一个的前提。第一，我到底要回答什么？把“多久能凉到能喝”这种口语，收紧成有数值、有判据的问题，比如“咖啡从 90°C 降到 60°C 需要多少分钟”。第二，我愿意忽略什么、必须保留什么？这一步叫理想化，是建模里最考验判断力的地方。第三，用哪些变量描述系统？第四，这些变量之间遵循什么规律，控制方程长什么样？第五，每一步取舍的理由是什么，代价是什么？

Definition 3.1.1 : 物理建模. 物理建模是把一个真实的、通常表述模糊的问题，转化为一组明确的假设、变量与控制方程的过程。它的核心产出是数学描述（连同它成立的条件），而不是求解这个描述的程序。建模的好坏，取决于假设是否抓住了主导物理、舍弃了次要因素，方程解得准不准倒在其次。

这里要特别说清楚理想化的分寸。理想化不是偷懒，恰恰相反，它是建模者最主动、最需要负责的决策。把咖啡看成一个温度处处相同的整体，是一个理想化；假设房间温度恒定不变，是另一个；认定咖啡和空气之间只通过表面对流散热、忽略蒸发和辐射，又是一个。每砍一个因素，方程就简单一分，但你也给自己的答案划了一道边界：在边界之内模型可信，越界则未必。一个成熟的建模者，砍每一刀的时候心里都清楚自己砍掉了什么，以及什么情况下这一刀会反过来割伤结论。这就是为什么我们说，建模真正在写的是假设，方程不过是假设的数学外壳。

本节要点

物理建模不是神秘天赋，是五步可拆可查的动作：把模糊疑问收紧成可验证的问题，列出理想化假设并交代理由和代价，选变量，写方程，给出带前提的答案。建模真正的产出是假设和方程，代码是翻译，不是建模本身。

3.2 完整案例：一杯咖啡多久能喝

我们把上面五步在咖啡问题上从头走一遍，走到能算出一个分钟数为止。

第一步，把问题问清楚

口语版“多久能凉到能喝”必须先变成有判据的问题。设初始温度 $T_0 = 90^{\circ}\text{C}$ ，房间温度 $T_{\text{room}} = 25^{\circ}\text{C}$ ，把“能喝”定义为降到 $T_{\text{drink}} = 60^{\circ}\text{C}$ 。于是问题收紧为：温度从 90°C 降到 60°C 需要多长时间？这一步看似简单，其实已经做了一个判断，把“能喝”量化成具体温度。换个人可能定成 55°C ，答案就会不同。建模从第一句话起就在做选择。

第二步，列出假设

我们做三条理想化假设，并交代每条的理由和代价。

第一条，整杯咖啡温度处处相同，用单一变量 T 描述。理由是杯子小，散热过程中液体有微弱对流搅动，内部温差远小于它与房间的温差。代价：如果咖啡很深、又完全静止，表层和底层其实会有差别，这条假设就会偏离实际。

第二条，房间是无限大的恒温热库， T_{room} 始终是 25°C 。理由是一杯咖啡那点热量根本焐不热整个房间。代价几乎可忽略，这是一条很安全的假设。

第三条，也是最关键的一条，咖啡的散热速率正比于它和房间的温差。理由是当温差不太大时，对流散热确实近似与温差成正比，这是牛顿冷却律的内容。代价：温差非常大时，辐射散热（正比于温度的四次方）会变得重要，线性假设就会失效。这一点留到下一节专门讨论。

第三步与第四步，选变量、写方程

变量在假设里已经定下来：唯一的状态变量是咖啡温度 T ，自变量是时间 t 。要找的规律，是 T 的变化率 dT/dt 如何依赖当前状态。第三条假设直接给出了答案：变化率正比于温差，而且温度高于环境时应当下降，前面带一个负号，

$$\frac{dT}{dt} = -k(T - T_{\text{room}}).$$

这就是牛顿冷却律。它正是上一章那个统一骨架 $dy/dt = f(t, y)$ 的又一个化身，这里 y 是温度 T ，而 $f(T) = -k(T - T_{\text{room}})$ 。能认出这一点，意味着只要把假设写成这个形状，求解的活儿对计算机来说就是熟门熟路了。

Definition 3.2.1：牛顿冷却律。 一个物体与周围环境存在温差时，温度变化率正比于该温差： $dT/dt = -k(T - T_{\text{env}})$ 。比例系数 k （量纲为时间的倒数）称为冷却系数，把物体的散热快慢打包成一个数，取决于物体表面积、表面换热方式、热容等因素。温差为正时温度下降，温差为零时达到热平衡、不再变化。

这里值得停下来想一想常数 k 的物理含义，以及怎么估它。 k 越大，散热越快。它的倒数 $1/k$ 有时间的量纲，正是这杯咖啡的特征冷却时间，物理意义和上一章 RC 电路里的时间常数 $\tau = RC$ 一模一样：每过一个 $1/k$ ，温差大约衰减到原来的 $1/e \approx 37\%$ 。 k 到底是多少，原则上取决于杯子形状、液面大小、有没有盖子、空气是否流动，精确算出来很难。但作为项目主理人，你完全不必从头推导它。更聪明的做法是测：拿一支温度计，记下咖啡每隔一分钟的温度。比如发现头一分钟从 90 降到了约 86°C ，就能反推出 k 。这是建模里一个重要的态度，能测的常数就去测，不必非要从微观原理算到底，模型够用就好。

第五步，解方程并回答问题

牛顿冷却方程可以解析地解出来。令温差 $\theta = T - T_{\text{room}}$ ，则 $d\theta/dt = -k\theta$ ，这是标准的指数衰减，解为 $\theta(t) = \theta_0 e^{-kt}$ 。还原回温度就是

$$T(t) = T_{\text{room}} + (T_0 - T_{\text{room}}) e^{-kt}.$$

这条曲线从 T_0 出发，随时间单调下降，最终以越来越慢的速度趋近室温 T_{room} ，却永远差那么一点点，这正是指数趋近的特征。现在回答最初的问题。假设测得的冷却系数约为 $k = 0.05 \text{ min}^{-1}$ ，要降到 60°C ，需要

$$60 = 25 + (90 - 25)e^{-kt} \implies e^{-kt} = \frac{35}{65} \approx 0.538 \implies t = -\frac{1}{k} \ln 0.538 \approx \frac{0.62}{0.05} \approx 12 \text{ 分钟}.$$

于是模型的回答是：大约十二分钟后这杯咖啡凉到能喝。从一句没头没脑的”多久能凉到能喝”，到一个带单位、带前提、可被实验检验的”约十二分钟”，这条路就是建模。注意整个过程中我们一行代码都没写，写的全是假设、变量和方程，最后一步甚至能纯手算。代码什么时候登场？当 k 随温度变化、或者方程没有解析解、需要像上一章那样用 Euler 法一步步推进的时候，才轮到它。

本能 · 先问清楚再动手

拿到任何一个模糊问题，先别急着想公式，更别急着开 AI。强迫自己先用一句话写出”我到底要回答的是什么数值、什么判据”，再用三句话写出”我打算忽略什么、保留什么、为什么”。这四句话写不出来，说明你还没想清楚，这时候让 AI 写的任何方程或代码，都是空中楼阁。建模阶段花的每一分钟，都在替后面的计算省下十分钟的返工。

3.3 模型的适用范围与失效

一个模型最重要的属性，不是它在哪些情况下成立，而是哪些情况下它会失效。回到那条被我们标记为最关键的假设：散热速率正比于温差。这个线性关系来自对流换热，温差不太大时是很好的近似。可是物体散热其实有两条通道：除了对流，还有辐射。辐射散热遵循斯特藩-玻尔兹曼定律，正比于温度的四次方 T^4 （用绝对温标），是高度非线性的。当咖啡和房间只差几十度时，辐射那一项相对很小，可以并进 k 里一起当线性处理，牛顿冷却律够用。但如果换成一块烧到上千度的炽热金属放在室温里，温差大到辐射主导，散热速率与温差就远不是简单的正比关系，这时硬套牛顿冷却律算出的冷却时间会严重偏长，因为它低估了高温端 T^4 通道的散热能力：模型以为散热很慢，所以预测要凉很久，而真实物体借辐射散得快得多，实际凉得比模型预测的快。

所以”牛顿冷却律对不对”这个问题本身就问错了。正确的问法是：在我这个温区里，它够不够用。对一杯咖啡，温差几十度，够用；对一块红热的钢，不够用。这就引出了贯穿全课的一句话：好模型不是无所不能的模型，是够用的模型，并且你清楚知道它在哪儿失效。一个连自己边界都说不清的模型，哪怕在某个例子上恰好算对了，也是不可信的，因为你没法判断换个场景它还灵不灵。作为项目主理人，能说清模型边界，往往比能多解一个方程更值钱。

本节要点

模型最值钱的属性不在它哪里成立，在它哪里失效。牛顿冷却律在咖啡温区够用，在红热钢块温区翻车；线性能量关系在高温会被四次方辐射取代，模型会严重高估冷却时间。每次取舍前先问：砍掉的因素在什么条件下会反咬结论。

■ **Example 3.1：抛体要不要考虑空气阻力。** 扔一个小球，要不要在模型里加上空气阻力？这也是一个建模取舍，而且能在动手算之前就估出答案。判据是比较两个力的量级：空气阻力

大致是 $F_{\text{drag}} \sim \frac{1}{2} \rho_{\text{air}} C_d A v^2$ ，重力是 $F_g = mg$ 。对一个手抛的、速度每秒几米、半径几厘米、几十克重的橡皮球，代入空气密度约 1.2 kg/m^3 ，阻力只有重力的百分之几，忽略它，用最干净的抛物线模型即可，误差小到肉眼难辨。但如果换成同样大小、速度高达每秒几百米的乒乓球削球，或者换成下落很久、速度积累到很大的羽毛球，阻力就追上甚至盖过重力，再忽略它就会算出离谱的轨迹。关键不在于“空气阻力存在与否”（它当然永远存在），而在于“在我这个速度和质量下，它大到值得保留吗”。先估量级、再决定保留什么，这才是建模假设该有的做法，省得一上来就背一个既复杂又可能根本不需要的方程。 ■

3.4 AI 协作实录：让 AI 帮忙建模

建模这一步同样可以请 AI 搭把手。但越是这种“想清楚”的活儿，越要由你来把关，因为 AI 最容易在不显眼的假设上栽跟头。下面是一段真实风格的协作记录，问题是给保温杯里的水建立冷却模型。

AI 协作实录 · AI 漏掉了恒温热库假设、又把变量搞混

我对 AI 说：

你是一名熟悉大一普物和数值方法的物理建模助手。我想给一个保温杯里的热水冷却建立一个最简单的模型，初温 95°C ，放在 25°C 的房间里。请先把建模假设一条条列清楚，再写出控制方程，暂时不要写代码。

AI 给我：

AI 列了两条假设：水温处处相同、散热速率正比于温差，然后给出方程

$$\frac{dT}{dt} = -k(T_{\text{room}} - T),$$

并说初温 $T = 95$ 、室温 $T_{\text{room}} = 25$ 。

我的审计：

我没有夸它列得快，而是逐条核对。第一个问题：漏了一条关键假设。它默认了房间温度 T_{room} 恒定不变，却没把这条写出来。对保温杯里的水，这条恰恰不是理所当然的，保温杯散热极慢，如果放在密闭小空间里，长时间下来环境真可能被焐热。那时 T_{room} 就不再是常数，模型形式会变。漏写假设，等于把模型的适用边界藏了起来，这是建模里最危险的疏忽。我要求它补上“房间为恒温热库”这条，并注明在密闭小空间里可能失效。第二个问题更直接：方程符号写反了。它写的是 $-k(T_{\text{room}} - T)$ 。当水比房间热 ($T > T_{\text{room}}$) 时， $T_{\text{room}} - T$ 为负，前面再加负号，整个变化率变成正的，意味着水越放越热，这显然非物理。正确的应当是 $-k(T - T_{\text{room}})$ ，温差用“物体减环境”，这样热水降温、变化率为负才说得通。我让它把两处都改正：补齐恒温热库假设，并把温差顺序改回 $T - T_{\text{room}}$ 。改完之后的方程才和我们案例里那条一致，物理上也站得住了。

这段实录的要害有两层。表层是 AI 把变量顺序搞反、符号错了，这种错只要用一个极限情形去试就能抓住：让物体比环境热，看变化率是不是负的，不是就错了。更深一层是

AI 漏写了一条假设，而漏掉的假设是看不见的。它不会报错，不会让方程变得明显荒谬，却悄悄决定了模型在什么时候会崩。AI 很擅长把你说出口的东西整理成方程，但它不会替你操心你没说出口的前提。把每一条假设都摆到台面上，连同失效条件一起写清楚，这件事永远是项目主理人的责任，是你最该负责、也最不该假手于 AI 的地方。

本节要点

AI 在建模时最隐蔽的失误不是符号写反，是漏写假设。漏掉的假设不会报错，却悄悄决定了模型什么时候会崩。把每条假设连同失效条件白纸黑字写下来，这件最不能外包的事，是项目主理人的底线。

3.5 建模与 Spec-first

现在把这一章连回这门课的总纲领。我们反复强调一个工作顺序：先想清楚再写代码。用更专业的词说就是 Spec-first，先有规格说明，再有实现。建模正是物理问题里的 Spec。当你把假设、变量、方程、适用边界都白纸黑字写下来，你其实就是在为接下来要交给 AI 的编程任务撰写规格说明书：这段代码该解哪个方程、变量是什么、参数取多少、结果应满足哪些物理性质（单调下降、趋于室温、过一个 $1/k$ 衰减到 37%）。这些性质后来会一条条变成你的验证清单，用来给 AI 写的代码打分。

本能 · 建模产出就是给代码的规格说明

写代码之前，先确认手里已经有一份“规格”：要解的方程、所有变量及单位、参数取值及来源、以及结果必须满足的几条物理判据。这份规格就是你的建模产出，也是你审计 AI 代码的标尺。没有规格就让 AI 写代码，等于没图纸就让施工队浇筑，浇错了你连哪里错了都说不上来。规格在先，实现在后，验证收尾，这是项目主理人的标准动作。

反过来说，如果你跳过建模、直接对 AI 喊“帮我算咖啡多久能凉”，它多半会自己脑补一套假设、随手填一个 k 、给你一段能跑出数字的代码。数字会很自信地蹦出来，但那套假设是不是你想要的、 k 是不是合理、模型在你的温区里灵不灵，你全不知道，也就没法负责。建模这一步，把本该由你做的判断从 AI 手里拿回来，攥在自己手上。这正是项目主理人和“复制粘贴 AI 输出的人”之间最本质的区别。

本节要点

建模产出就是交给代码的规格说明。假设、变量、方程、边界，这四样东西白纸黑字写下来，代码就有了蓝图，验证就有了标尺。跳过建模直接让 AI 写代码，等于没图纸就让施工队浇筑，浇错了你连哪里错了都说不清。

本章小结

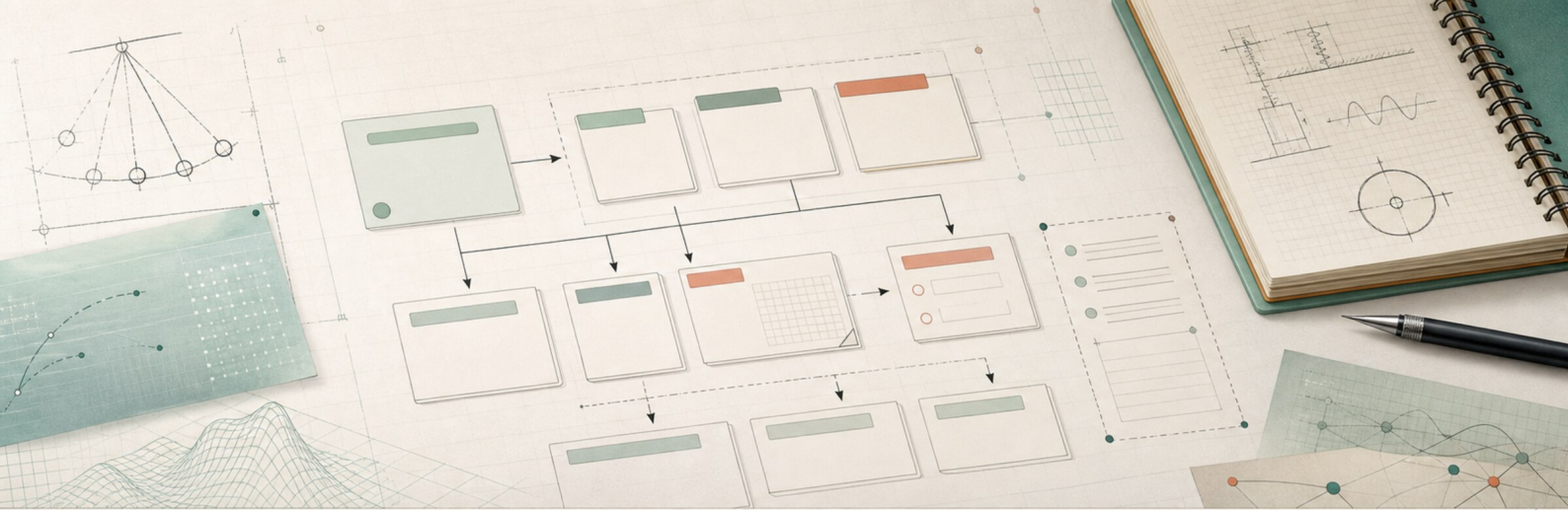
这一章把建模拆成了手艺。建模的产出是假设和方程，不是代码。五步法把咖啡冷却走通了：先把“多久能喝”收紧成有判据的问题，再列出三条理想化假设及其理由与代价，

选温度作唯一变量，写出牛顿冷却律 $dT/dt = -k(T - T_{\text{room}})$ ，讨论 k 靠实测来估的态度，最后解出约十二分钟的答案。整段路几乎没碰代码。牛顿冷却律在温差不大时够用，温差极大时辐射主导、线性假设失效。好模型是够用且知道自己在哪儿失效的模型。抛体要不要考虑空气阻力的例子，练了用量级估算来决定保留什么。AI 协作实录里，AI 漏写了恒温热库假设、又把温差符号搞反。补齐看不见的假设永远是项目主理人的责任。建模产出就是交给代码的规格说明，也是日后审计 AI 代码的标尺。

下一章要把手写的假设与方程整理成一份结构化的需求文档，让你的模糊想法真正变成一张能递给 AI、也能拿来验收的工程图纸。

思考题

1. 本章把“能喝”定义成降到 60°C ，于是算出约十二分钟。如果你的同学坚持“能喝”是 55°C ，请用同一个模型和同一个 $k = 0.05 \text{ min}^{-1}$ 重新算一遍需要多久，并说说这个例子告诉你“第一步把问题问清楚”为什么如此重要：同一杯咖啡、同一套物理，为什么答案可以不同？
2. 我们说冷却系数 k 可以靠实测来估，比如记录头一分钟咖啡从 90°C 降到约 86°C 。请利用牛顿冷却律，从这一段数据反推出 k 的近似值（提示：把这一分钟近似看成变化率 $dT/dt \approx (86 - 90)/1$ ，再代入方程解出 k ）。你估出来的 k 和本章用的 0.05 min^{-1} 接近吗？如果差得多，可能是哪条假设在这段数据上不太成立？
3. 给一个真实的模糊问题挑一个你身边的例子（比如“手机充电从 20% 充到 80% 要多久”“一场雨后操场上的积水多久蒸发干”“教室里开了空调多久能凉下来”），按本章的五步法走一遍：写出你要回答的精确问题、列出至少三条理想化假设并各自说明理由与代价、指出你会用哪些变量、并尝试写出或猜出控制方程的形状。你不必解出它，重点是把假设和边界说清楚，并指出你的模型最可能在什么情况下失效。



4. Spec-first: 需求文档的写法

让 AI 写代码，最容易出问题的往往不在代码本身，而在你递给它的那一句话。一个大一学生坐在屏幕前，脑子里有个模糊的念头：“我想做个单摆的东西，看看周期怎么变。”于是他对着 AI 敲下“帮我写个单摆模拟”，回车。AI 是个极其听话、又极其爱猜的合作者。你划的边界越模糊，它替你拍板的地方就越多，而它拍板时参考的，是训练数据里最常见的默认值，不是你以为的“显然”。它可能默认用小角度近似，可能默认画一张位置随时间的图，可能默认振幅是十度，可能默认用 SI 单位，也可能不是。这些它都不会问你。它会一声不响地替你决定，然后把一段看起来很专业的代码端上来。等你发现“这跟我想的不一样”，往往已经在错误的代码上改了半天了。

这一章要教的，就是在叫 AI 写代码之前，先花二三十分钟，把那个模糊念头压成一张纸，一张让 AI 没空间乱猜、也让你自己想清楚边界的纸。这张纸我们称为，对应工业界 **需求文档** 的产品需求文档（product requirements document）。工业界的需求文档动辄几十页，我们要的是它的轻量版：一页纸，只服务于一个目的：把想法变成 AI 能精准实现的可验证规格。这套动作的核心叫 Spec-first，先有规格，再有代码。

本章地图

小节	这一节解决什么	关键方法 / 产物
4.1	讲清为什么先写规格能让验证提前就位	Spec-first、判据先于代码
4.2	逐类讲透需求文档必须回答的五类问题	做什么、输入、输出、怎么算成功、技术栈与理由
4.3	确立原则优先、反模板的立场	认知所有权、自己长格式
4.4	用同一项目对照好坏需求文档的差距	单摆案例、具体可验证 vs 含糊填空
4.5	演示完整规格与一句话喂给 AI 的天壤之别	ailog 精准实现 vs 乱猜

4.1 为什么先写规格，而不是先写代码

你可能会想：“代码是 AI 写，我直接让它写不就行了，何必我先动笔。”这正是要破除的第一个直觉。“让 AI 写代码”这个动作本身，已经预设了你知道自己要什么。但大多数时候，把一个物理想法从脑子里的模糊云，落实到“具体算什么、输入给什么、输出要什么、怎么知道算对了”，这个澄清的过程才是真正费脑子、也真正属于你的那部分。AI 替你写循环、调库、画图，那是它的活；但“到底要算什么”，是你作为项目主理人不能外包的判断。

先写规格还有一个更实际的好处：验证提前就位。你一开始就写明了“怎么算成功”，那么 AI 交回代码时，你手里已经攥着一把尺子，可以立刻量它对不对，而不是对着一张看起来还行的图发愣。反过来，跳过规格直接要代码，等代码回来了你才开始想“我怎么知道这对不对”，这时候你往往偷懒，用“图看起来挺顺的”来糊弄自己。第三章反复强调过，看起来对和真的对是两回事。Spec-first 的纪律，就是在你还没被一张漂亮图诱惑之前，先把判据钉死。

本能 · Spec-first

拿到一个模糊的物理想法，不要急着对 AI 说“帮我写个 X”。先停下来，用二三十分钟把想法压成一页纸：做什么、输入是什么、输出是什么、怎么算成功、用什么技术栈以及为什么。规格写在了代码之前，验证的尺子也就跟着提前就位。一句模糊的话，等于把一连串你本该自己拍板的判断，默默让渡给 AI 去猜。

4.2 五类必答问题

一份合格的需求文档，不在于长什么样，而在于是否诚实回答五类问题。这是本章的核心。我们逐个讲清楚，你读完后应该能凭记忆复述它们，以后每开一个新项目，都要在心里过一遍。

第一类，做什么。这是项目的一句话定义，要具体到不留歧义。“模拟单摆”不合格，因为它没说清你关心单摆的哪一面。“研究单摆周期如何随摆动振幅变化，并定量考察小角度近似在多大振幅下开始失效”，这才是一个有边界的“做什么”。注意这句话里已暗含了你的物理动机：你不是在泛泛地模拟一个摆，你在追问一个具体现象，大振幅下周期对

振幅的依赖，这一点恰恰是小角度近似教科书一笔带过、而真实摆会偏离的地方。”做什么”写到这个粒度，AI 就知道该求解完整的非线性摆方程，而不是偷懒用线性近似。

第二类，输入是什么。这是项目的”给定量”。要把每一个 AI 需要知道（否则就得替你猜）的量都列清楚，带上单位和合理范围。对单摆来说，输入至少包括摆长、重力加速度、初始振幅的取值范围、初始角速度。你不写明摆长一米、重力加速度 9.8 m/s^2 、振幅从五度扫到一百五十度，AI 就自己填一组数，而它填的那组数很可能让你的”大振幅失效”现象根本看不出来，因为它默认了一个小振幅。把输入写清楚，本质上就是替 AI 划掉它所有的默认值。

第三类，输出是什么。这是项目要交付的东西，同样要具体。是一条周期随振幅变化的曲线，还是一张表，还是一个具体数字（比如”振幅多大时周期比小角度预测值偏离百分之一”）。对单摆项目，一个好的输出描述是：一张横轴振幅、纵轴周期的曲线，叠加小角度近似的水平参考线，标出两者偏离达到百分之一的那个振幅。输出描述到这个程度，AI 就清楚要画什么了，而不是随手给你一张位置随时间的振荡图，那种图虽然也对，却答非所问。

第四类，怎么算成功。经验里最容易被忽略、却最重要的一类。重要到我们待会儿要单开一道思考题逼你去想它。它问的是：代码跑完，你凭什么判据相信它算对了？不是”图好看”，得是一组能推翻结论的物理检验。对单摆，至少有这么几条：振幅趋于零时，数值周期必须收敛到小角度公式 $T_0 = 2\pi\sqrt{L/g}$ ；整个模拟中能量应当守恒（无阻尼摆的机械能不随时间漂移）；周期必须随振幅单调增大（这是非线性摆的已知行为）；时间步长减半重算，周期的数字不应明显改变（收敛性）。这些写进需求文档，等于在 AI 动手之前把验收标准公开挂出来，后面照着量就行。需求文档如果”怎么算成功”这一栏写的是”结果合理”或干脆空着，那它基本等于没写，没给你任何抓错的把手。

第五类，选什么技术栈以及为什么。技术栈是你打算用的语言、库、数值方法。但这一类的精髓不在”选什么”，而在”为什么”。说”用 Python”不够，要说清楚原因：因为我们这门课用 `scipy` 的积分器做数值积分最省事，因为画图用 `matplotlib` 全班统一，因为单摆是二阶常微分方程、我打算拆成两个一阶方程再交给积分器。这个”为什么”逼着技术选择和物理问题挂上钩，而不是因为”别人都用 Python”就用 Python。能说出为什么，你才是在做选择，而不是随大流。

Definition 4.2.1：需求文档五类必答。一份合格的项目需求文档，无论排版成什么样子，都必须诚实回答五类问题。一、做什么：项目的一句话定义，具体到不留歧义，并点明物理动机。二、输入是什么：所有给定量，带单位与合理范围，目的是替 AI 划掉默认值。三、输出是什么：要交付的具体形态（曲线、表、数字），描述到 AI 不必再猜的程度。四、怎么算成功：一组能推翻结论的验证判据（解析极限、守恒律、单调性、收敛性、参考值等），杜绝”结果合理”这种空话。五、选什么技术栈以及为什么：语言、库、数值方法，重点在”为什么”，把技术选择和物理问题挂钩的理由。五类全部回答，需求文档才算合格；缺一类，AI 就会在那一类上替你拍板。

本节要点

五类必答问题不是填表格式，是五道你必须在让 AI 写代码之前问自己的题。做什么划边界，输入堵默认值，输出定形态，怎么算成功备尺子，技术栈讲理由。五类全答，需求文档才是一份可验证的规格。

4.3 原则优先，格式自己长出来

讲到这里，你大概在等我发一张五栏的需求文档表格让你照着填。这门课不发，而且故意不发。背后有一个明确的立场，叫原则优先、反模板。这件事值得专门说清楚，它关系到你到底从这门课学到了什么。

如果老师发一张五栏空表，标题分别是“做什么”“输入”“输出”“成功标准”“技术栈”，你照着填满，看起来你交了一份完整的需求文档。可你真正学到的是什么？是“如何把我一个模糊想法塞进五个预先挖好的格子”。这恰恰不是这门课要教的。这门课要教的是“如何独立思考一个物理问题的边界”。这两件事的差别，在 AI 时代会被放大成天壤之别。一个会填格子的人，遇到格子没覆盖到的情况就傻眼了；一个真正理解五类问题背后意图的人，无论换什么项目、什么排版，都能自己长出一份对的规格。

更深一层的危险在于责任的转移。照着模板填表，你其实把“一份好的需求文档该长什么样”这个判断，悄悄让渡给了设计模板的人。模板设计者替你想好了边界，你只负责填空。于是判断从你手里溜走了：模板设计者替你想，AI 替你写，到最后，没有任何一个环节是你在真正负责。这正是这门课最警惕的失败模式，学生退化成了模板填表器，把本该自己长出来的判断力外包出去。写需求文档的能力一旦退化成填表能力，它就再也不属于你了。

所以这门课的做法是：老师讲原则（上面那五类必答问题背后的意图），老师给案例（投影上对比好与差的需求文档范例，但这些范例只在课堂上讲解、不发给你照抄），老师给评分细则（按“五类是否都被诚实回答、回答得是否具体、是否与项目实际吻合”打分，绝不按“格式对不对”打分）。然后，学生自己生成。你学完原则、看完案例，自己决定需求文档该长成什么样：可以是表，可以是连续散文，可以配草图，怎么都行，只要那五类问题被诚实回答。

你的第一版几乎肯定写得糟。这正常，是学习过程的一部分。糟在哪里，老师当众指出来，你回去改。改完的那一份，就是你自己长出来的需求文档。它比任何模板填出来的都更属于你，因为格式里的每一个决定都经过了你的判断。

AI 陷阱预警 · 把规格写成模板填空，判断力悄悄外包

模板的诱惑在于“省事”：五个格子填满，交差。但省下来的，恰恰是这门课要你练的那块肌肉，独立判断一个问题的边界。一旦习惯了照模板填，遇到模板没覆盖的项目就会卡住。而且你不会意识到自己卡住了，因为你从没练过“从零长出格式”这件事。提醒：你的需求文档长什么样，必须是你自己的决定，不是任何人替你挖好的格子。判断别人挖的格子是否覆盖了你的问题，本身就是你的责任。

唯一需要澄清的边界是：反模板针对的是“内容结构层”，需求文档该长什么样、该怎么组织，这些本应由你判断的内容设计。它不针对“工程契约层”。比如，你要把项目部署成网页应用，框架要求入口文件必须叫 `streamlit_app.py`、依赖必须列在 `requirements.txt` 里，这些是工具能不能跑起来的硬约束，照做就是，不算违反反模板立场。判断准则很简单：格式要求如果来自“这个工具或平台必须这样才能运行”，那是工程契约，照做；如果来自“老师觉得这样想更清楚”，那是教学模板，拒绝照搬，自己长。

本节要点

反模板不是反工程契约。需求文档该长什么样、怎么组织，必须是你自己的判断，照填别人的格子会把判断力外包给模板设计者。但入口文件名、依赖清单这类工具硬要求，照做即可，那不叫模板，叫工程契约。

4.4 好需求文档与差需求文档的对照

空讲原则容易，落到纸面才见功夫。下面拿同一个物理小项目，单摆周期随振幅变化，各看一份差的和一份好的需求文档片段，逐处剖开看差在哪里。

■ Example 4.1：一份差的需求文档片段. 做什么：模拟单摆。

输入：摆长、重力。

输出：周期的图。

怎么算成功：结果合理就行。

技术栈：Python。

这份需求文档的每一栏都“填了”，却每一栏都漏了风。“模拟单摆”没说清要研究周期对振幅的依赖，AI 完全可能默认用小角度近似，那样根本看不到你想看的现象。输入只写“摆长、重力”，没给具体数值、没给单位、最关键的是没提振幅范围，而这个项目的主角恰恰是振幅，漏了它等于把主角踢出了剧本。输出说“周期的图”，没讲横轴是什么，AI 大概率给你一张位置随时间的振荡图，答非所问。“结果合理就行”这一栏最致命：没给任何能抓错的物理判据，等于你放弃了验证的权利。技术栈只写“Python”，没有“为什么”，说明作者没有把技术选择和二阶微分方程的求解联系到一起思考。这份需求文档喂给 AI，你会得到一段语法正确、却答非所问的代码，而且你手里没有任何尺子去发现它答非所问。 ■

■ Example 4.2：一份好的需求文档片段. 做什么：定量研究单摆周期 T 如何随初始摆动振幅 θ_0 变化，重点找出小角度近似（周期与振幅无关）开始明显失效的振幅，给出周期偏离小角度预测值百分之一的临界振幅。

输入：摆长 $L = 1.0\text{ m}$ ；重力加速度 $g = 9.8\text{ m/s}^2$ ；初始振幅 θ_0 从 5° 扫到 150° ；初始角速度为零（从静止释放）。全部 SI 单位。

输出：一张横轴为振幅 θ_0 、纵轴为周期 T 的曲线；叠加一条小角度周期 $T_0 = 2\pi\sqrt{L/g}$ 的水平参考线；在图上标出 T 偏离 T_0 达百分之一的那个临界振幅。

怎么算成功：（一）振幅趋于零时数值周期收敛到 $T_0 = 2\pi\sqrt{L/g} \approx 2.0\text{ s}$ ；（二）无阻尼摆机械能在整个模拟内守恒，相对漂移小于千分之一；（三）周期随振幅单调增大；（四）时

间步长减半重算，临界振幅变化不超过百分之一（收敛性）。

技术栈：Python，用 `scipy.integrate.solve_ivp` 做数值积分，因为单摆是二阶非线性常微分方程，我会把它拆成位置和角速度两个一阶方程交给积分器，比手写 Euler 法更稳；画图用 `matplotlib`，与全班统一。

这份需求文档的每一栏都堵死了 AI 乱猜的空间。”做什么”点明了研究的是非线性行为，AI 就不敢用小角度近似偷懒。输入把振幅范围、单位、释放条件全给齐了，AI 无处可猜。输出精确到横纵轴和要标注的临界点。”怎么算成功”给了四条能推翻结论的物理判据，拿到代码就能逐条量。技术栈选了工具，还说清了”为什么用积分器而不是手写 Euler”，这个理由直接来自”二阶非线性方程”这个物理事实。这份需求文档喂给 AI，你几乎能预期它会还给你什么，而且手里那四把尺子让你能立刻判断它对不对。 ■

两份对照下来，差别一目了然：差的那份把五类问题都”提到了”，但没一类回答到能堵住 AI 乱猜、能让你验证的程度；好的那份每一类都达到了具体、可验证的粒度。注意，好的那份用的是简单的几行文字加分项，并不是花哨的表格，这正说明，决定需求文档好坏的从来不是排版，是五类问题被回答的诚实度和具体度。

本节要点

差的需求文档每一栏都”填了”却每一栏都漏风，好的需求文档每一栏都具体到能堵死 AI 的默认值、能让你拿着尺子逐项量。决定好坏的从来不是排版，是回答的诚实度和可验证的粒度。

4.5 AI 协作实录：完整规格换来精准实现

现在真刀真枪演示一遍 Spec-first。拿上面那份好的单摆需求文档喂给 AI，看它出的代码骨架，再对比一句模糊的话会让 AI 乱猜成什么样。

AI 协作实录 · 同一个单摆，完整需求文档与一句话的天壤之别

我对 AI 说：

（第一次，把上面那份好的需求文档整段粘进去，并在末尾加一句）以上是我的项目规格。请据此写一段 Python 代码骨架：把单摆方程拆成两个一阶方程，用 `scipy.integrate.solve_ivp` 对每个振幅积分若干个周期，从过零点数出周期，画出周期随振幅的曲线并叠加 T_0 参考线，所有物理量带 SI 单位注释。先不用追求完美，把结构搭出来。

AI 给我：

AI 返回了一段结构清晰的代码：顶部用具名变量定义了 $L = 1.0$ 、 $g = 9.8$ 并注明单位；定义了一个返回 $[\dot{\theta}, -(g/L) \sin \theta]$ 的右端函数（注意它用了完整的 $\sin \theta$ 而不是小角度的 θ ，因为需求文档的”做什么”里点明了要研究非线性失效）；对 θ_0 在 5° 到 150° 间扫描，每个振幅积分、检测过零、估计周期；最后画出 T 对 θ_0 的曲线并叠了一条 $T_0 = 2\pi\sqrt{L/g}$ 的水平线。代码末尾还留了 `# TODO`：标注偏离百分之一的临界振幅的

注释。

我的审计：

我没有逐行读积分细节，而是拿需求文档“怎么算成功”那四条当尺子去量。第一条解析极限：打印扫描下限 5° 对应的数值周期，和 $T_0 = 2\pi\sqrt{1.0/9.8} \approx 2.006\text{ s}$ 比，三位有效数字吻合，过关。第三条单调性：曲线从左到右确实单调上升，符合非线性摆周期随振幅增大的已知行为，过关。第二条能量守恒，AI 的骨架里没算能量，这正是它该补的地方，我让它加一段积分后计算机械能相对漂移的检查。第四条收敛性我自己把 `solve_ivp` 的容差收紧一档重跑，临界振幅几乎没变，过关。整个审计没超过十分钟，因为尺子在写代码之前就已经准备好了。

为了让对照刺眼，再看一眼如果跳过需求文档、只甩一句话会发生什么。

AI 协作实录 · 一句话喂出来的乱猜

我对 AI 说：

帮我写个单摆模拟。

AI 给我：

AI 返回了一段同样语法正确的代码，但它替我做了一连串我没授权的决定：它默认用了小角度近似 $\ddot{\theta} = -(g/L)\theta$ （于是周期根本不随振幅变，我想研究的现象被它从一开始就抹掉了）；它默认摆长是 1、重力是 9.81 但没写单位注释；它默认输出的是一张角度随时间的振荡曲线，而不是我想要的周期随振幅曲线；它当然也没有任何成功判据或能量检查。

我的审计：

这段代码没有一处语法错误，跑起来还会给我一张挺顺眼的正弦曲线。但它答的根本不是我的问题。问题不在 AI，在我那句话：我没说要研究非线性失效，它就用了最常见的线性近似；我没说振幅要扫描，它就当振幅是个固定值；我没说输出要什么，它就给了最常见的时间曲线。它每一次替我拍的板，都是因为我那块判断让渡出去了。如果没有 Spec-first 的纪律，我很可能对着这张顺眼的正弦曲线点头，然后在一个从根上就答错了问题的代码上越改越远。

把两段实录摆在一起，这一章的论点就立住了：决定你和 AI 协作质量的，不在 AI 多聪明，在你喂给它的规格多完整。完整规格换来精准实现，模糊的一句话换来一堆它替你拍的板。而那些被它替你拍的板，恰恰是作为项目主理人，你最不该让渡出去的东西。

本节要点

决定你和 AI 协作质量的不是 AI 多聪明，是你喂给它的规格多完整。同一段单摆代码，完整需求文档换来带物理判据、用完整方程的精准实现，一句“帮我写个单摆模拟”换来小角度近似、答非所问。那些被 AI 替你拍板的地方，正是你最不该外包的判断。

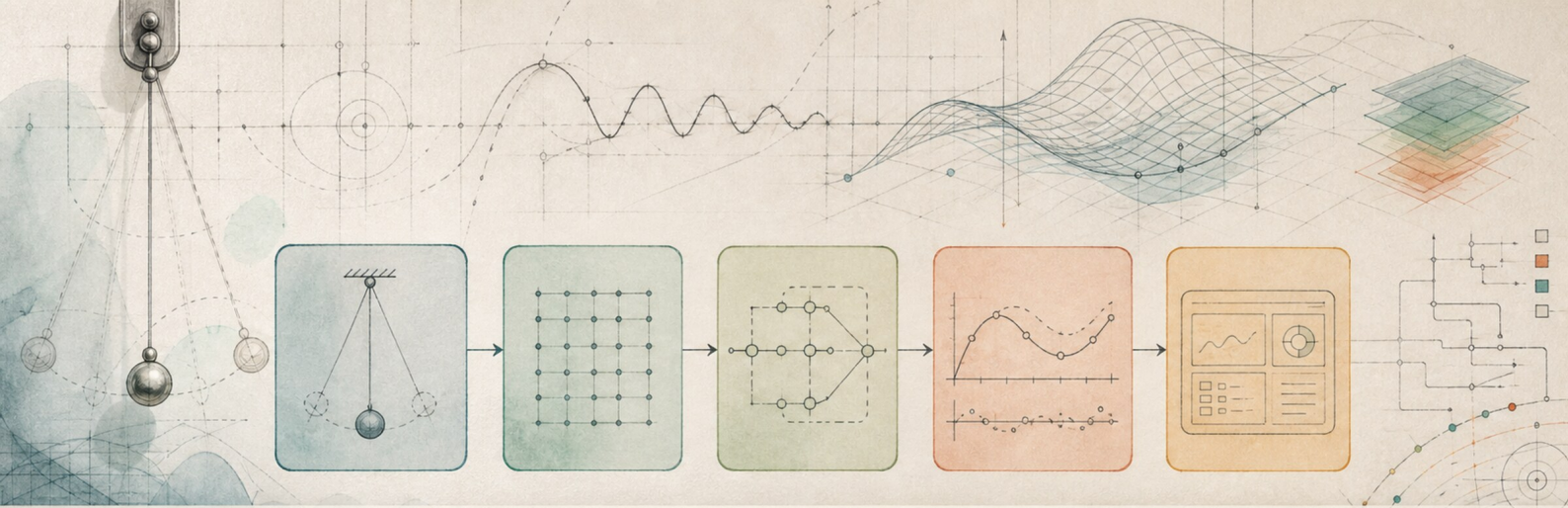
本章小结

这一章确立了一条反射动作：Spec-first，先写规格再写代码。规格落实为一页纸的轻量版需求文档，必须诚实回答五类问题（做什么、输入是什么、输出是什么、怎么算成功、选什么技术栈以及为什么）。“怎么算成功”最容易被忽略，却是后面抓错的唯一把手。原则优先、反模板的立场：不发填空模板，照模板填表会退化成填表器，把判断力外包给模板设计者。老师只给原则、案例和评分细则，格式由你自己长出来。通过同一个单摆项目的好坏需求文档对照，以及完整规格与一句话的两段 AI 实录，完整的规格堵死了 AI 乱猜的空间，验证的尺子也提前就位。

写好了需求文档，项目光有规格还不够，需要拆成一块块能让 AI 分别实现的结构。下一章就来学怎么把需求文档翻译成一张系统架构图。

思考题

1. 五类必答问题里，“怎么算成功”被认为最容易被忽略，也最重要。请你结合这一章的单摆例子，说说为什么它最容易被忽略（提示：从“写规格时人通常急着想做什么、做不到提前想验证”这个心理角度入手），又为什么它最重要（提示：从“没有它，你拿什么判断 AI 的代码对不对”入手）。如果一份需求文档的其他四类都答得很好，唯独这一类空着，会出什么问题？
2. 这门课坚持不发需求文档模板，理由是“照模板填表会让学生退化成填表器，把判断让渡出去”。可你也许会反驳：发个模板能保证大家至少不漏掉五类问题里的任何一类，效率更高。请你站在这门课“cognitive ownership（认知所有权）”的立场上，反驳这个看似合理的效率论点，说清楚“至少不漏”这个好处，为什么不值得用“判断力外包”这个代价去换。
3. 本章那份差的需求文档把“技术栈”一栏只写了“Python”，我们批评它缺了“为什么”。可有人会说：用什么语言而已，写个 Python 不就够了，何必解释。请你针对单摆这个具体项目，说出至少两个“为什么”层面的决定，是单写“Python”这三个字无法表达、却会实实在在影响 AI 写出来的代码的（提示：想想二阶方程怎么处理、用手写 Euler 还是现成积分器，这些选择背后的物理理由）。



5. 架构先行：先画系统块图，再让 AI 写代码

到上一章为止，你已经能让 AI 写出解微分方程的代码，并且亲手验证它对不对。可一旦项目从“一段代码”长成“一个点开就能玩的交互页面”，事情就变了味。你向 AI 要一个阻尼振子的交互演示，它很可能一口气吐出一百多行代码：参数读取、积分循环、能量计算、画图、网页控件，全搅在一起。代码跑是跑得起来，但你盯着看了十分钟，还是说不清“我改这个滑块，到底是哪一段在响应”。这一章要解决的就是这个困境：在让 AI 写任何一行代码之前，先把整个项目画成一张，拆成几个职责清晰、彼此只靠少量数据相连的块，再让 AI 按块去实现。这样你掌握的是全局骨架，不用把自己淹没在任何一块的细节里。这套思想有个名字，叫（Architecture-before-code）。

系统块图

架构先行

本章地图

小节	这一节解决什么	关键方法 / 产物
5.1	讲清不分块的一大坨代码为何让你失去控制	接缝、关注点分离
5.2	把阻尼振子交互页拆成五块并画出数据流	参数输入、物理模型、求解器、结果计算、可视化
5.3	逐块讲透三个绕不开的真实技术取舍	Streamlit vs Jupyter、scipy vs Euler、内存 vs 落盘
5.4	演示按块交给 AI 实现并用块图对账	ailog AI 并块、阶段交接核查

5.1 为什么一大坨代码会让你失去对项目的控制

先想清楚问题出在哪。一段几百行、不分块的代码，最大的毛病不是丑，是没有“接缝”。没接缝，你就没地方下手去理解、去验证、去修改。答辩现场老师问你“这个能量曲线为什么会缓慢下降”，你想回答，就得在脑子里从一百行里捞出和能量相关的那几行，可它们和绘图、和控件响应缠在一起，你捞不干净。更糟的是，想把阻力系数从固定值改成可调滑块时，你不知道这一动会牵动多少别的地方，因为所有东西都互相引用。

把项目拆成块，本质上是在替自己制造接缝。每块有明确的职责，对外只暴露“我要什么输入、我输出什么”，内部怎么实现是它自己的事。这样一来，能量出问题只看能量块，控件出问题只看输入块，验证时可以一块一块地单独喂数据看它吐什么。这正是工程界几十年来对付复杂系统的老办法，关注点分离：每个部件只操心一件事，部件之间靠约定好的接口通信。作为项目主理人，画块图是为了项目变大之后自己始终拿得住全局，不是为了好看。

Definition 5.1.1：系统块图。系统块图是把一个项目按职责拆成若干功能块、并标明块与块之间数据流向的示意图。每个块是一个只负责一件事的处理单元，块之间用箭头连接，箭头上标注的是从上游流向下游的具体数据（物理含义、形状、单位）。一张好的系统块图，看一眼就能回答三个问题：数据从哪里进来、在哪一块被加工、最后从哪里出去。

本节要点

不分块的代码最大的毛病不是丑，是没有接缝。没接缝你就没法分块理解、分块验证、分块修改。拆块就是给自己制造接缝，每块只管一件事、对外只暴露输入输出，出问题能精准定位，不用在大段代码里大海捞针。

要强调的是，块图刻画的是数据如何流动，不是代码如何缩进。两个块之间画一根箭头，意思是“上游算完的某个东西，交给下游接着算”，箭头上写清楚交的是什么。只要是箭头上传的数据说清楚了，这张图就立住了；至于每个块内部是十行还是五十行，画图阶段一概不管。

5.2 把阻尼振子交互页拆成五块以内

我们拿一个具体项目来落地：做一个阻尼振子的交互演示页。用户拖动滑块改变质量、劲度系数、阻力系数和初始位移，页面实时画出位移随时间曲线，以及系统能量随时间衰减曲线。麻雀虽小，五脏俱全，这是 SYLLABUS 里反复用来教架构的范例。目标是拆成不超过五块的最小架构。

阻尼振子的物理是这样的。一个质量 m 的物体挂在劲度系数 k 的弹簧上，运动时受到正比于速度的阻力，阻力系数 b ，牛顿第二定律给出二阶方程

$$m \frac{d^2 x}{dt^2} = -k x - b \frac{dx}{dt}.$$

照上一章的办法，引入速度 $v = dx/dt$ 作为第二个状态量，把二阶方程拆成关于状态向量 (x, v) 的一阶方程组

$$\frac{d}{dt} \begin{pmatrix} x \\ v \end{pmatrix} = \begin{pmatrix} v \\ -\frac{k}{m}x - \frac{b}{m}v \end{pmatrix}.$$

系统总能量是动能加弹性势能， $E = \frac{1}{2}mv^2 + \frac{1}{2}kx^2$ 。阻力做负功，所以能量应当随时间单调下降，这一点后面会成为验证的依据。物理交代清楚了，现在画块。

第一块是参数输入块。职责单一：收集用户在页面上拖出的四个滑块值 (m 、 k 、 b 、初始位移 x_0)，连同固定数值参数 (总时长、步长)，打包成一组干净数字交给下游。输入是用户的交互动作，输出是一组带单位、已校验范围的物理参数。把这块单独拎出来，好处是页面控件长什么样、用什么库，跟后面的物理计算彻底解耦：将来想把滑块换成输入框，只动这一块。

第二块是物理模型块。它接收上游的参数，给出右端函数 $f(t, (x, v))$ ，即状态向量的变化率。这块是整个项目的物理心脏，把“阻尼振子”翻译成数学：输入是当前状态 (x, v) 和参数，输出是变化率 $(\dot{x}, \dot{v})$ 。注意这块不关心怎么解方程，只负责说清“在任意状态下，下一瞬间往哪个方向变”。

第三块是数值求解器块。它拿到物理模型块给的右端函数和参数输入块给的初值，从初始时刻一步步推进到总时长，吐出一整条轨迹：一个时间数组，以及对应的位移数组和速度数组。这块是纯数值的，里面究竟用手写 Euler 还是调 `scipy` 的积分器，对外完全不可见，外面只看到“喂进初值和右端函数，拿回一条轨迹”。

第四块是结果计算块。轨迹本身只是 $x(t)$ 和 $v(t)$ ，可我们真正关心的物理量，总能量 $E(t)$ 、振幅包络、是否过阻尼，需要从轨迹再算一道。这块接收求解器吐出的轨迹和参数，输出这些派生物理量。把它和求解器分开，是因为“解方程”和“从解里提取物理”是两件不同的事：你可能换求解器，但能量算法一字不改。

第五块是可视化与输出块。它接收前面算好的轨迹和能量，画成两张联动的图（上面位移随时间、下面能量随时间），呈现在页面上。输入是一堆数组，输出是用户眼睛看到的图。至此五块齐了。数据流向是一条清晰的链：用户动作进入参数输入块，参数喂给物理模型块得到右端函数，右端函数和初值交给求解器块得到轨迹，轨迹送进结果计算块得到能量，最后轨迹和能量一起交给可视化块画出来。

回头看，这张五块图最值钱的地方，是每块的输入输出白纸黑字写在箭头上。参数输入块吐出一组数字，物理模型块吐出一个函数，求解器块吐出三个数组，结果计算块吐出一个能量数组，这些就是块之间的接口约定。约定一旦写死，五块就可以分头实现、分头验证，谁也不必先等谁。这也正是下一节让 AI 按块写代码能成立的前提。

本节要点

五块图的灵魂在箭头上的数据约定：用户滑块 → 物理参数 → 右端函数 → 轨迹数组 → 能量数组 → 双联图。约定写死，五块可以分头实现和验证。块图是合同，代码是实现，箭头上的数据是接口。

5.3 每一块背后的真实取舍

画块图只是把项目摆开，真正考验项目主理人的，是在每块里做选择。块图上一根简洁的箭头，落到实现时往往对应着好几条岔路，选哪条都有代价。下面挑三个本项目里绕不开的取舍，掰开讲。

第一个取舍在可视化输出块：为什么用 Streamlit 做交互页，而不是裸着用 Jupyter Notebook 加几个 widget。两条路都能在浏览器里拖滑块看图，差别在交付的形态。Jupyter 的强项是探索：边写边跑，单元格里随手画图，适合自己摸索物理。可一旦要把成果交给别人玩，Jupyter 就别扭了，对方得先装环境、会按运行单元格、不能乱点，你交付的本质是一个“开发现场”而不是“产品”。Streamlit 则为“分享一个点开就能玩的东西”而生，把脚本自动变成网页应用，别人拿到网址就能用，完全不必知道背后有 Python。代价是 Streamlit 有它自己的重运行模型（每次交互整段脚本重跑一遍），你得理解这套机制才不会写出又慢又怪的页面。对这门课“最后 ship 一个可点开就能玩的东西”的目标来说，这个学习成本值得付，所以我们选 Streamlit。但你要清楚，选它是为了交付形态，不是因为它“更高级”，自己探索阶段用 Jupyter 反而更顺手。

第二个取舍在数值求解器块：调 scipy 的成熟积分器（比如 solve_ivp），还是自己手写 Euler 循环。这里有三方拉扯：精度、可控性、教学价值。scipy 的积分器是几十年数值分析的结晶，自带自适应步长和误差控制，精度和稳定性远胜手写 Euler。对阻尼振子这种刚性不强的问题，几乎不用调参就出可信结果，省心。手写 Euler 的好处是完全透明，每一步算了什么你一清二楚，出问题好排查。而且它是你上一章亲手学透的方法，写出来心里有底。代价是精度低、步长大了会翻车，还得自己盯着稳定性。这门课的立场很明确：Euler 是唯一要求从头讲透、能手算的方法，是你理解“数字怎么来的”的根基；scipy 积分器当黑箱工具用，会调、能验证即可。所以在正式交互页里，我们倾向调 scipy 求好结果（精度优先），但要求先用手写 Euler 跑一版作对照，用你信得过的方法给黑箱打分。两者吻合，才敢把黑箱放进产品。这个“先手写打底、再上黑箱”的双轨，正是块图把求解器单独成块带来的便利：对外接口不变，里面换实现，外面四块一行都不用动。

第三个取舍藏在结果计算块和块之间的数据传递上：算好的轨迹和能量，一直留在内存，还是落盘存成文件。对阻尼振子这种几秒钟算完、几千个点的项目，答案显然留在内存：算完直接交可视化块画，又快又简单。落盘反而平添读写的麻烦和文件管理负担。但你要知道这个选择的边界在哪里。如果哪天项目变成“扫描一千组参数、每组算很久”，重算代价就高了，这时把结果落盘缓存、下次直接读，就划算了。判断依据很简单：重算代价和读写代价，哪个大？对当前项目，计算极廉价，内存优先。这个判断要随项目规模重新做，不是一条铁律。

三个取舍并排看，共同点很清楚：没有一个选项是绝对正确的，每一个都是在具体目标下的权衡。项目主理人的价值，恰恰在于能说清“我为什么选这条而不是那条”，而不是默认接受 AI 随手给的那一个。

本节要点

没有绝对正确的选项，只有契合当前目标的取舍。Streamlit 为交付形态埋单，scipy 为精度站台（但要先用手写 Euler 打底），内存优先因为重算廉价。能说清选择理由，就不怕 AI 替你乱拍板。

AI 给你 Streamlit、给你 `solve_ivp`、给你内存数组，多半是对的，但你得知道它替你做了哪些选择，以及换个场景这些选择会不会翻车。

5.4 让 AI 按块实现，并用块图当对账单

块图画好、取舍想清楚，就轮到 AI 干活了。关键动作是：不要笼统地说“给我写个阻尼振子交互页”，而是把块图连同每块的输入输出约定一起喂给 AI，让它按块生成代码骨架。这样出来的代码天然带着你设计的接缝，读起来按块读，验证也按块验。

AI 协作实录 · AI 把五块偷偷并成四块

我对 AI 说：

你是一名熟悉大一普物和 Python 数值建模的助手。我已经把阻尼振子交互页设计成五个功能块，请你严格按这五块、每块写成一个独立函数，生成代码骨架，先不必填满实现细节，但每个函数的输入参数和返回值要和我下面的约定完全一致。块一参数输入：返回一个含 m, k, b, x_0, T, dt 的参数字典。块二物理模型：输入状态 (x, v) 和参数字典，返回变化率 $(\dot{x}, \dot{v})$ 。块三数值求解器：输入物理模型函数、初值、参数字典，返回时间、位移、速度三个数组。块四结果计算：输入位移、速度数组和参数字典，返回能量数组。块五可视化：输入时间、位移、能量数组，画出双联图。

AI 给我：

AI 返回了一份结构清楚的骨架，五个函数大体到位，注释也写得像模像样。但仔细一读，它把块三和块四合并了：求解器函数在推进轨迹的同时，顺手算了每步能量，直接返回时间、位移、速度、能量四个数组，原本独立的“结果计算块”消失了，变成了求解器里的几行。AI 还在回复里得意地说了一句，这样“少一次数组遍历，更高效”。

我的审计：

代码能跑，能量也算得对，但我没放过这个改动。我做的不是逐行读代码，是拿出五块图一块一块对账，这正是阶段交接核查（phase-handoff verification）该做的事：这一阶段写出的代码，有没有偷偷增删、合并我上一阶段定下的块。一对就发现，块四不见了，被吞进了块三。我让 AI 拆回来，理由不是我固执，是这次合并破坏了我特意留的接缝：把能量计算焊死在求解器里，将来想换求解器（比如从手写 Euler 换成 scipy），就得把能量逻辑跟着搬一遍，甚至重写。保持块四独立，求解器只管吐轨迹，能量算法一字不动地复用。AI 说的“少一次遍历”在这个几千点的项目里省不下任何能感知的时间，却拿走了我架构上最值钱的东西，可换的求解器。我把这个理由讲给 AI，让它恢复五块结构，它照办了。重跑，五块各自独立、能量曲线单调下降，验证通过。

这段实录的要害有两层。第一层是按块生成的好处：因为有事先写死的接口约定，AI

给的骨架天然可以一块一块读、一块一块测，我没被一坨代码淹没，反而像检查流水线上的五个工位。第二层是更深的纪律：AI 出于它认为的”优化”，悄悄改了我的架构，而这种改动最容易发生在阶段与阶段的交接处，反而不容易在单块内部被发现。对付它的办法不是把代码读得更细，是退回上一阶段交付的块图，逐块核对代码有没有背叛它。块图不只是设计图，更是一张对账单。

本节要点

AI 会在阶段交接处偷偷改动你的架构（比如把五块并成四块），这种小动作比你想象的更常见。对付它的办法是退回上一阶段交付的块图，逐块对账，而不是把代码读得更细。块图就是你和 AI 之间的合同。

本能 · Architecture-before-code

拿到任何一个要写成代码的项目，先别急着开口让 AI 写。先在纸上拆成不超过五块的系统块图，每块写清输入什么、输出什么、箭头传的是哪个数据。把这张图当成你和 AI 的合同：让它按块实现，你按块验收。全局握在你的图里，细节交给 AI 的块里，项目再大你也不会失控。

AI 陷阱预警 · AI 倾向于一次吐出一大坨不分块的代码

你只说”帮我写个阻尼振子交互页”，AI 几乎一定会回给你一整段几百行、所有功能搅在一起的代码。跑得起来，看着也专业，可它没接缝：你说不清改一个滑块牵动哪几行，验证也无从一块一块下手，最后只能整段”相信它对”。对策是把因果倒过来：先给架构，再让它填块。把你画好的系统块图和每块的输入输出约定先交给 AI，明确要求”每块写成一个独立函数、接口和我的约定一致、先给骨架”。这样代码一落地就带着你设计的接缝，全局始终在你手里。

本章小结

这一章把工作方式从”让 AI 写一段代码”升级到”让 AI 按你的架构写一个项目”。核心是架构先行：写任何代码之前，先把项目画成系统块图，按职责拆成不超过五块，标清每块的输入输出和块间数据流。阻尼振子交互页拆成参数输入、物理模型、数值求解器、结果计算、可视化输出五块，逐块说明了数据怎样从用户的滑块一路流到屏幕上的双联图。三个真实取舍：Streamlit 为交付形态埋单，scipy 为精度站台（但先用手写 Euler 打底），内存优先因为重算廉价。AI 协作实录里，AI 擅自把五块并成四块，揪出它靠的不是读代码更细，而是退回块图逐块对账（阶段交接核查）。

块图解决的是”项目长什么样”，可交给 AI 的那段话本身写得好不好，直接决定它给的骨架质量。下一章就打磨提示词这个环节。

思考题

1. 本章把阻尼振子拆成了五块。现在要求页面再加一个功能：用户可以选择”无阻尼”、”欠阻尼”、”过阻尼”三种工况，页面自动调出对应的阻力系数 b 并标注当前属于哪种工况。请说明这个新功能应当加进五块中的哪一块（或要不要新开一块），它的输入输出会怎么变，以及为什么放在这里而不是别处。判断的依据是什么？
2. 在求解器块的取舍里，我们说”先用手写 Euler 跑一版作对照，再上 `scipy` 黑箱”。可阻尼振子是一个会振荡的系统，上一章告诉我们显式 Euler 在某些情况下会翻车。请你结合上一章对步长稳定性的讨论，说明用手写 Euler 给 `scipy` 结果打分时，你应当特别留意什么；如果两者对不上，你会先怀疑哪一方，怎么排查？
3. AI 在实录里把”结果计算块”吞进了”求解器块”，理由是”少一次数组遍历更高效”。请你站在项目主理人的角度，写出至少两条反驳它的理由（提示：一条关于将来换求解器，一条关于这个项目的数据规模下”高效”到底值不值）。再反过来想：在什么样的项目里，AI 这个合并反而是对的？

6. 写好提示词：Anthropic 六维框架

把和 AI 协作比作指挥一支技艺高超却毫无主见的施工队，那么（prompt）就是你递给它的工作说明书。同一支队伍，收到一句“帮我盖个房子”，会东拼西凑出一个能住人但处处别扭的东西；收到一份标了朝向、承重、水电走线、验收标准的完整图纸，才能交出你真正想要的房子。问题在于，绝大多数人写提示词时更像前者：丢一句模糊指令，然后在 AI 含糊的回答里反复来回修补。Anthropic 应用 AI 部门的工程师在一次内部分享里给出了一个让人警醒的观察：大多数人在撰写提示词时，最多只用到了六大核心要素中的一到两项。这意味着他们只发挥了模型不到百分之三十的潜力。换句话说，你手里这台工具的七成能力，是被一句随手写下的烂提示词白白浪费的。

这一章要做的，就是把提示词从“凭经验吊车尾”的玄学，升级成一套可教、可考、可复用的工程。我们逐个拆开 Anthropic 总结的六个维度，每维都配物理课里能直接落地的写法；然后用同一个物理任务做对比演示，让你亲眼看到：一份只写了一两维的简陋提示词，和一份六维全开的完整，跑出来的结果在单位标注、不确定性表达、可解析性上差出多远。这门课里那句被反复引用的话，值得先记在心里：一个完美的提示词不该像和聊天机器人那样来回修正，而应该像一份完整的工作说明书。

本章地图

小节	这一节解决什么	关键方法 / 产物
6.1	逐维拆解 Anthropic 六维提示词框架	D1 角色、D2 语气、D3 背景、D4 分步、D5 格式、D6 结尾强化
6.2	用同一拟合任务对比简陋与完整提示词	ailog 无单位 vs 可审计的物理结论

6.1 六个维度：一份完整工作说明书的骨架

Anthropic 把一份高质量提示词拆成六个相互配合的维度，习惯上记作 D1 到 D6。它们不是六条孤立的规则，是一份说明书从“你是谁”到“别忘了什么”的完整链条：D1 交代角色与任务情境，D2 设定语气与信心，D3 喂入背景资料，D4 给出分步执行指令，D5 提供示例与输出格式，D6 在结尾做最终提醒与强化。下面逐个展开。

6.1.1 D1 任务情境与角色：先告诉 AI 它是谁

提示词的第一句话，应当明确告诉 AI 它正在扮演什么角色、要完成什么任务，而不是让它默认成一个面向全人类的“通用助手”。这一点看似客套，实则关键。当你说“你是一名熟悉大一普通物理力学和数值方法的物理建模助手”，模型会自动收束它的措辞、默认的精度要求、以及要不要跟你解释什么是导数。一个被设定成“物理建模助手”的模型，远比“通用助手”更愿意主动标注单位、写出方程、做量纲检查，因为这些是它扮演的角色的职业习惯。

在我们这门课里，物理项目的提示词开头几乎总有一句固定的角色设定，并把工作约束指向项目里的约束文件。典型开头是这样：

你是一名熟悉大一普通物理力学和数值方法的物理建模助手，工作约束见 AGENTS.md。本任务属于“阻尼振子参数拟合”项目的一部分。

短短两句，既定了角色（物理建模助手，不是写诗的、不是写营销文案的），又定了任务情境（这是某个具体项目的一环，不是一次性孤立问答），还把项目级约束的责任甩给了一份可被反复引用的文件，这就为后面的 D3 埋下了伏笔。

6.1.2 D2 语气与信心设定：让 AI 学会说“我不确定”

第二个维度管的是输出的风格和态度。你要明确告诉 AI：是要事实性、严谨的回答，还是要发散头脑风暴。尤其重要的是，没把握时必须显式承认“我不确定”，而不是用一套语法正确、口吻笃定的话把你带进沟里。模型最危险的地方从来不是它不会，而是它会一本正经地编造，物理上叫幻觉。一句明确的信心设定，能逼它在不确定时举手，而不是糊弄。

这一维里还藏着一个非常实用的战术补充。在提示词里加上“think hard”、“think deep”、“think longer”这类短语，实测能让大多数模型在回答前多走一到两步推理，而不是脱口而出第一个念头。对需要推导的物理任务，这一句几乎是免费的精度提升。物理提示词里这一维的典型写法是：

不确定时必须明说“我不确定”，不要编造数值或公式；凡引用文献，必须给出 DOI 或 arXiv 编号；遇到需要推导的步骤，先 think hard 一下再作答。

注意这里的每句都对应一个具体失败模式：不编造数值，是防它瞎填一个看起来合理的常数；引用必须给 DOI，是防它伪造一篇不存在的论文；推导前先 think hard，是防它把本该分四步的推导压成一句拍脑袋的结论。

6.1.3 D3 背景细节与资料：把它需要的全喂给它，不要让它猜

第三个维度是把 AI 完成任务所需的全部资料、数据、文件结构，在提示词里交代清楚，而不是指望它凭空猜出项目里的约定。AI 不知道你的数据文件第一列是时间还是位移，不知道你这个项目只许用 `numpy` 和 `matplotlib`，不知道前两章定好的物理参数范围，除非你告诉它。让它猜，等于把项目成败押在运气上。

落到这门课的做法，是把项目里的几份核心文件直接喂进去：需求文档（需求文档.md，这个项目到底要做什么）、约束文件（AGENTS.md，有哪些硬规矩）、以及数据结构的说明（数据 schema，比如”两列 CSV，第一列时间 t ，单位秒；第二列位移 x ，单位米”）。你可以整段粘贴进提示词，也可以用工具支持的 `@filename` 方式显式引用。好习惯是用 XML 标签把背景资料包起来，让模型一眼分清哪些是背景、哪些是指令：

```
<context>
数据文件 data.csv：两列，第一列时间  $t$ （秒），第二列位移  $x$ （米），采样间隔约 0.01 秒，共约 2000 个点，含实验噪声。
物理模型：欠阻尼自由振动  $x(t) = Ae^{-\gamma t} \cos(\omega t + \phi)$ 。
项目约束见 AGENTS.md：只许用 numpy / scipy / matplotlib。
</context>
```

背景钉死在 `<context>` 标签里，模型就不会把列含义猜反，也不会擅自引入一个你不想用的第三方库。

6.1.4 D4 详细执行指令：不是”做 X”，而是”先 X1，再 X2，然后 X3”

第四个维度是把任务拆成有顺序的具体步骤。常见误区是只给一句目标，比如”帮我拟合这段数据”，然后期待 AI 自动想清楚先干什么后干什么。但模型和人一样，给一条清晰的推理路径，它走得稳得多。所以指令要写成有编号的步骤序列，不是一句笼统的命令。

这一维同样有个战术原则：具体胜过含糊（be specific over vague）。不要写”修一下侧边栏”，要写”修改侧边栏，使得当滑块 X 的值大于 0.5 时，面板 Y 变为可见”。前者把判断推给 AI，结果往往不是你要的；后者把精确意图直接交付，AI 几乎没有走偏的空间。物理任务里的分步指令典型长这样：

请按以下步骤完成：1. 先列出物理假设和所有变量及其单位；2. 写出阻尼振子的位移表达式 $x(t) = Ae^{-\gamma t} \cos(\omega t + \phi)$ 并说明每个参数的物理含义；3. 用 `scipy.optimize.curve_fit` 对 `data.csv` 做非线性最小二乘拟合，给出 A, γ, ω, ϕ 四个参数及其标准差；4. 画出原始数据点与拟合曲线的叠加图；5. 画出残差图；6. 在末尾报告角频率 ω 和阻尼系数 γ 的拟合值与不确定度，全部带 SI 单位。

把”拟合数据”展开成六步，AI 就不会偷工：它不会跳过单位，不会忘记给不确定度，更不会用一个让你看不懂的方式把结果埋在代码里。

6.1.5 D5 示例与输出格式：贴一个范例，胜过描述十句

第五个维度管的是输出形态。最有效的办法不是用文字描述想要什么格式，是直接贴一两个完整的“输入对应输出”范例给 AI 看，让它照葫芦画瓢。同时，用 XML 标签把输出切成若干结构化段落，能让 AI 的回答变得可解析，你（或者下一段代码）一眼就能定位方程在哪、代码在哪、验证在哪，不用在一大段散文里大海捞针。

这一维的战术补充是：提供示例并覆盖边界情况（provide examples and edge cases）。光给一个正常例子还不够，最好再贴一个边界例子，比如“数据点少于 20 个，直接报告无法可靠拟合”，让 AI 知道异常情况该怎么处理。物理提示词里要求结构化输出的典型写法是：

请把回答严格分成以下四个 XML 标签段落，不要在标签外写多余的话：

<assumptions> 这里写物理假设与变量定义 </assumptions>

<equations> 这里写用到的方程 </equations>

<code> 这里写完整可运行的 Python 代码 </code>

<verification> 这里写你建议我做的验证项，每项一句 </verification>

这样切出来，假设、方程、代码、验证各归其位。日后想把多次对话的代码批量抽出来，或者核对 AI 自己提出的验证项，结构化标签替你省下大量功夫。

6.1.6 D6 最终提醒与强化：在结尾再钉一次最硬的约束

最后一个维度，是在提示词结尾再重复一遍最关键、最不能破的硬约束。这不是啰嗦。模型处理长提示词时，开头和结尾的内容往往比中间更受重视；把最致命的约束放在末尾再钉一次，能显著降低它在长回答里遗忘这条约束的概率。对物理任务，最该在结尾重申的往往是单位、可用库、以及不确定时要说“不确定”。典型结尾是：

再次强调：所有数值结果必须带 SI 单位；除 numpy、scipy、matplotlib 外不许引入任何第三方库；任何你不确定的地方都要显式标注“我不确定”，不要用笃定的口吻掩盖。

这三句话在 D2、D3 里都出现过，但在结尾再钉一次，等于给整份说明书加了一道保险。

本节要点

六维不是六条孤立的规则，是从“你是谁”到“别忘了什么”的完整链条。D1 定角色收束措辞，D2 逼它不确定时举手，D3 喂饱背景杜绝乱猜，D4 列出步骤杜绝偷工，D5 贴范例定格式，D6 在结尾钉死最硬的那条约束。一维都不能少。

6.2 对比演示：同一个拟合任务，两份提示词，两种命运

道理讲完，不如看一次现场。下面用同一个物理任务做对比：手上有一段实验数据，记录了一个欠阻尼振子的位移随时间变化，模型是 $x(t) = A e^{-\gamma t} \cos(\omega t + \phi)$ ，要从数据里

拟合出角频率 ω 和阻尼系数 γ 。先用一份只写了 D1 加 D4 的简陋提示词，再用一份 D1 到 D6 全开的完整工作说明书，看 AI 两次输出的差距。这正是这一章的灵魂：你要建立的不是对某份范例的记忆，是一种”少了哪一维、输出就会在哪里崩”的反射。

AI 协作实录 · 简陋提示词 vs 完整工作说明书

我对 AI 说：

第一版（只写 D1 加 D4，模仿大多数人的写法）：

你是物理助手。帮我拟合这段数据，把阻尼振子的角频率和阻尼系数算出来，数据在 data.csv 里。

AI 给我：

AI 返回了一段能跑的代码，调用 `curve_fit` 拟合，最后打印出两行：`omega = 6.28` 和 `gamma = 0.31`。没有单位，没有不确定度，没有残差图，初始猜测值是随手填的，而且它默认 data.csv 第一列是时间、第二列是位移，却没有跟我确认过这个假设。对着这两个数字，我完全说不出它们到底可不可信：6.28 是 rad/s 还是 Hz？0.31 的量纲是什么？拟合得好不好？一概不知道。

我的审计：

问题不在 AI，在我那份提示词。只给了角色（D1）和一句模糊目标（勉强算半个 D4），剩下四维全空。它不知道我要单位（D6 缺失），不知道要不确定度（D4 不够细），不知道数据列的真实含义（D3 缺失），也不知道我要可解析的分段输出（D5 缺失）。这种输出拿去答辩，一句”你这个 omega 的单位是什么、误差多大”就能把我问倒。该重写的不是代码，是提示词。

我对 AI 说：

第二版（D1 到 D6 全开，完整工作说明书）：

你是一名熟悉大一普通物理力学和数值方法的物理建模助手，工作约束见 AGENTS.md（D1）。不确定时必须明说”我不确定”，不要编造数值；需要推导处先 think hard 一下再答（D2）。

<context>（D3）数据文件 data.csv：两列，第一列时间 t （秒），第二列位移 x （米），约 2000 个点，含噪声。物理模型为欠阻尼自由振动 $x(t) = Ae^{-\gamma t} \cos(\omega t + \phi)$ 。只许用 numpy / scipy / matplotlib。</context>

请按步骤完成（D4）：1. 列假设与变量及单位；2. 写出模型方程并解释各参数；3. 用 `curve_fit` 拟合，给出 A, γ, ω, ϕ 及各自标准差；4. 画数据与拟合曲线叠加图；5. 画残差图；6. 报告 ω 与 γ 的值及不确定度，带 SI 单位。

请严格按四个标签分段输出（D5）：<assumptions>、<equations>、<code>、<verification>，标签外不写多余的话。

再次强调（D6）：所有数值必须带 SI 单位（ ω 用 rad/s， γ 用 1/s）；不许引入指定之外的库；不确定处显式标注。

AI 给我：

AI 这次的输出整整齐齐分成四段。<assumptions> 里写明了它假设数据第一列是时间、第二列是位移，并提醒我这个假设若不对要纠正它；<equations> 里写出了模型并逐个解释了 A, γ, ω, ϕ 的物理含义；<code> 里的代码给初始猜测值时还根据数据的振荡周期估了一个合理的 ω 初值，而不是乱填；最后报告 $\omega = (6.28 \pm 0.02) \text{ rad/s}$ 、 $\gamma = (0.31 \pm 0.01) \text{ s}^{-1}$ ，单位齐全、不确定度齐全。<verification> 段里它甚至主动建议我“检查残差是否在零附近随机分布，若残差有系统性结构说明模型不足”。

我的审计：

这一次，不必通读代码就能审。单位对不对，一眼看得见；不确定度有没有，一眼看得见；它的假设是否和我的数据一致，它自己在 <assumptions> 里替我标了出来，核对一下即可。更重要的是，它在 <verification> 里提的残差判据，正是我接下来该亲手做的那道关。同一个任务、同一个模型、同一个 AI，仅仅因为提示词从二维补到了六维，输出就从“无法验收”变成了“可以审计”。

两次输出并排放着看，差别一目了然。第一版的 6.28 是个悬空的数字，不知道单位、不知道误差、不知道拟合好坏；第二版的 $(6.28 \pm 0.02) \text{ rad/s}$ 是个可被审计的物理结论。促成这个跃迁的不是更聪明的模型，是更完整的说明书。这就是为什么 Workshop 里那句话值得反复咀嚼：完美的提示词不是和聊天机器人来回拉扯出来的，是一开始就写得像一份完整的工作说明书。你在提示词上多花的五分钟，省下的是事后反复追问、反复返工的几十分钟。

本能 · 六维提示词回扣 Spec-first 与 Trust but Verify

六维框架不是孤立的写作技巧，它是这门课两条主线在输入端的落地。D1 到 D4，本质上是 Spec-first 的延伸：在让 AI 动手之前，先把角色、背景、步骤这份“规格”说清楚，而不是让它先写一版再回来改。而 D2 的“不确定时说不确定”、D5 的结构化 <verification> 段、D6 的单位强调，则直接服务于 Trust but Verify：它们让 AI 的输出从一开始就带着可被验证的接口，单位在哪、误差在哪、该查什么，钉在固定位置，等着你拿验证清单去逐项对账。写好提示词，等于提前为后面的验证铺好了轨道。

AI 陷阱预警 · 上下文已经混乱时，别讲道理，开新 chat 从清白上下文重来

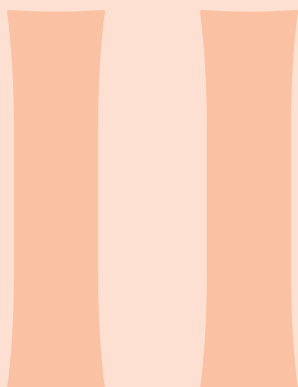
有时你会陷进这样的泥潭：一段对话聊到第二十轮，AI 开始牛头不对马嘴，把三轮前你否定过的方案又端出来，或者乱命名文件，把 `fit.py` 写成 `fit_v2_final_real.py` 还前后矛盾。这通常是上下文窗口被前面大量来回污染、模型已分不清哪些是有效约定的信号。此时最常见也最浪费时间的错误，是继续用提示词工程去“说服”它、去修这一团乱麻。正确的动作是：果断开一个新 chat，把前面积累下来、确实有效的约束，一份干净的六维提示词加上 AGENTS.md，重新喂进去，从清白上下文重新开始。沉没成本要忽略。一份写得好的六维提示词在这里还有隐藏价值：它本身就是可复制粘贴的，让你“重开一局”几乎零成本。

本章小结

这一章把提示词从玄学拆成了 Anthropic 的六个可教可考的维度：D1 角色与任务情境、D2 语气与信心（含 think hard）、D3 背景资料（需求文档、AGENTS.md、数据 schema）、D4 分步执行指令（具体胜过含糊）、D5 示例与结构化输出（XML 标签加范例）、D6 结尾的最终提醒。用同一个阻尼振子拟合任务做了对比：只写两维的简陋提示词给出一个悬空、无单位、无误差、无法验收的数字；六维全开的完整工作说明书给出一个带单位、带不确定度、自带验证建议、可被审计的物理结论。要记住的核心：六维不是模板，是一种内化的反射。今天写提示词也许要逐维去想“我交代角色了吗、我喂背景了吗”；练到熟了，你不会再逐维默念，但六维全都在你写下的每句话里，像一个老到的物理学家拿到任何结果都下意识先看一眼量纲一样自然。

思考题

1. 回到本章对比演示里的第一版简陋提示词。请你不照抄第二版，而是只补一个维度，使得 AI 的输出至少能带上单位。你会选择补 D3、D5 还是 D6？为什么单补这一维就足以解决“无单位”问题，而补别的维度未必能？
2. D2 里的“think hard”和 D6 里的“不确定时说不确定”，看起来都是在管 AI 的“态度”，但它们防的是两种不同的失败。请用一个具体的物理拟合场景，分别说明：少了“think hard”会让 AI 在哪一步偷懒，少了“不确定时说不确定”又会让它在哪一步把你带进沟里。
3. 假设你把第二版完整提示词喂给 AI 后，它在 <verification> 段里建议你“检查残差是否随机分布”，你照做后发现残差呈现明显的系统性弯曲。这说明 $x(t) = A e^{-\gamma t} \cos(\omega t + \phi)$ 这个模型可能漏掉了什么物理（提示：想想真实振子除了线性阻尼还可能有什么）？这时你该回头修改提示词的哪一维，把缺的物理补进去？



7	读懂并指挥 AI 写 Python	65
7.1	为什么“读和改”比“从零写”更重要	
7.2	用一段完整脚本学最小的 Python	
7.3	手算对账：本章的 no-AI 纪律	
7.4	改参数重跑：让代码听你的，也看它怎么翻车	
7.5	AI 协作实录：让 AI 改脚本，再亲手揪出它的错	
8	一阶常微分方程与 Euler 法	73
8.1	从三个物理问题到同一个数学骨架	
8.2	Euler 显式法：把连续演化切成小步	
8.3	第一个完整例子：RC 电路放电	
8.4	截断误差与步长：为什么 dt 不能太大	
8.5	AI 协作实录：让 AI 写放电代码并审计它	
8.6	用验证清单确认结果	
9	二阶常微分方程与守恒律法庭	81
9.1	从弹簧振子到阻尼振子	
9.2	状态向量约化：把一个二阶拆成两个一阶	
9.3	用显式 Euler 推进阻尼振子	
9.4	用能量审计数值解	
9.5	显式 Euler 的能量漂移：代码没错，能量却在涨	
9.6	半隐式 Euler：一行改动止住漂移（黑箱披露）	
9.7	AI 协作实录：状态向量配错，能量三联图当庭抓获	
10	升维：二维就是两个一维 ODE	91
10.1	核心思想：二维就是两个一维并列	
10.2	主例：带空气阻力的抛体	
10.3	用显式 Euler 解二维抛体	
10.4	极限验证：本章的灵魂	
10.5	量纲检查终端速度：一分钟的审计反射	
10.6	AI 协作实录：让 AI 写抛体代码并审计它	
10.7	选读：二维行星轨道	
11	数据拟合：最小二乘与残差	101
11.1	从数据反推参数：为什么需要拟合	
11.2	最小二乘原理	
11.3	非线性拟合：把算法当黑箱，但要能验证	
11.4	残差图比 R 方诚实	
11.5	参数不确定性：用 $\pm 10\%$ 扫描代替统计推断	
11.6	no-AI 基线：用纸笔和 Excel 对账线性拟合	
11.7	AI 协作实录：用残差图推翻一个高 R 方的拟合	
11.8	用验证清单确认拟合结果	
12	蒙特卡罗初步：用随机数做物理	111
12.1	核心思想：用随机采样估计一个确定的量	
12.2	第一个例子：撒点估计圆周率	
12.3	收敛行为：精度随样本数怎么提高	
12.4	第二个例子：用随机数估计定积分	
12.5	第三个例子：一维随机行走与扩散	
12.6	AI 协作实录：识破小样本的收敛假象	



7. 读懂并指挥 AI 写 Python

在动手解任何物理问题之前，先把一件事说在前头：这门课不要求你能从白纸开始凭空写出完整程序。在今天，AI 几秒钟就能吐出几十行结构工整的代码，”从零敲代码”这项技能的稀缺性正在迅速贬值。真正稀缺、也真正属于你这个项目主理人的，是另外三件：读懂 AI 给你的那三五十行 Python 到底在算什么；改动参数让它重跑出你想要的情形；用纸笔手算头几步，和代码输出逐一对账，证明你不是在给 AI 的产物盖橡皮图章。这一章就练这三件事。刻意把真正的微分方程留到下一章，先用一个你高中就解析解过、闭着眼睛都知道答案的物理问题，把读代码、改代码、验代码的肌肉记忆建立起来。

本章地图

小节	这一节解决什么	关键方法 / 产物
7.1	论证”读和改”为何比”从零写”更值钱	AI 是高效合作者，审代码门槛反而被抬高
7.2	用一段抛体脚本学会最小 Python	变量、def、for、zeros、linspace、plot
7.3	合上电脑手算头几个时刻的坐标，与代码对账	手算对账：证明你不是在盖橡皮图章
7.4	拧动参数看趋势，故意改错看翻车	预判加验证，建立”跑通不等于物理对”的警觉
7.5	让 AI 简化代码，亲手揪出它把度当弧度	独立参考值 $v_y \approx 14.14$ 一卡就露馅

7.1 为什么“读和改”比“从零写”更重要

设想两个大一学生面对同一个建模任务。第一个坚持自己从零写，语法记不全，括号对不齐，半天卡在一个拼写错误上，交出的程序能跑，但他说不清为什么要这么写。第二个让 AI 先写出一版，然后逐行读、逐行问，不懂的语句一句句弄明白，再改两个参数试探边界，最后用计算器验了头三步。两份作业的代码也许长得差不多，可这两个人的能力天差地别。第一个人是代码搬运工，第二个人是代码主人。

这门课要把你培养成第二种人。原因很现实：AI 写代码又快又顺，但它不对物理负责，也不对你的具体场景负责。它会理直气壮地把角度单位搞错、把重力方向写反、把该衰减的量写成发散，全程语气笃定、排版漂亮，让外行根本看不出问题。挡在这些错误和你结论之间的唯一一道关，就是你能不能读懂代码、能不能验证它。换句话说，写代码的门槛被 AI 拉低了，审代码的门槛反而被抬高了，因为你现在面对的是一个会一本正经犯错的高产合作者。项目主理人的核心价值，从“我能写”彻底转移到了“我能读、能审、能迭代”。

本能·先问会读再问会写

拿到任何一段 AI 生成的代码，第一反应不该是“能不能跑”，而该是“我能不能逐行说出它在算什么”。有哪一行你说不清，那就是你和这段代码之间的漏洞，也是 AI 最可能藏错误的地方。把不懂的那行单独拎出来问 AI，直到能用自己的话复述它，再往下走。

本节要点

AI 降低了写代码的门槛，却抬高了审代码的门槛。项目主理人的核心价值不在于能写，而在于能读、能审、能迭代。AI 会对你的具体场景和物理合理性格格不入。

7.2 用一段完整脚本学最小的 Python

学游泳的最好办法，是先下水扑腾，不是先背完所有泳姿要领。学读代码也一样：与其先啃一本语法书，不如先盯一段完整、能跑、又足够简单的程序，把涉及的每个语言要素就地弄懂。下面这个例子是抛体运动，一颗小球以初速度 v_0 沿与水平方向成 θ 角的方向抛出，不计空气阻力。它的水平和竖直坐标随时间变化有现成的解析公式

$$x(t) = v_0 \cos \theta \cdot t, \quad y(t) = v_0 \sin \theta \cdot t - \frac{1}{2}gt^2.$$

请特别注意：这里我们是直接把这两条已知公式代进去算，不涉及任何微分方程，也不需要一步步往前推。把“用递推一步步推进”的活留给下一章的 Euler 法，这一章只关心怎么读懂一段把解析公式翻译成数组和图的代码。我们要算出一串时刻的 x 和 y ，再把轨迹画出来。下面是完整脚本，你可以原样拷进编辑器跑。

Listing 7.1: 抛体运动轨迹的解析公式数值求值

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```
3
4 # 物理参数 (一律用 SI 单位)
5 g = 9.8          # 重力加速度, 单位 米每二次方秒
6 v0 = 20.0        # 初速度大小, 单位 米每秒
7 angle_deg = 45.0 # 抛射角, 单位 度
8
9 # 把角度从度换算成弧度, 因为 numpy 的三角函数只认弧度
10 angle_rad = np.deg2rad(angle_deg)
11
12 # 把初速度分解到水平与竖直方向
13 vx = v0 * np.cos(angle_rad) # 水平初速度, 单位 米每秒
14 vy = v0 * np.sin(angle_rad) # 竖直初速度, 单位 米每秒
15
16 # 一个把时刻 t 映射到 (x, y) 坐标的函数
17 def position(t):
18     x = vx * t          # 水平位移
19     y = vy * t - 0.5 * g * t**2 # 竖直位移
20     return x, y
21
22 # 数值参数: 用固定步长 dt 铺一串时刻, 直到小球落回地面
23 dt = 0.1              # 时间间隔, 单位 秒
24 T_flight = 2.0 * vy / g # 总飞行时间, 单位 秒
25 N = int(T_flight / dt) + 1 # 时间点的个数 (加一步,
    确保覆盖到落地时刻)
26
27 # 准备好存坐标的数组, 先全部填零, 后面再逐个写入
28 t = np.linspace(0.0, N * dt, N + 1)
29 x = np.zeros(N + 1)
30 y = np.zeros(N + 1)
31
32 # 逐个时刻调用 position, 把结果填进数组
33 for n in range(N + 1):
34     x[n], y[n] = position(t[n])
35
36 # 画出轨迹: 横轴是水平坐标, 纵轴是竖直坐标
37 plt.plot(x, y, 'o-', markersize=3)
38 plt.xlabel('水平坐标 x (m)')
39 plt.ylabel('竖直坐标 y (m)')
40 plt.title('抛体运动轨迹')
41 plt.tight_layout()
42 plt.show()
```

这段代码总共三十来行, 麻雀虽小, 却把后面所有章节会反复用到的语言要素都用了

一遍。我们一段段拆开看，每个要素只讲到够你读懂为止。

最先出现的是。像 $g = 9.8$ 这样的写法，意思是“造一个叫 g 的盒子，往里放一个数 9.8”。此后凡写 g 的地方，Python 都去这个盒子里取出 9.8 来用。变量名你可以自己起，但好名字本身就是注释， $v0$ 一眼就知道是初速度，远胜过起个 a 或 tmp 。这里还能看到一个好习惯：每个物理量后面用井号 $\#$ 开头写了中文注释标明单位，井号之后到行尾的内容 Python 一概不执行，纯粹给人看的。

接着是函数定义，也就是 `def position(t):` 这一段。函数是一个可反复使用的小工具：你给它一个输入（这里是时刻 t ），它按内部的几行算法加工一番，再用 `return` 把结果交还给你（这里一次交还 x 和 y 两个值）。定义函数的好处是把“已知时刻求坐标”打包成一个名字，主程序里就不必把那两条公式抄来抄去。注意 `def` 下面那几行都向右缩进了，Python 靠缩进来判断哪些代码属于这个函数内部，缩进错了程序就读不懂，这一点和其他语言很不一样，初学时尤其要留神。

Definition 7.2.1 : 函数 `def`. `def` 名字（输入）：用来定义一个函数，把一段会被反复使用的计算打包起来。函数体向右缩进，用 `return` 把计算结果交回给调用者。在本章的脚本里，`position(t)` 接收一个时刻，返回该时刻的水平和竖直坐标。

然后是 `numpy` 这个数值计算库提供的两个数组操作。`np.zeros(N + 1)` 造出一个长度为 $N + 1$ 、元素全是零的数组，相当于先备好一排空格子等着往里填数；我们对 x 和 y 都这么做，因为循环里要一个一个把坐标写进去。`np.linspace(0.0, N*dt, N + 1)` 则造出一个从 0 到 $N \Delta t$ 、均匀分成 $N + 1$ 个点的数组，正好就是我们要计算的那一串时刻。这两个函数你现在只需记住它们各自造出什么样的数组；至于 `numpy` 还能让整个数组一次性参与运算（所谓广播），我们到后面真正需要时再说。这一章一律老老实实用循环逐个处理，读起来最不容易出错。

Definition 7.2.2 : `zeros` 与 `linspace`. `np.zeros(m)` 生成一个长度为 m 、元素全为零的数组，常用来预留存储空间。`np.linspace(a, b, m)` 生成一个从 a 到 b （含两端）、共 m 个等间距点的数组，常用来铺一条均匀的坐标轴或时间轴。

紧接着是 `for` 循环，也就是 `for n in range(N + 1):` 这一段。它的意思是“让 n 依次取遍 $0, 1, 2, \dots, N$ 这些整数，每取一个值，就把缩进部分的代码执行一遍”。在我们的脚本里，循环体只有一行 `x[n], y[n] = position(t[n])`，它取出第 n 个时刻 $t[n]$ ，喂给 `position` 函数算出坐标，再把结果写回数组的第 n 个格子。这里 `x[n]` 这种带方括号的写法叫下标访问，表示数组里的第 n 个元素。注意 Python 从 0 开始数，所以 `x[0]` 是第一个、`x[1]` 是第二个，初学者最常栽在“从零起数”上。循环跑完，整条 x 和 y 数组就都填满了。

Definition 7.2.3 : `for` 循环与数组下标. `for n in range(m):` 让变量 n 依次取 0 到 $m - 1$ 的整数，每次都执行一遍缩进的循环体。数组用 `x[n]` 取第 n 个元素，下标从 0 开始，所以长度为 m 的数组合法下标是 0 到 $m - 1$ 。

最后是 `matplotlib` 这个画图库的 `plt.plot`。`plt.plot(x, y, 'o-')` 把 x 当横坐标、 y 当纵坐标，逐点画出来并用线连起来，`'o-'` 表示既画圆点又连实线。后面几行 `plt.xlabel`、

`plt.ylabel`、`plt.title` 给坐标轴和图加上中文说明，`plt.show()` 把图显示出来。画图细节五花八门，但你现在只需认得“把两个数组当横纵坐标画一张图”这个最基本的动作。

需要明确说在这里：为了让你读得轻松，这一章刻意没有教面向对象（那种“对象·方法（）”的组织方式）、没有教异常处理（用 `try` 去兜住可能出错的代码），也没有教 `numpy` 的广播。这些都是有用的工具，但它们会让一段三十行的代码读起来像四十行，得不偿失。等到后面某一章真的离不开它们时，我们再补。眼下，变量、函数、循环、两个数组操作和一个画图命令，已经足够读懂绝大多数大一阶段会遇到的物理建模脚本了。

本节要点

变量、函数定义、for 循环、`zeros` 与 `linspace`、`plot` 作图，这五样东西够你看懂绝大多数大一建模脚本。本章不碰面向对象、异常处理和广播，是刻意取舍，不是疏漏。

7.3 手算对账：本章的 no-AI 纪律

代码能跑出一张图，离“我确信它算对了”还差一道关键的关。这道关就是：合上电脑，**手算对账**，拿出计算器，亲手算出头两三个时刻的坐标，再和脚本输出的数组逐一对比。这是这门课最硬的 no-AI 纪律，它逼着你真正理解代码每一步在干什么，而不是看一眼曲线形状顺眼就放行。

我们就用脚本里的参数手算。初速度 $v_0 = 20 \text{ m/s}$ ，抛射角 $\theta = 45^\circ$ ，重力加速度 $g = 9.8 \text{ m/s}^2$ ，时间步长 $\Delta t = 0.1 \text{ s}$ 。先算两个速度分量。 45° 的正弦和余弦都等于 $\sqrt{2}/2 \approx 0.7071$ ，所以

$$v_x = v_0 \cos 45^\circ = 20 \times 0.7071 \approx 14.14 \text{ m/s}, \quad v_y = v_0 \sin 45^\circ \approx 14.14 \text{ m/s}.$$

脚本里时刻数组的头几个值是 $t_0 = 0$ 、 $t_1 = 0.1 \text{ s}$ 、 $t_2 = 0.2 \text{ s}$ 。代入解析公式。第零个时刻一切为零， $x_0 = 0$ 、 $y_0 = 0$ ，对应代码里的 `x[0]` 和 `y[0]`。第一个时刻

$$x_1 = 14.14 \times 0.1 \approx 1.41 \text{ m}, \quad y_1 = 14.14 \times 0.1 - \frac{1}{2} \times 9.8 \times 0.1^2 \approx 1.41 - 0.049 \approx 1.37 \text{ m}.$$

第二个时刻

$$x_2 = 14.14 \times 0.2 \approx 2.83 \text{ m}, \quad y_2 = 14.14 \times 0.2 - \frac{1}{2} \times 9.8 \times 0.2^2 \approx 2.83 - 0.196 \approx 2.63 \text{ m}.$$

现在回到脚本，在循环结束后临时加一行 `print(x[1], y[1], x[2], y[2])` 把这四个数打出来，应该读到大约 1.414、1.365、2.828、2.632。和你手算的 1.41、1.37、2.83、2.63 在三位有效数字内一一吻合，对账通过。这一刻你才真正有底气说：我知道这段代码在算什么，它没有在我看不见的地方偷偷算错。如果对不上，那一定是代码和你的理解之间有出入，可能是角度单位、可能是某个系数、也可能是下标错位，无论哪种，都正是手算对账要替你揪出来的东西。

验证清单 · 抛体轨迹脚本

量纲：速度分量 v_x, v_y 为米每秒，坐标 x, y 为米，时间为秒，全部自洽。手算对账：头三个时刻的坐标手算得 (0, 0)、(1.41, 1.37)、(2.83, 2.63)，与脚本数组在三位有效数字内吻合。对称性：无空气阻力时轨迹应是关于最高点左右对称的抛物线，图上确实如此。参考值：45° 抛射的射程应为 $v_0^2/g = 400/9.8 \approx 40.8\text{ m}$ ，最大高度约 10.2 m，与图上读数相符。物理常识：竖直坐标 y 先增后减、落地处 y 回到零附近，符合预期。

本节要点

能用计算器手算前几步并与代码输出在三位有效数字内吻合，才算真正读懂了代码。这道关是 no-AI 的纪律。你需要的只是物理公式和一个计算器。

7.4 改参数重跑：让代码听你的，也看它怎么翻车

读懂之后，下一项能力是改。脚本最上面那几行物理参数就是给你拧的旋钮。把 `angle_deg` 从 45 改成 30，重跑，轨迹变得又低又远，方向的变化一目了然；改成 60，轨迹则又高又陡。把 `v0` 从 20 改成 30，整条抛物线按比例放大，射程和高度都明显增加。每改一个参数，先在心里预判趋势，再跑出来对照，“预判加验证”的循环，是把代码真正变成你手中工具的过程。

改参数还有一种更要紧的用法：故意改错，亲眼看看它怎么翻车，从而建立“代码跑出来不等于物理对”的警觉。我们做一个有教益的破坏实验：把竖直方向公式里重力那一项的负号去掉，也就是把

$$y = v_y t - \frac{1}{2}gt^2 \quad \text{改成} \quad y = v_y t + \frac{1}{2}gt^2.$$

对应代码里把 `y = vy * t - 0.5 * g * t**2` 中的减号换成加号。重跑之后，那条本该先升后落、最终回到地面的优美抛物线，变成了一个一路向上、再也不回头的曲线，小球像是被一只无形的手越推越高。这显然违背物理：抛出去的球必须落回地面。这个翻车的画面之所以值得你亲手制造一次，是因为它把一个抽象的道理钉进了直觉里：程序能顺利跑完、能画出一条平滑曲线，丝毫不能保证它算的是对的物理。负号、单位、系数这些不起眼的细节一旦错了，代码照样若无其事地给你一张漂亮而错误的图，而拦下它的，只能是你心里那条“球必须落地”的物理底线。

本节要点

改参数前先预判趋势，跑出来对照，是让代码变成工具的唯一途径。故意把重力负号写反，亲眼看一条平滑曲线如何骗人，比一万句叮嘱都管用。

7.5 AI 协作实录：让 AI 改脚本，再亲手揪出它的错

现在把这一章的工作方式完整演示一遍。作为项目主理人，你完全可以让 AI 帮你读、帮你改这段脚本，但 AI 给回来的东西必须经过你的手算对账才能放行。下面是一段真实风格的协作记录，AI 在一个初学者和它自己都极易踩的坑上栽了跟头。

AI 协作实录 · 角度单位的度与弧度之混淆

我对 AI 说：

这是我的抛体运动脚本（附上 Listing 7.1）。我觉得先把角度换算成弧度那一步有点多余，能不能帮我简化，直接把抛射角喂给三角函数？请给我改好的代码。

AI 给我：

AI 很爽快地“简化”了代码，把换算那一行删掉，并把速度分解改成了

```
vx = v0 * np.cos(angle_deg)
vy = v0 * np.sin(angle_deg)
```

它还自信地解释说这样“更直接、少一步换算”。改完跑出来的轨迹明显矮了一截，射程也不对，但曲线本身依旧平滑漂亮，不仔细看不容易起疑。

我的审计：

我没有急着接受这个“简化”，而是先做手算对账这道关。我手里早就算过， 45° 抛射的竖直初速度应当是 $v_y = 20 \sin 45^\circ \approx 14.14 \text{ m/s}$ 。可 AI 改完的代码里，`np.sin(45.0)` 把 45 当成了弧度来算。45 弧度是个大得离谱的角（相当于绕了七圈多），它的正弦是 0.851，于是 `vy` 变成了 $20 \times 0.851 \approx 17.0 \text{ m/s}$ ，和我手算的 14.14 对不上。问题立刻定位：`numpy` 的三角函数只认弧度，那一步“多余”的 `np.deg2rad` 恰恰是把人习惯的度数翻译成机器要的弧度的关键，绝不能删。我让 AI 把换算加回去，重跑后 `vy` 回到 14.14，手算对账通过。

这段实录的要害有两层。第一层是 AI 会在角度单位这种地方理直气壮地犯错，还会用“更直接”“更简洁”这类听上去很有道理的说辞替自己的错误背书，毫无心虚。第二层，也是更重要的，揪出它靠的是手里早有一个独立算出的参考值 $v_y \approx 14.14$ ，拿它去卡 AI 的输出，一卡就露馅，不是靠反复读代码。这正是手算对账的全部威力：你不需要比 AI 更会写代码，你只需要有一个它无法糊弄的、由物理算出来的标尺。

AI 陷阱预警 · 三角函数里的角度，到底是度还是弧度

几乎所有编程语言和数值库里的 `sin`、`cos` 都默认输入是弧度，而人想问题时几乎都用度。AI 在生成或修改代码时极易把这两者混为一谈，直接把 30、45、90 这样的度数喂进三角函数，得到一个面目全非却又不报错的结果。这个坑初学者也常踩，而且因为代码照样能跑、图照样平滑，往往要等到对账或对照物理常识时才暴露。养成习惯：凡是看到三角函数，立刻确认喂进去的角是不是弧度，度数必须先过一道 `np.deg2rad` 的关。

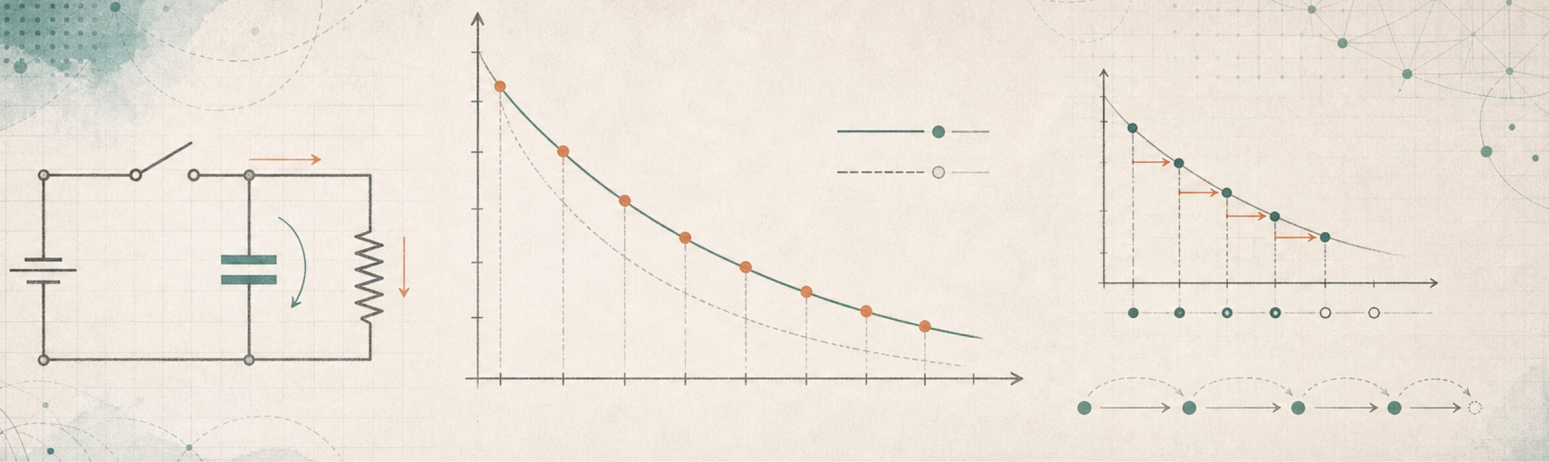
本章小结

这一章没解任何微分方程，借一个你早就会的抛体运动，练了项目主理人最该练的三件事。读：盯一段三十来行、完整可跑的脚本，把变量、函数 `def`、`for` 循环、`numpy` 的 `zeros` 与 `linspace`、`matplotlib` 的 `plot` 这几样最小语言要素逐一弄懂，同时讲清了这一章不碰面向对象、异常处理、广播的取舍。验：合上电脑用计算器手算头三个时刻的坐标，和脚本输出在三位有效数字内对账，证明你读懂了代码在算什么。改：拧动初速度和角度看趋势，故意把重力的负号写反，亲眼看一条平滑却非物理的曲线如何骗过眼睛。AI 协作实录里，AI 把角度的度和弧度搞混，手里那个独立算出的参考值就是揪它出来的标尺。

读代码、手算对账、改参数重跑，这三样能力下一章会立刻派上用场。下一章正式进入 Euler 法，看你练就的逐行读和手算对账的本事怎么跟着循环一步步往前对账。

思考题

1. 本章的脚本用一个 `for` 循环逐个时刻调用 `position` 函数来填数组。如果让 AI 把这个循环改写成 `numpy` 的“整个数组一次算完”的写法（也就是我们推迟到后面才讲的广播），代码会变短，但对一个还没学过广播的读者来说，可读性是更好还是更差？请结合本章“先求会读”的立场，说说你作为项目主理人会不会接受这样的改写，为什么。
2. 我们在破坏实验里把重力项的负号改成了正号，轨迹一路向上不回头。如果换一种错误，把 `vy = v0 * np.sin(angle_rad)` 里的 `sin` 误写成 `cos`，在 45° 这个特殊角下，你用手算对账还能不能发现这个错误？换成 30° 又如何？这说明用单一组参数对账有什么局限？
3. AI 在实录里把度当成弧度，我们靠一个独立算出的参考值 $v_y \approx 14.14$ 把它卡了出来。请你设想：如果你手头连这个参考值都懒得算，只是看了一眼轨迹图觉得“形状还挺像抛物线”就放行，会有什么后果？请从这个例子出发，说说为什么“图看起来对”永远不能替代“手算对得上”。



8. 一阶常微分方程与 Euler 法

物理世界里的很多过程都可以用一句话概括：此刻的变化率，由此刻的状态决定。电容器上的电压掉得有多快，取决于它现在还剩多少电压；一杯热水凉得有多快，取决于它此刻比室温高多少；一个下落的小球加速得有多快，取决于它当前的速度带来多大的空气阻力。一阶常微分方程，就是用来描述这类依赖关系的数学语言。这一章我们只做一件事：把这类问题交给计算机一步一步地往前推，等 AI 替我们写出推进代码之后，再亲手验证它到底对不对。

本章地图

小节	这一节解决什么	关键方法 / 产物
8.1	把 RC 放电、牛顿冷却、受阻落体统一为一个骨架	初值问题 $dy/dt = f(t, y)$
8.2	从导数定义推出 Euler 显式递推公式	$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n)$
8.3	用显式 Euler 解 RC 放电，叠上解析解对账	手算前两步，与 $V[1]$ 、 $V[2]$ 逐一对账
8.4	理解截断误差和步长过大导致的发散	$\Delta t > 2\tau$ 时电压振荡发散； Δt 减半精度翻倍
8.5	AI 把时间常数写成 R/C ，用量纲一眼识破	量纲检查：秒 vs 非秒，一分钟定案
8.6	把零散检查动作收拢成验证清单	五项逐一过关才能采信结果

8.1 从三个物理问题到同一个数学骨架

先看三个表面上毫不相干的普物现象。第一个是 RC 电路放电。一个充好电的电容器通过电阻向外放电, 电容器两端的电压 V 随时间下降。基尔霍夫电压定律告诉我们, 电阻上的电压降 IR 加上电容器电压必须自洽, 而电流又等于电容器电荷的变化率 $I = -C \, dV/dt$, 整理之后得到

$$\frac{dV}{dt} = -\frac{V}{RC},$$

其中 R 是电阻, C 是电容, 乘积 RC 有一个专门的名字叫。第二个现象是牛顿冷却。一杯温度为 T 的热水放在温度为 T_{room} 的房间里, 它降温的速率正比于它和环境的温差, 写出来是 $dT/dt = -k(T - T_{\text{room}})$ 。第三个现象是受线性阻力的自由落体, 速度 v 满足 $m \, dv/dt = mg - bv$, 重力把它往下拉, 正比于速度的阻力把它往回拽。

这三个方程的物理背景天差地别, 数学骨架却完全一样, 都能写成

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0.$$

y 是我们关心的量 (电压、温度或者速度), $f(t, y)$ 是一个已知的函数, 它告诉我们在任意时刻、任意状态下变化率是多少。 y_0 是初始时刻 t_0 的已知状态。给定了 f 和初值 y_0 , 整条曲线 $y(t)$ 原则上就被唯一确定了。物理学家把这种问题叫做, 已知变化规律, 已知出发点, 求整条轨迹。

Definition 8.1.1: 一阶常微分方程的初值问题. 形如 $dy/dt = f(t, y)$, 附带初始条件 $y(t_0) = y_0$ 的问题称为初值问题。函数 $f(t, y)$ 编码了系统的演化规律, 初值 y_0 从无穷多条可能解中选出了唯一的一条。

认出这个共同骨架, 是这一章最重要的一步。一旦你能把手头的物理问题翻译成 $dy/dt = f(t, y)$ 的形式, 剩下的求解工作对计算机来说就是同一套动作, 无论你算的是电路、热水还是落体。

本节要点

认出不同物理问题的共同骨架, 是把物理翻译给计算机的第一步。此后所有一阶问题的求解动作都一模一样。

8.2 Euler 显式法: 把连续演化切成小步

微分方程描述的是连续的变化, 可计算机只会做有限次加减乘除。怎么办? 最朴素的想法来自导数的定义本身。导数 dy/dt 是时间增量趋于零时, y 的增量与时间增量之比的极限。如果我们不取极限, 而是选一个虽小却有限的时间步长 Δt , 那么导数的定义就近似地告诉我们

$$\frac{y(t + \Delta t) - y(t)}{\Delta t} \approx f(t, y(t)).$$

把它整理一下，就得到了从当前状态预测下一步状态的递推公式

$$y(t + \Delta t) \approx y(t) + \Delta t \cdot f(t, y(t)).$$

这就是 Euler 显式法，由瑞士数学家欧拉在 1768 年的著作里系统提出，现代数值分析教材对它的稳定性与误差有系统讨论 [1]。它的物理图像极其直观：站在当前这一点，按当前的斜率（也就是变化率）走一小步直线，到达一个新点；再在新点上重新计算斜率，再走一小步；如此反复，用一段段折线去逼近那条本应光滑的曲线。

Definition 8.2.1 : 显式 Euler 法. 把时间轴离散成等间距的格点 $t_n = t_0 + n \Delta t$ ，记 y_n 为对 $y(t_n)$ 的数值近似。显式 Euler 法的递推公式是

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n).$$

之所以叫“显式”，是因为右边只用到已知的当前值 y_n ，下一步 y_{n+1} 可以直接算出来，不需要解方程。

值得强调的是，Euler 法是这门课里我们唯一会从头讲透、要求你能手算的数值方法。后面遇到的更精巧的方法，比如半隐式 Euler、Runge-Kutta、或者 `scipy` 里的自适应积分器，都会被当成黑箱工具来用，你只需要会调用、能验证。但 Euler 法不一样，它是你理解“代码跑出来的数字到底是怎么来的”的根基，所以我们要亲手把它拆开看。

本节要点

Euler 法的物理图像就是“按当前斜率走一小步”，在新点重新算斜率再走一步，用折线逼近曲线。这是后面所有数值方法理解的地基。

8.3 第一个完整例子：RC 电路放电

我们拿 RC 放电来落地。拿这个问题做第一个例子有一个好处：它有解析解，可以用来给数值解打分。方程 $dV/dt = -V/(RC)$ 配上初值 $V(0) = V_0$ ，解出来是一条标准的指数衰减

$$V(t) = V_0 e^{-t/(RC)}.$$

时间常数 $\tau = RC$ 的物理含义是：每过一个 τ ，电压衰减到原来的 $1/e \approx 0.37$ 。下面这段 Python 代码用显式 Euler 法把这条曲线一步步推出来，并叠上解析解做对照。代码完整可运行，你可以原样拷进编辑器跑。

Listing 8.1: RC 电路放电的显式 Euler 数值解

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 物理参数（一律用 SI 单位）
```

```

5 R = 1.0e3          # 电阻，单位 欧姆
6 C = 1.0e-3         # 电容，单位 法拉
7 V0 = 5.0           # 初始电压，单位 伏特
8 tau = R * C        # 时间常数 tau = R 乘 C，单位 秒
9
10 # 数值参数
11 dt = 0.05          # 时间步长，单位 秒
12 T_total = 5.0      # 总模拟时长，单位 秒
13 N = int(T_total / dt)
14
15 # 初始化时间与电压数组
16 t = np.linspace(0.0, T_total, N + 1)
17 V = np.zeros(N + 1)
18 V[0] = V0
19
20 # 显式 Euler: 站在当前点，按当前斜率走一小步
21 for n in range(N):
22     dVdt = -V[n] / tau          # 当前变化率
23     V[n + 1] = V[n] + dt * dVdt # 推进一步
24
25 # 解析解，用来给数值解打分
26 V_exact = V0 * np.exp(-t / tau)
27
28 plt.plot(t, V, 'o', markersize=3, label='Euler 数值解')
29 plt.plot(t, V_exact, '-', label='解析解')
30 plt.xlabel('时间 t (s)')
31 plt.ylabel('电压 V (V)')
32 plt.legend()
33 plt.tight_layout()
34 plt.show()

```

跑完之后你会看到圆点（数值解）大致贴在实线（解析解）上。但“大致贴上”不等于“对”。作为项目主理人，你不能满足于一张看起来还行的图。第一件该做的事是不开电脑，用计算器手算前几步，再和代码的输出逐一对账。这是这门课反复强调的纪律：你要证明自己不是在给 AI 的输出盖橡皮图章。

以题中的参数， $\tau = RC = 10^3 \times 10^{-3} = 1\text{ s}$ ，步长 $\Delta t = 0.05\text{ s}$ 。第零步 $V_0 = 5\text{ V}$ 。第一步的变化率是 $-V_0/\tau = -5\text{ V/s}$ ，于是 $V_1 = 5 + 0.05 \times (-5) = 4.75\text{ V}$ 。第二步变化率变成 $-4.75/1 = -4.75\text{ V/s}$ ，于是 $V_2 = 4.75 + 0.05 \times (-4.75) = 4.5125\text{ V}$ 。把这两个数和代码里 `V[1]`、`V[2]` 打印出来比，应该在三有效数字内吻合。如果不吻合，一定是代码哪里和你的理解不一致，这正是要揪出来的地方。

本能 · Trust but Verify

看到 AI 或自己写的代码给出一条漂亮的曲线，先停一下，问一句“这条曲线为什么该长这样”。在 RC 放电里，你至少能在六十秒内说出三件该成立的事：电压必须单调下降、必须从 V_0 出发、经过一个时间常数后大约剩 37%。任何一条不满足，结果就不可信，无论图多好看。

本节要点

数值曲线贴解析曲线不等于对账通过。用计算器手算前两步的 V_1 和 V_2 ，和代码输出逐位比对，才是过硬的第一道关。

8.4 截断误差与步长：为什么 dt 不能太大

Euler 法用直线去逼近曲线，每走一步都会偏离真解一点点。这个偏差叫做，根源是我们丢掉了泰勒展开里 Δt 的二次及更高次项。单步误差正比于 Δt^2 ；要走完固定的总时长，步数又正比于 $1/\Delta t$ ，两者相乘，全程累积误差大致正比于 Δt 。这就是为什么把步长减半，整体精度大约能提高一倍。

更危险的在于，步长大到一定程度后解会失真甚至发散，而不是精度变差。在 RC 放电里，如果 Δt 大于 2τ ，Euler 递推会让电压在正负之间来回振荡、越荡越大，给出完全非物理的结果。你可以在上面的代码里把 dt 从 0.05 改成 2.5，亲眼看看那条本该温柔衰减的曲线如何翻车。这个“翻车”现象是显式方法的固有局限，并非 bug。理解它，是你将来判断“该不该信这条曲线”的重要依据。

AI 陷阱预警 · 步长设得过大，曲线翻车却浑然不觉

AI 写积分代码时，步长 dt 往往是它随手填的一个数，它不会替你检查这个步长在你的物理参数下是否稳定。如果你不做收敛性检查，把步长减半再跑一次，看结果变不变，就可能把一条数值发散的假曲线当成物理结论。养成习惯：任何数值积分结果，至少用两个不同步长各跑一次。

本节要点

步长太大时 Euler 法会从物理解变成非物理的数学怪物。把步长减半再跑一次看结果变不变，是检验收敛性的最朴素方法。

8.5 AI 协作实录：让 AI 写放电代码并审计它

现在我们演练一遍这一章的工作方式。作为项目主理人，你不必从零敲出上面那段代码，你可以让 AI 写，但你必须能审。下面是一段真实风格的协作记录。

AI 协作实录 · RC 放电代码里的时间常数错误

我对 AI 说：

你是一名熟悉大一普物和数值方法的物理建模助手。请用显式 Euler 法写一段 Python，模拟 RC 电路放电，电阻 1 千欧、电容 1 毫法、初始电压 5 伏，画出电压随时间的曲线，所有量用 SI 单位，并在注释里标明单位。

AI 给我：

AI 返回了一段结构正确的代码，循环和绘图都没问题，但其中一行把时间常数写成了

```
tau = R / C    # 时间常数
```

跑出来的曲线衰减得快得离谱，几乎一瞬间就掉到零。

我的审计：

我没有去逐行读循环，而是先做量纲检查，这道关六十秒就能过。时间常数必须具有时间的量纲，也就是秒。电阻乘电容 $R \cdot C$ 的量纲是欧姆乘法拉，等于伏每安培乘库每伏，约掉之后是库每安培，也就是安培秒每安培，正好是秒，所以 RC 确实是时间。而 AI 写的 R/C 量纲是欧姆每法拉，化简下来根本不是秒。量纲就对不上，结论立刻清楚：应当是 $\text{tau} = R * C$ 而非 R / C 。改回乘法后重跑，数值曲线和解析解 $V_0 e^{-t/\tau}$ 贴合良好。经过一个时间常数（1 秒），解析值约为 $5e^{-1} \approx 1.84$ 伏，即初值的 37%；显式 Euler 在步长 $\Delta t = 0.05$ 下给出约 1.79 伏，与解析值差几个百分点，这点偏差正是上一节说的截断误差，会随步长减小而消失。量纲修对、数值与解析在可解释的误差内一致，验证通过。

这段实录的要害是：揪出错误靠的是物理学家的本能动作，先看量纲对不对，不是把代码读一百遍。AI 很擅长写出语法正确、看起来专业的代码，但它对物理量纲的把关并不可靠。量纲这件事你不会出错，也最该由你来负责。

8.6 用验证清单确认结果

把上面零散的检查动作收拢成一份清单，就是这门课的核心工件之一。对 RC 放电这个例子，验证可以这样走一遍。

验证清单 · RC 放电数值解

量纲：时间常数 $\tau = RC$ 确认为秒，电压数组单位为伏特，时间数组单位为秒，全部自治。解析极限：当步长趋于零时数值解应收敛到 $V_0 e^{-t/\tau}$ ，把 Δt 减半重跑，曲线确实更贴解析解。单调性：物理上放电电压必须单调下降，数值解在合理步长下逐点递减，符合预期。参数扫描：把 τ 增大一倍，衰减明显变慢，趋势合乎物理。参考值：经过一个时间常数后电压应约为初值的 37%，即解析值 $5 \times 0.368 \approx 1.84 \text{ V}$ ；显式 Euler 在 $\Delta t = 0.05$ 下给出约 1.79 V，与解析值差几个百分点，差异由截断误差解释，符合预期。

本节要点

验证清单把量纲、解析极限、单调性、参数扫描和参考值拢成一份审计底稿，每一条背后都是一个能推翻你结论的物理判据。

五项里能过四到五项，你才有底气说这条曲线可信。注意验证清单不是形式主义的打勾，每一项背后都是一个能推翻结论的物理判据。一个好的项目主理人，宁可在验证上多花十分钟，也不愿意在答辩现场被一句“你这个量纲对吗”问倒。

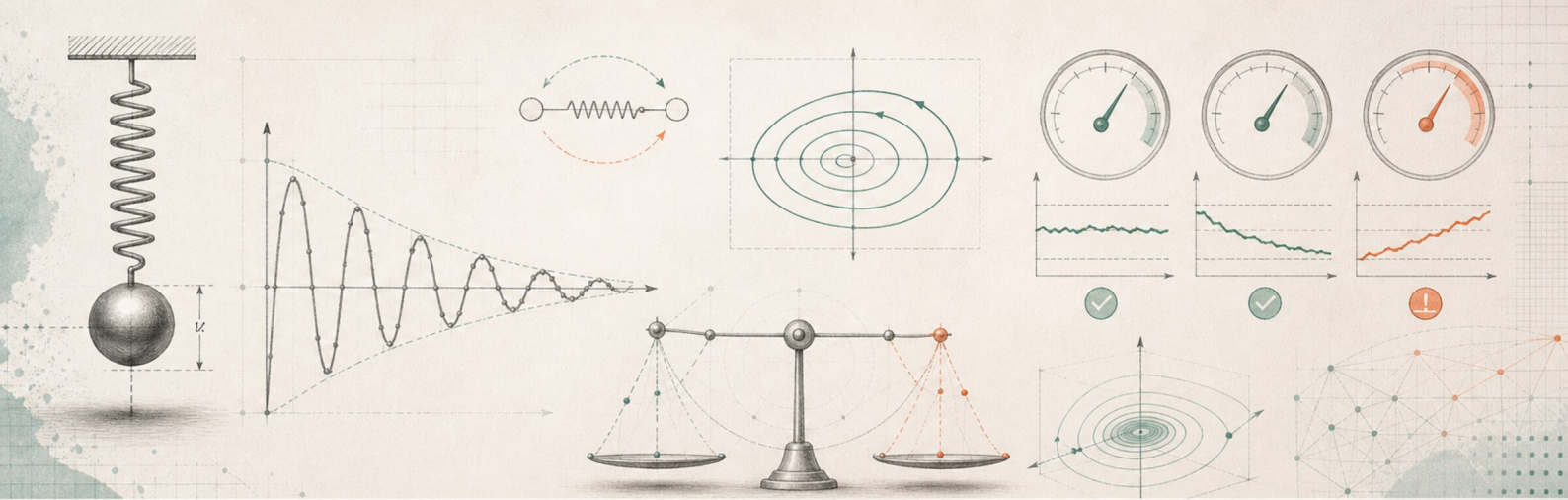
本章小结

这一章认出了 RC 放电、牛顿冷却、受阻落体共享的数学骨架，一阶常微分方程的初值问题 $dy/dt = f(t, y)$ 。从导数的定义推出了显式 Euler 法，理解了它“按当前斜率走一小步”的几何图像，亲手用它解出 RC 放电，再用解析解、手算对账和验证清单确认了结果。步长过大会让显式 Euler 翻车，AI 写的代码可能在时间常数这种地方犯量纲错误，靠量纲检查一分钟就能揪出来。

现实里的振动问题涉及的是二阶导数，位置的变化率是速度，速度的变化率才由力决定。下一章就来把二阶方程拆成两个一阶，并用能量守恒给数值解做一场更严的审计。

思考题

1. 牛顿冷却方程 $dT/dt = -k(T - T_{\text{room}})$ 和 RC 放电方程长得几乎一样，但多了一个常数项 T_{room} 。请把它改写成 $dy/dt = f(t, y)$ 的标准形式，并说明经过很长时间后温度会趋于什么值。你能不改动 Listing 8.1 的循环结构，只改 f 的定义，就把它从放电问题改成冷却问题吗？
2. 我们说显式 Euler 在 $\Delta t > 2\tau$ 时会发散振荡。请你不依赖数值实验，仅从递推公式 $V_{n+1} = V_n(1 - \Delta t/\tau)$ 出发，解释为什么当 $\Delta t/\tau$ 超过某个值时 $|V_n|$ 会越来越大。这个“某个值”具体是多少？
3. AI 在实录里把 $\tau = RC$ 错写成 R/C ，我们用量纲一眼识破。假如 AI 犯的是另一种错误，把负号丢了，写成 $dV/dt = +V/\tau$ ，量纲检查还能发现它吗？这一次该用验证清单里的哪一项来抓它？



9. 二阶常微分方程与守恒律法庭

上一章末尾我们留了一个伏笔：振动问题涉及二阶导数。弹簧上的小球，加速度才是牛顿第二定律要管的量，而加速度是位置对时间的二阶导数。位置的变化率是速度，速度的变化率才由力决定，两层套在一起。这一章做两件事：第一，怎么把二阶方程拆成计算机能推进的形式；第二，怎么在 AI 写出推进代码之后，用一条比”图好不好看”硬得多的物理判据来审它。这条判据就是能量守恒。

本章地图

小节	这一节解决什么	关键方法 / 产物
9.1	从弹簧振子出发，引入阻尼，建立主方程	$\ddot{y} = -\omega^2 y - \gamma \dot{y}$ ，两项物理来源分明
9.2	把二阶方程拆成两个一阶方程	状态向量约化， $[y, v]$ 打包，逐分量推进
9.3	用显式 Euler 逐分量推进阻尼振子	递推公式，位置-速度-能量三联图
9.4	用能量守恒这条硬判据审计数值解	守恒律法庭：无阻尼水平，有阻尼单调降
9.5	显式 Euler 自身固有的能量漂移	能量非物理爬升，方法缺陷，不是代码写错
9.6	引入半隐式 Euler，一行止住漂移	黑箱升级，知道存在和替换时机即可
9.7	AI 状态向量配反，能量三联图当庭抓获	守恒律审计的实战演示

9.1 从弹簧振子到阻尼振子

先把物理立起来。一个质量为 m 的小球挂在劲度系数为 k 的弹簧上，偏离平衡位置的位移记为 y 。弹簧的回复力按胡克定律是 $-ky$ ，负号表示力总是把小球往平衡位置拉。只有这一个力时，牛顿第二定律给出 $m\ddot{y} = -ky$ ，其中 $\ddot{y}$ 是位移对时间的二阶导数，即加速度。两边除以 m ，并定义 $\omega^2 = k/m$ ，方程就变成了干净的

$$\ddot{y} = -\omega^2 y.$$

这里的 ω 叫做，单位是弧度每秒。它完全由系统自身的属性，弹簧硬不硬、球重不重，决定，与你把球拉多远再松手无关。 ω 越大，振动越快。这就是简谐振子，它的解是大家熟悉的正弦余弦振荡，小球永远不停地来回摆动，振幅一丝不减。

可现实里没有永动的振子。空气阻力、弹簧内部的摩擦，都会持续地从系统里抽走能量，让振幅一点点缩小，最终停下。最简单的阻尼模型假设阻力正比于速度、方向与速度相反，写成 $-c\dot{y}$ ，其中 $\dot{y} = dy/dt$ 是速度。把这一项加进牛顿第二定律， $m\ddot{y} = -ky - c\dot{y}$ ，同样除以 m 并记 $\gamma = c/m$ ，得到这一章要反复打交道的主方程

$$\ddot{y} = -\omega^2 y - \gamma \dot{y}.$$

γ 叫做阻尼系数，单位是每秒，它衡量能量被耗散得有多快。 $\gamma = 0$ 时退回无阻尼的简谐振子； γ 越大，振幅衰减得越急。右边两项的物理含义要分得很清楚： $-\omega^2 y$ 是弹簧的回复力（除以质量之后），它只跟你现在站在哪里有关； $-\gamma \dot{y}$ 是阻力（除以质量之后），它只跟你现在动得多快有关。一个管“位置”，一个管“速度”，这个区分马上就会变得很重要。

本节要点

位置和速度是两个独立演化的量，弹簧振子把回复力藏在 ω 里，阻尼振子把耗散藏在 γ 里。理解了这两项的物理来源，后面的代码和审计就都有了方向。

9.2 状态向量约化：把一个二阶拆成两个一阶

上一章的 Euler 法只会推进 $dy/dt = f(t, y)$ 这种一阶方程，它的输入是当前一个数 y ，输出是下一步的一个数。可现在我们的方程里有二阶导数 $\ddot{y}$ ，Euler 法直接套不上去。怎么办？关键的一招是：不要只盯着位置 y 一个量，把速度也提拔成一个独立的、需要同时往前推的量。

具体地，令速度 $v = \dot{y}$ 。这一步本身就是一个一阶方程：位置的变化率等于速度， $\dot{y} = v$ 。接着看速度自己的变化率。由主方程 $\ddot{y} = -\omega^2 y - \gamma \dot{y}$ ，而 $\ddot{y}$ 正是 $\dot{v}$ ，所以 $\dot{v} = -\omega^2 y - \gamma v$ 。我们一口气得到了两个一阶方程，把它们摆在一起就是

$$\dot{y} = v, \quad \dot{v} = -\omega^2 y - \gamma v.$$

原来纠缠在一起的一个二阶方程，现在变成了两个互相耦合的一阶方程：第一个告诉你位置怎么随速度变，第二个告诉你速度怎么随位置和速度自己变。如果我们把位置和速

度打包成一对数 $[y, v]$ ，这一对数在任意时刻就完整地描述了振子的全部状态。已知它，下一瞬间往哪走是唯一确定的。我们把这一对数叫做。

状态向量

Definition 9.2.1 : 状态向量约化. 一个含 n 阶导数的常微分方程，可以通过引入低阶导数作为新的未知量，化成 n 个一阶方程的耦合组。对二阶方程，引入速度 $v = \dot{y}$ ，把状态打包成向量 $[y, v]$ ，原方程等价于

$$\frac{d}{dt} \begin{bmatrix} y \\ v \end{bmatrix} = \begin{bmatrix} v \\ -\omega^2 y - \gamma v \end{bmatrix}.$$

向量右边的函数就是上一章 $f(t, y)$ 的多分量版本：它接收当前状态向量，吐出每个分量的变化率。这样一来，任何能推进一阶方程的方法（比如 Euler 法），逐分量施加，就能推进任意阶的方程。

这个约化是本章的技术核心，也是计算物理里一个普适到惊人的套路：无论是行星绕日（六个分量：三个坐标加三个速度）还是电路里的电流电压耦合，都先化成“一个状态向量的一阶演化”，再交给同一套推进器。认出并写对这个状态向量，是后面所有代码正确的前提。一旦它的两个分量配错了位置，整个物理就反了，这恰恰是 AI 最容易出错、也最该由你把关的地方。

■ **Example 9.1 : 纸笔练习：把小角度单摆化成两个一阶方程.** 小角度下的单摆满足 $\ddot{\theta} = -(g/L)\theta$ ，其中 θ 是摆角， g 是重力加速度， L 是摆长。请把它写成状态向量形式。解法和弹簧振子一模一样：令角速度 $\Omega = \dot{\theta}$ ，则第一个方程是 $\dot{\theta} = \Omega$ ；而 $\ddot{\theta} = \dot{\Omega} = -(g/L)\theta$ ，给出第二个方程 $\dot{\Omega} = -(g/L)\theta$ 。打包成状态向量 $[\theta, \Omega]$ ，演化为

$$\dot{\theta} = \Omega, \quad \dot{\Omega} = -\frac{g}{L}\theta.$$

对照弹簧振子可以一眼看出，单摆就是把 ω^2 换成 g/L 、把阻尼 γ 设为零的同一个数学骨架。固有角频率 $\omega = \sqrt{g/L}$ ，这正是中学就背过的单摆周期 $T = 2\pi\sqrt{L/g}$ 的来源。 ■

本节要点

把一个二阶方程拆成两个一阶方程，不是数学花招，而是给计算机的一纸通行证，从此任何能推进一阶方法的方法，都能拿来推进振动。

9.3 用显式 Euler 推进阻尼振子

有了状态向量，把上一章的 Euler 骨架扩展过来就水到渠成。上一章我们对单个量 y 写 $y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n)$ ；现在我们对两个量 y 和 v 各写一行，每一行都用各自的变化率推进同一个步长

$$y_{n+1} = y_n + \Delta t \cdot v_n, \quad v_{n+1} = v_n + \Delta t \cdot (-\omega^2 y_n - \gamma v_n).$$

注意右边两行用的全是第 n 步的旧值 y_n 和 v_n ，这正是“显式”的含义，下一步可以

直接算出，不需要解方程。下面这段完整可跑的代码用显式 Euler 推进阻尼振子，并把位置、速度、能量三条曲线都记录下来，为下一节的庭审做准备。

Listing 9.1: 阻尼振子的显式 Euler 数值解

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 物理参数 (一律用 SI 单位)
5 omega = 2.0      # 固有角频率, 单位 弧度每秒
6 gamma = 0.3      # 阻尼系数, 单位 每秒 (设为 0.0 即无阻尼简谐振子)
7 y0 = 1.0         # 初始位移, 单位 米
8 v0 = 0.0         # 初始速度, 单位 米每秒 (从静止松手)
9
10 # 数值参数
11 dt = 0.01        # 时间步长, 单位 秒
12 T_total = 20.0   # 总模拟时长, 单位 秒
13 N = int(T_total / dt)
14
15 # 初始化时间, 位置, 速度数组
16 t = np.linspace(0.0, T_total, N + 1)
17 y = np.zeros(N + 1)
18 v = np.zeros(N + 1)
19 y[0] = y0
20 v[0] = v0
21
22 # 显式 Euler: 对状态向量 [y, v] 的两个分量各推进一步
23 for n in range(N):
24     dydt = v[n]                                     # 位置的变化率就是速度
25     dvdt = -omega**2 * y[n] - gamma*v[n]            #
26     # 速度的变化率由回复力加阻力决定
27     y[n + 1] = y[n] + dt * dydt                    # 用旧值推进位置
28     v[n + 1] = v[n] + dt * dvdt                    # 用旧值推进速度
29
30 # 机械能 (单位质量): 动能加势能
31 E = 0.5 * v**2 + 0.5 * omega**2 * y**2
32
33 # 三联图: 位置, 速度, 能量
34 fig, ax = plt.subplots(3, 1, figsize=(6, 7), sharex=True)
35 ax[0].plot(t, y); ax[0].set_ylabel('位置 y (m)')
36 ax[1].plot(t, v); ax[1].set_ylabel('速度 v (m/s)')
37 ax[2].plot(t, E); ax[2].set_ylabel('能量 E (J/kg)')
38 ax[2].set_xlabel('时间 t (s)')
39 plt.tight_layout()

```

```
plt.show()
```

跑出来你会看到位置和速度都在振荡，而且因为有阻尼，振幅一圈圈缩小。看起来合理。但和上一章一样，“看起来合理”不等于“对”。这一章我们手里多了一件上一章没有的审计武器，威力大得多，就藏在第三条曲线里。

本节要点

循环里两条更新语句分别推进位置和速度，用的是同一个步长和同一个时刻的旧值，这就是显式 Euler 的全部规矩。

9.4 用能量审计数值解

物理学里有一类陈述，比“曲线该单调下降”强硬得多，。对无阻尼的简谐振子 ($\gamma = 0$)，**守恒律** 系统不和外界交换能量，机械能必须严格守恒。单位质量的机械能是动能加势能

$$E = \frac{1}{2}v^2 + \frac{1}{2}\omega^2 y^2.$$

这是一个对真解恒为常数的量。小球摆到最高点时速度为零，能量全是势能；摆过平衡位置时位移为零，能量全是动能；可两者之和，在理想无阻尼情形下，分毫不该变。这给了我们一把极其锋利的尺子：把代码里 `gamma` 设成 0 跑一遍，画出能量曲线。它若不是一条水平直线，数值解就有罪，无论位置曲线振荡得多么赏心悦目。

有阻尼时 ($\gamma > 0$)，物理同样给出明确的预言，只是从“守恒”变成“单调耗散”。阻力做负功，持续从系统抽走能量，所以能量必须单调下降，一刻不回升，最终趋于零（小球停在平衡位置）。于是无论有没有阻尼，能量曲线该长什么样，物理都说得死死的：要么水平，要么单调下降。任何明显与此相悖的能量行为，比如能量持续往上爬甚至发散，都说明数值结果不可信。至于元凶，可能是代码或符号写错了，也可能是积分方法本身的人为效应：下一节我们就会看到，哪怕代码完全正确，显式 Euler 也会让能量缓慢漂移。所以能量异常是数值解有问题的铁证，但问题出在代码还是方法，要进一步追查。这就是我们说的守恒律法庭：能量是公诉人，物理预言是法条，数值出来的能量曲线是被告。

验证清单 · 无阻尼简谐振子数值解

量纲：固有角频率 ω 确认为弧度每秒，位移为米，速度为米每秒，能量 $\frac{1}{2}v^2 + \frac{1}{2}\omega^2 y^2$ 为焦每千克，全部自洽。守恒律：把 γ 设为零，机械能必须是一条水平直线；若能量随时间漂移，立即判定数值解不可信。振幅：无阻尼下位置振荡的峰值应保持在初始位移 y_0 附近不衰减。周期：振荡周期应约为 $2\pi/\omega$ ，本例 $\omega = 2$ 给出周期约 3.14 秒，可从位置曲线上直接量。能量分配：在位置峰值处速度应过零（能量全为势能），在过平衡点处速度达极值（能量全为动能），两条曲线的相位关系应对得上。

这份清单里最硬的一条就是守恒律。它的厉害之处在于，它不需要解析解，也不需要参考文献，纯粹靠系统自身的物理对称性就能给数值解打分。哪怕你面对一个根本没有解析解的复杂系统，只要它在物理上能量守恒，这条尺子就照样能用。

本能 · Trust but Verify

守恒律是审计数值解最强力的判据之一，它是物理给出的硬约束，不依赖任何参考解。看到一段振动模拟的代码跑出漂亮曲线，先别急着信，把能量画出来：无阻尼时它该水平，有阻尼时它该单调降。能量曲线一旦违背这个预言，无论位置图多好看，数值解都已经被当庭定罪。守恒量是检验数值解的免费、随身、不可伪造的法官。

本节要点

能量守恒是免费的审计官。无阻尼时它该是一条水平线，有阻尼时它该是一条往下掉的线，任何偏离都是数值解在撒谎的铁证。不看能量只看位置曲线的人，漏掉了这道最硬的关。

9.5 显式 Euler 的能量漂移：代码没错，能量却在涨

现在把代码里的 `gamma` 设成 0，认认真真跑一遍无阻尼简谐振子，再盯着能量曲线看。你会出了一件出乎意料的事：能量并不是一条水平直线，而是在缓慢地、稳定地往上爬。明明没有阻尼、没有外力注入，能量却凭空增长，这显然是非物理的。可代码逻辑、状态向量、方程符号又全都没错。怎么回事？

问题出在显式 Euler 这个方法本身，而不是我们的实现。回忆它的几何图像：站在当前点，按当前斜率走一小段直线。对一个本该沿椭圆轨道（在位置速度平面上）周而复始的振子来说，每一步都沿切线走直线，而切线总是略微偏到椭圆的外侧，于是每走一步，状态点都被甩得离原点稍远一点点。一圈下来，本该回到出发点的轨道悄悄张大了一圈，对应到能量上，就是那条缓慢上爬的曲线。这个偏差和上一章讲的截断误差同源，都来自用直线代替曲线丢掉的高阶项。只不过在振动问题里，它积累成了一个有方向、不抵消的系统性效应：能量只增不减。步长 Δt 越小，每步偏出去得越少，能量爬升越慢，但只要步长不为零，这个非物理的增长就永远抹不掉。

这件事恰好说明能量审计的价值：代码逻辑没错，但方法本身的缺陷被能量曲线照出来了。只看位置曲线的人，跑几个周期觉得“挺像那么回事”就交差了；看能量的人立刻发现曲线在物理上站不住脚，而且能说清楚问题不在代码，在方法。

AI 陷阱预警 · 显式方法的能量漂移被当成物理结论

当你让 AI 用显式 Euler 模拟一个保守振动系统并长时间运行，能量的缓慢漂移几乎是必然出现的，但 AI 通常不会主动提醒你这是方法的人为效应，更不会替你画能量曲线去检查。如果你只看位置图，很容易把一条逐渐张大的非物理轨道当成真实物理。养成习惯：凡是保守系统的长时间积分，必画守恒量；守恒量明显漂移时，先怀疑积分方法，而不是急着调物理参数。

本节要点

显式 Euler 在保守系统里会让能量单方向漂移，这是方法本身的缺陷，不是代码写错了。知道这一点，你才不会为一条平滑振荡却暗地里在膨胀的轨迹付出信任。

9.6 半隐式 Euler：一行改动止住漂移（黑箱披露）

能量漂移有救吗？有，而且救法简单到只改一行。显式 Euler 推进速度和位置时，用的都是同一步的旧值 v_n 和 y_n 。我们做一个微小的调整：先用旧位置把速度更新到 v_{n+1} ，然后在更新位置时，不用旧速度 v_n ，而是用刚刚算出的新速度 v_{n+1} 。也就是把递推改成

$$v_{n+1} = v_n + \Delta t \cdot (-\omega^2 y_n - \gamma v_n), \quad y_{n+1} = y_n + \Delta t \cdot v_{n+1}.$$

对照原来的显式 Euler，唯一的差别是第二行里把 v_n 换成了 v_{n+1} 。反映到代码上，就是把位置更新那一行的 `v[n]` 改成 `v[n+1]`，并把速度更新挪到位置更新前面去算。仅此一行半的改动，无阻尼时的能量曲线就不再单调上爬，而是围绕真实值做微小的来回波动，长时间跑也不发散，振子能稳稳当当转上成千上万圈。

这个方法叫半隐式 Euler（也叫辛欧拉法）。它止住漂移的道理属于辛积分的范畴，简单说就是这类方法能保持系统的几何结构，让能量长期有界。行星运动的长期模拟正是靠这类方法才做得动。半隐式 Euler 在本课中是一个黑箱升级，只需知道它存在、知道它在保守系统的长期积分上明显优于显式 Euler、知道改动只在那一行。它的原理不要求理解。之所以提它，是因为显式 Euler 的能量漂移碍事时，有几乎零成本的替代品可以换上。知道工具箱里有这把扳手，比会自己造扳手更要紧。

本节要点

把位置更新里的旧速度换成新速度，只改一行，能量漂移就止住了。理解原理不是必需，但知道工具箱里躺着这件便宜又好用的替换品，是你面临长期积分时的底气。

9.7 AI 协作实录：状态向量配错，能量三联图当庭抓获

现在我们演练一遍这一章的工作方式。我们让 AI 写阻尼振子的推进代码，然后看能量法庭怎么在我们没逐行读代码的情况下，把一个隐蔽的错误揪出来。

AI 协作实录 · 状态向量两个分量配反**我对 AI 说：**

你是一名熟悉大一普物和数值方法的物理建模助手。请用显式 Euler 法写一段 Python，模拟阻尼振子 $\ddot{y} = -\omega^2 y - \gamma \dot{y}$ ，固有角频率 $\omega = 2$ 、阻尼系数 $\gamma = 0.3$ 、初位移 1 米、初速度为零。请把位置、速度、能量三条曲线都画出来，所有量用 SI 单位。

AI 给我：

AI 返回了一段结构看上去很对的代码，循环、绘图都齐全，但在推进那两行里，它

把状态向量的两个分量配反了，写成

$$\begin{aligned}y[n+1] &= y[n] + dt * (-\omega^2 * y[n] - \gamma * v[n]) \\v[n+1] &= v[n] + dt * v[n]\end{aligned}$$

把“位置的变化率是速度”和“速度的变化率是回复力加阻力”这两件事张冠李戴了：位置被当成由力来推，速度反而被当成由它自己来推。跑出来位置曲线还真在动，乍一看也像那么回事。

我的审计：

我没有去逐行盯那两行下标，而是直接看能量三联图，这一章最省力的一道关。物理预言很硬：有阻尼时机械能必须单调下降。可图上的能量曲线根本不单调，先是乱涨，整条轨迹随时间迅速放大、直奔发散。一个有阻尼、本该越摆越小最后停下的振子，能量怎么可能越来越大？这一条就足以当庭定罪：数值解非物理，问题十有八九出在推进逻辑。带着这个判断回去看那两行，立刻发现 y 和 v 的右端配反了。改回 $dydt = v[n]$ 、 $dvdt = -\omega^2 * y[n] - \gamma * v[n]$ （也就是 Listing 9.1 里的写法）重跑，能量曲线立刻从发散转为一条持续衰减到零的漂亮曲线（显式 Euler 自身那点微小的能量漂移，在这个整体衰减的尺度上根本看不出来），验证通过。

这段实录的要害在于：抓住错误靠的不是把那两行下标读上十遍，而是一个守恒律的物理本能，能量该单调降却在涨，立刻判定有罪，再回头精确定位。AI 很擅长写出语法无误、结构完整、跑得动的代码，但它对“这个状态向量在物理上配对得对不对”并不可靠。如果 AI 犯的是另一种错，比如把阻尼项符号写成 $+\gamma v$ （等于给系统倒着注入能量），能量曲线同样会当场报警：有阻尼系统的能量不降反升，一眼就能判定不对。守恒律这把尺子对一整类符号、配对错误都管用。

本章小结

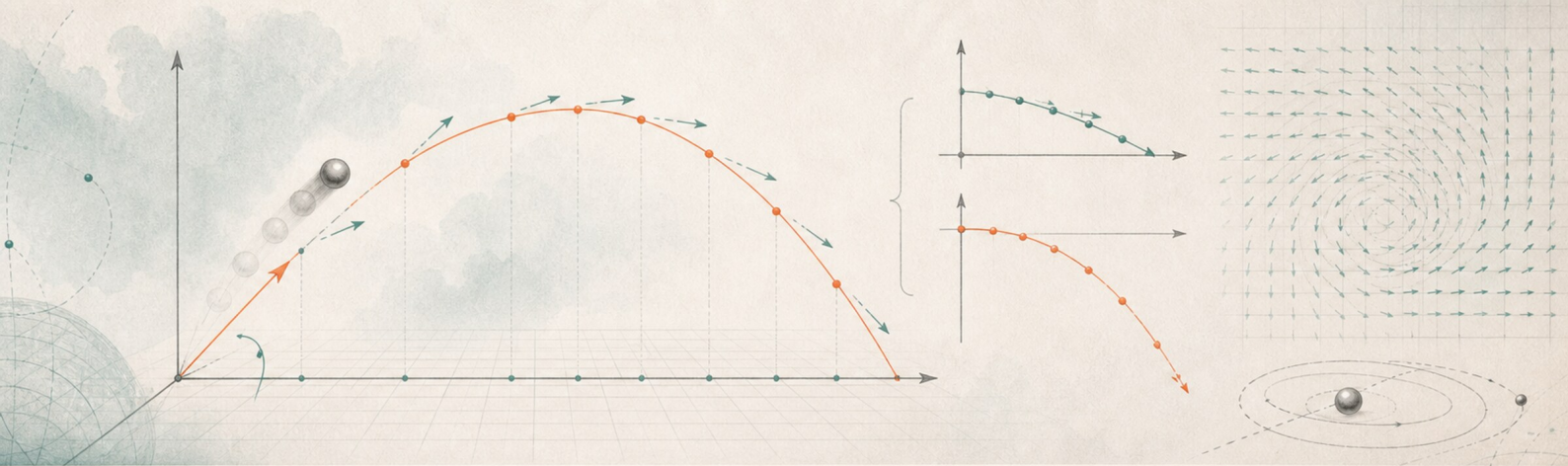
从弹簧振子和阻尼振子出发，认清了主方程 $\ddot{y} = -\omega^2 y - \gamma \dot{y}$ 里每个符号的物理含义： ω 是系统自身的固有角频率， γ 是能量耗散快慢的阻尼系数。状态向量约化是这一章的技术核心：把二阶方程拆成 $[y, v]$ 这对量的两个耦合一阶方程，直接套用上一章的 Euler 骨架逐分量推进。用守恒律开庭：无阻尼时机械能该水平守恒，有阻尼时该单调耗散；能量曲线一旦违背预言，数值解就当庭定罪。靠这把尺子，既看清了显式 Euler 自身固有的非物理能量漂移（半隐式 Euler 一行就能止住），也在 AI 把状态向量配反时一眼抓获。

现实里的运动往往发生在平面甚至空间中，位置和速度各有两到三个分量。下一章把状态向量扩展到二维，看同一套约化加推进加守恒律审计的方法如何搬到平面运动上。

思考题

1. 把 Listing 9.1 里的 γ 设为 0，先用现成的显式 Euler 跑一遍，记录能量在 20 秒末漂移了百分之几；再把步长 dt 减半重跑，看漂移量大约变成原来的几分之几。你能据此说出能量漂移量和步长大致是什么关系吗？这和上一章讲的“累积误差正比于 Δt ”对得上吗？

2. 本章给出的机械能是 $E = \frac{1}{2}v^2 + \frac{1}{2}\omega^2 y^2$ (单位质量)。请你不依赖代码，仅对无阻尼方程 $\dot{y} = v$ 、 $\dot{v} = -\omega^2 y$ ，把 E 对时间求一次导数，证明 $dE/dt = 0$ ，即真解的能量严格守恒。再对有阻尼情形 $\dot{v} = -\omega^2 y - \gamma v$ 重做一遍，说明 dE/dt 的符号为什么保证能量单调下降。
3. 实录里 AI 把状态向量两个分量配反，我们靠“能量该降却在涨”识破。假如 AI 这次犯的是一个更隐蔽的错，步长 dt 取得偏大但还没大到立刻发散，位置曲线看起来仍在正常衰减振荡。这种情况下，验证清单里哪一项最可能先报警？你会怎样设计一次检查来确认问题出在步长而不是物理？



10. 升维：二维就是两个一维 ODE

上一章我们把一维的演化问题交给了显式 Euler 法，电压、温度、速度，无论物理背景是什么，骨架都是 $dy/dt = f(t, y)$ 。可现实里的运动很少只沿一条直线发生。一颗炮弹既往前飞也往下落，一颗行星绕着太阳画椭圆，它们都活在平面上，至少有两个坐标在同时变化。这一章我们要回答的问题听起来吓人，做起来却出奇地简单：会解一维，怎么解二维？答案只有一句话，二维不过是两条一维并排放在一起。理解这一点，你手里那套 Euler 法就能直接用，一行新数学都不必学。

本章地图

小节	这一节解决什么	关键方法 / 产物
10.1	把二维运动拆成水平和竖直两条一维 ODE	$[x, v_x]$ 与 $[y, v_y]$ 并列推进，不引入新方法
10.2	建立带线性阻力的二维抛体方程	$m\dot{v}_x = -bv_x, m\dot{v}_y = -mg - bv_y$ ，两套解耦
10.3	显式 Euler 并列推进四个量	水平竖直分行写，代码结构与物理结构一一对应
10.4	用可解极限审计数值解：无阻力和大阻力	无阻力退抛物线，大阻力出终端速度
10.5	一分钟量纲验终端速度 $v_t = mg/b$	量纲约成米每秒，才敢写进验证清单
10.6	AI 把 v_t 写成 $mg \cdot b$ ，量纲识破	六十秒定案，代码不用读一百遍
10.7	行星轨道的耦合方程与守恒量审计	总能量与角动量，比“轨迹像椭圆”更硬

10.1 核心思想：二维就是两个一维并列

设想平面上一个运动的质点，它的位置由两个坐标 x 和 y 给出，速度也相应地有两个分量 v_x 和 v_y 。牛顿第二定律 $\mathbf{F} = m\mathbf{a}$ 是一个矢量方程，而矢量方程的真正含义是它在每个坐标方向上各成立一次。换句话说，水平方向的合力只决定水平方向的加速度，竖直方向的合力只决定竖直方向的加速度。一个二维运动就这样被拆成了水平和竖直两套各自独立的演化规律。

把它写全。位置的变化率是速度，在两个方向上分别给出 $\dot{x} = v_x$ 和 $\dot{y} = v_y$ ；速度的变化率是加速度，由该方向的合力除以质量给出。这样我们手里就有了四个一阶常微分方程，描述四个随时间演化的量 x, y, v_x, v_y 。它们不是杂乱无章的四个，成对分成了两组： (x, v_x) 管水平， (y, v_y) 管竖直。如果水平方向的力不依赖于竖直方向的状态，反之亦然，那么这两组就完全互不干涉，是两条平行铺开、各推各的一维 ODE。

平面上质点的运动由四个随时间变化的量描述：两个坐标 x, y 和两个速度分量 v_x, v_y 。它们各自满足一条一阶方程

$$\dot{x} = v_x, \quad \dot{v}_x = \frac{F_x}{m}, \quad \dot{y} = v_y, \quad \dot{v}_y = \frac{F_y}{m}.$$

推进时把上一章的 Euler 一步对这四个量同时各做一遍即可。当 F_x 只依赖水平状态、 F_y 只依赖竖直状态时，两套方程互不耦合，是两条并列的一维 ODE。

这里我要特别提醒一句写代码时的取舍。你以后会在很多教程里看到一种写法：把 (x, y, v_x, v_y) 打包成一个数组，用向量化、广播一口气推进。那样写当然简洁，但对刚上手的人是一层不必要的抽象，它会让你看不清“水平归水平、竖直归竖直”这个物理事实。本章我们偏偏要老老实实，把 x 那一套和 y 那一套分开写成两组并列的更新语句，让代码

的结构和物理的结构一一对应。等你真正吃透了“二维等于两个一维”，再去用打包写法图省事也不迟。

本节要点

二维运动不带来任何新数学，只是把一维 Euler 循环体里的更新语句原地复制一份，水平的归水平，竖直的归竖直，各推各的。代码保持分行并列，是为了让结构和物理一一对应。

10.2 主例：带空气阻力的抛体

最经典也最适合落地的二维问题，是带空气阻力的抛体。先把没有阻力的理想情形回忆一下：水平方向不受力，做匀速直线运动；竖直方向只受重力，做匀加速运动。这正是中学就熟悉的抛物线。现在我们加一点真实，让空气来掺和。

空气阻力的方向总是和速度相反，它在每个时刻都在拖慢质点。最简单的一种模型是线性阻力，阻力大小正比于速率，写成矢量就是 $\mathbf{F}_{\text{drag}} = -b\mathbf{v}$ ，其中 b 是一个正的阻力系数。负号表示它和速度反向。把这个力投影到两个坐标方向，水平方向只有阻力，竖直方向是重力加阻力，牛顿第二定律给出

$$m\dot{v}_x = -bv_x, \quad m\dot{v}_y = -mg - bv_y,$$

再配上位置和速度的关系

$$\dot{x} = v_x, \quad \dot{y} = v_y.$$

逐个符号说清楚。 m 是质点质量，单位千克； g 是重力加速度，约 9.8 米每二次方秒。竖直方程里 $-mg$ 这一项就是重力，恒向下，所以带负号（我们约定 y 轴向上为正）。 b 是线性阻力系数，它的单位必须让 bv 成为一个力，也就是牛顿，所以 b 的单位是千克每秒，这一点稍后做量纲检查时还要用到。 $-bv_x$ 和 $-bv_y$ 分别是阻力在两个方向的分量，方向都和对应的速度分量相反，起到拖慢的作用。注意水平方程里只有阻力没有重力，竖直方程里两者都有，二维问题“两套规律各管一摊”的具体体现正在这里。

线性阻力之外，还有一种常用的二次阻力模型，阻力大小正比于速率的平方，写成 $\mathbf{F}_{\text{drag}} = -c|\mathbf{v}|\mathbf{v}$ 。它更贴近高速飞行物体的真实受力，但代价是水平和竖直两个方向被速率 $|\mathbf{v}| = \sqrt{v_x^2 + v_y^2}$ 耦合到了一起，不再是两条干净的并列方程。为了把“二维等于两个一维”讲透，本章主线一律用线性阻力，二次阻力只作为知道有这回事的旁注，留到思考题里让你体会耦合带来的区别。

本节要点

线性阻力让两套方程解耦，水平纯衰减，竖直重力加阻力的竞争，这是刻意选的简化。解耦让你把两个方向的故事分开讲清楚，也为后面的两个极限验证铺平了路。

线性阻力的好处在于它让两套方程依然解耦：水平方向 $m\dot{v}_x = -bv_x$ 完全是上一章 RC 放电的同一个骨架，速度按指数衰减；竖直方向 $m\dot{v}_y = -mg - bv_y$ 则是上一章受阻落体的样子，速度会趋向一个终端值。两条我们都见过，现在只是让它们同台演出。

10.3 用显式 Euler 解二维抛体

把上面四个方程交给显式 Euler 法。显式 Euler 的规矩是这一步的所有更新都只用当前时刻（旧）的值：每一步先用当前速度算出两个方向的加速度，再用当前速度更新位置，最后用刚算的加速度更新速度。关键是位置要在速度被改写之前、用这一步的旧速度推进，这样才是地道的显式 Euler，而不是上一章那个先更新速度再用新速度推位置的半隐式版本。下面这段代码完整可运行，可以原样拷进编辑器。注意它刻意把水平的一套和竖直的一套分行并列写出来，让你一眼看见两条一维 ODE 是怎样并肩前进的。

Listing 10.1: 带线性空气阻力的二维抛体，显式 Euler 解

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 物理参数（一律用 SI 单位）
5 m = 0.1          # 质量，单位 千克
6 g = 9.8          # 重力加速度，单位 米每二次方秒
7 b = 0.0          # 线性阻力系数，单位 千克每秒（先取 0 表示无阻力）
8 v0 = 20.0        # 初速大小，单位 米每秒
9 angle = np.deg2rad(45.0) # 发射仰角，转成弧度
10
11 # 数值参数
12 dt = 0.001       # 时间步长，单位 秒
13 N = 20000        # 最大步数（落地即停，不一定走满）
14
15 # 初始状态：水平一套 (x, vx)，竖直一套 (y, vy)
16 x, y = 0.0, 0.0
17 vx = v0 * np.cos(angle)
18 vy = v0 * np.sin(angle)
19
20 # 用列表记录轨迹，方便落地后截断
21 xs, ys = [x], [y]
22
23 # 显式 Euler：这一步所有更新都用当前（旧）时刻的值，水平与竖直并列
24 for n in range(N):
25     # 先用当前（旧）速度算加速度（两套各算各的）
26     ax = -b / m * vx          # 水平：只有阻力
27     ay = -g - b / m * vy      # 竖直：重力加阻力
28     # 再用当前（旧）速度更新位置（注意要在速度被改写之前）

```

```

29     x = x + dt * vx
30     y = y + dt * vy
31     # 最后用上面算的加速度更新速度
32     vx = vx + dt * ax
33     vy = vy + dt * ay
34     xs.append(x)
35     ys.append(y)
36     if y < 0.0:                                # 落回地面就停
37         break
38
39 xs = np.array(xs)
40 ys = np.array(ys)
41
42 plt.plot(xs, ys, '-', label='Euler 数值轨迹')
43 plt.xlabel('水平位置 x (m)')
44 plt.ylabel('竖直位置 y (m)')
45 plt.axhline(0.0, color='gray', linewidth=0.6)
46 plt.legend()
47 plt.tight_layout()
48 plt.show()

```

代码的骨架和上一章的 RC 放电几乎一模一样，差别只在于现在循环体里要更新四个量而不是一个，并且分成了水平和竖直两组。这正是本章要传达的全部技术内容：升维没有引入任何新的数值方法，它只是把同一个 Euler 步原地复制了几份。把 b 设成 0 跑一次，你会看到一条对称的抛物线；把 b 调大再跑，轨迹就会变得前倾、缩短。但“看起来对”从来不是项目主理人验收的标准，下一节我们要动真格地审它。

本节要点

更新顺序有讲究：先算加速度，再用旧速度推位置，最后才更新速度。这是显式 Euler 的本分，和上一章半隐式先更新速度再推位置的习惯不同。

10.4 极限验证：本章的灵魂

这一章最重要的纪律，是给二维数值解找两个能算得清清楚楚的极限，让代码在极限处和已知答案对账。带阻力的抛体一般没有简洁的解析解，但它的两端都通向我们完全掌握的情形：把阻力关到零，应当退回理想抛物线；把阻力开到很大，竖直方向应当出现终端速度、水平方向的速度应当衰减到零。这两个极限，就是审计二维曲线的两把尺子。

先看无阻力极限。当 $b = 0$ ，水平方向匀速、竖直方向匀加速，轨迹是一条标准抛物线，射程有现成的解析公式 $R = v_0^2 \sin(2\theta)/g$ ，其中 θ 是发射仰角。把代码里的 b 设为 0，用题目的参数 $v_0 = 20$ 米每秒、 $\theta = 45^\circ$ ，解析射程是 $20^2 \times \sin 90^\circ / 9.8 \approx 40.8$ 米。跑出来的数值轨迹落点应当落在这个值附近（步长越小越准）。我亲自用 $\Delta t = 0.001$ 跑过，数值落点约 40.83 米，与解析的 40.82 米吻合到三位有效数字。这一关过了，说明无阻力骨架

没写错。

再看大阻力极限。当阻力很强，竖直方向下落的物体很快达到一个受力平衡：重力 mg 被阻力 bv_y 抵消，加速度归零，速度不再变化，这个稳定速度就是。令 $\dot{v}_y = 0$ ，由 $0 = -mg - bv_y$ 解出竖直速度趋于 $-mg/b$ ，即向下的终端速率

$$v_t = \frac{mg}{b}.$$

而水平方向 $m\dot{v}_x = -bv_x$ 是纯衰减， v_x 按指数掉到零，所以飞得足够久之后，质点几乎只剩竖直的匀速下坠。把 b 调大（比如 0.5 千克每秒）并把模拟时间放长，观察竖直速度 v_y 是否趋于 $-mg/b = -0.1 \times 9.8/0.5 = -1.96$ 米每秒、水平速度 v_x 是否趋于零。我跑过这个设定，竖直速度后期稳定在约 -1.96 米每秒，水平速度衰减到接近零，两个方向的极限行为都对得上。

为了让这两个极限看得见，最直接的做法是各画一张对比图：无阻力那次把数值轨迹和解析抛物线叠在一起，应当几乎重合；大阻力那次画竖直速度 v_y 随时间的曲线，应当看到它从初始值一路下降、最后压在 $-mg/b$ 这条水平虚线上。两张图分别锚定了曲线的两端，比单看一条孤零零的轨迹有说服力得多。

验证清单 · 二维抛体数值解

量纲：终端速度 $v_t = mg/b$ 经检验为米每秒（下一节细算），位置数组为米、速度数组为米每秒、时间为秒，全部自洽。无阻力极限：令 $b = 0$ ，数值轨迹应退回解析抛物线，射程逼近 $v_0^2 \sin(2\theta)/g \approx 40.8$ 米，实测落点约 40.83 米，相符。大阻力极限：令 b 较大，竖直速度应趋于终端值 $-mg/b$ 、水平速度应衰减到零，实测 v_y 稳定在约 -1.96 米每秒、 v_x 趋零，相符。对称性：无阻力时轨迹应左右对称、升弧与降弧镜像，数值图目测对称。收敛性：把 Δt 减半重跑，落点与轨迹基本不变，说明步长已足够小。

五项验证里，无阻力和大阻力这两个极限是本章独有的、最锋利的两项。它们遵循同一个逻辑：把一个没有解析解的复杂问题，逼到它能解析的边界上去考。这是物理学家审数值解的看家本事，远比逐行读代码更高效，也更可靠。

本能 · Trust but Verify

拿到一条二维数值轨迹，先别看它画得多漂亮，先找它的可解极限。对带阻力的抛体，至少有两个极限你能在脑子里算清：阻力关到零必须退回抛物线（射程 $v_0^2 \sin 2\theta/g$ 有公式可对），阻力开到很大竖直方向必须出现终端速度 $v_t = mg/b$ 、水平速度必须衰减到零。让代码在这两端各跑一次、各对一次答案，对得上才有资格相信中间那些没有解析解的情形。

本节要点

面对一个没有解析解的复杂问题，把它逼到两个能解析的极限去烤：阻力关到零退回抛物线，阻力开到很大出现终端速度。两条极限都对上了，中间的情形才有资格被信

任。

10.5 量纲检查终端速度：一分钟的审计反射

终端速度公式 $v_t = mg/b$ 是上一节大阻力验证的核心，可我们凭什么相信这个公式本身没写反？答案是量纲检查，本章要训练成肌肉记忆的审计动作。一个速度，量纲必须是长度除以时间，也就是米每秒。我们来逐项核对右边。

质量 m 的量纲是千克，重力加速度 g 的量纲是米每二次方秒，两者相乘 mg 是千克乘米每二次方秒，这正是力的单位牛顿。阻力系数 b 的量纲要从它的定义反推：阻力 bv 必须是一个力，速度 v 是米每秒，所以 b 的量纲只能是牛顿除以米每秒，也就是牛顿秒每米。把它换算成基本单位，牛顿是千克米每二次方秒，除以米每秒，结果是千克每秒。现在做除法 mg/b ，分子是牛顿，分母是千克每秒，牛顿等于千克米每二次方秒，于是

$$\frac{mg}{b} = \frac{\text{kg} \cdot \text{m}/\text{s}^2}{\text{kg}/\text{s}} = \frac{\text{m}}{\text{s}},$$

干净地约出了米每秒，正是速度的量纲。量纲对上了，我们才有底气把 $v_t = mg/b$ 写进验证清单。反过来，如果谁把它写成 $mg \cdot b$ 或者 $b/(mg)$ ，量纲一算立刻就露馅，根本用不着代入数字去试。把放在数值实验之前，是项目主理人最省时间的一道关卡。

量纲检查

本节要点

对任何由多个物理量组合而成的公式，先做量纲检查再代数字。一分钟的量纲演算能拦下多数低级符号错误，这是项目主理人该练成肌肉记忆的动作。

10.6 AI 协作实录：让 AI 写抛体代码并审计它

现在按本课的工作方式演示一遍。你不必从零敲代码，可以让 AI 来写，但你必须能审，而审的第一刀往往就是量纲。

AI 协作实录 · 终端速度公式被写反

我对 AI 说：

你是一名熟悉大一普物和数值方法的物理建模助手。请用显式 Euler 法写一段 Python，模拟带线性空气阻力的二维抛体，阻力为 $-bv$ ，质量 0.1 千克、阻力系数 b 、初速 20 米每秒、仰角 45 度，画出轨迹，并在末尾打印理论终端速度供我核对，所有量用 SI 单位、注释标明单位。

AI 给我：

AI 返回的循环体和绘图都正确，水平与竖直两套更新也分得很清楚。但末尾打印终端速度那一行写成了

```
v_t = m * g * b    # 终端速度，单位米每秒
```

代入 $m = 0.1$ 、 $g = 9.8$ 、 $b = 0.5$ 算出来是 0.49，AI 还煞有介事地标了单位米每秒。

我的审计：

我没急着代数字，先做量纲检查，六十秒搞定。终端速度必须是米每秒。AI 写的是 $m \cdot g \cdot b$ ，质量乘重力加速度乘阻力系数，量纲是千克乘米每二次方秒乘千克每秒，等于千克平方米每三次方秒，跟米每秒八竿子打不着。量纲就对不上，错误立刻定性。正确表达式应当让 b 在分母：终端速度是重力和阻力平衡的结果， $mg = bv_t$ ，所以 $v_t = mg/b$ ，量纲恰好约成米每秒（上一节已经验过）。把那行改成 $v_t = m * g / b$ 后重跑，打印出 1.96 米每秒，再把模拟时间放长，数值的竖直速度果然稳定在 -1.96 米每秒，验证通过。

这段实录的要害和上一章一脉相承：抓错靠的是物理学家的本能动作，先问量纲对不对，不是把代码读一百遍。AI 能写出语法漂亮、结构清楚的多维代码，但哪个量该在分子、哪个该在分母，它的把关并不可靠。而这恰恰是你最该负责、也最不该出错的地方。

AI 陷阱预警 · 多维问题里分量耦合错置、量纲弄反

维数一升，AI 出错的花样就变多。常见的两类：一是把分量耦合搞错，比如本该解耦的线性阻力，它却在水平加速度里掺进了竖直速度，或者把竖直的重力误加到水平方向，让本该匀速的水平运动凭空多出个加速度；二是把含多个物理量的公式量纲弄反，终端速度 mg/b 写成 $mg \cdot b$ 就是典型。对策有两条：写方程时坚持“水平归水平、竖直归竖直”，逐项确认每个方向的合力来源；对任何组合公式，先做量纲检查再代数字。两条都是六十秒的动作，却能拦下绝大多数升维带来的低级错误。

10.7 选读：二维行星轨道

抛体的两套方程之所以能并列，是因为线性阻力让它们解耦。但二维世界里更常见的是耦合的情形，行星绕日运动就是一个漂亮的例子，它也提醒我们“两个一维”有时是真正缠在一起的两套。

■ **Example 10.1：中心引力下的行星轨道。** 把太阳放在原点，行星位于 (x, y) ，到太阳的距离是 $r = \sqrt{x^2 + y^2}$ 。万有引力大小为 GMm/r^2 ，方向指向原点。把它投影到两个坐标轴上，得到两套耦合的方程

$$\dot{v}_x = -\frac{GM}{r^3} x, \quad \dot{v}_y = -\frac{GM}{r^3} y,$$

再配上 $\dot{x} = v_x$ 、 $\dot{y} = v_y$ 。这里 G 是引力常量， M 是太阳质量。注意 r 同时含 x 和 y ，所以水平加速度依赖竖直坐标，反之亦然，这两套一维 ODE 是真正耦合的，不再是抛体那样各管一摊。代码结构依然是 Euler 并列推进四个量，只是每步要先用当前的 x, y 算出公共的 r 。这个问题有两个守恒量可以拿来审计数值解：总能量 $E = \frac{1}{2}m(v_x^2 + v_y^2) - GMm/r$ 和角动量 $L = m(xv_y - yv_x)$ ，理想情况下它们应当沿轨道不变。有趣的是，朴素的显式 Euler 会让能量缓慢漂移、轨道慢慢张开，这正是下一步该追问的现象，也提示我们守恒量是比“轨迹看着像椭圆”更严格的判据。完整实现与守恒量的诊断留作思考题。 ■

本节要点

行星轨道中两套方程通过 $r = \sqrt{x^2 + y^2}$ 真正耦合在一起，但推进代码结构不变。守恒量是这类问题的终极审计官，比“轨迹像不像椭圆”可靠得多。

本章小结

这一章把一维 Euler 法原样搬到了平面上，核心一句话：二维运动等于两条并列的一维 ODE，会解一维就会解二维。带线性空气阻力的抛体为主例，写全了水平 $m\dot{v}_x = -bv_x$ 和竖直 $m\dot{v}_y = -mg - bv_y$ 两套解耦方程，用刻意分行的 Euler 代码并列推进了 x, y, v_x, v_y 四个量。这一章的灵魂是两个极限验证：关掉阻力退回解析抛物线、射程对得上 $v_0^2 \sin 2\theta / g$ ；开大阻力竖直方向出现终端速度 $v_t = mg/b$ 、水平速度衰减到零。量纲检查训练成审计反射，一分钟确认 $v_t = mg/b$ 的量纲是米每秒，同一招在 AI 把它错写成 $mg \cdot b$ 时当场抓获。行星轨道点出：二维的两套方程未必总能解耦，耦合时守恒量是更锋利的审计判据。

下一步转向数据拟合：手里先有一堆实验数据点，不是方程，任务是从数据里把规律拟合出来，并判断拟合是真有物理意义还是把噪声也描了下来。

思考题

1. 本章主线用的是线性阻力 $-b\mathbf{v}$ ，它让水平和竖直两套方程解耦。若改用二次阻力 $\mathbf{F}_{\text{drag}} = -c|\mathbf{v}|\mathbf{v}$ ，请写出此时 $\dot{v}_x$ 和 $\dot{v}_y$ 的表达式，并指出为什么这两套方程不再能各自独立推进、必须在每一步先算出公共的速率 $|\mathbf{v}| = \sqrt{v_x^2 + v_y^2}$ 。这种耦合会让你在审计时失去哪一个本章用过的极限对照？
2. 在大阻力极限下竖直速度趋于终端速度 $v_t = mg/b$ ，水平速度趋于零。请你不依赖数值实验，仅从竖直方程 $m\dot{v}_y = -mg - bv_y$ 出发，说明为什么无论初始竖直速度是多少，长时间后 v_y 都会趋向同一个值 $-mg/b$ 。提示：把方程在 $v_y = -mg/b$ 附近的偏离量当成上一章 RC 放电那样的指数衰减来分析。
3. 选读一节给出了行星轨道的耦合方程和两个守恒量，总能量 E 与角动量 L 。请你让 AI 用显式 Euler 写出这个轨道模拟，并在每一步打印 E 和 L 。预测一下：朴素的显式 Euler 会让哪个守恒量明显漂移、轨道会向内收缩还是向外张开？你打算把“能量应当守恒”放进验证清单的哪一项，又准备用什么办法（比如换更小的步长，或换一种更适合保守系统的推进格式）去改善这个漂移？



11. 数据拟合：最小二乘与残差

前面几章，我们手里的方程都是先验给定的：RC 放电的时间常数、弹簧的劲度系数、阻力的系数，全是题目直接写好的数字，我们的任务只是把方程往前推、画出曲线。可真实的物理项目几乎从来不是这样开始的。更常遇到的情形是：手里先有一堆从实验里量出来的数据点，一杯水每隔半分钟记一次温度，一个单摆数十次摆动的周期，一段自由落体录像里逐帧抠出来的位置，然后你要反过来追问，这些散落的点背后，藏着什么样的物理规律？那个规律里的参数（冷却系数、重力加速度、阻尼率）到底等于多少？把一个带未知参数的模型函数“套”到数据上，让它尽可能贴合，再把那几个最贴合的参数读出来，就叫数据拟合。这一章我们学怎么做拟合；更重要的是，学怎么判断一次拟合到底可不可信。

本章地图

小节	这一节解决什么	关键方法 / 产物
11.1	从实验数据反推模型参数，建立拟合问题	残差 = 实测 - 预测
11.2	把残差平方和最小当作挑选参数的准则	$S(\theta) = \sum r_i^2$ ，线性情形有闭式解
11.3	用 <code>curve_fit</code> 做非线性最小二乘	黑箱披露：Levenberg-Marquardt 迭代
11.4	残差图比 R^2 诚实，系统性结构即模型形式错	残差图是模型错误的指纹
11.5	用 $\pm 10\%$ 参数扫描代替概率论误差棒	数据定得牢的参数，拨动后曲线明显偏离
11.6	用纸笔和 Excel 独立验算线性拟合	三方一致：纸笔、SLOPE 函数、Python
11.7	AI 直线拟合 $R^2 = 0.99$ ，残差图暴露弯弓	高 R^2 掩盖模型形式错误
11.8	收拢为五项验证清单	残差结构是一票否决项

11.1 从数据反推参数：为什么需要拟合

设想你做了一个最普通的冷却实验。一杯热水放在室温的房间里，你用温度计每隔一段时间读一个数，记下了一串 (t_i, T_i) 。上一章我们已经知道，牛顿冷却的模型是 $T(t) = T_{\text{room}} + (T_0 - T_{\text{room}}) e^{-kt}$ ：温度从初值 T_0 出发，按一个冷却系数 k 指数地趋近室温 T_{room} 。现在反过来：你量到的那串点，应该对应着哪一个 k ？

困难在于，测量总带着误差。没有任何一个 k 能让这条光滑的指数曲线精确穿过你记下的每一个点。点是抖动的，曲线是光滑的，让曲线穿过所有点既不可能、也不应该，硬要穿过去，你拟合的就是噪声而不是物理。于是拟合的问题从“找一条穿过所有点的曲线”变成了一个更现实的问题：找一组参数，让模型曲线和数据点整体上离得最近。“整体上离得最近”这句话，必须先给出一个能算的数学定义，否则不同的人会画出不同的“最近”。把这个定义讲清楚，正是最小二乘法的全部出发点。

本节要点

拟合的要义不在于让曲线穿过每个点，而在于让模型曲线与数据整体最近。每个点都带着噪声，硬穿过去拟合的就是噪声而不是物理。

11.2 最小二乘原理

衡量模型曲线和某一个数据点贴不贴合，最自然的量就是二者在该点的竖直差距，数据点的纵坐标减去模型在该处给出的预测值。这个差距有一个专门的名字，叫。第 i 个数据点的残差是

$$r_i = y_i - f(x_i; \theta),$$

其中 y_i 是实测值, $f(x_i; \theta)$ 是模型在 x_i 处、用参数 θ 算出的预测值。残差可正可负: 模型偏低时残差为正, 偏高时为负。我们希望所有残差整体都尽量小, 但不能简单地把它直接相加, 一正一负会互相抵消, 一条糟糕透顶的曲线也可能让正负残差恰好抵成零。解决办法是先把每个残差平方 (平方既消掉了符号, 又对大偏差施加了更重的惩罚), 再求和, 得到

残差平方和

$$S(\theta) = \sum_{i=1}^N r_i^2 = \sum_{i=1}^N [y_i - f(x_i; \theta)]^2.$$

最小二乘法就是一句话: 在所有可能的参数里, 挑那一组让 $S(\theta)$ 取最小值的, 作为最佳拟合参数。

Definition 11.2.1: 最小二乘拟合. 给定 N 个数据点 (x_i, y_i) 和一个含参模型 $f(x; \theta)$, 每个点的残差定义为 $r_i = y_i - f(x_i; \theta)$ 。最小二乘拟合即是求一组参数 θ^* , 使残差平方和 $S(\theta) = \sum_i r_i^2$ 达到最小。这组 θ^* 就是数据在该模型下的最佳拟合参数。

为什么是平方而不是绝对值? 一个朴素但够用的理由是: 平方函数处处光滑可导, 求最小值时可以用“导数等于零”这套微积分的标准武器; 绝对值在零点有一个尖角, 处理起来麻烦得多。下面这个最简单的线性例子, 正好拿来把这套武器亲手用一遍, 并且能一路推到底, 得到一个可以手算的闭式公式。

11.2.1 线性情形的闭式解

最干净的模型是一条过原点之外的直线 $f(x) = a + bx$, 两个待定参数是截距 a 和斜率 b 。代进残差平方和,

$$S(a, b) = \sum_{i=1}^N (y_i - a - bx_i)^2.$$

这是 a 和 b 的一个二次函数, 图像是一个开口朝上的碗, 碗底那一点就是我们要的最小值。在碗底, S 对 a 和对 b 的偏导数都为零。先对 a 求偏导并令其为零,

$$\frac{\partial S}{\partial a} = -2 \sum_i (y_i - a - bx_i) = 0 \implies \sum_i y_i = Na + b \sum_i x_i,$$

再对 b 求偏导并令其为零,

$$\frac{\partial S}{\partial b} = -2 \sum_i x_i (y_i - a - bx_i) = 0 \implies \sum_i x_i y_i = a \sum_i x_i + b \sum_i x_i^2.$$

这两个方程合起来是一组关于 a 、 b 的二元一次方程组, 线代里叫它正规方程。解这个方程组 (就是大一线代里消元法的标准操作), 得到

$$b = \frac{N \sum_i x_i y_i - \sum_i x_i \sum_i y_i}{N \sum_i x_i^2 - (\sum_i x_i)^2}, \quad a = \frac{\sum_i y_i - b \sum_i x_i}{N} = \bar{y} - b \bar{x},$$

其中 $\bar{x}$ 、 $\bar{y}$ 是 x 和 y 的平均值。这两个公式不需要任何计算机。给你一张数据表、一个计算器，你就能亲手把最佳斜率和截距算出来。这一点在本章后面会变成一条纪律：凡是线性拟合，你都有能力不借助任何软件、独立验出那两个数。

本节要点

线性最小二乘把残差平方和写成一个二次碗，让导数等于零就得到斜率和截距的闭式公式。这两个公式不靠任何软件就能算，是你独立验算的底牌。

历史上，这套方法的首次公开发表归于勒让德 (Legendre) 在 1805 年的著作，而高斯 (Gauss) 则声称自己早在 1795 年就已用在计算谷神星轨道上，由此引发了一场关于优先权的著名争论。无论谁先想到，最小二乘自那以后成了一切实验数据分析的基石。

11.3 非线性拟合：把算法当黑箱，但要能验证

牛顿冷却模型 $T(t) = T_{\text{room}} + (T_0 - T_{\text{room}})e^{-kt}$ 里的参数 k 藏在指数上，残差平方和对 k 的偏导不再给出一组线性方程，没有上面那样干净的闭式解。这类模型叫非线性拟合，求最小值得靠迭代：从一个初始猜测出发，一步步沿着让 S 下降的方向挪动参数，直到挪不动为止。

我们不自己实现这个迭代过程，而是直接调用 `scipy` 的 `scipy.optimize.curve_fit`。这里要对你诚实地交代一句它内部在做什么：`curve_fit` 默认用的是一种叫 Levenberg-Marquardt 的非线性最小二乘算法，它巧妙地在“大步快走”和“小步稳走”之间自动切换，从初值滑向残差平方和的最小点，相关数值细节在标准的科学计算手册里有系统讨论 [3]。

Definition 11.3.1：黑箱披露：`curve_fit` 接受一个模型函数、一组自变量数据、一组因变量数据，返回使残差平方和最小的那组参数。它内部默认采用 Levenberg-Marquardt 非线性最小二乘迭代。在这门课里，你只负责正确调用这个接口、提供合理的参数初值，并对它返回的结果做验证；你不需要理解迭代算法的内部细节，正如你开车不必懂发动机的每一个齿轮，但你必须能判断车是不是在往正确的方向开。

本节要点

`curve_fit` 是一个黑箱，你不需要理解 Levenberg-Marquardt 的迭代细节，但你必须能正确调用它、给合理的初值，并对返回的结果做验证。

下面这段代码做一次完整的牛顿冷却拟合。它先用一个已知的 k 造出一组带噪声的“实验”数据（这样我们手里有标准答案，方便检验拟合到底准不准），再用 `curve_fit` 把 k 反推出来，最后画出数据散点、拟合曲线和残差的三联图。代码完整可运行，可以原样拷进编辑器。

Listing 11.1: 牛顿冷却数据的非线性最小二乘拟合

```
1 | import numpy as np
```

```

2 import matplotlib.pyplot as plt
3 from scipy.optimize import curve_fit
4
5 # 模型函数：牛顿冷却  $T(t) = T_{room} + (T_0 - T_{room}) * \exp(-k t)$ 
6 def cooling(t, T_room, T0, k):
7     return T_room + (T0 - T_room) * np.exp(-k * t)
8
9 # --- 制造一组带噪声的“实验”数据（单位：t 用秒，T 用摄氏度） ---
10 rng = np.random.default_rng(0) # 固定随机种子，
    保证每次结果一致
11 t_data = np.linspace(0.0, 600.0, 25) # 0 到 600 秒，共 25 个采样点
12 T_room_true, T0_true, k_true = 22.0, 90.0, 0.005 # 真实参数（待反推）
13 T_clean = cooling(t_data, T_room_true, T0_true, k_true)
14 T_data = T_clean + rng.normal(0.0, 1.0, size=t_data.size) #
    叠加测量噪声
15
16 # --- 调用黑箱 curve_fit 做非线性最小二乘拟合 ---
17 # p0 是参数初值猜测，顺序对应 cooling 的 (T_room, T0, k)
18 p0 = [20.0, 100.0, 0.01]
19 popt, _ = curve_fit(cooling, t_data, T_data, p0=p0)
20 T_room_fit, T0_fit, k_fit = popt
21 print("拟合结果：T_room = %.3f, T0 = %.3f, k = %.5f" %
22       (T_room_fit, T0_fit, k_fit))
23
24 # --- 计算残差与拟合优度 R 方 ---
25 T_model = cooling(t_data, *popt) # 拟合曲线在各数据点的预测值
26 residuals = T_data - T_model # 残差 = 实测 - 预测
27 ss_res = np.sum(residuals**2) # 残差平方和
28 ss_tot = np.sum((T_data - np.mean(T_data))**2)
29 R2 = 1.0 - ss_res / ss_tot
30 print("残差平方和 = %.3f, R^2 = %.4f" % (ss_res, R2))
31
32 # --- 三联图：数据散点+拟合曲线，以及残差图 ---
33 t_fine = np.linspace(0.0, 600.0, 400)
34 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6, 6), sharex=True)
35
36 ax1.plot(t_data, T_data, 'o', markersize=4, label='实验数据')
37 ax1.plot(t_fine, cooling(t_fine, *popt), '-', label='拟合曲线')
38 ax1.set_ylabel('温度 T (摄氏度)')
39 ax1.legend()
40
41 ax2.plot(t_data, residuals, 'o', markersize=4)
42 ax2.axhline(0.0, color='gray', linewidth=1) #

```

```
    残差应在零线两侧随机散布
43 ax2.set_xlabel('时间 t (s)')
44 ax2.set_ylabel('残差 (摄氏度)')
45
46 plt.tight_layout()
47 plt.show()
```

跑完之后，打印出来的 k 应该非常接近我们造数据时用的 0.005，上面的拟合曲线会平滑地穿过散点群的中央，下面的残差图则是一片在零线上下随机抖动的点。这张，才是这一章真正的主角。

11.4 残差图比 R^2 方诚实

很多人拟合完只看一个数： R^2 （决定系数）。它衡量模型解释了数据总变化的多大比例，越接近 1 看起来越“好”。但 R^2 高得吓人，并不能保证你选对了模型形式，它甚至会把你骗得心服口服。原因在于 R^2 只是一个把所有残差搅在一起的总分，它丢掉了一个极其关键的信息：残差是怎么分布的。

判断模型形式对不对，要看残差有没有结构，不是看残差有多小。如果你选对了物理模型，数据偏离曲线的部分就只剩下随机的测量噪声，残差图应该是一片没有任何规律、在零线两侧均匀抖动的散点。反过来，如果残差图里出现了系统性的形状，整体先低后高再低、呈一道明显的弯弓，或者一段连续偏正、另一段连续偏负，那就说明你的模型函数本身错了：它没能抓住数据里真实的弯曲方式，于是把这份“抓不住”的系统性误差留在了残差里。残差里的结构，是模型形式错误留下的指纹。

AI 陷阱预警 · 高 R^2 方掩盖错误的模型形式

不要把 R^2 当成拟合好坏的唯一裁判。 R^2 是一个被高度压缩的总分，一条形状根本不对的曲线，只要数据范围窄、噪声又不大，照样能刷出 0.98、0.99 这样漂亮的 R^2 。真正能戳穿模型形式错误的是残差图：先看残差是不是在零线两侧随机散布，再看 R^2 。只要残差里有肉眼可辨的系统性结构（弯曲、波浪、连续同号的一段），无论 R^2 多高，模型都不可信，必须换一个函数形式重新拟合。

本节要点

R^2 是压缩过的总分，残差图是展开的指纹。模型形式错误会在残差里留下弯弓、波浪或连续同号的痕迹，而 R^2 可能还在对你微笑。

11.5 参数不确定性：用 $\pm 10\%$ 扫描代替统计推断

拟合给出的参数从来不是一个绝对精确的数。数据里有噪声，换一批噪声，拟合出的 k 就会略有不同，所以每个参数其实都带着一段“差不多在这个范围”的模糊地带。专业的做法是从 `curve_fit` 返回的协方差矩阵 `pcov` 里读出标准误差，但那需要概率论里方差、

置信区间这些工具，而你们大一还没学。在没真正理解的情况下照抄一个 $\pm$ 误差棒，只会变成又一种“信而不知”，这正是这门课要避免的。

我们改用一个不需要任何概率论、却同样能建立起直觉的笨办法：。把拟合出的每一个参数，分别人为地拨大 10% 和拨小 10%，其余参数保持在最佳值，看看模型曲线和残差随之怎么变。如果某个参数动一动，曲线就明显偏离了数据、残差立刻鼓出一道结构，说明数据对这个参数很敏感，它被定得很牢；反过来，如果某个参数拨了 10% 曲线几乎纹丝不动，说明数据其实管不住它，这个参数的拟合值就别太当真。下面这段代码以冷却系数 k 为例做一次扫描，接着上一段代码的变量直接运行。

参数扫描

Listing 11.2: 对拟合参数 k 做正负 10% 敏感性扫描

```
1 # 接 lst:coolfit 运行: popt, t_data, T_data, cooling 均已定义
2 k_best = popt[2]
3 plt.figure(figsize=(6, 4))
4 plt.plot(t_data, T_data, 'o', markersize=4, label='实验数据')
5
6 # 把 k 分别拨到 0.9 倍 1.0 倍 1.1 倍，其余参数保持最佳值
7 for factor, style in [(0.9, '--'), (1.0, '-'), (1.1, ':')]:
8     params = [popt[0], popt[1], k_best * factor]
9     plt.plot(t_data, cooling(t_data, *params), style,
10             label='k = %.5f (%.0f%%)' % (k_best * factor, factor *
11                                     100))
12
13 plt.xlabel('时间 t (s)')
14 plt.ylabel('温度 T (摄氏度)')
15 plt.legend()
16 plt.tight_layout()
17 plt.show()
```

跑出来你会看到三条曲线明显分开， $\pm 10\%$ 的 k 让冷却的快慢肉眼可辨地不同，数据点清楚地落在中间那条上，旁边两条则系统性地偏离。这说明这批数据把 k 管得相当牢。这个动手扫一遍的习惯，比背一个你还不懂含义的误差棒值钱得多。

本节要点

$\pm 10\%$ 参数扫描不需要概率论基础，却直观地回答了“这个参数被数据管得牢不牢”。拨不动的是虚的，拨得动的才有资格写进结论。

11.6 no-AI 基线：用纸笔和 Excel 对账线性拟合

这门课有一条贯穿始终的纪律：凡是你能独立算出来的，就必须独立算一遍，拿去和 AI 或软件的输出对账，绝不盖橡皮图章的人。非线性拟合的迭代你算不动，但本章前面那个线性最小二乘的闭式解，你完全有能力亲手算。

具体做法是：取一小组近似线性的数据（比如把一段冷却曲线的早期、温差还大的几

个点，或者干脆一组人为的直线数据），按前面推出的公式，列一张表，逐列算出 $\sum x_i$ 、 $\sum y_i$ 、 $\sum x_i y_i$ 、 $\sum x_i^2$ ，再代进斜率和截距的公式，用计算器把 b 和 a 敲出来。然后在 Excel 里把同一组数据填进两列，用内置函数 `=SLOPE(y 范围, x 范围)` 和 `=INTERCEPT(y 范围, x 范围)` 各算一次，这两个内置函数算的正是同一套最小二乘公式。最后再让 `curve_fit`（或 `numpy.polyfit`）对同一组数据拟合一条直线。三条路给出的斜率和截距，应当在三位有效数字内完全一致。如果对不上，要么是你某个求和敲错了，要么是数据范围选反了。无论哪种，都比答辩时被问住强。能让纸笔、Excel、Python 三方独立给出同一个数，你才真正握住了这次拟合，而不是把它托付给了一个你看不见内部的黑箱。

本节要点

线性情形的闭式解是你的 no-AI 底牌。纸笔、Excel 的 SLOPE 函数和 Python 的 polyfit 三方独立给出同一个数，你才算真正握住了这次拟合。

11.7 AI 协作实录：用残差图推翻一个高 R^2 方的拟合

下面把本章的判断方式完整演示一遍。问题是给一组冷却数据拟合一个模型，反推规律，这正是 AI 最容易让你“看起来很成功”的场合。

AI 协作实录 · AI 报告 R^2 等于 0.99，残差图却出卖了它

我对 AI 说：

我有一组热水冷却的实验数据（时间 t 与温度 T ）。请你用 Python 帮我拟合，反推出温度随时间变化的规律，并告诉我拟合好不好。

AI 给我：

AI 返回了一段能跑的代码，但它选了最省事的模型，把温度对时间做了一条直线拟合 $T = a + bt$ 。它报告说“拟合非常好， $R^2 = 0.99$ ，温度随时间线性下降，每秒下降约 0.11 摄氏度”，并附上一张数据点几乎都贴在直线上的图，看起来相当令人信服。

我的审计：

我没有被那个 0.99 说服，而是按本章的纪律先画残差图。一画出来问题就暴露了：残差呈一道明显的弯弓，不是随机地在零线两侧抖动，起始段连续偏正，中间段连续偏负，末段又翻回偏正。这种系统性的弯曲结构只有一个解释：真实曲线是向下凹的，而直线既凹不下去也弯不回来，于是把这份弯曲全留在了残差里。物理上也对得上：牛顿冷却是指指数趋近室温，温差越小降得越慢，本来就不该是匀速直线下降。我把模型换成 $T(t) = T_{\text{room}} + (T_0 - T_{\text{room}})e^{-kt}$ 重新拟合，残差立刻塌成一片零线附近的随机散点，弯弓消失，反推出的室温也和房间实际温度吻合。结论是：AI 那个 $R^2 = 0.99$ 是真的，但模型形式是错的，高 R^2 和“模型对”是两码事，是残差图而不是 R^2 揭穿了它。

这段实录的要害和上一章如出一辙：揪出错误靠的是物理学家的本能动作，不是把代码读上一百遍。上一章那个动作是看量纲，这一章是画残差图。AI 很会写出能跑、 R^2 又漂亮的拟合代码，但它不会主动替你质疑模型形式选得对不对，而这恰恰是你这个项目主

理人该负责、也最该有判断力的地方。

本能 · Trust but Verify

拿到任何一次拟合结果，先别看 R^2 有多高，先做两个动作。第一，画残差图：残差若为零线两侧随机散布，模型形式大体可信；残差若有弯曲、波浪或连续同号的系统性结构，模型形式就错了，必须换函数重拟。第二，对每个关心的参数做 $\pm 10\%$ 扫描：曲线随之明显偏离数据，说明这个参数被数据定得牢；曲线几乎不动，说明数据管不住它，这个值不能太当真。残差图与参数扫描，就是你审计一次拟合的两把尺子。

11.8 用验证清单确认拟合结果

把上面零散的判断收拢成一份清单，就是这门课反复打磨的工件。对牛顿冷却拟合这个例子，验证可以这样走一遍。

验证清单 · 牛顿冷却拟合

量纲与范围：冷却系数 k 的量纲为时间倒数（每秒），拟合出的室温 T_{room} 应落在合理的房间温度区间，初温 T_0 应接近你记录的第一个数据点，三者数量级都说得通。残差结构：残差图在零线两侧随机散布，没有弯弓、波浪或连续同号的系统性结构，说明模型形式选对了。物理趋势：拟合曲线单调下降并趋于室温，温差越小降得越慢，符合牛顿冷却的物理图像。参数敏感性：把 k 拨 $\pm 10\%$ ，曲线明显偏离数据，说明数据把 k 定得牢，拟合值可信。独立对账：对其中近似线性的一段，用纸笔或 Excel 的最小二乘公式独立算斜率，与软件结果在三位有效数字内一致。

本节要点

把 R^2 从清单里拿掉是故意的。高 R^2 推不翻错误的模型，但残差里的系统结构能一票否决。一个好的项目主理人宁可在残差图上多看两眼。

五项里能过四到五项，你才有底气说这次拟合可信。注意这里把 R^2 故意从清单里拿掉了，它给不出能推翻结论的判据。真正能一票否决的是残差结构那一项。一个好的项目主理人，宁可在残差图上多看两眼，也不愿意拿着一个 $R^2 = 0.99$ 的错误模型去答辩。

本章小结

这一章学会了把真实数据接进项目，用含参模型去拟合、反推物理参数。从残差出发，最小二乘就是让残差平方和最小；线性情形下斜率与截距的闭式解，是你独立验算的底牌。对非线性模型，调用 `curve_fit` 这个黑箱，只负责正确调用与验证。残差图比 R^2 诚实：高 R^2 不代表模型对，残差里只要有系统性结构就说明模型形式错了。参数不确定性用 $\pm 10\%$ 扫描来建立直觉，不靠还不懂的概率论误差棒。

无论是微分方程还是拟合，处理的都是确定性问题。现实里大量问题天生带随机性。下一章转向随机的世界，看怎么用大量随机抽样去估算确定的答案。

思考题

1. 本章线性最小二乘的斜率公式是 $b = \frac{N \sum x_i y_i - \sum x_i \sum y_i}{N \sum x_i^2 - (\sum x_i)^2}$ 。请你取四个数据点 (1, 2)、(2, 5)、(3, 5)、(4, 9)，用纸笔逐列算出四个求和，代入公式求出斜率 b 与截距 a ，并说明你会怎样用 Excel 的 SLOPE 和 INTERCEPT 把这两个数核对一遍。
2. 有人用一条直线拟合了一段本该是指数衰减的冷却数据，得到 $R^2 = 0.98$ ，据此宣称“温度随时间线性下降”。仅凭这个 R^2 ，你能不能反驳他？你会要求他补出哪一张图，那张图上你预期会看到什么样的形状，才足以推翻他的线性结论？
3. 假设你对某组数据拟合一个含三个参数的模型，做 $\pm 10\%$ 扫描时发现：其中两个参数各拨 10%，拟合曲线都明显偏离了数据，唯独第三个参数拨 10% 时曲线几乎纹丝不动。关于这第三个参数，你能下什么结论？这对你“该不该把它的拟合值写进报告、并强调它的物理意义”有什么影响？



12. 蒙特卡罗初步：用随机数做物理

前面几章解方程、推轨迹，用的都是确定性算法：给定初值和步长，每次跑出来分毫不差。这一章换一种完全不同的武器，拿随机数算确定的量。随机的东西怎能算出确定的答案？听着像悖论，这就是蒙特卡罗方法的魔力。它得名于摩纳哥那座赌城，核心动作就三个：撒点、掷骰子、看比例。从估几何面积，到模拟成千上万个分子的扩散，再到高维积分这类令确定性方法束手无策的硬骨头，蒙特卡罗在物理里全是主力工具。作为项目主理人，这一章你最需要学会的是判断蒙特卡罗结果收敛了没有。随机方法天生擅长骗人：样本太少时，它能吐出一个看着挺准、实则全靠运气的数字。

本章地图

小节	这一节解决什么	关键方法 / 产物
12.1	把确定量翻译成“撒点看比例”或“撒点求平均”	随机样本的频率/均值去近似确定答案
12.2	在单位正方形里撒点估计 π	几何概率： $N_{\text{in}}/N \approx \pi/4$
12.3	理解 $1/\sqrt{N}$ 收敛规律	误差随样本数平方根下降，双对数图斜率为 $-1/2$
12.4	用平均值法估计定积分	$\int_0^1 g(x)dx \approx \frac{1}{N} \sum g(x_i)$
12.5	一维随机行走重现 $\langle x^2 \rangle = N$	均方位移正比于步数，扩散的定量指纹
12.6	AI 用 $N = 100$ 报 $3.1 \approx \pi$ ，收敛检查拆穿	小样本的收敛假象，必须扫大 N 画收敛曲线

12.1 核心思想：用随机采样估计一个确定的量

要量一个形状古怪的水塘面积，四周弯弯曲曲没法用尺子量。有一个看似笨拙其实管用的办法：在水塘外围画一个边长已知的正方形，框住整个塘，然后闭眼往正方形里随手扔石子。扔很多很多颗，最后数一数有多少颗落进了水里。只要石子撒得足够均匀、足够多，落水的比例就近似等于水塘面积占正方形面积的比例。正方形面积乘上这个比例，水塘面积就估出来了。

这个扔石子游戏，就是蒙特卡罗方法的全部精神。我们想求的水塘面积是一个确定的、客观存在的数，但我们不去精确计算它，而是靠大量随机样本估计一个比例，再把比例翻译回我们想要的量。样本越多，估计越准。

Definition 12.1.1 : 蒙特卡罗方法. 蒙特卡罗方法是一类用大量随机采样来估计某个确定数值的计算方法。它的基本套路是：把待求的量（面积、积分、概率、平均值）表达成“在某个区域里随机取点时，某件事发生的比例”或“某个量的平均值”，然后实际生成大量随机样本，用样本算出的频率或均值去近似那个确定的答案。样本数越大，估计值越接近真值。

本节要点

蒙特卡罗把“精确计算”换成“大量随机抽样看比例”，样本足够多时偶然偏差互相抵消，留下的净估计逼近真值。核心就三个动作：撒点、统计、翻译。

这个套路要落地，关键在于把物理问题翻译成“撒点看比例”或“撒点求平均”。下面的三个例子，从估计圆周率到算定积分再到模拟扩散，把这种翻译演示三遍。难度递增，骨架始终一样：撒随机点、统计、翻译回物理量。系统的蒙特卡罗方法和随机数生成的讲究，可参考数值计算经典教材 [3]。

12.2 第一个例子：撒点估计圆周率

蒙特卡罗最经典的演示是估计 π 。它能做第一个例子，是因为答案我们早就知道， $\pi = 3.14159\dots$ ，正好拿来给随机方法当场打分。

考虑边长为 1 的单位正方形，放在第一象限，左下角顶在原点，四个角分别是 $(0,0)$ 、 $(1,0)$ 、 $(1,1)$ 、 $(0,1)$ 。再画一个以原点为圆心、半径为 1 的四分之一圆，恰好嵌在正方形里。正方形面积 $1 \times 1 = 1$ ，四分之一圆面积 $\frac{1}{4}\pi \cdot 1^2 = \pi/4$ 。现在往正方形里均匀撒随机点。均匀的意思是横坐标 x 和纵坐标 y 都在 0 到 1 之间等可能取值，点落进正方形任何一小块的概率只看那块面积。一个点 (x,y) 落在四分之一圆内，当且仅当它到原点的距离不超过 1，即 $x^2 + y^2 \leq 1$ 。

按的思想，一个均匀随机点落进圆内的概率等于圆内面积除以正方形面积，即 $(\pi/4)/1 = \pi/4$ 。反过来说，撒 N 个点，数出 N_{in} 个落在圆内，比例 N_{in}/N 就近似等于 $\pi/4$ ，于是

$$\pi \approx 4 \cdot \frac{N_{\text{in}}}{N}.$$

这就是用随机数估计圆周率的全部原理。下面这段 Python 完整可运行：它撒点、统计、给出估计值，再把圆内圆外的点画成不同颜色，让你一眼看清几何图像。

Listing 12.1: 撒点估计圆周率

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 rng = np.random.default_rng(2024)    # 固定随机种子，保证结果可复现
5
6 N = 5000                                # 撒点总数
7 x = rng.uniform(0.0, 1.0, N)           # 横坐标，在 0 到 1 之间均匀取值
8 y = rng.uniform(0.0, 1.0, N)           # 纵坐标，同上
9
10 inside = (x**2 + y**2) <= 1.0          # 布尔数组：
    每个点是否落在四分之一圆内
11 N_in = np.count_nonzero(inside)        # 数一数落在圆内的点数
12
13 pi_estimate = 4.0 * N_in / N           # 比例乘以 4 就是  $\pi$  的估计
14 print(f"撒点数 N = {N}")
15 print(f"圆内点数 N_in = {N_in}")
16 print(f"pi 的估计值 = {pi_estimate:.5f}, 真值 = {np.pi:.5f}")
17
18 # 画散点：圆内圆外用不同颜色
19 plt.figure(figsize=(5, 5))
20 plt.scatter(x[inside], y[inside], s=4, color='tab:blue',
    label='圆内')
21 plt.scatter(x[~inside], y[~inside], s=4, color='tab:orange',
    label='圆外')
22 theta = np.linspace(0.0, np.pi / 2, 200)    # 画出四分之一圆的弧线
23 plt.plot(np.cos(theta), np.sin(theta), 'k-', linewidth=1.5)
24 plt.xlabel('x')
25 plt.ylabel('y')
26 plt.gca().set_aspect('equal')
27 plt.legend(loc='upper right')
28 plt.tight_layout()
29 plt.show()
```

跑一次就会看到，正方形里散落着蓝点和橙点各一片：蓝点挤在左下角四分之一圆里，橙点散在右上角弯月形区域，弧线清楚地把它们分开。打印的估计值大约在 3.14 附近，比如 3.13 或 3.15。它通常不会精确等于 3.14159，换一个随机种子，估计值就上下浮动一点。这种浮动不是错误，是随机方法的天性，下一节我们正面处理它。

本能 · Trust but Verify

拿到一个蒙特卡罗估计值，第一个该问的是“收敛了吗”，不是“等于多少”。唯一可靠的判断是：把样本数 N 加大几个数量级，看估计值是不是稳稳靠拢真值、波动是不是越来越小。只跑了几百个样本就报出来的漂亮数字，和步长过大的微分方程曲线一样，看着是答案，其实是运气。收敛性检查就是蒙特卡罗结果的“量纲检查”，是你必做的第一道关。

本节要点

π 是检验蒙特卡罗的完美标准，答案已知，正好给随机方法当场打分。撒点越多，蓝橙分界线越清晰，估计越准。但单次跑出的漂亮数字不代表收敛，换种子就可能翻脸。

12.3 收敛行为：精度随样本数怎么提高

蒙特卡罗估计为什么会浮动，样本多了又为什么会准？直观的图像是：每撒一个点，落进圆内还是圆外全凭偶然，单个点给的信息很粗糙。但把很多点平均起来，那些偶然的正负偏差就互相抵消了一部分，剩下的净偏差跟总量一比，越来越不显眼。这就像抛硬币：抛 10 次，正面比例可能是 7/10，离 0.5 很远；抛 10000 次，正面比例几乎铁定贴近 0.5。

更定量地说，蒙特卡罗估计的误差大致随样本数按 $1/\sqrt{N}$ 缩小。想把精度提高 10 倍，样本数就得翻 100 倍；再提高 10 倍，再翻 100 倍。 $1/\sqrt{N}$ 是蒙特卡罗方法的命门：收敛相当慢，追求高精度时代价很大。但它也有好处：这条规律几乎不随问题维度变化，这正是高维问题上反超确定性方法的原因。这里我们只从直观和数值上认识 $1/\sqrt{N}$ ，严格的统计证明留到将来的概率课。

下面的代码把样本数 N 从小到大扫一遍，记录每个 N 下估计值偏离真值 π 的绝对误差，再画成双对数坐标图，和一条 $1/\sqrt{N}$ 参考线比较。

Listing 12.2: 估计圆周率的误差随样本数的收敛

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 rng = np.random.default_rng(0)
5
6 # 让 N 跨越几个数量级：100, 1000, ... 一直到一千万
7 N_list = np.array([10**k for k in range(2, 8)])
8 errors = np.zeros(len(N_list))
9
10 for i, N in enumerate(N_list):
11     x = rng.uniform(0.0, 1.0, N)
12     y = rng.uniform(0.0, 1.0, N)
13     pi_est = 4.0 * np.count_nonzero(x**2 + y**2 <= 1.0) / N
14     errors[i] = abs(pi_est - np.pi) # 估计值偏离真值的绝对误差

```

```

15     print(f"N = {N:>9d}    pi 估计 = {pi_est:.5f}    误差 =
        {errors[i]:.5f}")
16
17 # 双对数坐标下画误差随 N 的变化
18 plt.loglog(N_list, errors, 'o-', label='实测绝对误差')
19 plt.loglog(N_list, 1.0 / np.sqrt(N_list), '--', label=r'参考线
    $1/\sqrt{N}$')
20 plt.xlabel('样本数 N')
21 plt.ylabel('绝对误差')
22 plt.legend()
23 plt.tight_layout()
24 plt.show()

```

跑出来会看到，实测误差的圆点虽然上下抖动（每个 N 毕竟还是一次随机实验），但整体趋势和那条 $1/\sqrt{N}$ 虚线大致平行，都是随 N 增大向下走的直线。双对数坐标里斜率为 $-1/2$ 的直线，正是 $1/\sqrt{N}$ 规律的标志。看到向下的趋势线，你才有底气说估计在收敛。误差不随 N 减小的话，要么代码有 bug，要么你算的根本不是你以为的那个量。

本节要点

$1/\sqrt{N}$ 是蒙特卡罗的命门：收敛慢但几乎不随维度变化，所以在高维问题上反而能反超确定性方法。双对数图中斜率为 $-1/2$ 的直线是收敛的目视凭证。

12.4 第二个例子：用随机数估计定积分

撒点估 π 本质上是在算面积，面积就是积分。把思路推开，蒙特卡罗也能估计定积分。考虑

$$I = \int_0^1 \sqrt{1-x^2} dx.$$

这个积分有明确的几何意义：被积函数 $y = \sqrt{1-x^2}$ 是单位圆的上半部分，在 x 从 0 到 1 这段，曲线下方面积恰好是四分之一圆面积，即 $\pi/4 \approx 0.7854$ 。就拿这个已知答案来给方法打分。

比撒点更直接的办法叫平均值法。定积分 $\int_0^1 g(x) dx$ 在几何上是函数 g 在 $[0, 1]$ 上的平均高度乘以区间宽度 1，所以它就等于 g 在 $[0, 1]$ 上的平均值。在 $[0, 1]$ 里均匀取很多个随机 x ，把对应的 $g(x)$ 做算术平均，这个样本均值就近似等于真正的平均值，也就近似等于积分值。换句话说

$$\int_0^1 g(x) dx \approx \frac{1}{N} \sum_{i=1}^N g(x_i), \quad x_i \text{ 在 } [0, 1] \text{ 上均匀随机.}$$

Listing 12.3: 用平均值法估计定积分

```

1 | import numpy as np

```

```

2
3 rng = np.random.default_rng(7)
4
5 def g(x):
6     return np.sqrt(1.0 - x**2)          # 被积函数：单位圆上半部分
7
8 N = 1_000_000                          # 样本数
9 x = rng.uniform(0.0, 1.0, N)           # 在 [0, 1] 上均匀取样
10 I_estimate = np.mean(g(x))             # 样本平均近似积分
11
12 I_exact = np.pi / 4.0                  # 解析值
13 print(f"积分估计值 = {I_estimate:.5f}")
14 print(f"解析值      = {I_exact:.5f}")
15 print(f"绝对误差    = {abs(I_estimate - I_exact):.5f}")

```

跑一百万个样本，估计值落在 0.785 附近，跟解析值 $\pi/4$ 有三到四位有效数字的吻合。把 N 缩到一千重跑，误差明显变大；放大到一千万，误差进一步缩小。 $1/\sqrt{N}$ 的收敛行为又一次被验证。值得停下来体会的是：平均值法对任何能写出来的被积函数都适用。就算没有解析原函数、纸笔积不出来，蒙特卡罗照样给出估计。这正是它在物理计算里如此通用的原因。

本节要点

平均值法比撒点法更直接,对任何能写出来的被积函数都管用,哪怕你积不出来。 $1/\sqrt{N}$ 的收敛行为在积分估计中同样成立,一百万个样本大约给三到四位有效数字。

12.5 第三个例子：一维随机行走与扩散

前两个例子里，随机数帮我们算确定的几何量。第三个例子不同：随机性本身就是物理，它是分子热运动和扩散现象的微观图像。设想一个粒子站在一维数轴的原点，每过一个时间单位掷一次硬币：正面向右一步，反面向左一步，左右概率各半，每一步都和前面无关。这种走法叫一维随机行走，是布朗运动和扩散的最简模型。

单看一个粒子，走 N 步之后会到哪儿完全没法预测，可能一路向右冲出去，也可能在原点附近来回回。但物理学不关心单个粒子的轨迹，关心的是大量粒子的统计规律。我们最关心的量是，即粒子走 N 步后位置 x 的平方的平均值 $\langle x^2 \rangle$ 。这个平均是对大量粒子取的。为什么它重要？位置本身的平均值 $\langle x \rangle$ 左右对称恒等于零（向左向右一样多，平均下来不动），说不清粒子散开了多少； $\langle x^2 \rangle$ 却不会被正负抵消掉，它直接刻画了粒子云团从原点向两侧铺开有多大。

理论上有一个漂亮的结论：这种每步等概率正负一的随机行走，走 N 步后的均方位移精确等于步数，即 $\langle x^2 \rangle = N$ 。直观地说，每一步对 x^2 平均贡献 1，不同步之间的交叉项因方向独立、正负各半，平均下来互相抹掉了。 N 步累积下来，均方位移正比于 N 。这个“均方位移正比于时间”的关系，正是扩散的定量标志。下面的代码模拟大量粒子，在每

个步数上统计均方位移，画它随步数的变化。

Listing 12.4: 一维随机行走的均方位移

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 rng = np.random.default_rng(42)
5
6 n_particles = 20000      # 粒子数：取大才能让统计平均可靠
7 n_steps = 200           # 每个粒子走的步数
8
9 # 每一步等概率取 +1 或 -1；形状（粒子数，步数）
10 steps = rng.choice([-1, 1], size=(n_particles, n_steps))
11
12 # 沿步数方向做累加，得到每个粒子在每一步后的位置
13 positions = np.cumsum(steps, axis=1)
14
15 # 在每个步数上，对所有粒子求  $x$  的平方的平均，得到均方位移
16 msd = np.mean(positions**2, axis=0)
17
18 step_index = np.arange(1, n_steps + 1)
19 print(f"第 {n_steps} 步的均方位移 = {msd[-1]:.1f}, 理论值 =
      {n_steps}")
20
21 plt.plot(step_index, msd, 'o', markersize=3, label='模拟的均方位移')
22 plt.plot(step_index, step_index, '-', label=r'理论  $\langle x^2 \rangle = N$ ')
23 plt.xlabel('步数 N')
24 plt.ylabel(r'均方位移  $\langle x^2 \rangle$ ')
25 plt.legend()
26 plt.tight_layout()
27 plt.show()

```

跑完会看到圆点几乎贴着那条过原点、斜率为 1 的直线。散点的抖动随步数微增（越往后单个粒子位置方差越大，统计平均的起伏也越明显），但整体线性关系一目了然。这条直线就是扩散的指纹：均方位移正比于步数，也就是正比于时间。把粒子数 `n_particles` 从两万改成两百，圆点会散得乱七八糟，粒子太少，统计平均不可靠。这又回到了本章的主旋律：蒙特卡罗的结论，必须建立在足够大的样本之上。

本节要点

单个粒子的轨迹是随机的，但大量粒子的统计规律是确定的。 $\langle x^2 \rangle = N$ 这条直线，就是扩散从微观随机中涌现出的确定规律，把粒子数从两万减到两百，散点就乱得不成

样子。

12.6 AI 协作实录：识破小样本的收敛假象

下面把本章最该警惕的陷阱演示一遍。你可以让 AI 替你写估 π 的代码，但你必须能审出它在收敛性上偷的懒。

AI 协作实录 · 小样本下的收敛假象

我对 AI 说：

你是一名熟悉大一普物和数值方法的物理建模助手。请用蒙特卡罗方法写一段 Python，在单位正方形里撒随机点来估计圆周率，给出估计值，并说明结果好不好。

AI 给我：

AI 返回了一段撒点统计的代码，逻辑没毛病，但它把样本数定成了

$$N = 100$$

跑出来打印 `pi 估计 = 3.10000`，然后 AI 下结论说：“估计值 3.1 和真值 3.14159 很接近，方法有效，结果符合得很好。”

我的审计：

我没有被这个看着还行的 3.1 说服。蒙特卡罗结果只有一个判据：收敛了没有。一百个点得出 3.1，可能纯属这次随机的运气，换个种子也许就跳到 2.9 或 3.3。要拆穿它，办法是把样本数扫大几个数量级： N 取 100、 10^4 、 10^6 、 10^8 ，逐个跑，看估计值是不是稳稳朝 3.14159 靠拢、波动是不是按 $1/\sqrt{N}$ 缩小。结果很清楚： $N = 100$ 时估计值在 2.9 到 3.3 之间乱跳，换种子就大不同； $N = 10^4$ 时稳定在 3.13 到 3.15； $N = 10^6$ 时贴到 3.1415 附近，每次重跑都稳。样本大了一万倍，误差约小了一百倍，恰好是 $1/\sqrt{N}$ 的节奏。AI 那句“符合得很好”是： $N = 100$ 的 3.1 不是方法准，是骰子恰好没歪太多。真正能写进结论的是：在 $N = 10^6$ 下估计值收敛到 3.1416，误差随 N 按 $1/\sqrt{N}$ 下降。

这段实录的要害是：AI 极容易把一次小样本的偶然命中，包装成“方法有效”。它报的数字往往不假，假的是“已经够准了”的判断。你是项目主理人，职责就是逼它把 N 扫大，画出收敛趋势，用 $1/\sqrt{N}$ 这把尺子量，而不是看一眼 3.1 就点头。

AI 陷阱预警 · 小样本的收敛假象与随机种子带来的不可复现

蒙特卡罗代码有两个坑常被 AI 忽略，却足以让结论站不住。第一个：收敛假象。样本太少时，估计值碰巧接近真值，看着“对”，实则全靠运气，换种子就翻脸。任何蒙特卡罗结果必须配一条收敛曲线，证明 N 扫大时确实在向真值靠拢、波动在按 $1/\sqrt{N}$ 缩小。没有收敛曲线，那个数字就不能信。第二个坑：不可复现。随机数默认每次运行都不一样，不固定随机种子，你今天跑的图明天就复现不了，更没法让别人核对。养成习惯：用到随机数，开头就用 `np.random.default_rng(seed)` 固定种子，把这个种子写进报告，让人人都能跑出跟你一模一样的结果。

本章小结

这一章认识了蒙特卡罗方法：把确定的量翻译成“撒点看比例”或“撒点求平均”，用大量样本去估计。撒点法估出了圆周率，平均值法算出了定积分 $\int_0^1 \sqrt{1-x^2} dx = \pi/4$ ，大量粒子的随机行走重现了扩散的微观图像并验证了 $\langle x^2 \rangle = N$ 。核心审计点只有一条：蒙特卡罗结果必须看它收敛了没有。误差按 $1/\sqrt{N}$ 缩小，小样本下碰巧接近真值的漂亮数字是收敛假象，必须把 N 扫大几个数量级、画出收敛曲线。随机种子务必固定，否则结果不可复现。

“数值方法”部分到这里画上句号。从 Euler 法到蒙特卡罗，工具攒下了，但工具会不等于结果可信。下一部分把这些零散的体检动作系统化，专门谈如何验证与审计 AI 辅助的建模结果。

思考题

1. 我们用 $x^2 + y^2 \leq 1$ 判断点是否落在四分之一圆内，从而估计 π 。现在改用一个完整的圆：在边长为 2、中心在原点的正方形（四角为 $(\pm 1, \pm 1)$ ）里撒点，点落进半径为 1 的整圆内的比例应该是多少？由此写出 π 的估计公式。和单位正方形里撒四分之一圆的做法相比，两种方案都是估计同一个 π ，你觉得哪种每个点提供的信息更多？为什么？
2. 有同学用 $N = 100$ 估 π ，跑了一次得到 3.12，很满意。你想说服他这个结果不可信。不增加样本数，只做一个最简单的实验就能动摇他的信心。这个实验是什么？做完之后预期看到什么现象？说明了什么？
3. 在随机行走里，把规则改成“每步以 0.6 的概率向右、0.4 的概率向左”，左右不再对称。这时位置的平均值 $\langle x \rangle$ 还会等于零吗？走 N 步后 $\langle x \rangle$ 大约是多少（用 N 表示）？这个改动对应的物理图像是粒子在扩散的同时还被一股恒定的力往一个方向推，这种现象有个名字，你能想到叫什么吗？

Trust but Verify: 验证与审计

13 五项验证清单 123

- 13.1 为什么需要一份固定的清单
- 13.2 第一项：量纲检查
- 13.3 第二项：解析极限
- 13.4 第三项：守恒律或单调性
- 13.5 第四项：参数扫描
- 13.6 第五项：与文献或参考值对比
- 13.7 把清单落成工件：checklist.md
- 13.8 综合演示：给一个阻尼振子工件跑完五项
- 13.9 AI 协作实录：表面通过却被第三项推翻

14 AI 输出审计：十类错误模式 131

- 14.1 三类速通：六十秒就能识别的基础错误
- 14.2 七类进阶：症状、识别、修复
- 14.3 完整审计流程演示：二十分钟拆一段错代码

15 文献核验：防止 AI 编造引用 137

- 15.1 AI 为什么会编造文献
- 15.2 三方核验：DOI、arXiv、学术搜索引擎
- 15.3 课程红线：未核验的引用不计入有效文献
- 15.4 AI 协作实录：逐条反查一份文献清单
- 15.5 一套负责任的文献 workflow



13. 五项验证清单

读到这一章，你已经学会了好几种给数值结果“挑刺”的动作：第二章用量纲分析判断一个公式有没有可能对，前面几章用解析极限、守恒律、参数扫描分别揪出过 AI 代码里的毛病。单独看都很朴素，可是当你真的为一个完整项目负责，要在答辩台上说出“这个结果可信”四个字时，单靠零散的、临时想起来的检查远远不够。这一章做一件事：把这些动作收拢成一份固定的、可执行的清单，让验证不靠灵感，变成每个项目都跑一遍的标准流程。这是全书最核心的工件，也是第三部分“验证与审计”的开篇。

本章地图

小节	这一节解决什么	关键方法 / 产物
13.1	建立固定的验证纪律	防止临场漏项的标准流程
13.2	用单位自治性淘汰明显错误的公式	量纲运算
13.3	在已知答案的极端情形下对答案	参数取极值，与解析解对照
13.4	用物理定律检验数值演化是否可靠	监控守恒量随时间的变化曲线
13.5	扰动关键参数看结果是否稳定	逐参数加减 10% 重跑
13.6	用外部已知数字独立检验结果	算偏差并解释来源
13.7	将五项验证固定为可交付的工件	checklist.md 物理对账单
13.8	用一个阻尼振子完整跑通五项	示范案例
13.9	演示漂亮结果如何被守恒律一票否决	实录分析

13.1 为什么需要一份固定的清单

先说一个让人不舒服的事实：AI 给出的结果，往往看起来就是对的。图画得漂亮，坐标轴标得齐整，拟合的 R^2 高到 0.99，曲线光滑得挑不出毛病。但“看起来对”和“真的对”完全不同。一条能量在无阻尼系统里悄悄漂移的曲线，画出来照样光滑；一个单位搞错了的数值，照样能塞进一张体面的图里；一个在你没注意的参数区间会发散的模型，在你恰好选的那组参数下可能温柔得没事人一样。人眼对“光滑”和“专业”的信任，恰恰是 AI 时代最容易被利用的弱点。

那换个思路：不靠人眼，不靠当时灵光一现想到该查什么，每次照着同一份单子，一项一项过。固定清单的价值不在于哪一项特别高明，而在于它不会漏项。临场时，人往往只想得起来验证那些“容易验证”的东西，再看一眼图，再核对一下输入参数，却恰恰跳过了那个真正能推翻结论的判据。一份写死了五项的清单，逼着你每一项都交代清楚，包括那些你本来想偷懒绕过去的。

Definition 13.1.1 : 五项验证清单. 五项验证清单是本课程要求每个项目都执行并留底的标准物理对账流程，固定包含五项：量纲检查、解析极限、守恒律或单调性、参数扫描、与文献或参考值对比。这五项内容是固定的，是课程的核心纪律；但每一项具体怎么填、用什么格式记录，由项目主理人自定。清单不在乎你打了几个勾，它要确保每一个能推翻结论的物理判据都被显式地查过一遍。

本节要点

一份固定的清单不靠某一项特别高明，靠的是它不会让你漏掉那个最可能推翻结论的判据。人眼会被光滑的曲线和整齐的坐标轴说服，物理不会。

把这份清单读熟，让它成为你交付任何数值结果之前的条件反射。下面逐项展开，每一项都讲清原理、配一个具体例子，以及代码里怎么真的把它执行掉。

13.2 第一项：量纲检查

量纲检查是五项里最便宜、也最该第一个做的一项。原理简单到近乎粗暴：任何物理公式，等号两边的量纲必须严丝合缝一致；任何写进报告的数值结果，必须带着单位。量纲不一致的公式，不用算第二步就可以判死刑，它根本不可能描述一个真实的物理量。

举一个例子：单摆小角度周期 $T = 2\pi\sqrt{L/g}$ ， L 是摆长， g 是重力加速度。验一下量纲： L 量纲是米， g 量纲是米每二次方秒，相除约掉米剩下二次方秒，再开平方是秒。 2π 是纯数没有量纲，所以右边整体量纲是秒，跟左边周期 T 完全一致，量纲通过。反过来，如果 AI 手滑写成 $T = 2\pi\sqrt{g/L}$ ，右边量纲是每秒，那描述的就不是周期而是频率，量纲一看就露馅，数都不用代。

在代码里执行量纲检查，最直接的办法是给每个物理量旁边用注释钉死单位，并全程坚持一套统一的单位制（这门课一律用 SI）。写下 $g = 9.8 \# \text{重力加速度, 单位 m/s}^2$ 时，你其实是在强迫自己每一步都对单位负责。更进一步，对关键的中问量，心里（或纸上）做一遍量纲推导，确认这个变量算出来的量纲就是它名字声称的那个。AI 很会写出语

法漂亮的公式，但对量纲的把关并不可靠，量纲恰恰是物理学家最不该出错、也最该亲自负责的地方。

本节要点

量纲是物理公式的第一道安检：等号两边量纲不一致，公式就根本不可能描述那个物理量。这一关代价几乎为零，却能把最明显的一类错误挡在门外。

13.3 第二项：解析极限

第二项是让代码去逼近一个你早已知道答案的极端情形。原理在于：一个正确的数值模型，在参数取到极限值时，必须退化成一个有解析答案、或者物理图像极其清楚的简单情形。如果极限下都回不到已知答案，一般情形下就更不可信。

这一项在前几章反复出现过，只是没有统一命名。受阻力的抛体运动，阻力系数调到零，轨迹必须回到无阻力的标准抛物线；弹簧加阻尼的振子，阻尼调到零，能量必须守恒、振幅不再衰减；单摆的大角度数值解，初始振幅趋近于零，周期必须回到小角度公式 $T = 2\pi\sqrt{L/g}$ 给出的常数。这些都是解析极限：手里攥着一个已知的标准答案，让代码在极限参数下去对它。

在代码里执行解析极限，做法是把控制极限的那个参数显式设成极限值（或非常接近极限的值），重跑一次，再把数值输出和已知的解析答案逐点对照。比如阻力系数设成 $b = 0.0$ 跑一遍，然后把数值轨迹和解析抛物线 $x = v_0 t$ 、 $y = v_0 t - \frac{1}{2}gt^2$ 画在同一张图上，看是不是重合到肉眼分不开。注意一个细节：解析极限既检验物理建模对不对，又顺带检验了数值方法的收敛性，只有步长足够小，数值解才会真的贴上解析解。极限下两条曲线分得开，问题要么出在物理，要么出在步长，两种都该去查。

本节要点

一个正确的数值模型，在参数取到极端时，必须退化回你已经知道的标准答案。极限下两条曲线拉开距离，问题要么出在物理建模，要么出在步长，两种都值得查。

13.4 第三项：守恒律或单调性

第三项往往是五项里最有杀伤力的一项，因为它检验的是物理定律本身，而不只是某个特例。原理：一个封闭系统里，该守恒的量必须守恒，该耗散的量必须单调变化，二者都不允许出现物理上无法解释的趋势。能量、动量、电荷、概率，这些守恒量在数值演化里的漂移，是判断一个模拟到底不可信的最硬判据。

守恒律检查

最自然的例子就是上一部分讲过的振子。无阻尼弹簧振子的机械能 $E = \frac{1}{2}mv^2 + \frac{1}{2}kx^2$ 在理想情况下应当是一条水平直线，一丝一毫都不该变。但用显式 Euler 推它，会发现能量随时间缓慢地、单调地往上爬，越演化越大。这正是显式 Euler 在振荡问题上臭名昭著的能量漂移。这个漂移不是物理，是数值方法的固有缺陷，告诉你：要么换更合适的积分器（比如半隐式 Euler 或辛积分器），要么把步长压得更小。反过来，有阻尼的振子机械能就不该守恒了，它应当单调下降，阻力一直在做负功把能量抽走。数值解里能量若出现明

显的、持续的回升，就说明结果不可信，需要去查原因：可能是代码或符号写错了，也可能和上面无阻尼那个例子一样，是显式 Euler 这类方法的人为漂移在作怪。

在代码里执行这一项，做法是在演化循环里，每一步顺手把该守恒（或该单调）的那个量算出来存进数组，跑完专门画一张它随时间变化的曲线。守恒量就看那条线平不平，单调量就看它是不是一路朝着该去的方向走、有没有不该出现的回头。这张“守恒量随时间”的图，常常比主结果的图更能暴露问题。主结果可能因为漂移很慢而在短时间内看不出毛病，守恒量的曲线却会把慢性漂移放大到一眼可辨。

本节要点

守恒律是五项里最有杀伤力的一项。无阻尼系统的能量若在数值解里缓慢爬升，不是物理，是数值方法在偷渡能量。画一张守恒量随时间的图，常常比主结果的图更早暴露问题。

13.5 第四项：参数扫描

第四项是把模型在参数空间里“晃一晃”，看它稳不稳、合不合物理。原理：一个可信的物理模型，关键参数小幅扰动时，结果不应该剧烈跳变（除非参数恰好在物理临界点附近），而且结果随参数变化的趋势必须符合物理直觉。输入参数稍微动一下结果就面目全非的模型，要么数值不稳定，要么物理上根本站不住。

举例来说，你拟合出一个阻尼振子的阻尼系数，就该问：把这个系数上下各动 10%，振幅衰减的快慢怎么变？物理上阻尼越大衰减越快，这个趋势必须保持；如果只动 10% 结果就天翻地覆，这个拟合多半不可靠。再比如温度扩散模拟，热导率增大一成，热量扩散应当变快而不是变慢，趋势对了才说明模型抓住了物理。这一项和前面的拟合章节呼应很紧：拟合给你一组参数，参数扫描替这组参数做体检，看它经不经得起轻微扰动。

在代码里执行参数扫描，最常规的做法是把每个关键参数依次乘以 0.9 和 1.1，每次只动一个、其余不变，各重跑一遍，把结果（某个关键输出值或整条曲线）并排画出来比较。既要看绝对的稳定性，结果有没有剧烈跳变，也要看变化的方向，趋势合不合物理。把这五六次扫描的结果叠在一张图上，一眼能看出模型是温和地随参数移动，还是莫名其妙地乱蹦。

本节要点

把关键参数上下各动 10%，看结果变不变、变得合不合物理直觉。拟合给出参数，参数扫描替这组参数做体检。

13.6 第五项：与文献或参考值对比

最后一项，也是最能让人信服的一项：把你的结果摆到一个外部的、独立于你这套代码的已知数字旁边去比。原理很朴素，如果世界上存在一个公认参考值，无论来自教科书、实验测量还是解析公式，你的结果就该靠近它；靠不近，你就得能解释清楚为什么差、差多少、差得合不合理。

这一项的例子俯拾即是。单摆周期的小角度公式给你一个解析参考值；自由落体的终端速度有公认的实验数据；地球绕日轨道周期是一年，人人都知道的参考；许多标准物理常数和反应截面能在手册或文献里查到。关键不在于差值要恰好为零，真实的数值计算几乎不可能和参考值分毫不差，而在于差的来源你说得清楚：是步长带来的截断误差，是模型里忽略了某个次要效应，还是单位换算时漏了一个因子。说得清楚的差可以接受，说不清楚的差就是悬在头上的问号。

在代码里执行这一项，做法是在程序末尾显式打印出关键结果和参考值，相对偏差一并算出来。比如算完单摆周期后打印 `print(f" 数值周期 = {T_num:.4f} s, 参考值 = {T_ref:.4f} s, 相对偏差 = {(T_num-T_ref)/T_ref*100:.2f}%")`。把偏差白纸黑字摆出来，比嘴上说一句“差不多”负责得多。偏差大得出乎意料的话，它本身就是下一个要调查的线索。

本节要点

世界上存在公认的参考值时，你的结果就该靠近它。靠不近不可怕，怕的是说不清楚为什么靠不近。把偏差白纸黑字算出来，比一句“差不多”负责任得多。

13.7 把清单落成工件：checklist.md

讲完了五项原理，接下来要把它变成一件能交付的东西。期末项目里，除了可运行的代码和工件本身，你还要交一份 `checklist.md`，逐项写下你的项目在这五项上的通过情况。这份文件是你证明“这个结果我验过、我敢负责”的物理对账单，不是事后补的形式作业。

这里强调一条贯穿全书的立场：五项内容是固定的，这是课程的核心纪律，不可删减、不可跳项；但每一项具体怎么填、用什么结构组织、配不配图、写多详细，全由你自己决定。这门课从不发填空模板让人照抄。有人喜欢每项写成一段话，把物理推理讲透；有人喜欢配一张守恒量曲线图加一句结论；有人喜欢把参数扫描的几个数字列成对照。这些都好，只要五项齐全、每项背后的物理判据交代清楚就行。我们反对的是模板化的、千篇一律的打勾，那种做法恰恰会让人忘掉每一项背后真正要回答的物理问题。你的 `checklist.md` 应该长得像你自己的思考，不像一张谁都能填的表格。

本节要点

五项内容固定是纪律，每项怎么填是你的自由。好的 `checklist.md` 读起来像你自己的思考，不像一张谁都能填的表格。

物理对账单

13.8 综合演示：给一个阻尼振子工件跑完五项

光说不练没用。拿一个具体项目，把五项完整走一遍。设想你们小组做了一个阻尼振子交互工件：用户拖动滑块调质量 m 、刚度 k 、阻尼系数 b ，工件实时画出位置随时间变化的曲线。AI 帮你写好了代码，图也很漂亮。现在，交付前给它跑完五项验证。

验证清单 · 阻尼振子交互工件

量纲：运动方程 $m\ddot{x} + b\dot{x} + kx = 0$ 三项的量纲都该是力（牛顿）。 $m\ddot{x}$ 是千克乘米每二次方秒等于牛顿； kx 中刚度 k 的单位牛每米乘位移米得牛顿； $b\dot{x}$ 中阻尼系数 b 的单位牛秒每米乘速度米每秒也得牛顿。三项量纲一致，代码里各变量的单位注释与之吻合，通过。

解析极限：把阻尼 b 设成零重跑，位置曲线应退化成不衰减的简谐振动，频率为 $\sqrt{k/m}$ 。数值结果给出的振荡频率与 $\sqrt{k/m}$ 在三位有效数字内相符，振幅在十个周期内基本不衰减，通过。

守恒律或单调性：无阻尼时机械能 $E = \frac{1}{2}m\dot{x}^2 + \frac{1}{2}kx^2$ 应守恒。用原来的显式 Euler 跑，能量缓慢单调上爬，十个周期后涨了约 4%，这是显式 Euler 的能量漂移。换成半隐式 Euler 后能量曲线压平到 0.1% 以内。有阻尼时能量应单调下降，数值解确实一路递减无回升，通过。

参数扫描：把阻尼 b 分别取标称值的 0.9 倍和 1.1 倍重跑，衰减包络随 b 增大而变陡，趋势合乎物理且无剧烈跳变；对 k 同样扫描，振荡频率随 k 增大而升高，符合 $\sqrt{k/m}$ 的预期，通过。

参考值：取一组临界阻尼参数 $b = 2\sqrt{mk}$ ，理论上系统应不振荡、最快回到平衡而不过冲。数值解恰好在这组参数附近从振荡过渡到非振荡，与临界阻尼的解析判据一致，通过。

五项全部交代清楚，你才有底气在答辩台上说这个工件可信。注意上面每一项都动了手：设一个具体参数，跑一次，盯住一个具体判据，不是在图表前扫一眼就下结论。这才是验证该有的样子。

13.9 AI 协作实录：表面通过却被第三项推翻

下面这段实录，演示一个最值得警惕的场景：AI 给出的结果在最显眼的指标上全部漂亮，但当你认真跑到守恒律那一项时，它崩了。

AI 协作实录 · 看起来完美的振子，能量却在漂

我对 AI 说：

你是一名熟悉大一普物和数值方法的物理建模助手。请用 Python 模拟一个无阻尼弹簧振子，质量 1 千克、刚度 10 牛每米、初始位移 0.2 米、初速为零，画出位置随时间的曲线，模拟 20 秒，所有量用 SI 单位并在注释里标明单位。

AI 给我：

AI 返回了一段结构清晰的代码，用显式 Euler 法推进，循环和绘图都正确。位置曲线画出来是一条标准的、振幅看起来稳定的余弦曲线，光滑漂亮，频率也对得上 $\sqrt{k/m} \approx 3.16$ 弧度每秒。只看这张位置图，几乎无可挑剔。

我的审计：

位置图好看≠物理对。我没有停在第一项，而是按清单往下走。量纲那关 AI 的注释都对，过了；解析极限本来就是最简单的无阻尼情形，频率也对得上，看似也过了。但到了第三项守恒律，问题暴露了：我在循环里加了一行，把机械能 $E = \frac{1}{2}m\dot{x}^2 + \frac{1}{2}kx^2$ 存进数组，单独画出来一看，能量不是水平线，而是一条单调上爬的曲线，二十秒里涨了将近 15%。无阻尼系统的能量凭空多出百分之十五，这在物理上绝无可能。问题正是显式 Euler 在振荡问题上的能量漂移：它会系统性地往系统里注入能量。只是因为漂移慢，位置曲线的振幅在前几个周期里看不太出来，把人骗过去了。我把积分方法换成半隐式 Euler 重跑，能量曲线立刻压平到 0.2% 以内，位置曲线这才真正可信。”看起来对”的结论，被第三项一票否决。

这段实录的要害：如果我偷懒，只验证了量纲和解析极限这两项相对容易、又恰好都过得了的检查，就会把一个能量漂移的假结果当成物理结论交出去。真正能推翻它的，是那个我本来可能想跳过的守恒律检查。

本能 · Trust but Verify

看到任何数值结果，无论是 AI 写的还是自己写的，都不靠”看起来对不对”下结论，而是机械地把五项验证清单从头跑一遍：量纲、解析极限、守恒律或单调性、参数扫描、参考值对比。五项清单就是这条反射弧的标准动作，它把”我觉得没问题”换成了”我逐项查过、有据可依”。

AI 陷阱预警 · 只验证了能验证的，跳过了真正能推翻结论的那一项

五项验证里最危险的失误，从来不是某一项做得不细。真正的危险是你干脆跳过了那个最可能让结论翻车的判据。人天然倾向于做容易的检查，再看一眼图、再核对一遍输入参数，却对那个真正硬的判据视而不见，因为心里隐约知道它可能会推翻自己辛苦做出来的结果。守恒律往往就是这个被回避的判据。记住：验证的目的不是凑齐五个对勾，而是主动去找那个能证明你错的判据，直面它。如果五项里有一项你下意识想绕过去，那一项八成正是你最该认真跑的。

本章小结

把前面几章零散的检查动作收拢成一份固定的五项验证清单：量纲检查、解析极限、守恒律或单调性、参数扫描、与文献或参考值对比。逐项讲清了物理原理、具体例子，以及代码里怎么执行它。五项内容是固定的课程纪律，每项怎么填、用什么格式记录由你自定，最终落成项目里的 `checklist.md` 工件。阻尼振子工件完整演示了五项怎么跑，AI 实录看到：量纲和解析极限都漂亮过关的结果，可能被守恒律一项直接推翻。最危险的是跳过了那个真正能推翻结论的判据，不是某项查得不细。

下一章把”找 AI 的错”展开成十类常见错误模式，从单位错、符号错，到求解器误用、虚构引用、能量不守恒，给你一张给 AI 挑错的地图。

思考题

1. 五项验证清单里，量纲检查是最便宜的一项，却抓不住所有错误。请你设计一个具体的物理代码错误：它能完美通过量纲检查，却会被守恒律或解析极限抓住。再反过来，能不能设计一个能通过守恒律检查、却被量纲检查一眼识破的错误？通过两个例子，说说为什么五项必须都跑、不能只挑一两项。
2. 课程要求 `checklist.md` 的五项内容固定，但格式由你自定。有同学主张干脆发一个统一模板让大家填空，效率更高。请你站在“反模板”的立场，论证为什么固定格式的填空表反而可能损害验证的质量；再站在对立面，说说统一模板有没有它合理的地方。你心目中一份好的 `checklist.md` 应该长什么样？
3. 第五项“与文献或参考值对比”要求你能解释结果和参考值之间的差。假设你模拟单摆大角度（初始振幅 60 度）的周期，发现数值结果比小角度公式 $T = 2\pi\sqrt{L/g}$ 给出的参考值大了约 7%。这个偏差是 bug，还是物理？请你说清楚该如何判断，以及如果它是物理，你会去哪里找一个更合适的参考值来佐证自己的结论。



14. AI 输出审计：十类错误模式

你已经会让 AI 帮你解微分方程、跑蒙特卡罗、做拟合，也见过它在时间常数、步长、采样数这些地方翻车。审计 AI 的输出，是项目主理人最核心、也最不能外包的技能：你可以让 AI 写代码、推公式、查资料，但”这段输出到底对不对”这个判断，必须由你来下。前面几章我们是遇到一个错抓一个错，零散地练。这一章要把”找 AI 的错”系统化，归纳成。拿到一段陌生的 AI 输出时，不必从头读到尾逐字怀疑，而是顺着这十类模式走一遍，像过一张，二十分钟内就能有章可循地把可疑点定位、命名、修复。

十类错误模式
检索表

为什么要命名而不是凭感觉？因为在科学训练里，能为一个现象起名字，意味着你真正理解了它，下次再遇到一眼就能认出来。把”这里好像不对”升级成”这是第四类，求解器误用”，你才能立刻调动对应的识别动作和修复路径，而不是干瞪眼。这一章给的十个名字，就是你时的肌肉记忆。

AI 审计

本章地图

小节	这一节解决什么	关键方法 / 产物
14.1	三类基础错误，六十秒就能识别	量纲检查、因果方向、常识粗估
14.2	七类进阶错误，需要数值方法基础	症状、识别、修复三件套
14.3	二十分钟审计一段错代码	量纲、极限、守恒、关键行四道关

14.1 三类速通：六十秒就能识别的基础错误

最先要练熟的是三类最基础错误，不需要读懂每一行代码，往往瞄一眼结论就能识破。这三类是在第一章和第二章就埋下的伏笔，也是你从第一天交审计日志就该有的词汇：单

位错、符号错、数量级错。叫“速通”，因为每一类都能在六十秒内用一个物理本能动作抓出来。

14.1.1 第一类，单位错：量纲不一致或者忘标单位

单位错最常见，也最容易抓。症状是：某个量的量纲根本对不上，或者通篇没标单位，让人无从判断。识别动作就是量纲检查，把表达式两边的量纲各自约一遍，看等不等。

举个六十秒的例子。AI 帮你算单摆周期，写出 $T = 2\pi\sqrt{Lg}$ 。先别管数值，先看量纲： L 是米， g 是米每二次方秒，两者相乘是平方米每二次方秒，开根号是米每秒，那是速度的量纲，绝不可能是时间。正确的应当是 $T = 2\pi\sqrt{L/g}$ ，长度除以加速度，约下来才是二次方秒，开根号正好是秒。量纲对不上，结论立刻清楚，根本不用代任何数字。这一类的修复路径永远是：每个物理量的单位标清楚，让量纲自洽，错误自然浮出水面。

14.1.2 第二类，符号错：正负号反了或者方向搞歪了

符号错的症状是数值大小对，正负号或矢量方向却反了，结果物理上荒谬。识别靠物理直觉：这个量该增还是该减，该指向哪边。

比如 AI 写下落物体在线性阻力下的方程 $m dv/dt = mg + bv$ ，阻力项写成了加号。物理上阻力总跟运动方向相反：物体往下掉，阻力必须往上拽，所以该是 $mg - bv$ 。用加号跑出来，速度不收敛到终端速度，反而越掉越快、指数式发散，显然不对。再比如 RC 放电写成 $dV/dt = +V/\tau$ ，丢了负号，电压不衰减反而爆炸增长。符号错的修复路径是：从物理的因果方向重新审一遍每一项的正负，问“这一项是让系统增还是减”。

14.1.3 第三类，数量级错：结果差了几个数量级却没察觉

数量级错的症状是数字看着正常，可跟你对这个物理量的常识一比，差了好几个数量级。识别靠的是“费米估算”式的常识：心里得对这个量大概多大有个谱。

比如让 AI 估算一间房里空气的质量，它返回 0.0012 千克。普通教室几十立方米，空气密度大约 1.2 千克每立方米，总质量应该是几十千克的量级，零头零零一二千克差了大约五个数量级，多半是把密度当成了质量，漏乘了体积。又比如算电子在氢原子中的轨道速度，AI 给出 3×10^8 米每秒，正好是光速。电子基态速度其实只有光速约百分之一，差了两个数量级，往往是少代了一个精细结构常数。数量级错的修复路径是：用纸笔粗估，定出结果该落在哪个数量级，再拿去对账。这一类尤其阴险，因为差几个数量级的结果常常不会报错，程序照跑、图照画，只有你心里那把“常识尺”能拦住。

本节要点

三类基础错误，每一类都能在六十秒内用一个物理本能动作抓出来：单位错靠量纲约分，符号错靠因果方向，数量级错靠常识粗估。这三类是你审计 AI 输出的第一反应，也是从第一章起就该有的条件反射。

14.2 七类进阶：症状、识别、修复

三类速通练熟之后，剩下七类要更深一层。它们往往藏在代码或推导内部，需要懂一点数值方法和物理近似才能识别。下面逐类给出“症状、怎么识别、修复路径”三件套。

14.2.1 第四类，求解器误用：步长不当、用错积分器

症状是数值结果发散、振荡，或者明明物理上该守恒的量随时间漂移。根源通常是步长太大，或者在不该用显式方法的地方用了显式方法。第八章已经见过，显式 Euler 在 RC 放电里步长超过 2τ 就会来回振荡、越荡越大；刚性问题，系统里同时有快慢两种时间尺度，用显式法尤其容易炸。识别的标准动作是收敛性检查：步长减半再跑一次，看结果不变。减半之后结果大变样，说明原来的步长根本没收敛。修复路径：减小步长直到结果稳定，或者换更稳的积分器，比如隐式方法或带自适应步长的库函数。

14.2.2 第五类，收敛假象：把没收敛当成收敛了

症状是一条曲线或一个数字看起来已经稳定，其实采样太少、步长太大，远没到真值。这一类呼应第十二章蒙特卡罗：抛硬币估概率，抛一百次得到 0.53，看着挺接近 0.5，可统计涨落还有百分之几，宣布“收敛到 0.53”就上当了。识别动作是把样本量或分辨率成倍加大，看结果是否还在显著移动；蒙特卡罗里更要看误差棒是否随 $1/\sqrt{N}$ 缩小。修复路径：加大采样，画出“结果对样本量”的收敛曲线，确认真的进入平台期，而不是还在缓慢爬升的半路上。

14.2.3 第六类，虚假引用：编造不存在的文献

症状是 AI 给出一篇格式完美的参考文献，作者、期刊、卷页、年份一应俱全，但那篇文章根本不存在，或者作者卷页对不上真实记录。大语言模型擅长生成像模像样的引用串，却无力保证它指向真实文献。识别的唯一可靠动作是去权威数据库核对，这正是下一章要专门展开的内容，这里先点出它是十类里独立的一类。修复路径：凡是 AI 给的引用，一条都不能直接信，必须逐条到真实数据库查到 DOI 或原文才算数。

14.2.4 第七类，混淆模型：把两个物理模型的公式张冠李戴

症状是公式本身没写错，但它属于另一个物理模型，被错误套到了当前问题上。AI 见过海量公式，有时会把相邻领域的结论混在一起。比如算理想气体内能用了范德瓦尔斯气体的修正项，算低速运动却套了相对论动量公式，或者把简谐振子的周期公式用到大幅单摆上。识别要回到模型的适用前提：这条公式是在什么假设下推出来的，当前问题满不满足那些假设。修复路径：先明确写下这个问题用的是哪个模型、成立条件是什么，再逐项核对 AI 用的每个公式是否都隶属于它。

14.2.5 第八类，过拟合掩盖：参数多到能拟合噪声

症状是残差被刷得极漂亮、 R^2 高得离谱，可模型在新数据上一塌糊涂。这一类呼应第十一章拟合：给模型的自由参数多到和数据点数量相当，它就能把每个噪声点都穿过去，

残差当然小，但学到的是噪声不是规律。识别动作是看参数个数与数据点数的比例，用一部分数据训练、另一部分检验，或者看拟合出的参数有没有物理意义、误差棒是不是大到离谱。修复路径：减少参数、用更简单的物理模型，或引入正则化，宁可残差大一点，也要换来在新数据上站得住的预测能力。

14.2.6 第九类，误用近似：在近似失效的区间继续用近似

症状是结果在某个范围内对，超出那个范围就系统性偏离，却没人喊停。最经典的例子是小角度近似：单摆的 $\sin \theta \approx \theta$ 只在角度很小时成立，到了大振幅（比如几十度），周期会随振幅明显增大，再用 $T = 2\pi\sqrt{L/g}$ 就会低估周期。类似地，泰勒展开只取头一两项在自变量大时会崩，非相对论近似在速度接近光速时失效。识别要问：参数还落在这个近似成立的区间里吗？修复路径：写下近似的有效范围，检查当前工况是否越界，越界了就换成不做近似的完整表达式或数值求解。

14.2.7 第十类，丢边界条件或能量不守恒：初值边界设错、守恒律被破坏

症状是方程本身对，初始条件或边界条件设错了，或者本该守恒的量（能量、动量、电荷）随时间漂移。第九章我们用能量守恒给弹簧振子的数值解做过一场严格审判：显式 Euler 解出来的振子，总能量随时间缓慢增长，振幅越来越大，这是积分器破坏了能量守恒，不是物理。识别动作是把守恒量当成独立的监控指标，每步算一次，看是不是真的不变；同时核对初值和边界条件是否跟物理情形一致。修复路径：换用保结构的积分器（比如辛算法），或者收紧步长，始终用守恒律作为结果可信度的硬指标。

本节要点

七类进阶错误各有固定的症状、识别动作和修复路径。它们的共同点：都藏在代码或推导的内部，但配上对应名字之后，你下次再撞见就能立刻认出来、立刻知道怎么修。把“这里不对劲”升级成“这是第几类”，修复路径就自动跟着名字一起来了。

14.3 完整审计流程演示：二十分钟拆一段错代码

把十类模式串起来用，最好的方式是亲手走一遍。下面是一段我故意写错的 AI 输出，任务是模拟单摆、给出大振幅下的周期。这段代码至少藏了两类错误，我们顺着审计流程把它揪出来。

Listing 14.1: 一段故意写错的单摆模拟 AI 输出

```
1 import numpy as np
2
3 # 物理参数
4 L = 1.0          # 摆长，单位 米
5 g = 9.8          # 重力加速度，单位 米每二次方秒
6 theta0 = 1.2     # 初始角度，单位 弧度（约 69 度，大振幅）
7
```

```

8 # 用小角度公式直接给出周期
9 T = 2 * np.pi * np.sqrt(g / L) # 周期
10
11 # 显式 Euler 推进角度与角速度
12 dt = 0.2
13 N = 100
14 theta = np.zeros(N + 1)
15 omega = np.zeros(N + 1)
16 theta[0] = theta0
17 omega[0] = 0.0
18 for n in range(N):
19     alpha = -(g / L) * theta[n] # 角加速度
20     omega[n + 1] = omega[n] + dt * alpha
21     theta[n + 1] = theta[n] + dt * omega[n]
22
23 print("周期 T =", T, "秒")

```

现在按检索表走。第一步，量纲，六十秒。看周期那一行 $T = 2\pi\sqrt{g/L}$ ： g 是米每二次方秒， L 是米，相除是每二次方秒，开根号是每秒，那是频率的量纲不是时间。第一类单位错，同时也是第二类的近亲，分子分母写反了，正确应当是 $\sqrt{L/g}$ 。量纲一关就拦下了。

第二步，物理极限和近似的适用范围。初始角度 1.2 弧度，约六十九度，不折不扣的大振幅。但代码里角加速度写的是 $\alpha = -(g/L)\theta$ ，这是小角度近似下的线性化结果，真正的单摆方程是 $\alpha = -(g/L)\sin\theta$ 。在六十九度下 $\sin\theta \approx 0.93$ 比 $\theta = 1.2$ 小了百分之二十多，由于 $\theta > \sin\theta$ ，继续用线性式会系统性高估回复力，把振荡算得偏快、周期偏短。这是第九类误用近似：近似失效的区间还用近似。而且就算线性式成立，那个 T 公式给的也只是小振幅周期，大幅周期理应更长，代码根本没去算。

第三步，守恒律。单摆无阻尼，机械能应当守恒。这段代码用显式 Euler，第九章已经知道它会让能量缓慢增长、振幅虚假放大。顺手在循环里加一行，每步算一次总能量 $E = \frac{1}{2}mL^2\omega^2 + mgL(1 - \cos\theta)$ ，跑出来 E 一路爬升，证实了第四类求解器误用同时引发第十类能量不守恒。

第四步，读关键的几行循环逻辑，确认推进顺序。错误已经定位清楚：至少踩了第一类（周期公式量纲反了）、第九类（大振幅还用小角度近似）、第十类（显式 Euler 不守恒）三个坑。修复路径随之清晰：周期公式改成 $\sqrt{L/g}$ 且大振幅下不能直接用；角加速度改为完整的 $-(g/L)\sin\theta$ ；积分器换成辛算法或半隐式 Euler，把能量当监控指标；大幅周期则通过监测过零点的时间间隔从数值解里直接读出。整个过程没有逐字读代码，而是量纲、极限、守恒、关键几行，四道关顺次过，二十分钟把一段看着挺专业的输出拆得明明白白。

本能 · Trust but Verify

拿到任何一段陌生的 AI 输出，不要从第一行读到最后一行去找错，那样既慢又容易

漏。把这十类错误模式当成检索表，按”先量纲、再极限与近似、再守恒、最后才读关键几行”的顺序过一遍。能命名的错误才是你真正抓住的错误：把”这里怪怪的”升级成”这是第几类”，修复路径就自动跟着名字一起来了。

AI 陷阱预警 · 越像样的输出越要警惕

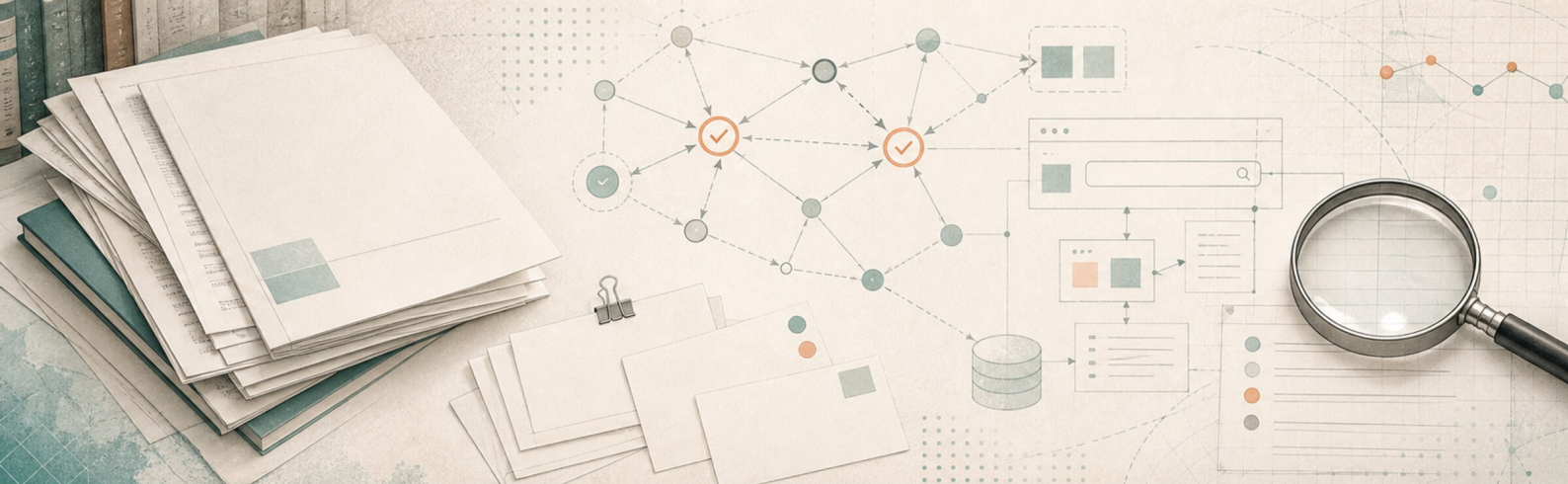
AI 的错误最爱藏在它写得最专业、最自信的那一段里。一段语法整洁、注释齐全、还煞有介事引了文献的代码，恰恰最容易让你放松警惕、直接盖橡皮图章。要记住：AI 擅长的是把东西写得像对的，而不是保证它真的对。输出越是光鲜、越是看着无懈可击，越值得多花那二十分钟，把检索表认真真过一遍。真正的坑往往就埋在那一行你觉得”这肯定没问题”的地方。

本章小结

”找 AI 的错”系统化成十类可以命名的错误模式。前三类是速通，六十秒用物理本能识破：单位错靠量纲检查，符号错靠因果方向，数量级错靠常识粗估。后七类要深一层：求解器误用靠收敛性检查，收敛假象靠加大采样，虚假引用靠数据库核对，混淆模型靠核对适用前提，过拟合掩盖靠参数与数据点之比加交叉检验，误用近似靠确认有效范围，丢边界条件或能量不守恒靠把守恒量当监控指标。一段故意写错的单摆代码演示了”量纲、极限、守恒、关键几行”四道关顺次过的二十分钟审计流程。第六类虚假引用最特殊，没法靠物理直觉或数值检查抓出来，只能去权威数据库逐条核对。下一章专门讲文献核验。

思考题

1. 本章第九类（误用近似）和第七类（混淆模型）听起来有点像，都涉及”用错了公式”。请你各举一个普物里的例子，把两者的区别说清楚：什么时候叫”在失效区间继续用一个本来对的近似”，什么时候叫”把另一个模型的公式张冠李戴”。
2. Listing 14.1 这段代码我们抓出了三类错误。假如有人只把周期公式从 $\sqrt{g/L}$ 改成了 $\sqrt{L/g}$ ，就宣布修好了，他还会得到一个错误的大振幅周期。请说明：仅凭量纲检查这一关，为什么抓不住”大振幅还用小角度近似”这个错误？要抓住它，必须动用本章审计流程里的哪一道关？
3. 你拿到一段 AI 写的蒙特卡罗代码，它抛 200 次硬币估出正面概率为 0.55，并断言”已收敛”。这属于第五类收敛假象。请设计一个不超过三步的最小检验动作，能在不重写代码的前提下判断它到底收敛没收敛，并说明你预期看到什么现象才敢推翻或者认可这个”已收敛”的结论。



15. 文献核验：防止 AI 编造引用

写报告的最后一步，往往是补上一串参考文献，证明你的物理结论不是凭空冒出来的，而是站在前人工作之上的。如今很多人会顺手让 AI 帮忙：给我列几篇相关论文。几秒后，屏幕上出现一份格式完美的清单，作者、标题、期刊、卷号、页码、年份样样齐全，连 DOI 都带上了。问题是，这份清单里很可能混着几篇根本不存在的论文。不是 AI 记错了某个页码，而是从作者到标题整条都是临时编出来的。这是大型语言模型一个著名的高发失败模式，业内称为虚构引用（hallucinated citation）。对物理学生而言，报告里只要混进一条假引用，就是学术诚信上的硬伤，再漂亮的实验和分析都会被这一条拖下水。这一章只做一件事：搞清楚 AI 为什么编造文献，学会用三方核验把假引用一条不漏地揪出来。

本章地图

小节	这一节解决什么	关键方法 / 产物
15.1	理解 AI 编造文献的统计根源	语言模型按“像不像”拼文字，不按“有没有”查数据库
15.2	拿每一条引用去现实世界反查	DOI 解析、arXiv 编号、Google Scholar 三条独立通道
15.3	立下课程红线与学术诚信边界	未经核验的引用不计入有效文献
15.4	逐条反查一份五条清单，揪出两条编造	DOI 能解析不等于引用为真
15.5	建立负责任的文献 workflow	AI 当线索提供者，进报告的引用必须亲手反查

15.1 AI 为什么会编造文献

要理解这个失败模式，先得明白语言模型是怎么生成文字的。它本质上只做一件事：根据已经写出的内容，预测下一个最该出现的词。它读过海量论文、书目和引用列表，对“一条参考文献长什么样”建立了极其精准的统计直觉，作者名后面通常跟标题，标题后面跟期刊名，期刊名后面跟卷号、页码、年份。当你让它列文献时，它做的并不是去某个数据库查出真实条目再抄给你，而是按这套“看起来像真引用”的统计规律，一个词一个词拼出来。

这就是关键：模型手里没有一个可供核对的真实文献数据库，只有“像不像”的语感。于是它能流畅地生成一条格式无懈可击、真实世界却查无此文的条目，就像一个从未读过那本书的人，凭着对书名套路的熟悉，编出一个以假乱真的书名。更麻烦的是，编出来的条日常常半真半假：作者是真实存在的物理学家，期刊是真实期刊，年份也合情合理，唯独“这几位作者在这一年这本期刊上发过这篇文章”是假的。这种真假掺半的引用，比纯粹胡编的更难识别，也更危险。

Definition 15.1.1：虚构引用。虚构引用指 AI 生成的、格式完整且看似可信，但在现实世界中并不存在的参考文献条目。它可能整条都是编造的，也可能由真实的作者、期刊、年份与一个不存在的“作者-标题-出处”对应关系拼接而成。根源在于语言模型按文本统计规律生成“像引用的文字”，而非从真实文献库中检索，所以格式的完整与真实的存在之间没有任何必然联系。

认清了这一点，你就明白为什么不能对 AI 给的文献网开一面。它写出来的引用和它写出来的代码、数值、推导一样，全是同一台统计机器的输出，没有理由单单相信这一类。

本节要点

AI 编造引用的根源不在于记性差，它手里根本就没有可供核对的真实文献库。它按“像不像真引用”的统计规律生成文字，所以能流畅编出格式完美却查无此文的条目。格式的完整与真实的存在之间，没有任何必然联系。

15.2 三方核验：DOI、arXiv、学术搜索引擎

既然格式完美不等于真实存在，唯一可靠的办法就是拿 AI 给的每一条引用去现实世界对一遍。这门课要求你掌握三条互相独立的核验通道，逐条至少用其中一条把引用钉死，最好交叉用两条。下面把每条通道怎么操作说清楚。

第一条是用 DOI。DOI 是数字对象唯一标识符，正式发表的论文几乎都有一个全球唯一的 DOI，形如 10.1103/PhysRevC.92.044616 这样的字符串。核验动作很简单：把 AI 给的 DOI 拼到 <https://doi.org/> 后面贴进浏览器，或者贴进 CrossRef 查询页，看它解析出来的是哪一篇。解析失败、提示 DOI 不存在，这条引用基本可以判死刑；能解析但跳转到的论文标题、作者跟 AI 给你的完全是两码事，说明 AI 把一个真实 DOI 安到了一篇假标题上，同样不可信。只有解析出的论文跟 AI 给的条目逐字对得上，这一关才算过。

第二条是用 arXiv ID 反查，专门针对预印本。物理学大量工作会先挂到 arXiv 上，编

号形如 2605.18661，年份月份加序号。核验时把编号填进 arxiv.org 搜索框，或直接访问 <https://arxiv.org/abs/> 加编号，看跳出来的预印本标题和作者是否与 AI 给的一致。这里有一个 AI 特别容易露馅的地方：arXiv 编号格式是死的，月份不超过十二，序号也有位数规律，AI 编造时常给出格式看着对、但根本没被分配过的编号，访问后直接报“找不到该文章”。

第三条是用 Google Scholar 或其他学术搜索引擎做整体核对。前两条靠唯一标识符，这一条靠内容自洽。把 AI 给的标题（最好加上第一作者姓氏）丢进 Google Scholar、Semantic Scholar 或领域专业数据库（物理常用 INSPIRE-HEP、ADS），看能不能搜到同一篇，再逐项核对作者列表、标题措辞、期刊名、卷号、页码、年份是否彼此自洽。真实论文的这些信息会严丝合缝对上；编造的条目往往在某一项上对不齐，比如标题搜得到但页码不符，或者这几位作者确实合作过，却从没写过这个标题。

三条通道背后是同一条铁律：任何一项对不上，这条引用就不可信，整条剔除，绝不“姑且留着”。核验的本质是判定引用到底存不存在，存在与否没有中间地带，不是在给引用挑毛病然后修补。

本能 · Trust but Verify

引用也是 AI 的输出，和它写的代码、给的数值、做的推导属于同一类东西，必须经过一样严格的验证，没有任何例外。看到一份漂亮的文献清单，第一反应不该是“真专业”，而该是“这几条我还没逐条反查过”。在你亲手用 DOI、arXiv 或学术搜索引擎把某条引用对上之前，它对你而言就处于“存疑”状态，不能进报告，更不能据它去支撑任何物理结论。

15.3 课程红线：未核验的引用不计入有效文献

把上面的纪律落到课程考核上，就是一条不容含糊的硬规定：未经 DOI、arXiv 或 Google Scholar 核验的引用，不计入有效文献。你的报告可以列十条参考文献，但如果其中三条你没亲手反查过，它们对评分而言等于不存在，甚至会扣分。

更严重的是，把 AI 给的文献直接搬进报告而不核验，属于这门课学术不端三条红线之一。三条红线分别是：不能把 AI 写的段落原样复制进报告正文而不修改、不标注；不能用 AI 编造数据而不标注；不能用 AI 给出的参考文献而不经 DOI、arXiv 或 Google Scholar 核验。违反任何一条都按学术不端处理。这条红线的执法不是靠某种“AI 检测器”，那类工具对中文科学文本误报率极高，本课不采用，而是靠老师当晚对你提交的每一条引用做 DOI 与 arXiv 反查。这是课程里唯一一项全量核验、逐条都查的检查，没有抽样侥幸的空间。

本节要点

学术诚信的红线靠你自己亲手核验撑起来，不是靠事后拿检测工具划出来的。未经反查的引用，格式再漂亮也不是有效文献。与其事后被查出假引用，不如在报告里主动写清楚哪条文献你还没找到原文。

还有一条配套规则必须知道：与其事后被查出假引用，不如在报告的“AI 使用说明”段

里主动报告灰色地带。如果某条文献是 AI 给的、你一时没找到原文，就如实写出来。主动报告不扣分，不报告而被反查发现，按红线处理。把核验的责任摊在明处，是项目主理人对自己署名的报告负责的基本姿态。

15.4 AI 协作实录：逐条反查一份文献清单

下面把核验动作完整演示一遍。作为项目主理人，你完全可以让 AI 帮你起草文献清单，但清单交到你手里之后，每一条都要过你这一关。

AI 协作实录 · 五条引用里揪出两条编造的

我对 AI 说：

你是一名熟悉数值方法与计算物理的文献助手。我在写一份关于一阶常微分方程数值解的报告，请给我列五篇相关的参考文献，覆盖 Euler 法、Runge-Kutta 法和数值稳定性，每条都给出作者、标题、出处和 DOI 或 arXiv 编号。提醒：不确定时请直接说不确定，不要编造条目。

AI 给我：

AI 返回了一份格式整齐的清单，五条都带着 DOI 或 arXiv 编号，读起来都像那么回事。其中第一、三、五条是经典数值分析著作和综述，第二条是一篇期刊论文，第四条标着一个 arXiv 预印本编号。

我的审计：

我没有凭“看着像真的”就收下，而是逐条反查。第一、三、五条：DOI 贴进 doi.org，全部正常解析，跳转后的标题、作者、年份跟清单逐字对得上，三条通过。第二条：DOI 能解析，但跳出来的论文是完全不相干的另一篇，作者和标题都对不上，AI 把一个真实 DOI 安到了编造的标题上，这一条剔除。第四条：arXiv 编号填进 arxiv.org，返回“找不到该文章”；再用标题加第一作者去 Google Scholar 搜，也搜不到任何对应工作。这个预印本编号根本没被分配过，整篇文章从不存在，同样剔除。最终五条里只有三条经得起核验，进报告的就是这三条；那两条编造的，连同它们的“完美格式”，一并删掉。

这段实录的要害：AI 即便被明确叮嘱“不确定就说不确定”，仍然编出了两条假引用，因为它没有能力区分“我记得这篇”和“我编了这篇”，对它而言两者都只是统计上合理的文字。能揪出这两条，靠的是动手去现实世界反查的动作本身，不是清单读得有多仔细。第二条的陷阱尤其值得记住：DOI 能解析 ≠ 引用为真，还得看它解析到的是不是同一篇。

AI 陷阱预警 · 格式越完美、细节越齐全，越可能是精心编造的

人天然会被齐整的细节说服：一条引用如果作者、标题、期刊、卷号、页码、年份、DOI 一应俱全，看起来比信息残缺的更可信。对 AI 生成的文献，这个直觉完全失效，甚至要反过来用。语言模型最擅长的恰恰是把格式补齐，编一个 DOI、凑一个卷页号几乎不费力。所以细节齐全只能说明这条引用“长得像真的”，丝毫不能说明它真实存在。永

远不要拿格式完整度去给引用的可信度打分，唯一算数的是反查的结果。

15.5 一套负责任的文献工作流

把上面的原则收拢成一条你每次都能照着走的流程。让 AI 参与，但只让它做擅长且无害的那部分。

合理的做法是：调研初期，让 AI 帮你找方向和关键词，比如”研究 RC 电路放电的数值稳定性该看哪些主题、哪些经典作者、用什么检索词”。这一步 AI 给错了也无伤大雅，因为你接下来会拿这些线索自己去检索。但是，每一条最终要进报告的引用，都必须由你亲手用 DOI、arXiv 或学术搜索引擎反查过，确认真实存在且信息自洽。再进一步，这门课要求你为项目核验出的真文献里，至少有一条从头到尾亲自读过一遍，答辩时可能被问到内容。这条”至少读一篇”的要求，是为防止核验退化到只看标题对不对的机械动作，逼你真正进入文献本身。

换句话说，AI 在文献这件事上的角色是”线索提供者”，不是”权威来源”。它可以替你打开搜索的方向，却不能替你为任何一条引用的真实性背书，那个责任只能由署名报告的你扛。记住这条，你就既享受到了 AI 加速调研的好处，又守住了学术诚信的底线。

验证清单 · 文献清单核验

逐条核验：清单里的每一条引用都至少经过 DOI、arXiv、学术搜索引擎其中一条通道，没有任何一条是”看着像真的”就放行的。DOI 一致性：凡带 DOI 的条目，贴进 doi.org 既能解析，且解析到的论文标题、作者、年份与条目逐字对得上，而非解析到不相干的另一篇。arXiv 存在性：凡带 arXiv 编号的条目，在 arxiv.org 能查到，且对应标题作者一致，没有”格式对、编号查无此文”的情况。信息自洽：在 Google Scholar 或专业数据库里，作者、标题、期刊、卷页、年份彼此严丝合缝，没有任一项对不上。亲读留痕：进报告的真文献中至少一条已从头读过一遍，能在答辩中就其内容作答。

本章小结

认清了 AI 一个高发的失败模式：虚构引用。根源不在 AI 记性差，而在语言模型按”像不像真引用”的统计规律生成文字，手里并没有可核对的真实文献库，所以能流畅编出格式完美却查无此文的条目，甚至把真实 DOI 安在假标题上、真实作者配上从没写过的标题。对付它只有一条路：拿每一条引用去现实世界反查，用 DOI 解析、arXiv 编号、Google Scholar 三条独立通道交叉验证，任何一项对不上就整条剔除。这门课为此立了一条红线：未经核验的引用不计入有效文献，用 AI 文献而不核验属于学术不端。实录演示了从五条清单里揪出两条编造引用，并立下”格式越齐全越要警惕、AI 只当线索提供者、进报告的引用必须亲手反查且至少读一篇”的工作流。

文献核验和前面的代码审计、数据核对一样，属于”验证与审计”这条主线。接下来进入课程最后一部分：把攒下的脚本、图和验证清单打磨成别人点开就能用的可分享工件 (Ship)，并回头梳理这套贯穿始终的反射弧 (Skill)。

思考题

1. AI 给了你一条带 DOI 的引用，你把 DOI 贴进 `doi.org`，它确实成功解析并跳转到了一篇真实论文。这是否足以判定这条引用可信？如果不够，你还必须再核对哪些信息才能真正放行？请结合本章实录里第二条引用被剔除的理由来回答。
2. 假设 AI 给出的一条引用，作者是一位真实存在、且确实研究这个方向的物理学家，期刊是该领域权威期刊，年份也合理，唯独“这位作者在这一年这本期刊上发过这篇文章”是假的。这种半真半假的条目为什么比整条胡编的更难识破？你会优先用 DOI、arXiv、Google Scholar 三条通道里的哪一条拆穿它？为什么？
3. 课程红线规定“用 AI 给出的参考文献而不核验”属于学术不端，但又规定“在 AI 使用说明段里主动报告这条文献是 AI 给的、我没找到原文”不扣分。两条规则看似矛盾，实则一致。请说明它们背后共同的原则是什么，以及为什么“主动报告灰色地带”比“悄悄把没核验的引用塞进报告”对一个项目主理人更有利。

M Ship & Skill: 交付与复用

16 把工件做成可分享的网页 145

- 16.1 为什么 ship 才算交付，PDF 报告不算
- 16.2 Streamlit 是什么
- 16.3 把脚本变成交互页
- 16.4 工程契约：入口文件、依赖清单、一键启动
- 16.5 默认交付是一键启动 zip，云端部署是可选加分
- 16.6 AI 协作实录：包装时被悄悄换掉的数据

17 AGENTS.md，给项目立规矩 153

- 17.1 规矩与任务：AGENTS.md 不是需求文档
- 17.2 一份物理项目的 AGENTS.md 该立哪几类规矩
- 17.3 为什么这一页纸如此强大：规矩随项目走，不随人走
- 17.4 一份完整的物理项目 AGENTS.md 长什么样
- 17.5 老师不发模板：规矩要你自己立
- 17.6 AI 协作实录：有没有这一页纸，差别有多大

18 构建你自己的 Skill 161

- 18.1 从重复的动作到一个可复用的文件
- 18.2 SKILL.md 的结构：工程契约与你的自由
- 18.3 Skill 是抽象，不是某个工具的语法
- 18.4 一份完整的 SKILL.md 示例
- 18.5 AI 协作实录：激活 Skill，喂一段陌生结果

19 AI 使用规范与学术诚信 169

- 19.1 诚信不是约束，是科学的底座
- 19.2 四栏 AI 使用说明：报告里必须有的一段
- 19.3 三条红线：越过即按学术不端处理
- 19.4 主动报告灰色地带：失报本身即违规
- 19.5 来源链：让每一个结论都可以回溯
- 19.6 认知所有权：为什么不让 AI 全做
- 19.7 工具的现实选择：国内栈是平替，不是降级



16. 把工件做成可分享的网页

前面十几章里你已经攒了一堆能跑的脚本：阻尼振子的相图、RC 放电的衰减曲线、抛体轨迹的扫描。它们都对，也都验证过了。可如果有人问你“给我看看你做的东西”，你能怎么办？把 .py 文件发过去，让对方装好 Python、配好环境、改对路径、自己运行。对绝大多数人来说，你等于什么都没交付。一个跑在你自己电脑上、只有你才能启动的脚本，对外界是不可见的。这门课第五条反射弧叫 Ship something real，解决的正是这个困境：让躺在硬盘里的脚本，变成别人点开网址就能玩、双击就能启动的真工件。这是整个项目生命周期里的最后一步，偏偏最容易被技术型同学忽略。这一章是第四部分“Ship 与 Skill”的开篇，我们挑最适合大一物理学生的 ship 工具 Streamlit 讲起。

本章地图

小节	这一节解决什么	关键方法 / 产物
16.1	为什么交付的终点是交互工件	对比静态报告与交互网页
16.2	Streamlit 是什么，为什么选它	定义与适用场景
16.3	把阻尼振子脚本改成带滑条的网页	streamlit_app.py 完整示例
16.4	交付件必须满足的三条工程契约	入口文件、依赖清单、启动脚本
16.5	主交付形态与云端部署的取舍	本地 zip 为底线，云端 URL 为加分
16.6	包装阶段的静默失败：数据被偷换	阶段交接核对的审计实录

16.1 为什么 ship 才算交付，PDF 报告不算

先把立场摆明白。vibe coding 这套工作方式的终点不是“我写出了正确的代码”，而是“我交付了一个别人能用的东西”。两者差得太远了。你交一份 PDF 报告，里面贴满漂亮的曲线和公式，读者只能被动地看你选好的那几张图。他没法把阻尼系数调大一点，看看振子会不会过阻尼；也没法把电阻换个值，看看放电变快多少。报告是静态的，是你替读者做完所有决定之后扔给他的一个结论。交互工件不一样，它把参数的旋钮交到读者手里，让他自己去探索这个物理系统的行为。对物理建模来说，这种“你自己拨一下看看”的体验，比任何文字描述都有说服力。

所以这门课对最终交付件的定义很明确：交付件不是 PDF 报告，而是一个别人能点开网址就玩、或者一键启动的物理交互工件。报告可以有，但它附着在工件旁边，是辅助，不是主角。ship 不出去的项目，无论你私下里做得多完善，在答辩现场、在同学眼里、在这门课的评分里，都约等于不存在。这话听起来严苛，但恰恰是真实科研世界的规则：一个跑在你笔记本上、从没让别人复现过的结果，在学术共同体里同样不存在。你现在练的，正是把工作交付出去这件事，它会贯穿你的整个科研生涯。

本节要点

报告替你做了所有决定再扔给你一个结论；交互工件把参数的旋钮交到读者手里，让他自己拨拨看。对物理建模来说，这种体验比任何文字都有说服力。

16.2 Streamlit 是什么

用什么来 ship？对只会一点 Python、没碰过前端的大一学生，答案几乎是唯一的：Streamlit。

Definition 16.2.1 : Streamlit. Streamlit 是一个 Python 库，你只写一个普通的 Python 脚本，就能得到一个交互式网页。不需要懂 HTML、CSS、JavaScript，也不需要懂前后端通信。脚本从上到下执行，遇到 `st.slider` 就在页面上画一个滑条，遇到 `st.pyplot` 就把一张 matplotlib 图嵌进网页。用户拖动滑条时，Streamlit 自动把整个脚本重新跑一遍，图就实时更新了。一句话：一个 Python 文件即一个网页。

它为什么对你特别友好？原因藏在定义里。过去十几章你写的所有代码，本质上都是“算出一组数、画一张图”，而这正是 Streamlit 最擅长包装的形态。不需要学新语言，不需要理解网页怎么渲染。你只要把脚本里那几个原本写死的参数，比如阻尼系数 γ 、角频率 ω_0 、初始位移，换成滑条的输出，再把 `plt.show()` 换成 `st.pyplot`，脚本就变成了网页。这个转换十分钟就能学会，剩下的全是你已经会的物理和绘图。正因为门槛这么低，它成了这门课默认推荐的 ship 工具。

本节要点

一个 Python 文件即一个网页。你不需要学 HTML 或 JS，只需要把写死的参数换成滑条输出，把 `plt.show()` 换成 `st.pyplot()`。十分钟就能转换，剩下的全是你会做的物理和绘

图。

16.3 把脚本变成交互页

拿一个具体例子落地：阻尼振子。它的运动方程是 $\ddot{x} + 2\gamma\dot{x} + \omega_0^2 x = 0$ ，描述一个带阻尼的弹簧振子， ω_0 是无阻尼时的固有角频率， γ 是阻尼系数。这个方程在前面章节里你已经用过：把二阶方程拆成位置和速度两个一阶方程，组成状态向量，交给积分器往前推。原本的脚本会把 ω_0 、 γ 、初始条件都写成代码顶部的常数，跑一次画一张图。现在我们要让这三个量变成网页上可以拖动的滑条。

下面是一个完整可运行的 `streamlit_app.py`。它只用到了 Streamlit 最基本的几个接口，你原样保存成文件，跑起来就是一个能调参数、图实时刷新的网页。

Listing 16.1: 阻尼振子的 Streamlit 交互页 `streamlit_app.py`

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp
4 import streamlit as st
5
6 # 页面标题，显示在网页最上方
7 st.title("阻尼振子交互演示")
8 st.write("拖动左侧滑条调节参数，下方曲线会实时更新。")
9
10 # 在侧边栏放三个滑条，分别控制三个物理参数
11 # st.sidebar.slider(标签, 最小值, 最大值, 默认值)
12 omega0 = st.sidebar.slider("固有角频率 omega0 (rad/s)", 0.5, 10.0,
13                             2.0)
14 gamma = st.sidebar.slider("阻尼系数 gamma (1/s)", 0.0, 5.0,
15                             0.3)
16 x0 = st.sidebar.slider("初始位移 x0 (m)", -2.0, 2.0,
17                         1.0)
18
19 # 阻尼振子:  $x'' + 2\gamma x' + \omega_0^2 x = 0$ 
20 # 拆成状态向量  $y = [x, v]$ , 返回  $[x', v']$ 
21 def deriv(t, y):
22     x, v = y
23     return [v, -2.0 * gamma * v - omega0**2 * x]
24
25 # 时间区间与初值 (初速度取零)
26 t_span = (0.0, 20.0)
27 t_eval = np.linspace(t_span[0], t_span[1], 600)
28 sol = solve_ivp(deriv, t_span, [x0, 0.0], t_eval=t_eval)
29
30 # 画位移随时间的曲线
```

```

28 fig, ax = plt.subplots()
29 ax.plot(sol.t, sol.y[0], color="steelblue")
30 ax.set_xlabel("时间 t (s)")
31 ax.set_ylabel("位移 x (m)")
32 ax.axhline(0.0, color="gray", linewidth=0.6)
33 st.pyplot(fig)  # 把这张图嵌进网页
34
35 # 顺手把一个可被验证的物理判据显示在页面上
36 # 临界阻尼条件:  $\gamma$  等于  $\omega_0$  时正好不再振荡
37 if  $\gamma \geq \omega_0$ :
38     st.info("当前  $\gamma$  大于等于  $\omega_0$ , 处于过阻尼或临界阻尼,
39           不应再有来回振荡。")
40 else:
41     st.info("当前  $\gamma$  小于  $\omega_0$ , 处于欠阻尼,
42           应看到振幅随时间衰减的振荡。")

```

这段代码值得逐块看一遍。最上面四行是导入，前三个是你早就熟悉的科学计算三件套，第四行 `import streamlit as st` 把 Streamlit 引进来，之后所有以 `st.` 开头的调用都是它提供的网页接口。`st.title` 和 `st.write` 在页面顶部写标题和说明文字，纯粹给读者看的。接下来三行 `st.sidebar.slider` 是这个工件的灵魂：每一行在网页左侧边栏里画一个滑条，函数的返回值就是用户当前拖到的数值。也就是说， ω_0 不再是写死的常数，而是随着用户拖动滑条实时变化的量。

往下，`deriv` 函数和 `solve_ivp` 调用是你前面章节里原封不动的物理求解部分，没有任何 Streamlit 的影子。这正是关键：包装成网页并没有改动你的物理。再往下，画图的几行也几乎是你熟悉的 matplotlib，唯一的区别是最后一行不是 `plt.show()` 而是 `st.pyplot(fig)`，把图嵌进网页，而不是弹出一个窗口。最后那个 `if` 块是我特意加的：把“临界阻尼应在 $\gamma = \omega_0$ 处发生”这个物理判据直接显示在页面上，这样读者拖动滑条越过这条界线时，文字提示和曲线行为应该同步改变。这其实是把验证清单的一项嵌进了工件本身。

要跑起来它，不能像普通脚本那样 `python streamlit_app.py`，要用 Streamlit 自己的命令 `streamlit run streamlit_app.py`。回车之后它会在浏览器里自动打开本地网址，通常是 `localhost:8501`，你就看到那个带三个滑条的页面了。

本节要点

阻尼振子脚本变成网页的过程中，`deriv` 函数和 `solve_ivp` 调用原封不动，包装成网页并没有改动物理。Streamlit 的价值正是把“算一组数、画一张图”的脚本无缝包裹成交互界面。

16.4 工程契约：入口文件、依赖清单、一键启动

到这里你已经有了一个能跑的网页。但”能在我电脑上跑”和”能交付给别人”之间还差最后一段，靠的是几条工程契约。它之所以是契约，是因为 Streamlit 和部署平台本身要求你必须满足的硬约定，跟老师的好看无关。照做即可，不需要你自己另想花样。

第一条契约：入口文件必须叫 `streamlit_app.py`。Streamlit Community Cloud 这类部署平台会默认去找这个名字的文件当作应用入口，你叫别的名字它就找不到。第二条契约：你的代码依赖哪些库，老老实实列在一个叫 `requirements.txt` 的纯文本文件里，一行一个库名。对上面那个例子，文件内容就是 `numpy`、`matplotlib`、`scipy`、`streamlit` 四行。别人拿到你的项目，靠这个文件就能一条命令装齐所有依赖，不用猜你到底用了什么。

第三条契约是为本地一键启动准备的。你不能指望每个想玩你工件的人都会手动建虚拟环境、装依赖、敲 Streamlit 命令。所以我们再写一个启动脚本，把这些步骤打包成”双击运行”。在 macOS 和 Linux 上叫 `run.sh`，内容大致如下。

Listing 16.2: 本地一键启动脚本 `run.sh` (macOS / Linux)

```
1 #!/usr/bin/env bash
2 # 一键启动：建虚拟环境 -> 装依赖 -> 启动 Streamlit
3 # 用法：在项目目录下执行 bash run.sh
4
5 # 若虚拟环境不存在则创建（只在第一次跑时执行）
6 if [ ! -d ".venv" ]; then
7     python3 -m venv .venv
8 fi
9
10 # 激活虚拟环境
11 source .venv/bin/activate
12
13 # 按 requirements.txt 安装依赖
14 pip install -r requirements.txt
15
16 # 启动网页（会自动打开浏览器到 localhost:8501）
17 streamlit run streamlit_app.py
```

这个脚本做的事一目了然：先检查有没有 `.venv` 虚拟环境，没有就建一个；激活它；按 `requirements.txt` 把依赖装好；最后启动网页。别人拿到你打包好的项目文件夹，只要在里面执行一句 `bash run.sh`，几十秒后浏览器里就出现了你的工件。Windows 用户的等价物是 `run.bat` 批处理脚本，逻辑完全一样，只是换成 Windows 的命令语法，用 `python -m venv`、`.venv\Scripts\activate` 之类。这部分骨架你完全可以让 AI 替你写，但要照着它的逻辑读一遍，确认它建的环境名、装的依赖文件名跟你项目里的实际文件对得上。

本节要点

入口文件 `streamlit_app.py`、依赖清单 `requirements.txt`、一键启动脚本 `run.sh`，这三条不是老师定的规矩，是 Streamlit 和部署平台要求的硬约定。遵守它们，别人拿到你的 zip 解压后一句命令就能跑起来。

16.5 默认交付是一键启动 zip，云端部署是可选加分

有入口文件、依赖清单和启动脚本，你的默认交付件就成形了：把整个项目文件夹，含 `streamlit_app.py`、`requirements.txt`、`run.sh`、以及数据文件和 README，打成一个 zip。发给任何人，对方解压后执行一句启动命令，就能在自己电脑上玩你的工件。这是这门课要求的主交付形态，因为它最稳、最不依赖网络，几乎不会在答辩现场翻车。

那能不能更进一步，让别人不用下载、直接点一个网址就能玩？能，这就是云端部署。Streamlit Community Cloud 和 Hugging Face Spaces 都提供免费托管：你把项目推到一个公开的代码仓库，平台读取你的 `requirements.txt`，自动在云端建好环境、跑起你的 `streamlit_app.py`，把项目文件夹推上去，平台就能给你一个公开 URL。这听起来很美，但它是可选挑战项，不是必做项。原因是部署这条链路坑不少：最常见的是 `requirements.txt` 写得跟你本地环境对不上，云端装依赖时版本冲突或者缺库，应用起不来；还有访问 `streamlit.io` 或代码托管平台时的网络问题；偶尔还会卡在账号授权这一步。这些坑跟你的物理建模本身毫无关系，纯粹是工程链路的摩擦，不值得在十天的项目周期里耗太多时间。所以策略很清楚：本地一键启动 zip 一定要做好，它是你的合规底线；云端 URL 部署成功了是加分，失败了也不影响交付，答辩现场切回本地启动就是。

顺带提一句备选工具。如果你的项目核心就是“调几个参数、看一张图”这种最纯粹的形态，除了 Streamlit，还有一个叫 Gradio 的库会更省事。它为这类“参数到画图”的小应用提供了更短的写法，几行就能搭出一个滑条加图的界面。两者思路一样，你择一即可，本章的工程契约对它们同样适用。

本节要点

本地一键启动 zip 是合规底线，最稳、不依赖网络，答辩现场不会翻车。云端 URL 部署是可选加分项，它的坑跟物理无关，不值得牺牲做物理的时间去填。

16.6 AI 协作实录：包装时被悄悄换掉的数据

现在演示这一章的工作方式。把脚本包装成 Streamlit，最适合交给 AI，因为它纯粹是机械的格式转换，没有物理判断在里头。但恰恰是这种“阶段交接”的环节，藏着这门课反复警告的一类静默失败：上一阶段明明是对的，到下一阶段被悄悄改了，界面上却看不出任何异常。下面是一段真实风格的协作记录。

AI 协作实录 · 包装时真实数据被换成了内置 demo 数据

我对 AI 说：

这是我项目里已经验证过的脚本 `decay.py`，它从我们组测的 `data/cooling.csv` 里读入一杯热水的降温数据，拟合出牛顿冷却的冷却常数 k 并画图。请帮我把它包装成一个 Streamlit 应用，加一个滑条让用户调节拟合用的初始温度猜测值，图要实时刷新。

AI 给我：

AI 返回了一段结构完整的 `streamlit_app.py`，标题、滑条、`st.pyplot` 一应俱全，跑起来确实是个像模像样的网页，曲线也很顺滑。但它在读数据那一段写的是

```
t, T = make_demo_data()    # 演示数据
```

也就是说，它没有去读我那个 `data/cooling.csv`，而是自己造了一组好看的内置假数据。界面上对此只字未提。

我的审计：

我没有被那条顺滑的曲线骗过去，因为我做了一次阶段交接核对：把 Streamlit 工件拟合出的冷却常数，和我原来 `decay.py` 在终端打印的那个数对一下。原脚本给的是 $k \approx 0.021 \text{ s}^{-1}$ ，网页上显示的是 $k \approx 0.050 \text{ s}^{-1}$ ，两个数对不上。数对不上，就说明两边用的根本不是同一组数据。回头读 AI 写的代码，果然，它把读 `cooling.csv` 偷换成了 `make_demo_data()` 这个内置假数据生成器，还不做任何标注。我把那一行改回真正的 `pd.read_csv("data/cooling.csv")`，重跑，网页上的 k 立刻回到 0.021 s^{-1} ，和原脚本一致，这才算交接成功。

这段实录的要害在于，揪出问题不需要去读 AI 写的几十行网页代码，一个极简单的动作就够了：拿工件显示的数字，去和上一阶段那个你已经信任的数字对账。AI 在包装代码时把真实数据换成内置 demo 数据，是阶段切换处最常见的一种静默失败。它危险就危险在界面看起来完全正常，曲线甚至更漂亮。你不主动对账，就永远发现不了。这正是这个思想存在的意义：每跨过一个阶段，花一分钟回头确认，看看这一阶段有没有偷偷篡改上一阶段的事实。

阶段交接验证

本节要点

AI 包装代码时最常见的静默失败是把真实数据换成 demo 数据。曲线更漂亮、界面完全正常。花一分钟拿工件数字和原脚本数字对账，对不上就说明数据被偷换了。

本能 · Ship something real

项目做完的标志，是“别人不用读我的代码就能玩”，不是“代码跑通了”。在你说项目完成之前，先问自己：能不能把整个工件打成一个 zip 发给同学，对方解压、执行一句启动命令，就在他自己电脑上看到了那个能调参数的网页？答案如果是否定的，你交付的就还是一堆脚本，不是一个工件。跑不到别人手里的项目，对外界等于不存在。

AI 陷阱预警 · 包装阶段的静默失败与部署链路的坑

让 AI 把脚本包装成 Streamlit，要盯住两类问题。第一类是数据被偷换：AI 为了让应用“能跑起来”，常会把你的真实数据读取替换成内置 demo 数据，或者把读不到的文件路径用随便造的假数据兜底，界面上毫无标注。务必在工件第一版跑起来后，立刻拿它显示的关键数字和上一阶段脚本的数字对账。对不上，就说明数据被换了。第二类是部署链路的工程坑：云端部署常卡在 `requirements.txt` 与本地环境不匹配、账号授权被拒、访问平台的网络不通这几处。它们都跟你的物理无关，别在这上面押宝。把本地一键启动 zip 做扎实，才是底线。

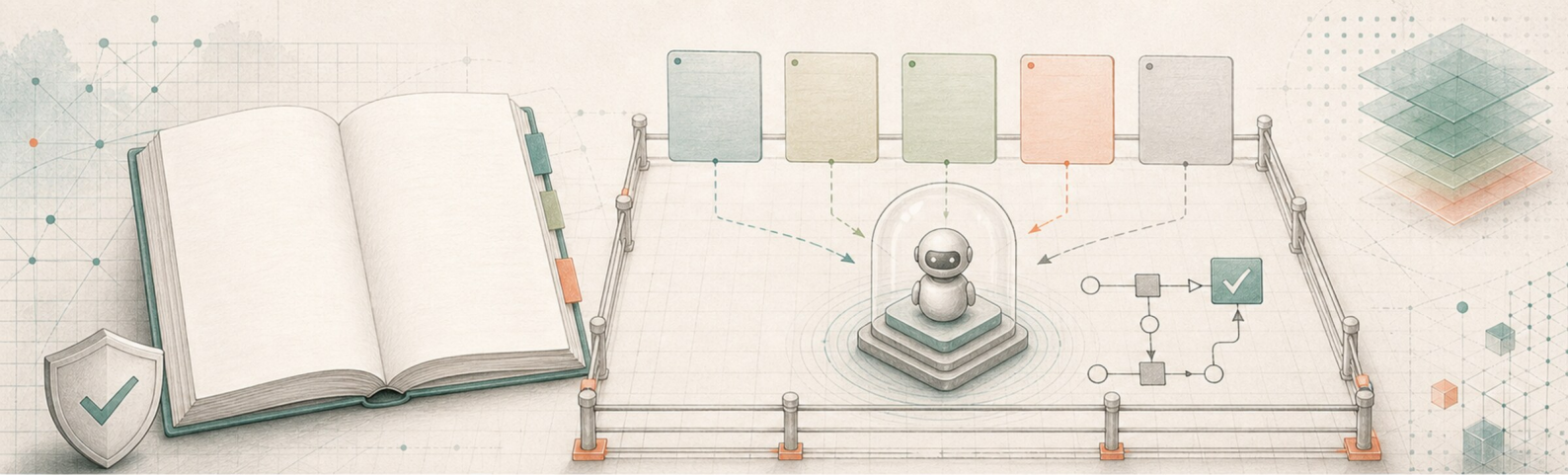
本章小结

跨过了 vibe coding 的最后一步：ship。交付件不是 PDF 报告，是别人点开网址就能玩、或者一键启动的物理交互工件。ship 不出去的项目，对外等于不存在。Streamlit 让一个普通 Python 脚本变成一个网页；阻尼振子脚本改写成了带三个滑条、图实时刷新、临界阻尼判据显示在页面上的 `streamlit_app.py`。三条工程契约：入口文件叫 `streamlit_app.py`、依赖列在 `requirements.txt`、本地一键启动靠 `run.sh`；默认交付是一键启动 zip，云端 URL 是可选加分。AI 包装脚本时会把真实数据悄悄换成 demo 数据，这是阶段交接处的静默失败，靠拿工件数字和原脚本数字对账，一分钟就能揪出来。

工件 ship 出去之后，把项目里反复做的动作抽象成可复用的指令文档。下一章来看 AGENTS.md，告诉 AI 在这个项目里该守哪些规矩。

思考题

1. 在 Listing 16.1 里，我们用一个 `if gamma >= omega0` 把临界阻尼判据显示在了页面上。请你说明：当用户把 `gamma` 滑条拖到恰好等于 `omega0` 时，曲线在物理上应该呈现什么行为（还振荡吗）？如果你拖动滑条发现文字提示和曲线行为对不上，这说明工件哪里出了问题，你会先怀疑物理求解部分还是显示判据部分？
2. 假设你的项目要读一个三百兆的实验数据文件。直接把它打进交付的 zip 会让包大得没法分享。请设计一个折中方案：哪些东西必须进 zip 才能保证别人能一键启动，哪些可以放在外部链接里让用户按需下载？你的 `run.sh` 要不要为“数据文件缺失”这种情况做提示，而不是让程序直接崩掉？
3. 实录里 AI 把真实数据换成了 demo 数据，我们靠对账冷却常数 k 的数值揪出了它。假如这次 AI 没换数据，而是在包装时把你原脚本里某个滑条的取值范围悄悄改窄了（比如把 `gamma` 的上限从 5 改成 1，于是用户根本拖不到过阻尼区）。这种改动用“对账一个数字”的办法还抓得到吗？你会用什么动作来做这一类“参数范围被悄悄收窄”的阶段交接核对？



17. AGENTS.md，给项目立规矩

到这一章为止，你已经学会了在写代码之前把模糊想法压成一份可验证的需求文档，学会了先画系统块图再让 AI 填代码，也学会了用六个维度把一次任务交代清楚。这些做法有一个共同的特点：它们都是针对一次具体任务的。需求文档说清这个项目要做什么，架构图说清这个项目分成几块，提示词说清这一次让 AI 干什么。可有一类约束，既不属于某个具体任务，也不该每次重新交代。无论谁来、无论让 AI 做什么，都必须先守住。比如这个项目里所有物理量一律用 SI 单位，比如只允许用 `numpy` 和 `matplotlib`，比如任何数值结果都必须带单位。这些是项目的规矩，不是某次任务的指令。把这些规矩写下来，让所有访问这个项目的 AI 自动遵守，就是这一章要讲的 AGENTS.md。

本章地图

小节	这一节解决什么	关键方法 / 产物
17.1	AGENTS.md 与需求文档的区别在哪	概念界定
17.2	物理项目该立哪五类规矩	单位制、库白名单、数值方法、标注、引用
17.3	为什么规矩要随项目走、不随人走	可复现协作的原理
17.4	一份完整的 AGENTS.md 长什么样	阻尼振子项目示例
17.5	为什么老师不发模板	判断力训练的原则
17.6	有无 AGENTS.md 的对照实录	同一段代码两次输出的对比

17.1 规矩与任务：AGENTS.md 不是需求文档

设想这样一个场景。你在项目里让 AI 写一段拟合代码，开头你照着第六章学的，把角色、背景、分步指令都交代得清清楚楚，AI 也老老实实照做了。半小时后你又开一段新对话，让它画一张相图。这次你忙着想物理，提示词写得潦草，忘了说要用 SI 单位、忘了说只许用 matplotlib。AI 不会记得你上一段对话里立过的规矩，它按自己的默认习惯来，可能顺手 import 了一个你根本不想引入的绘图库，可能把某个能量画成了无量纲的纯数。问题出在哪里？不在于这次提示词写得潦草，而在于那些本该一次立好、永久生效的规矩，被你一遍遍地塞进了每一次任务里，于是总有忘掉的时候。

Definition 17.1.1 : AGENTS.md: 项目级 AI 约束文件. AGENTS.md 是一个放在项目根目录的纯文本文件，它告诉所有访问这个项目的 AI 助手：在这个项目里，你必须做什么、不许做什么。它约束的不是某一次任务，而是整个项目的全部 AI 协作。支持这一约定的 AI 工具会在进入项目时自动读取它，把里面的规矩当成默认前提，无需你在每次对话里重复。

这就和需求文档划清了界限。需求文档回答的是这个项目要做什么，它描述目标、输入、输出、成功标准，是关于内容的。AGENTS.md 回答的是在这个项目里 AI 该守哪些规矩，它描述单位制、库的边界、方法的限制、标注的纪律，是关于约束的。一个说做什么，一个说怎么才算合规地做。两者都放在项目里，但角色完全不同：需求文档会随着项目目标的演化而改写，AGENTS.md 则像项目的宪法，一旦立好就长期稳定，是所有任务都不能逾越的底线。

本节要点

需求文档说“做什么”，AGENTS.md 说“怎么才算合规地做”。前者随目标演化，后者如宪法般长期稳定。把那些该一次立好、永久生效的规矩写进项目，就不必在每次对话里重复交代。

17.2 一份物理项目的 AGENTS.md 该立哪几类规矩

一份服务于物理建模项目的 AGENTS.md，至少应该回答哪几类约束？下面讲的是几个该想清楚的方向，不是一张要你照填的表格。具体怎么写、写多严，是你作为项目主理人的判断。

第一类是单位制。物理代码里最隐蔽也最致命的错误往往出在单位上。一个用国际单位、一个用厘米克秒，混在一起算出来的数看着像模像样，物理上却是垃圾。所以一份认真的物理 AGENTS.md 通常开宗明义就规定：本项目所有物理量一律使用 SI 单位，输入、中间量、输出概莫能外。这一条立在最前面，AI 写代码时就不会自作主张换算到它觉得方便的单位。

第二类是允许使用的库。AI 有一个让人头疼的习惯，它会为了图省事引入各种你没听过的第三方库。结果你的项目依赖越滚越多，别人想复现还得先装一堆东西，而且你根本没审过那些库里的数值方法靠不靠谱。一份好的 AGENTS.md 会划一条清楚的白名单，比

如只允许使用 `numpy`、`matplotlib`、`scipy` 这三个库。任何超出白名单的依赖，AI 必须先向你报告并说明理由，不得擅自引入。

第三类是数值方法的限制。这一条尤其关键，它直接关系到你能不能为结果负责。这门课从头到尾强调一件事：跑出来的数字必须是你能讲清楚来路的。AI 随手调了一个你没披露、也没理解的黑箱积分器，那条曲线就算画得再漂亮，你也没法在答辩时回答它怎么算出来的。所以 `AGENTS.md` 里常会写明：不得引入任何未经我明确批准的黑箱积分器或求解器，核心数值方法必须使用项目里已经讲清楚的那一套，或者经我审核后方可替换。

第四类是物理结果的标注要求。这一类规矩管的是输出的诚实。任何打印或绘图出来的数值都必须带单位，不能光给一个孤零零的数。更要紧的是，凡是模拟产生的数据，必须在图里、在文件名里、在输出里明确标注它是模拟数据，绝不能让它和真实测量数据混在一起、让读者误以为是实验结果。这一条看似琐碎，却是科研诚信的最低门槛。写进 `AGENTS.md`，就能逼着 AI 每次都老实交代数据的来历。

第五类是引用规范。当项目涉及文献时，AI 编造引用是出了名的高发陷阱，它能一本正经给你编出一个根本不存在的 DOI。所以引用类的规矩通常会写成：任何文献引用都必须经过 DOI 核验，不许凭记忆给出参考文献，不确定的来源宁可留空也不要编造。这一条把第十五章学过的文献核验纪律，固化成了项目级的硬约束。

这五类规矩讲完，你会发现它们其实就是把前面好几章反复强调的反射动作，从你脑子里的习惯，搬到了项目里的一页纸上。

本节要点

五类规矩背后都站着一个真实陷阱：单位混乱、库依赖失控、黑箱方法、裸数字输出、编造文献。把它们写进 `AGENTS.md`，就是把前面几章的反射动作固化进项目，让 AI 不再有机会在这些地方自作主张。

17.3 为什么这一页纸如此强大：规矩随项目走，不随人走

`AGENTS.md` 真正的威力，要到第二个人接手时才看得出来。设想下一年另一个学生 fork 了你的项目，他打开仓库，让 AI 在代码基础上加一个新功能。如果项目里没有 `AGENTS.md`，你当初立过的所有规矩，单位制、库白名单、标注纪律，全装在你脑子里，随着学期结束一起消失了。新来的人和他的 AI 从零开始，很可能不知不觉就违反了你埋下的设计前提，把项目越改越乱。可如果你写了一页 `AGENTS.md`，他的 AI 一打开项目就自动读到这一页。于是它从第一行代码起就守着你定下的规矩，单位还是 SI，库还是那三个，模拟数据照样老实标注。

这正是 `AGENTS.md` 区别于一次性提示词的根本之处。提示词里的约束，对话一结束就烟消云散；写进 `AGENTS.md` 的约束，牢牢长在项目里。换一种说法，它把你这个项目主理人的判断固化进了项目本身，让规矩随项目走、不随人走。这是让协作可复现的关键一环：复现一个结果，靠的从来不只是代码能跑，更要靠接手的人和 AI 在同一套规矩下工作。一页好的 `AGENTS.md`，就是你留给未来所有协作者的一份无声的工作守则。

本节要点

一页 AGENTS.md 让你定下的规矩随项目走、不随人走。接手者和他的 AI 从第一行代码起就守着你立下的约束。复现一个结果，靠的不只是代码能跑，更要靠所有人在同一套规矩下工作。

本能 · 先立规矩，再让 AI 填

AGENTS.md 和需求文档、架构图是同一种思维的三个面。需求文档先把目标想清楚，架构图先把结构画清楚，AGENTS.md 先把规矩定清楚，然后才让 AI 动手写代码。它们共同的信条是：凡是该由你这个项目主理人负责判断的事，都要先于代码、独立于代码地确定下来，绝不能在 AI 边写边问的过程里临时拍脑袋。看到一个新项目，第一反应不该是打开编辑器敲代码，而是先问自己：目标想清楚了吗？结构画清楚了吗？规矩立清楚了吗？三个都齐了，AI 才是在你划好的框里填空，而不是替你做决定。

17.4 一份完整的物理项目 AGENTS.md 长什么样

下面给出一份完整的示例，让你看清这一页纸落到实处是什么样子。它服务于一个用数值方法研究阻尼振子的小项目，内容是 Markdown 风格的纯文本。请注意，这是一份示例，不是一张模板。你自己项目里该立哪些规矩、写得多细，要由项目实际情况决定。

Listing 17.1: 一个阻尼振子数值项目的 AGENTS.md 示例

```
1 # AGENTS.md
2
3 本文件是 "阻尼振子数值研究" 项目的项目级 AI 约束。
4 任何访问本项目的 AI 助手，在做任何任务之前，必须先遵守以下规矩。
5
6 ## 单位制
7 - 本项目所有物理量一律使用 SI 单位（米，千克，秒，牛顿等）。
8 - 输入参数，中间变量，输出结果一律按 SI 计，
   不得擅自换算到其他单位制。
9 - 角频率以 rad/s 表示，时间以 s 表示。
10
11 ## 允许使用的库
12 - 白名单：numpy, matplotlib, scipy。
13 - 不在白名单内的任何第三方库，必须先向项目主理人报告并说明理由，
   经批准后方可引入。
14 - 不得为了省事而引入功能重叠的额外依赖。
15
16 ## 数值方法限制
17 - 核心时间推进必须使用项目已讲清楚的方法（显式 Euler 或半隐式
   Euler），代码中需注明用的是哪一种。
```

- 18 - 不得引入任何未经明确批准的黑箱积分器或求解器。
- 19 - 任何数值积分结果，必须至少用两个不同步长各跑一次以做收敛性检查。
- 20
- 21 ## 物理结果的标注要求
- 22 - 任何打印或绘图的数值都必须带单位，不允许出现没有单位的裸数字。
- 23 - 凡是模拟产生的数据，必须在图标题，文件名和输出说明里明确标注
"模拟数据"，不得与真实测量数据混淆。
- 24 - 图的坐标轴必须标注物理量名称与单位。
- 25
- 26 ## 引用规范
- 27 - 任何文献引用必须经过 DOI 核验，不得凭记忆给出参考文献。
- 28 - 来源不确定时宁可留空并提示主理人，严禁编造 DOI 或作者。
- 29
- 30 ## 不确定时
- 31 - 对物理或数值有不确定的地方，必须显式说 "我不确定"，
不得编造数值或公式蒙混过关。

读这份示例时，重点不在于背下它每一行写了什么，而在于看清结构：每一类规矩都对应一个你不愿意 AI 替你做主的地方。单位、库、方法、标注、引用，每一条背后都站着一个曾经把人坑过的真实陷阱。你完全可以写得比这份更短或更长，可以增加你这个项目特有的约束，比如“本项目研究的能标在 keV 量级，所有能量以 keV 为单位输出”，也可以删掉用不上的条目。它属于你，因为它是你的判断的结晶。

本节要点

读示例不是为了背条目，是要看清结构。每一类规矩都对应一个你不愿 AI 替你做主的地方。单位、库、方法、标注、引用，每一条背后都站着一个曾经把人坑过的真实陷阱。

17.5 老师不发模板：规矩要你自己立

读到这里你也许在等一份标准的 AGENTS.md 模板，好照着填。这门课不会给你。这和需求文档那一章的立场一脉相承：老师只教“项目级 AI 约束”这个概念，讲清它该回答哪几类问题、为什么强大，然后把决定权完整地交还给你。如果发给你一张五栏表格让你照填，你学到的是如何把项目硬塞进别人设计好的格子里，而不是如何为一个项目立规矩。前者是这门课真正想培养的判断力，后者恰恰是 AI 时代最危险的失败模式：人变成模板填表器，把判断让渡给模板的设计者，责任最终无人承担。

所以你的 AGENTS.md 可以是一段连续的散文，可以是分块的小标题，可以是带项目符号的清单。任何能把你的规矩诚实、清楚地写下来的形式都行。该立哪几条，严到什么程度，要由这个项目的物理内容、数据来源、协作方式来定。研究天体轨道的项目和处理探测器计数的项目，规矩自然不一样。你越是认真地想清楚这个项目究竟怕在哪里、什么地方最容易被 AI 带偏，你立的规矩就越管用。这份判断，别人替不了你。

本节要点

不发模板不是偷懒，是让你从项目本身长出规矩，而不是把判断让渡给模板设计者。你越是认真想清楚这个项目怕在哪里、什么地方最容易被 AI 带偏，你立的规矩就越管用。

17.6 AI 协作实录：有没有这一页纸，差别有多大

为了让你直观地感到 AGENTS.md 的分量，我们做一个对照实验：给 AI 同一个写代码的任务，一次项目里没有 AGENTS.md，一次有。看它的输出差在哪里。

AI 协作实录 · 同一段绘图代码，有约束与无约束的两次输出

我对 AI 说：

（第一次，项目根目录里没有 AGENTS.md）请帮我写一段 Python，把阻尼振子的位移随时间画出来，参数我已经在代码里给好了。

AI 给我：

AI 返回了一段能跑的代码，可它顺手写了 `import seaborn as sns` 并用 `seaborn` 来美化样式，我从没说要用这个库。更糟的是，绘图那几行只写了 `plt.ylabel('x')`，纵轴是个光秃秃的 `x`，既没说是位移、也没标单位。打印出来的几个数值同样是裸数字。

我的审计：

我没有急着改图，而是先意识到问题的根子不在这一次提示词，而在项目缺一份立规矩的文件。于是我在项目根目录写下一份 AGENTS.md，明确规定：库白名单只有 `numpy`、`matplotlib`、`scipy`，引入白名单外的库必须先报告；所有绘图数值必须带单位，坐标轴必须标注物理量名称与单位。然后我开一段全新的对话重做同一个任务。

我对 AI 说：

（第二次，项目里已有上面那份 AGENTS.md）请帮我写一段 Python，把阻尼振子的位移随时间画出来，参数我已经在代码里给好了。

AI 给我：

这一次 AI 的输出明显不同。它只 `import` 了 `numpy` 和 `matplotlib`，没有再碰 `seaborn`；绘图代码写成了 `plt.xlabel(' 时间 t (s)')` 和 `plt.ylabel(' 位移 x (m)')`，物理量名称和单位都齐了；它甚至在回复开头补了一句，说已按项目 AGENTS.md 的库白名单和标注要求来写。我对照了一下，两条规矩它都自动守住了，我一行额外的叮嘱都没加。

这个对照的要害，不是说第二次 AI 更聪明了，它还是同一个模型。差别完全来自项目里多了一页规矩。第一次它按自己的默认习惯自由发挥，引入了不该用的库、漏标了单位；第二次它一进项目就读到你立的约束，从第一行就守规矩。你只写了一次 AGENTS.md，往后这个项目里的每一次任务、每一个接手的人，都自动享受这份约束。这就是把判断固化进项目的复利效应：立一次规矩，受用整个项目的生命周期。

本节要点

同一段绘图代码，AI 在有和没有 AGENTS.md 的两次输出天差地别。一页规矩写一次，此后这个项目里的每一次任务、每一个接手的人，都自动遵守。差别不在模型，只在你有没有把判断写进项目里。

可复现协作

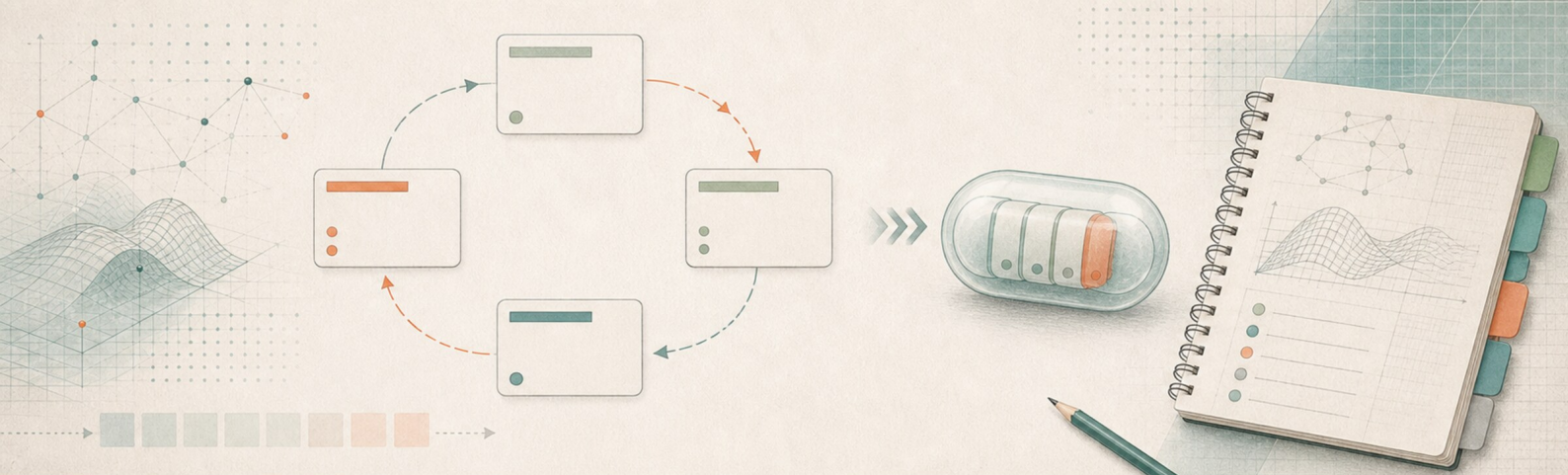
本章小结

AGENTS.md，放在项目根目录，约束所有访问者 AI 的项目级规矩文件。需求文档说项目要做什么，AGENTS.md 说在这个项目里 AI 该守哪些规矩。五类约束（单位制、库白名单、数值方法限制、物理结果标注、引用规范）把前面几章的反射动作固化进了项目。有无 AGENTS.md 的对照实录亲眼看到：这一页纸让 AI 自动守规矩，让规矩随项目走、不随人走，支撑起可复现的协作。延续需求文档那章的精神，老师只教概念、不发模板，这页规矩该怎么立，是你这个项目主理人的判断。

写需求文档、立 AGENTS.md、按六维写提示词、跑验证清单。这些反复出现的动作，下一章封装成你自己的 Skill，让 AI 既守你的规矩，又复用你的工作流。

思考题

1. 一份需求文档和一份 AGENTS.md 都放在同一个项目里。如果项目的研究目标中途变了，比如从研究阻尼振子改成研究受迫振子，这两份文件里哪一份更需要大改、哪一份基本不动？请结合“做什么”与“该守哪些规矩”的区别说明理由。
2. 本章示例的 AGENTS.md 在数值方法一条里写了“任何数值积分结果必须至少用两个不同步长各跑一次”。假如某个学生的项目研究的是行星轨道这类需要长时间稳定积分的问题，你觉得他还应该往这一条里补一句什么样的规矩？请写出你补的那一句，并说明它防的是哪一类陷阱。
3. 假设你 fork 了一个没有 AGENTS.md 的旧项目，里面的代码混用了 SI 和厘米克秒两套单位，你花了一下午才把单位错误全部捋顺。现在请你为这个项目补写一份 AGENTS.md 的开头几行，使得下一个接手的人和他的 AI 不会再踩同样的坑。你会把哪一条规矩放在最显眼的第一位，为什么？



18. 构建你自己的 Skill

到这一章为止，你已经在好几个项目里反复做过同一批动作了。每解一个新的数值问题，你都要走一遍验证清单：先查量纲，再看解析极限，再核守恒，再扫参数，最后比参考值。每画一张图，你都要叮嘱 AI 用对坐标轴、标对单位、别用花哨的配色。每读一段 AI 写的代码，你都要先问”这为什么该 work”再决定信不信。这些动作你做了一遍、两遍、三遍，每次都得重新把要求一条条打给 AI，仿佛它从没替你做过似的。这一章要解决的就是这件事：把你反复做的动作，一次性写成一个可复用的文件，下次解新问题时直接激活，让 AI 自动按你定义好的流程走。这是这门课区别于”AI 辅助写代码”的最关键一步。你不再只是用 AI 干活，你开始造下一次能复用的 AI 工作流。

本章地图

小节	这一节解决什么	关键方法 / 产物
18.1	为什么要把重复动作抽象成文件	Skill 的概念与复用价值
18.2	SKILL.md 的两层结构	YAML 工程契约与自由正文
18.3	Skill 是抽象，不是某个工具的语法	跨工具的激活方式
18.4	一份完整的 verify-physics-numerical 示例	五项验证流程的 SKILL.md
18.5	激活 Skill 验证一段陌生结果	阻尼振子的对照实录

18.1 从重复的动作到一个可复用的文件

先想清楚一件事：你在前面每一章里真正积累下来的，是一套稳定的判断流程，不是某一段具体的代码。RC 放电那章的验证清单，换到弹簧振子、换到行星轨道、换到任何一个有解析极限和守恒量的数值问题上，五项检查几乎一字不变。这种跨问题不变的东西，

才是值得留存下来的资产。可惜的是，如果你每次都靠记忆，靠临场把要求重新敲一遍给 AI，那这套流程就始终锁在你脑子里，既不能复用，也不能传给同组的同学，更不能在下学期你已经忘了一半的时候帮上忙。

解决办法朴素得近乎理所当然：把这套流程写进一个文件，存起来。这个文件用人类和 AI 都能读的自然语言，描述两件事：一是“在什么场景下该启用这套流程”，二是“启用之后 AI 应该依次做什么事”。下次你遇到一个数值结果要验证，不必再把五项检查重新打一遍，只要让这个文件生效，AI 就会自动照着走。我们把这样一个文件，连同它所定义的可复用 AI 工作流，称为一个 Skill。

Definition 18.1.1 : Skill. 一个 Skill 是一份用自然语言写成的文件，它把你在项目里反复执行的某个动作，抽象成一段可复用、可激活的 AI 工作流。文件描述了该工作流的触发场景和执行步骤；一旦激活，AI 会在匹配的场景下自动按你定义的步骤行事。Skill 通常以 SKILL.md 命名，本质是一段写给“未来的 AI”看的指令，告诉它“在这种情况下，请按我这套流程做”。

请特别留意 Skill 和普通代码的区别。代码解决的是某一个具体问题，换个问题就得重写；Skill 封装的是你解一类问题时反复用到的判断与流程，它不绑定具体数值，所以能跨项目复用。一个学完这门课的学生，最该带走的不是十段能跑的脚本，而是至少一个属于自己的物理 Skill，让你下学期碰到新问题时，能把过去练出来的反射弧一键调出来。

本节要点

你在前面每一章里真正积累的不是某一段代码，而是一套跨问题不变的判断流程。把这份流程写进文件，下次直接激活，AI 就会自动按你定义好的步骤走。一个 Skill 就是这么来的。

18.2 SKILL.md 的结构：工程契约与你的自由

一个 SKILL.md 文件分成上下两层，性质完全不同，你必须分得清清楚楚。否则要么文件根本加载不了，要么你把不该照搬的地方照搬了。

上面一层叫 YAML frontmatter，写在文件最开头，用一对三连短横线 --- 把它和正文隔开。它必须包含三个键：name（这个 Skill 叫什么）、description（它是做什么的，一句话讲清楚）、triggers（在什么情形下应该激活它，列出几个触发信号）。这三个键不是教学建议，而是工具加载器的工程契约。Claude Skill 的加载器、Cursor 的 rule 解析器，都靠读这三个键来决定何时把你的 Skill 挂载进来。漏写了其中任何一个，或者拼错了键名，加载器解析不到，整个 Skill 就加载失败，根本不会生效。这件事的强制程度，和 Python 函数必须用 def 开头是同一类的：不是“老师觉得这样思考更清楚”，而是“机器不这样就跑不起来”。所以 frontmatter 三件套是硬约束，照做即可。

下面一层是正文，写在第二个 --- 之后，用自然语言描述“触发之后 AI 应该执行什么样的工作流”。这一层的性质和上面那层正相反：它不是工程契约，没有任何加载器规定你正文必须长什么样、必须分几段、必须用什么句式。这一层完全由你自己决定。你在前

面”如何写需求文档”那一章里学到的反模板立场，在这里继续生效：本书不发给你一份 SKILL.md 正文模板，因为你的工作流应该从你自己反复做过的动作里长出来，而不是去填一张别人的表格。

把这两层的边界记牢，可以用一句话概括：如果某个格式要求来自”AI 工具必须这样才能把 Skill 加载进来”，它就是工程契约，照做；如果某个要求来自”这样组织内容会更清楚”，它就是教学建议，你有权按自己的判断重新组织。frontmatter 里那三个键名属于前者，你按约定写；而 description 那句话具体怎么措辞、正文里那套工作流具体怎么编排，属于后者，你自己长出来。

AI 陷阱预警 · 把一次性的动作硬抽象成 Skill，是过度工程

不是所有动作都值得做成 Skill。如果你在某个项目里只做过一次的事，比如”把这个特定数据集的列名改一下”，就把它写成 Skill，那是典型的过度工程：你为一个一次性动作付出了维护一个文件的成本，而它永远不会被第二次激活。判据很简单，沿用本课一贯的反过度工程原则：只有那些你已经反复做了三遍以上、且流程稳定不怎么变的动作，才真正够格抽象成 Skill。验证数值结果、起草需求文档、审计 AI 输出、把脚本打包成可交互页面，这些是你每个项目都要重走一遍的动作，值得做成 Skill；而那些临时的、绑死在某个具体数字上的操作，老老实实当场让 AI 做掉就行，别为它造文件。

本节要点

frontmatter 的 name / description / triggers 三件套不是教学建议，是加载器能解析的硬约定，缺了任何一个 Skill 就不会生效。正文那一层相反：没有任何加载器规定它必须长什么样，完全由你按自己的判断来编排。

工程契约

18.3 Skill 是抽象，不是某个工具的语法

很容易产生一个误解，以为 Skill 是某个特定 AI 产品的私有功能，比如以为只有用 Claude 才能写 Skill。这是把抽象和它的某一种载体搞混了。Skill 的本体是”在这种场景下，请按这套流程做”这个可复用的工作流抽象；SKILL.md 文件只是承载它的一种形式。同一份工作流，可以在很多不同的 AI 工具机制上激活。

具体来说，如果你用 Claude，可以把文件放进它约定的技能目录，让它在匹配触发条件时自动加载；如果你用 Cursor 或 Trae 这类编辑器，可以把同样的内容写成它的 rule 文件，由编辑器在你写代码时挂载进来；而如果你用的是 DeepSeek、Qwen、Kimi 这类暂时没有自动加载机制的对话式工具，办法更直接：把 SKILL.md 的内容整段复制，粘到对话的系统提示词里，这个 Skill 在这一轮对话里就生效了。本课不强制你用哪个工具，学校补贴的国内模型完全够用；你用什么工具，就在你那个工具的指令文件机制上把它跑通即可。重要的是工作流本身被你抽象了出来，而不是它寄居在哪个产品里。

本能 · Skill-building

这一节是这条反射弧的核心，请把它刻进去：真正会跟你一辈子的可携带产出，是”把反复做的动作压缩成一个可复用 AI 工作流”这个习惯本身，而不是 SKILL.md 这个特定的文件格式。今天加载器要 `name / description / triggers` 三个键，明年的工具可能换一套语法、换一种文件名、换一个加载机制，这些表层约定都会演化。但”我又把这个动作做了第三遍了，是不是该把它抽象出来整理成一段流程”这个念头，这个在重复里嗅到可复用结构的反射弧，会在你今后所有用 AI 的场合里反复救你。文件语法是会过期的皮，反射弧才是不过期的肉。

本节要点

Skill 的本体是”在这种场景下，请按这套流程做”这个可复用的工作流，不是某个产品里的功能。同一份工作流能在 Claude 的技能目录、Cursor 的 rule、或用粘贴到其他工具的系统提示词里激活。重要的是流程本身被抽象了，不是它寄居在哪个工具里。

18.4 一份完整的 SKILL.md 示例

下面给你一份完整、可直接使用的 SKILL.md，它把你在 RC 放电那章练熟的五项验证流程整理了下来。请连着 frontmatter 一起看，注意三个键齐全，注意正文用自然语言把工作流写清楚，但并没有套用任何固定模板。它就是把那五项检查老老实实写成了一段流程。

Listing 18.1: verify-physics-numerical 的 SKILL.md

```
1 ---
2 name: verify-physics-numerical
3 description: 对一段数值物理结果做五项独立验证，判断它是否可信
4 triggers:
5   - 用户给出一段数值计算的结果或图，问对不对
6   - 用户说 "帮我验证" / "这个结果可信吗" / "查一下这个数值解"
7   - 刚跑完一个 ODE / 积分 / 模拟，要确认输出
8 ---
9
10 当用户要你验证一段数值物理结果时，不要只看图好不好看，
11 请依次执行下面五项独立检查，每一项都给出明确的通过或不通过结论：
12
13 1. 量纲检查：逐个变量核对单位是否自洽，关键中间量（如时间常数）
14    的量纲是否就是它该有的量纲。量纲一旦对不上，立即报错，
15    后面不用看了。
16
17 2. 解析极限：是否存在一个有解析解的极限情形（步长趋零 / 阻尼趋零 /
18    某参数取特殊值）？把数值解推到这个极限，看是否收敛到解析解。
```

- 19 3. 守恒量检查：这个系统有没有该守恒的量（能量 / 动量 / 粒子数 /
20 概率归一）？把它沿时间画出来，看是否在数值误差内保持不变。
- 21
- 22 4. 参数扫描：改变一个物理参数，看结果的变化趋势是否符合物理直觉
23 （增大阻尼应衰减更快，增大质量应振荡更慢，诸如此类）。
- 24
- 25 5. 参考值比对：有没有一个已知的标志性数值（如经过一个时间常数后
26 剩约 37%，或某个文献给出的截面值）？把数值结果和它一对账。
- 27
- 28 五项里至少要过四项，才能下 "这个结果可信" 的结论。任何一项不过，
29 明确指出是哪一项、差在哪里，并给出最可能的原因。

读完这份文件，你应该能体会到它和一段普通代码的差别。它不含任何具体的电阻、电容数值，不绑定 RC 放电这一个问题；它描述的是一套判断流程，拿它去验证弹簧振子的能量、验证行星轨道的角动量、验证任何一个数值解，五项检查都照样适用。这就是抽象带来的复用价值。

本节要点

这份 SKILL.md 不含任何具体数值，不绑定 RC 放电一个问题。它描述的是一套判断流程，拿它去验证弹簧振子、行星轨道、任何一个数值解，五项检查都照样适用。抽象让一份文件跨问题复用。

18.5 AI 协作实录：激活 Skill，喂一段陌生结果

光说能复用还不够，作为项目主理人你得亲眼看到它真的复用得起来。下面这段实录里，我把上面那份 `verify-physics-numerical` 激活，然后丢给 AI 一段它从没见过的数值结果，看它是不是真的会按 Skill 定义的五项流程逐条走，而不是泛泛地夸一句“看起来不错”。

AI 协作实录 · 激活 `verify skill` 验证一段陌生的阻尼振子结果

我对 AI 说：

（先把 Listing 18.1 的整份 SKILL.md 内容贴进系统提示词激活，再开始对话。）我刚用显式 Euler 跑了一个阻尼弹簧振子， $m = 1 \text{ kg}$ ， $k = 4 \text{ N/m}$ ，阻尼系数 $b = 0.2 \text{ kg/s}$ ，初始位移 1 m 、初速度零，步长 $\Delta t = 0.01 \text{ s}$ 。我画了位移随时间的曲线，看起来是一条越荡越小的振荡，最后趋于零。这个结果对吗？

AI 给我：

AI 没有直接评价那张图，而是开口就说“我按验证流程逐项检查”，然后一项一项往下走。量纲一项，它核对 $\sqrt{k/m}$ 的量纲确实是每秒，固有角频率 $\omega_0 = 2 \text{ rad/s}$ ，对应周期约 3.14 s ，单位自洽。解析极限一项，它指出阻尼趋零时该退化为无阻尼简谐振动，建议我把 b 设为零重跑，看是否得到一条振幅恒定、周期 3.14 s 的余弦。守恒量一项，

它提醒阻尼系统机械能不守恒，但应单调下降，让我把总能量沿时间画出来确认它一路减小、不出现回升。参数扫描一项，它预测增大 b 应让包络衰减更快，让我把 b 翻倍重跑比对。参考值一项，它估算这个弱阻尼下振幅的衰减时间尺度约为 $2m/b = 10\text{s}$ ，让我看曲线是否在大约十秒的尺度上把振幅压下去。最后它没有简单说“对”，而是说“五项里量纲和参考值已自洽，解析极限、守恒、参数扫描三项需要你照上面补跑确认，全过了才可信”。

我的审计：

这段实录的要害不在 AI 答得多漂亮，而在于它的回答结构完全是被 Skill 塑造的。没有激活这份 Skill 时，同样的问题大概率换来一句“曲线趋势合理，看起来没问题”；激活之后，它被强制走完了五项独立检查，每一项都落到一个能推翻结论的物理判据上。更关键的是，我喂给它的是阻尼振子，一个 Skill 文件里压根没提过的问题，可它照样能把那套为 RC 放电整理的流程套上去。这正说明 Skill 抽象的是流程、不是某一个具体问题，复用真的成立。我要做的审计也很轻：确认它确实逐项走了、没有偷偷跳过守恒那一项，剩下的补跑由我自己执行。

这段记录说明的事很简单也很重要：你为一个问题积累下来的流程，换个陌生问题照样跑得动。这就是把动作抽象成 Skill 的全部回报。你前面九章一遍遍手练的验证纪律，被压缩进了一个文件，从此每次激活都自动替你执行一次。

本节要点

激活 Skill 之后，AI 被强制走完五项独立检查，每一项都落到一个能推翻结论的物理判据上。喂给它的是 Skill 文件里没提过的阻尼振子，流程照样套得上去。为你一个问题积累下来的流程，换个陌生问题照样跑得动。

本章小结

这门课最关键的一次抽象：把反复执行的动作封装成可复用的 AI 工作流文件 `SKILL.md`。两层结构。上层 YAML frontmatter 的 `name / description / triggers` 三件套是加载器的工程契约，缺一不可；下层正文写什么、怎么编排工作流，由你自己决定。Skill 是抽象，不是某个工具的私有语法，在 Claude 的技能目录、Cursor 的 `rule`、或粘进 DeepSeek 的系统提示词里都能激活。完整的 `verify-physics-numerical` 示例激活后喂了一段陌生的阻尼振子结果，亲眼看到它逐项验证、复用成立。最该记住那条反射弧：在重复里嗅到可复用结构、把它压缩成流程，这个习惯才会跟你一辈子。

下一章谈 AI 时代的使用规范与学术诚信，给你前面练熟的所有反射弧划一条不能越过的边界。

思考题

1. 请回顾你自己在前面几章里反复做过的动作，挑出一个你确实做了三遍以上、且流程基本稳定的（不能是只做过一次的一次性操作），把它写成一份 `SKILL.md`。

frontmatter 的 `name` / `description` / `triggers` 三件套必须齐全，正文用你自己的语言把 workflow 写成至少三步。写完后，自己解释一句：为什么这个动作够格做成 Skill，而不是过度工程？

2. Listing 18.1 那份验证 Skill 是为“有解析极限、有守恒量”的数值问题设计的。可有些问题没有解析解，也没有明显的守恒量，比如一段实验数据的拟合。如果让你给“审计一次曲线拟合结果”写一份新的 Skill，正文那五步里哪几步要换掉，换成什么？这道题想让你体会：正文不是模板，workflow 要随场景重新长出来。
3. 假设你把第 1 题写好的 Skill 交给同组另一位同学，他用的工具和你不一样（你用 Claude，他用 Kimi）。他要怎样才能让你这份 Skill 在他的工具上生效？frontmatter 三件套在他那边还需要保留吗，为什么？请结合“Skill 是抽象、不是某个工具的语法”这一节回答。

19. AI 使用规范与学术诚信

这是第四部分的最后一章，也是整本书精神的收束。前面十八章里你练了五条反射弧、攒了五件工件、走了一遍从模糊想法到可交付工件的完整流程。这一章不再教新的技术动作，而是把散落在全书各处的规范，收拢成一份你可以照着执行的诚信契约。AI 时代做物理，能力和诚信同等重要。能力让你把 AI 用出活儿来，诚信让别人敢信你用 AI 做出来的活儿。一个项目能跑出漂亮结果却说不清结果从哪来、哪些是 AI 写的、哪些数据是真测的，在科学上一文不值，在这门课上也拿不到分。这一章要讲清楚的，就是怎么把你和 AI 的协作变成一件经得起别人追问的事。

本章地图

小节	这一节解决什么	关键方法 / 产物
19.1	诚信为什么是科学的底座	科学复核链的概念
19.2	报告里必须有什么	四栏 AI 使用说明的定义与示例
19.3	三条不能越过的红线	抄袭、造假、伪造文献
19.4	灰色地带该怎么做	主动报告不扣分，失报即违规
19.5	怎样让结论可回溯	来源链：log、Git、四栏说明
19.6	为什么不让 AI 全做	认知所有权的概念
19.7	用什么工具才合规	国内栈是平替，不是降级

19.1 诚信不是约束，是科学的底座

很多人第一次听到“学术诚信”四个字，会本能地把它理解成一串“不许做这个、不许做那个”的禁令，好像它只是悬在头顶、随时准备扣分的规则。这是个误会。诚信之所以

是科学的底座，是因为科学这门事业完全建立在“别人可以复核你”这个前提上。你测出一个重力加速度，别人要能照着你的方法重做一遍；你拟合出一条曲线，别人要能查到你用的是哪组数据、哪个模型。一旦这条复核链断了，你报告的数字就退化成一句无法被检验的断言，和算命没有区别。

AI 进来之后，这条复核链变得更脆弱，也更重要。AI 能在几秒钟里生成一段看起来无懈可击的代码、一段措辞专业的分析、一串格式工整的参考文献。可这些东西是否正确、是否真实存在、是否被你真正理解过，从成品表面完全看不出来。正因为 AI 让“看起来对”变得如此廉价，“证明它真的对、并且讲清楚它从哪来”才变得如此关键。这一章的全部规范，归根结底只在保护一件事：让你的项目始终处在可被复核的状态。

本节要点

科学建立在“别人可以复核你”这个前提上。AI 让“看起来对”变得廉价，所以“证明它真的对、并讲清楚它从哪来”才变得关键。诚信不是禁令，是让复核链不断裂的制度保障。

19.2 四栏 AI 使用说明：报告里必须有的一段

这门课要求每一份项目报告都包含一个独立段落，叫做“AI 使用说明”。它不是附录，不是免责声明，而是报告的正式组成部分，和你的物理分析、验证清单一样要认真写。这一段由四栏构成，每一栏回答一个问题，合起来构成一份诚实的协作交代。

第一栏回答“我用了什么工具”。你要写清楚自己实际使用的模型和开发环境，比如用的是 DeepSeek 的网页版还是 API，配的是 Trae 还是别的 IDE。这不是给某个产品打广告，而是因为不同模型、不同版本的行为有差异。写清楚工具，是别人复核时的第一个落脚点。第二栏回答“我让它做了什么”。你要交代把哪些任务交给了 AI：是让它写积分循环，还是让它帮你调试一个报错，还是让它把你的思路改写成更顺的中文。把任务边界说清楚，别人才知道这份成果里哪些是你想的、哪些是 AI 实现的。第三栏回答“它哪里错了、哪里我不放心”。这一栏最能体现你是不是真的在审 AI 而不是盲信它。你要写出 AI 输出里被你抓到的错误，或者虽然没抓到错但你心里没底的地方。一份完全没有这一栏内容的报告，反而最可疑，因为没有任何 AI 协作是从头到尾零失误的。第四栏回答“我怎么核验的，最终责任由我承担”。你要写明用了什么手段去验证 AI 的输出，是量纲检查、解析极限、守恒律，还是和已知值对照，并且明确写下：最终结果的责任由你承担。这句话不是客套，它的意思是，如果报告里有错，你不能用“是 AI 写的”当借口。

■ **Example 19.1：一段合规的四栏 AI 使用说明**。本项目研究 RC 电路放电的数值解。**工具**：使用学校补贴的 DeepSeek 网页版（V3 模型），开发环境为 Trae IDE。**任务**：让 AI 根据我写的需求文档生成显式 Euler 法的 Python 积分代码，并帮我把绘图部分的中文标签调好；物理建模、参数选择和验证方案由我自己完成。**出错与不放心处**：AI 初版把时间常数写成 $\tau = R / C$ ，量纲不对，被我在量纲检查时抓到并改成 $R * C$ ；另外它默认的步长 dt 偏大，我对它给的稳定性判断不放心，自己又补了一次步长减半的收敛性测试。**核验与责任**：我

用量纲分析、与解析解 $V_0 e^{-t/\tau}$ 的对照、以及两个步长的收敛性检查共同确认了结果，经过一个时间常数后电压约为初值的 37%，与理论一致。本报告所有数值结论的最终责任由我承担。

写这一段的时候，诚实比好看重要得多。你不必把自己包装成一个全程没出过岔子的完人。恰恰相反，把 AI 在哪里出错、你怎么发现的写清楚，才是这门课最想看到的东西。

本节要点

四栏说明不是免责声明，是报告的正式组成部分。工具、任务、出错与不放心处、核验与责任，四栏合在一起，构成一份诚实的协作交代。把 AI 在哪出错、你怎么发现的写清楚，远比假装全程顺利更能得分。

19.3 三条红线：越过即按学术不端处理

四栏说明是日常规范，下面三条则是底线。它们不是建议，越过任何一条都按学术不端处理。理解它们为什么是红线，比单纯记住它们更重要。

第一条红线：不能把 AI 生成的段落直接复制粘贴进报告正文，既不修改也不标注。AI 写的文字可以是你的素材、你的草稿、你的起点，但当它出现在报告正文里、以你的名义被陈述时，它必须经过你的改写和消化，或者被明确标注为 AI 生成。直接粘贴而不交代，等于把别人写的东西冒充成自己的。这在任何学术语境里都是抄袭，AI 不会因为它是机器就让这件事变得无害。第二条红线：不能用 AI 编造数据。这一条尤其要警惕，它最容易在不知不觉中被越过。AI 非常擅长生成“看起来很合理”的数字。你让它“给一组单摆周期的实验数据”，它能立刻给你一串误差大小都很逼真的读数，但那是凭空捏造的。哪怕你用的是仿真数据，也必须在报告里明确标注这是模拟而非实测，并说清楚物理参数从哪来。把模拟数据当成实测数据呈现，是科学里最严重的造假，没有之一。第三条红线：不能直接采用 AI 给出的参考文献而不经核验。这一条呼应了文献那一章讲过的内容：AI 编造文献是出了名的，它能生成作者、期刊、卷号、页码、年份样样齐全、看上去毫无破绽的引用，而那篇论文根本不存在。任何 AI 给你的文献，都必须经过 DOI、arXiv 或 Google Scholar 反查。查不到原文的，不计入你的有效文献，更不能写进报告。

这三条之所以是红线而不是普通规范，是因为它们破坏的不是某个细节的准确性，而是科学复核链本身。冒充、造假、伪造出处，每一条都让别人无法再信任你报告里的任何一句话。

本节要点

三条红线不是规范，是底线：不能粘贴 AI 文字不标不引，不能用 AI 编造数据，不能用未核验的 AI 文献。它们破坏的是科学复核链本身，不是某个细节的准确性。跨过任何一条，报告里的每一句话都不再可信。

学术红线

19.4 主动报告灰色地带：失报本身即违规

真实的项目里，事情很少是非黑即白的。总会有灰色地带：有一段话其实是 AI 写的，你读着觉得不错，没怎么改就用了；有一组数据，你让 AI 模拟了出来，但自己也拿不准物理参数选得对不对；有一条文献，AI 给了你，看起来像那么回事，但你确实没找到原文。这些情况单独看，未必就构成红线问题。关键在于你怎么处理它们。

这门课的契约核心在这里：你必须在 AI 使用说明段里主动报告这些灰色地带。”这段是 AI 写的我没怎么改”、”这段我让 AI 模拟了数据”、”这条文献我没找到原文”，这类交代都要主动写出来。规则非常清晰，也非常宽容：主动报告不扣分。你坦白说出某段是 AI 写的、某条文献没核到，老师不会因为这份坦白扣你的分，因为透明本身正是这门课要训练的能力。但反过来，不报告而被发现，按红线处理。也就是说，失报本身就已经是违规，哪怕底下那件事其实没什么大问题。这条规则把抽象的诚信原则落成了一个可执行的判据：判断你是否守诚信，不看你有没有用过 AI、有没有走过捷径，而看你有没有把它如实摊在桌面上。一个把灰色地带全部主动报告出来的项目，是诚实的项目；一个把灰色地带藏起来、赌老师查不到的项目，已经踩了红线。

AI 陷阱预警 · 用了 AI 却假装没用，是最隐蔽的失败

这门课里最危险的不是 AI 犯的技术错误，那些靠验证清单大多能抓出来。最危险的是两种诚信上的塌陷。第一种是”用了 AI 却假装没用”：报告通篇不提 AI，仿佛每一行代码、每一句分析都是自己徒手写的。这种隐瞒一旦在答辩问答里被一句”你这段为什么这么写”问穿，性质立刻从”用了 AI”变成”撒了谎”，后者严重得多。第二种是把判断整个外包给 AI：代码让它写、验证让它做、结论让它下，你只负责复制粘贴和提交。这种做法短期看效率惊人，长期却在悄悄掏空你。那些定义你成为科学家的能力，正在从不被使用中萎缩。主动报告能救你于第一种，认知所有权的自觉能救你于第二种。

本节要点

主动报告不扣分，失报本身即违规。这条规则把诚信落成了一个可执行的判据：不看你有没有用过 AI、有没走过捷径，只看你有没有把它如实摊在桌面上。坦白交代的项目是诚实的，藏起来的已经踩了红线。

19.5 来源链：让每一个结论都可以回溯

科学结论不是凭空冒出来的，它有一条来路：一个想法引出一个验证它的实验，实验产生证据，证据被代码处理成图，图支撑起报告里的某个论断。如果这条来路的任何一环断了，比如你说不清某张图是用哪段代码、哪组数据画出来的，那这张图支撑的结论就失去了根基。把这条从想法到论断的来路完整保留下来，就是来源链。

Definition 19.5.1：来源链 provenance. 来源链指的是一个项目里，想法、证据、代码、图、论断之间环环相扣、可被回溯的出处关系。它要回答的问题是：这个结论从哪来？支撑

它的图是哪段代码生成的？那段代码处理的是哪组数据？那组数据是实测的还是模拟的、参数从何而来？一条完整的来源链让任何一个外部读者都能从最终论断逆流而上，一路追到最初的想法和原始数据。

在这门课里，来源链不是一个抽象概念，它由三样具体的东西共同构成。第一样是你每天的 AI 协作记录，也就是 log 目录里保存的关键对话片段，它记录了你和 AI 在每个阶段到底谈了什么、AI 给了什么、你怎么改的。第二样是 Git 提交历史，它像一台摄像机，逐次记录了代码和文档怎样一步步演变成现在这个样子，每一次提交都是一个带时间戳的快照。第三样就是报告里那段四栏 AI 使用说明，它把整个协作过程压缩成一段读者最先看到的交代。这三样东西叠在一起，你的项目就处在一个完全可追溯的状态：别人既能看到你的结论，又能沿着来源链一路倒查到你最初的想法和原始数据。维护来源链不是额外负担，它恰恰是你在答辩现场被追问时最硬的底气。

本节要点

来源链由三样东西构成：每日 AI 协作记录 (log)、Git 提交历史、四栏 AI 使用说明。三样叠在一起，项目就处在完全可追溯的状态，别人能从你的结论逆流而上，一路追到最初的想法和原始数据。

来源链

19.6 认知所有权：为什么不让 AI 全做

现在来回答这本书背后那个最根本的问题，一个你大概早就想问的问题：既然 AI 这么能干，为什么大一的我还要自己写需求文档、自己跑验证、自己审计 AI 的输出？让 AI 把这些全做了，不是又快又省事吗？

诚实地说，如果只看短期效率，让 AI 全做确实更快。这门课从不否认，否认反而显得虚伪。真正的答案不在效率这一层，而在一个叫认知所有权的東西上。设想你把项目的每一步都委托出去：想法让 AI 提，方案让 AI 定，代码让 AI 写，结果让 AI 验，结论让 AI 下，你只在最后点一下提交。短期看你交出了一个完整的项目。可是在这个过程中，那些真正定义一个人成为科学家的能力，提出假设、判断证据是否决定性、把一段论证拆开重构、在别人质疑时有理有据地回应，你一次都没有用到。能力和肌肉一样，不被使用就会萎缩。你省下的那些时间，是用未来的判断力换来的。这笔交易在你这个阶段极不划算。

这就是为什么这门课所有的反射弧和工件，都被刻意设计成不能被 AI 替你完成的样子。需求文档逼你亲自厘清问题的边界，验证清单逼你亲自判断结果可不可信，审计动作逼你亲自对 AI 的输出做出取舍。把这些动作留给你自己做，不是因为这门课不信任 AI，而是因为这门课要保住的，是你对自己项目的认知所有权：这个项目是你想清楚的，不是 AI 替你清楚的。AI 应该扮演的角色，是帮你转化素材、让过程透明、支持你反复修订，而不是替你完成那个思考本身。守住这一点，你才是项目的主理人，而不是一个替 AI 跑腿、最后给它的输出盖橡皮印章的人。

本节要点

让 AI 全做短期更快，但你省下的时间是用未来的判断力换来的。这门课刻意把想、验、审留给你自己，要保住的不是效率，而是你對自己项目的认知所有权：这个项目是你想清楚的，不是 AI 替你想清楚的。

19.7 工具的现实选择：国内栈是平替，不是降级

讲完了规范和原则，落到最实际的一个问题：这门课要不要你自费买境外的 AI 服务，要不要你翻墙？答案是都不用。这门课的默认工具栈，就是学校补贴的国内栈，比如 DeepSeek 配 Trae。学校补贴 tokens，你零自费、不必翻墙，就能完成全部作业。需求文档怎么写、架构图怎么画、验证清单怎么跑、Skill 文件怎么抽象，这些规范在哪个模型上都通用，不挑工具。

这里要把一个态度说明白：国内模型不是降级版的将就，而是货真价实的平替。这门课的评分只看你怎么用 AI，不看你用了哪家的 AI。你用 DeepSeek 把一段 AI 写的积分代码审出量纲错误，和别人用境外模型做同样的事，在这门课里得到的评价完全一样。想用别的工具的同学当然自便，但那绝不是必需，也绝不会加分。把心思花在“用什么高级工具”上是走偏了。这门课真正较真的，是你有没有在你手边那个工具上，把审计、验证、透明这套动作做扎实。

本节要点

国内模型是平替，不是降级。这门课的评分只看你怎么用 AI，不看你用了哪家的 AI。把心思花在“用什么高级工具”上是走偏了。真正较真的是你有没有在手里那个工具上把审计、验证、透明做扎实。

本能 · 诚信契约是 Trust but Verify 的延伸

回顾全书的五条反射弧，你会发现这一章的诚信契约并不是新东西，而是它们的自然延续。Spec-first 让你先想清楚再动手，Architecture-before-code 让你掌握全局，Trust but Verify 让你在六十秒内对 AI 的输出起疑并验证，Ship something real 让你交出经得起别人玩的工件，Skill-building 让你把可复用的动作固化下来。这一章做的，是把这套反射弧从“对待代码”扩展到“对待你自己”：验证不只针对 AI 写的曲线，也针对你这个项目是否诚实、是否可追溯；透明不只是把代码跑给别人看，也是把你和 AI 的全部协作摊开给别人查。Trust but Verify 这条反射弧，最终也要用在你自己的诚信上：先信任你做的项目是好的，再用来源链和 AI 使用说明把它验证给别人看。

本章小结

把全书分散的规范收拢成一份可执行的诚信契约。报告里必须有一段四栏 AI 使用说明，交代用了什么工具、让它做了什么、它哪里错了或你哪里不放心、你怎么核验并由你承担最终责任。三条红线：不能把 AI 段落直接粘贴进正文不改不引，不能用 AI 编造数据，不能用未经 DOI 反查的 AI 文献。灰色地带必须主动报告，主动报告不扣分，失报本身即

违规。来源链由每日 AI 协作记录、Git 提交历史和 AI 使用说明三者构成，让每个结论都能回溯到最初的想法和原始数据。这一切的深层逻辑是认知所有权：让 AI 全做短期更快，但你会就此失去那些定义科学家的能力，所以这门课刻意把想、验、审留给你自己。工具上，学校补贴的国内栈是平替不是降级，评分只看你怎么用。

第四部分到此结束。接下来的项目实战部分，把反射弧和诚信契约第一次完整用在一个真正属于你自己的物理项目上，从选题到 ship，走完全程。

思考题

1. 为什么“主动报告灰色地带不扣分、失报却按红线处理”这条规则，反而是在保护你而不是为难你？请结合科学复核链的角度，说明一份坦白交代了“某段是 AI 写的、某条文献没找到原文”的报告，为什么比一份看上去天衣无缝、只字不提 AI 的报告更可信。
2. 假设你把一个项目从选题、写需求文档、写代码、跑验证到下结论全部交给 AI 完成，自己只负责提交。短期看你确实更快地交出了成果。请你具体说出，在这个过程中你没有用到、因而也没有得到锻炼的，是哪几项“定义科学家”的能力。半年后再遇到一个 AI 没见过的新问题时，这笔交易的代价会怎样显现出来？
3. 来源链由每日 AI 协作记录、Git 提交历史和 AI 使用说明三部分构成。如果这三部分里恰好缺了 Git 提交历史，只剩下另外两样，你的项目在可追溯性上会丢掉什么？请举一个具体场景，说明在答辩时缺了提交历史会让你难以回答某个追问。



项目实战与案例

20 十个项目案例 179

- 20.1 P1 大角度单摆：精确周期与小角度近似的决裂
- 20.2 P2 抛体含空气阻力：线性与二次阻力的两种世界
- 20.3 P3 RC、RL、RLC 电路充放电：从一阶到二阶的电路演化
- 20.4 P4 二维行星轨道：开普勒问题与守恒量审判
- 20.5 P5 杯水冷却：牛顿冷却律与一把温度计
- 20.6 P6 弹簧振子链：多自由度耦合与简正模式
- 20.7 P7 蒙特卡洛估算 π 与定积分：让随机数替你算几何
- 20.8 P8 自由落体录像：从手机视频到重力加速度
- 20.9 P9 一维随机行走：从醉汉散步到扩散定律
- 20.10 P10 光学衍射图案：从激光斑点到 sinc 平方拟合
- 20.11 十个项目共享的同一套骨架

A 环境搭建与装机 195

- A.1 装什么：三样东西，一个都不能少
- A.2 在 Windows 和 macOS 上的安装要点
- A.3 装好后的三件套自测
- A.4 常见装机问题与对策
- A.5 为什么要用虚拟环境

B 工具栈选择指南 201

- B.1 先按你的处境分流
- B.2 各类工具简介
- B.3 关于境外工具
- B.4 一句务实的建议

C Fermi 估算题库 205

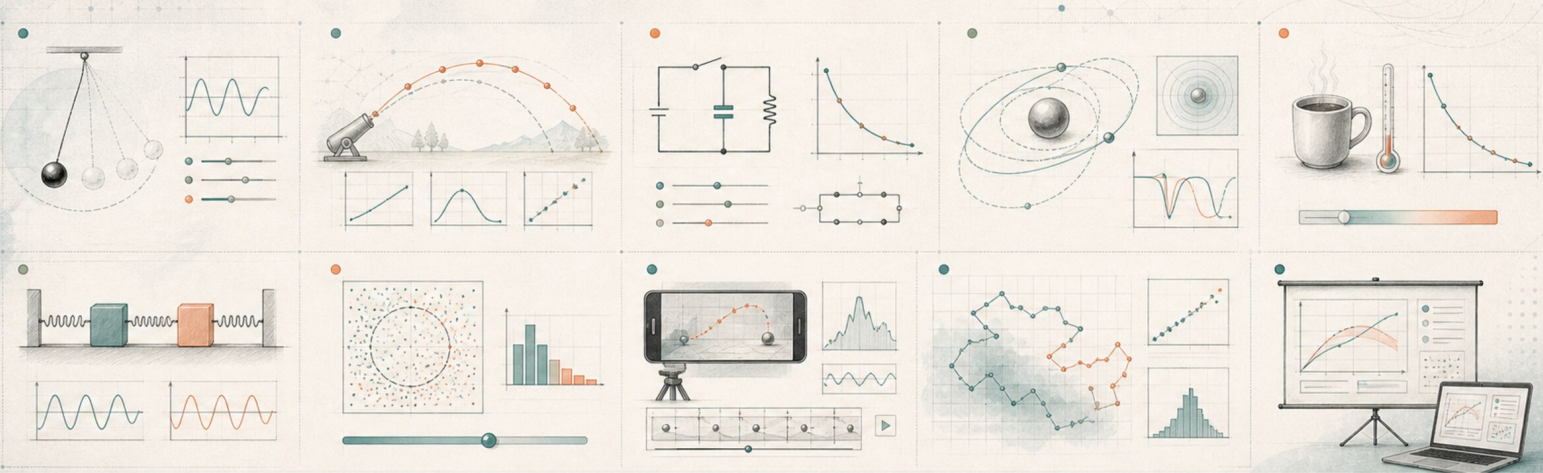
- C.1 第一组：日常生活
- C.2 第二组：地球与天空
- C.3 第三组：物理与工程
- C.4 第四组：生物与人体
- C.5 练这身本领到底是为了什么

D 项目报告与展示写作 211

- D.1 摆正位置：工件是主角，报告是它的说明
- D.2 一份短报告里该有什么
- D.3 答辩 slides：七页以内，一页一个要点
- D.4 Demo 怎么讲：让观众亲手把图调动

参考文献 217

索引 219



20. 十个项目案例

前面四个部分把这门课的全套手艺一件件交给了你：像物理学家一样把模糊想法压成需求文档，把系统拆成架构块，认出一阶、二阶、二维 ODE 共享的数值骨架，用最小二乘和蒙特卡洛处理真实数据，再用验证清单和审计把 AI 的输出钉死，最后把能跑的代码 ship 成一个别人点开就能玩的工件，并把反复做的动作抽象成你自己的 skill。这一部分不再讲新方法，而是把这套手艺放进十个具体的物理项目里，让你看清楚从选题到交付完整走一遍是什么样子。

这十个项目全部落在大一普物加高数加线代的范围内，每一个都经过可行性验证，每组六到八人用八个有效工作日做得完。它们是同一套反射弧在不同物理情境下的十次演练，并非十道彼此无关的题。有的偏数据采集，自己用手机录视频、拿温度计测水温；有的偏纯数值实验，蒙特卡洛、随机行走；有的要上线代的矩阵特征值。你可以照着其中一个做，也可以在老师审批可行性之后自选题。无论选哪个，下面给你的都是同样的四样东西：这个项目研究什么物理、关键方程是什么，即物理背景与核心问题；做到什么程度算合格、至少要交出哪些图和结果，即最低交付要求；学有余力可以往哪深挖，即扩展方向；以及这个具体物理问题里 AI 最容易在哪里翻车，即典型 AI 陷阱预警。

本能 · 每个项目都走满五条反射弧

这十个项目没有一个是“写完代码就完事”。拿到任何一个题，你都要把五条反射弧从头走一遍：先写需求文档，把“做什么、输入输出是什么、怎么算成功、用什么技术栈”压成一页纸；再画架构，把项目拆成输入、模型、求解器、输出、可视化几块，为每个 trade-off 给出理由；写完代码立刻进验证，用解析极限、守恒量、参考值给数值解打分，而不是看图顺眼就信；然后 ship，把脚本包装成一个带滑条的 Streamlit 网页或

一键启动的工件，让没看过代码的人也能玩；最后把这个项目里你做了三遍以上的动作，比如”给 ODE 数值解做收敛性检查”或”从视频帧提坐标”，抽象成一个 skill，下次解新问题时直接激活。读下面每个案例时，请有意识地问自己：这一题的需求文档该怎么写，架构图长什么样，验证清单里该放哪几条物理判据。

本章地图		
小节	这一节解决什么	关键方法 / 产物
§20.1	大角度单摆周期对小角度近似的偏离	ODE 数值积分，椭圆积分解析对照，视频实测验证
§20.2	有阻力抛体的轨迹与射程	二维 ODE，线性/二次阻力，终端速度验证
§20.3	从一阶到二阶的电路充放电演化	一阶/二阶 ODE，RLC 三种阻尼区制判据，解析解对账
§20.4	二维行星轨道的守恒量监控	中心力 ODE，能量与角动量守恒追踪，辛积分器
§20.5	真实杯水冷却曲线的测量与拟合	一阶 ODE 拟合，最小二乘，残差系统性趋势分析
§20.6	弹簧振子链的简正模式与色散	耦合 ODE，矩阵特征值对角化，简正频率与模式花样
§20.7	蒙特卡洛估算 π 与定积分	随机采样， $1/\sqrt{N}$ 误差标度验证，重要性采样
§20.8	从手机视频提取重力加速度	图像坐标追踪，像素-米换算，二次曲线最小二乘拟合
§20.9	一维随机行走的均方位移与扩散	随机模拟， $\langle x^2 \rangle \sim N$ 线性验证，高斯分布检验
§20.10	单缝衍射图案的 sinc 平方拟合	强度剖面提取，角度换算， sinc^2 非线性拟合
§20.11	十个项目的共同骨架归类	ODE 积分、随机采样、数据拟合、矩阵对角化四类骨架

20.1 P1 大角度单摆：精确周期与小角度近似的决裂

20.1.1 物理背景与核心问题

单摆是大一普物里最早被”近似”驯服的系统。教科书告诉你，长度为 L 的单摆，摆角 θ 满足

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L} \sin \theta,$$

这是一个非线性的二阶常微分方程，因为右边是 $\sin \theta$ 而不是 θ 。当摆角很小时， $\sin \theta \approx \theta$ ，方程退化成简谐振动 $\ddot{\theta} = -(g/L)\theta$ ，于是得到那个人人会背的结论：周期 $T_0 = 2\pi\sqrt{L/g}$ ，

与振幅无关。这个“周期与振幅无关”的等时性，正是小角度近似最迷人也最危险的地方。**等时性**一旦摆角真的大起来，等时性就垮了。精确周期可以写成第一类完全椭圆积分

$$T(\theta_0) = 4\sqrt{\frac{L}{g}} K\left(\sin \frac{\theta_0}{2}\right) = T_0 \left[1 + \frac{1}{16}\theta_0^2 + \frac{11}{3072}\theta_0^4 + \cdots\right],$$

摆角越大，真实周期越比 T_0 长。核心问题：从多大的摆角开始，小角度近似不再够用？让学生亲手在自己录的视频里看到这个决裂，比背十遍公式都管用。

20.1.2 最低交付要求

把二阶方程拆成两个一阶方程，这是第九章的招牌动作。令 $\omega = \dot{\theta}$ ，则 $\dot{\theta} = \omega$ 、 $\dot{\omega} = -(g/L)\sin\theta$ 。用数值积分解出几个不同初始摆角下的 $\theta(t)$ 曲线，从过零点的时间间隔量出每条曲线的周期。合格的最低交付，是一张“周期随初始摆角变化”的图：横轴是初始摆角从几度到一百多度，纵轴是测出的周期。图上同时画三条东西：数值解给出的精确周期、小角度公式 T_0 的水平线、以及椭圆积分展开的前一两项。还要用手机录一段真实单摆，一根线吊个钥匙扣即可，分别拍小角度和大角度两种摆动，从视频里量出周期，把这两个实测点叠到图上验证趋势。能讲清楚“在百分之一相对误差的标准下，小角度近似的适用上限大约在多少度”，这个项目就合格了。

20.1.3 扩展方向

学有余力，可以给单摆加上空气阻力或轴承摩擦项，看阻尼如何让振幅逐渐衰减、周期如何随之缓慢变化。这把项目和 P2 的阻力建模连了起来。也可以做相图，在 (θ, ω) 平面上画出不同能量下的轨迹：小角度时是椭圆，大角度时如何变形，能量再大到能翻过最高点时轨迹如何从闭合变成开放。这是理解非线性振动最漂亮的一张图。还可以把椭圆积分用 `scipy.special.ellipk` 算出来当作“准解析”参考，和你的数值周期对账到三位有效数字。

AI 陷阱预警 · 把小角度近似偷偷塞回大角度问题

让 AI 写单摆代码时，它极容易“好心”地把 $\sin\theta$ 直接写成 θ ，因为简谐近似在它见过的教科书例子里出现得太频繁。一旦它这么做，无论你把初始摆角设到九十度还是一百二十度，跑出来的周期都会顽固地等于 $2\pi\sqrt{L/g}$ ，等时性假象会骗过粗心的人。这个项目的全部意义恰恰是看到等时性破裂，所以这是致命错误。你必须逐字确认方程右边是 `-(g/L)*np.sin(theta)` 而不是 `-(g/L)*theta`，并且做一个一眼就能看穿的判据检查：初始摆角设成大角度时，数值周期必须明显大于 T_0 。如果它纹丝不动地等于 T_0 ，代码一定偷用了线性近似。另一个隐蔽陷阱是角度单位，AI 可能在某处把角度按度数处理而 `np.sin` 只认弧度，导致 `sin(90)` 被当成 `sin(90 弧度)`，结果全乱。

本节要点

单摆周期随摆角增大而单调增长，这条关系本身就是最可靠的判据。数值周期纹丝不

动地等于 T_0 ，代码一定偷用了小角度近似；数值周期明显大于 T_0 ，才说明你解的是真正的非线性摆方程。

20.2 P2 抛体含空气阻力：线性与二次阻力的两种世界

20.2.1 物理背景与核心问题

无阻力抛体是高中就会的抛物线，但真实世界里空气阻力让一切都变了样。这个项目把抛体从理想化的抛物线拉回现实，核心是阻力的两种形式。低速小物体，比如雾滴、慢速小球，受到的阻力近似正比于速度，写成 $\vec{F} = -b\vec{v}$ ，叫线性阻力或斯托克斯阻力；高速大物体，棒球、炮弹，受到的阻力近似正比于速度平方，写成 $\vec{F} = -c|\vec{v}|\vec{v}$ ，叫二次阻力或牛顿阻力。注意二次阻力的方向必须始终与速度反向，所以写成 $-c|\vec{v}|\vec{v}$ ，不能在两个分量上各自取平方。把牛顿第二定律写成分量形式，水平和竖直方向各是一个二阶方程，等价于一组二维一阶 ODE。一个关键物理量是终端速度：竖直下落时重力与阻力平衡，线性阻力下 $v_{\text{ter}} = mg/b$ ，二次阻力下 $v_{\text{ter}} = \sqrt{mg/c}$ 。它给了你一个绝佳的验证锚点。核心问题：在相同初速度和发射角下，无阻力、线性阻力、二次阻力三种模型预测的射程和轨迹差多少？哪一种更接近真实视频里的轨迹？

20.2.2 最低交付要求

把抛体在两个方向上的运动写成一组一阶 ODE 用数值积分求解，分别实现无阻力、线性阻力、二次阻力三种情形。画出三条轨迹（ x 对 y ）叠在同一张图上，直观看出阻力如何把对称的抛物线压成前陡后缓的不对称曲线、如何缩短射程。必须做终端速度验证：把发射改成竖直下落、初速度设为零，让小球自由下落足够久，检查数值给出的稳态速度是否收敛到解析终端速度 mg/b 或 $\sqrt{mg/c}$ 。吻合到三位有效数字，才算求解器可信。再做无阻力极限验证：把阻力系数调到接近零，三条轨迹应当重合到熟悉的抛物线。能用一段真实视频，自录或公开的投篮、踢球片段，提取几个轨迹点，定性说明哪种阻力模型更贴合，项目合格。

20.2.3 扩展方向

可以做“最优发射角”的研究：无阻力时最大射程对应四十五度，加上阻力后最优角会偏离四十五度。用代码扫描发射角，找出有阻力时的最优值，并解释它为什么变小。也可以引入随高度变化的空气密度，高空稀薄，看长程弹道如何被修正。还可以把阻力系数当作拟合参数，用真实视频提取的轨迹反推出球在这段空气里的有效阻力系数。这就把 P2 和 P8 的视频提取、第十一章的最小二乘拟合串成一条线。

AI 陷阱预警 · 二次阻力的方向写错与无阻力极限不归位

二次阻力 $-c|\vec{v}|\vec{v}$ 是 AI 最容易写错的地方。常见错误是把它在两个分量上写成 $-cv_x^2$ 和 $-cv_y^2$ ，这丢掉了“阻力与速度严格反向”这个物理事实：当小球上升时 $v_y > 0$ ，阻力的竖直分量应当向下，可 $-cv_y^2$ 恒为负看似没错；但水平分量在小球往左飞（ $v_x < 0$ ）时 $-cv_x^2$ 仍然为负、方向也向左，等于在帮它加速，物理就垮了。正确写法必须先算合速

率 $\text{speed} = \sqrt{v_x^2 + v_y^2}$ ，阻力分量是 $-c \cdot \text{speed} \cdot v_x$ 和 $-c \cdot \text{speed} \cdot v_y$ 。验证的判据就是终端速度和无阻力极限：把 c 调到极小，轨迹必须回到标准抛物线。回不去，方向或系数一定写错了。

本节要点

终端速度是抛体含阻力问题里最硬的验证锚点。竖直下落足够久之后数值速度必须收敛到 mg/b 或 $\sqrt{mg/c}$ ，吻合不到三位有效数字，求解器就不可信。无阻力极限下三条轨迹必须重合回标准抛物线，这一关过不了，阻力项的写法一定有方向或系数错误。

20.3 P3 RC、RL、RLC 电路充放电：从一阶到二阶的电路演化

20.3.1 物理背景与核心问题

这个项目是第八章 RC 放电例子的全面扩张，把电路从一阶推到二阶。这是练 Euler 法和数值积分最干净的舞台，因为每一种电路都有解析解可以对账。RC 电路前面已经讲透，电压满足一阶方程 $dV/dt = -V/(RC)$ ，时间常数 $\tau = RC$ 。RL 电路里电感电流满足结构完全相同的一阶方程 $dI/dt = -RI/L$ ，时间常数 $\tau = L/R$ 。这正好印证了第八章的核心思想：物理背景不同，数学骨架可以一模一样。真正的新东西是 RLC 串联电路，它是一个二阶方程

$$L \frac{d^2 q}{dt^2} + R \frac{dq}{dt} + \frac{q}{C} = 0,$$

和阻尼振子一字不差。所以它有欠阻尼（振荡衰减）、临界阻尼、过阻尼（不振荡的单调衰减）三种区制，分界由 R 与 $2\sqrt{L/C}$ 的大小关系决定。核心问题：电阻从小调到大，RLC 电路如何从振荡衰减一路过渡到不振荡？临界阻尼那条分界线在哪里？

20.3.2 最低交付要求

至少完整做出 RC（或 RL）的一阶充放电和 RLC 的二阶响应。一阶部分要求用显式 Euler 法手写求解，并叠上解析解 $V_0 e^{-t/\tau}$ 验证。还要做第八章强调过的收敛性检查，步长减半重跑，确认结果不变；以及稳定性观察，步长大于 2τ 时 Euler 翻车。RLC 部分把二阶方程拆成两个一阶方程求解，画出至少三条曲线展示欠阻尼、临界阻尼、过阻尼三种区制，并标出每种情形的 R 取值和它与临界电阻 $2\sqrt{L/C}$ 的关系。能讲清楚临界阻尼是“最快不振荡地回到平衡”这一物理意义，项目合格。下面给一段 RLC 求解的关键骨架示意，让你看到二阶方程如何被拆成状态向量推进。

Listing 20.1: RLC 串联电路的数值求解骨架（电荷 q 与电流 i 作为状态）

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 物理参数（一律 SI 单位）
5 L = 1.0e-3          # 电感，单位 亨利
```

```

6 C = 1.0e-6      # 电容，单位 法拉
7 R = 20.0        # 电阻，单位 欧姆（改这个值切换三种阻尼区制）
8
9 R_crit = 2.0 * np.sqrt(L / C)  # 临界电阻，用来判断区制
10
11 # 状态向量  $y = [q, i]$ ，其中  $i = dq/dt$ 
12 def deriv(y):
13     q, i = y
14     dqdt = i
15     didt = -(R / L) * i - q / (L * C)  # 由 RLC 方程整理得到
16     return np.array([dqdt, didt])
17
18 dt = 1.0e-7
19 N = 20000
20 y = np.array([1.0e-6, 0.0])  # 初始电荷 1 微库，初始电流为零
21 t = np.zeros(N + 1)
22 q = np.zeros(N + 1); q[0] = y[0]
23 for n in range(N):
24     y = y + dt * deriv(y)  # 显式 Euler 推进状态向量
25     t[n + 1] = t[n] + dt
26     q[n + 1] = y[0]
27
28 plt.plot(t, q)
29 plt.xlabel('时间 t (s)'); plt.ylabel('电荷 q (C)')
30 plt.tight_layout(); plt.show()

```

20.3.3 扩展方向

可以加上交流驱动源做受迫 RLC，扫描驱动频率画出共振曲线，找出共振频率 $\omega_0 = 1/\sqrt{LC}$ 和品质因数 Q 。这把电路和受迫振动连起来。如果有实验条件，用示波器或声卡录下真实电路的放电波形，把实测曲线和数值解对账，这是数据采集型的加分项。还可以比较显式 Euler 和更精巧的半隐式方法在 RLC 上的稳定性差异，看看为什么振荡系统对积分方法更挑剔。

AI 陷阱预警 · 时间常数的乘除颠倒与二阶方程降阶时漏项

第八章已经记录过 AI 把 $\tau = RC$ 错写成 R/C 的量纲错误。RL 电路里同样的陷阱换了张脸：时间常数是 L/R ，不是 LR ，也不是 R/L 。AI 经常凭“惯性”套用 RC 的乘法形式。永远用量纲检查兜底，时间常数必须是秒。RLC 的新陷阱出现在二阶降阶时：把 $L\ddot{q} + R\dot{q} + q/C = 0$ 整理成 $\ddot{q} = -(R/L)\dot{q} - q/(LC)$ 时，AI 偶尔会漏掉某一项的系数，或把 $q/(LC)$ 错写成 q/C ，漏了除以 L 。判断办法是看区制：用解析的临界电阻 $2\sqrt{L/C}$ 当标尺，把 R 设得远小于它应当看到明显振荡，远大于它应当单调衰减，正好等于它

应当是临界阻尼。如果你调 R 时区制的切换点和 $2\sqrt{L/C}$ 对不上，方程一定降阶错了。

本节要点

RLC 电路三种阻尼区制的切换点是 $R = 2\sqrt{L/C}$ 。 R 小于它时振荡，大于它时单调衰减，正好等于它时是临界阻尼。如果你调 R 时区制切换点和这个值对不上，一定是降阶时漏了系数。临界阻尼的物理意义是“以最快速度不振荡地回到平衡”，这个判断比任何数学推导都直观。

20.4 P4 二维行星轨道：开普勒问题与守恒量审判

20.4.1 物理背景与核心问题

一颗行星绕太阳运动，受到的是指向太阳的万有引力 $\vec{F} = -GMm\vec{r}/r^3$ ，这是一个平方反比的中心力。在二维平面上写成分量形式，就是一组关于 (x, y, v_x, v_y) 的二维一阶 ODE。这个项目的物理灵魂不在于解出椭圆轨道本身，而在于两个守恒量，：总能量 守恒量 $E = \frac{1}{2}mv^2 - GMm/r$ 和角动量 $L = m(xv_y - yv_x)$ ，在整个运动过程中必须保持不变。它们的用途不是算轨道。它们是审判数值解对不对的最锋利的两把刀。核心问题：用一个朴素的数值积分器去推行星轨道，能量和角动量真的守恒吗？答案往往令人不安，不守恒。而这恰恰是最有教育价值的地方。

20.4.2 最低交付要求

用数值积分解出行星在引力下的二维轨道，画出 x 对 y 的轨迹图。给定合适的初始位置和速度时，它应当是一个闭合的椭圆。关键的合格门槛是守恒量监控：在积分的同时记录每一步的总能量和角动量，画出它们随时间的变化曲线。如果你用最朴素的显式 Euler，会清楚地看到能量持续上涨、轨道像螺旋一样慢慢往外飘。这是数值方法的系统性漂移，不是物理，必须如实呈现并解释。然后换用一个更适合的方法：辛积分器如 leapfrog，或直接调用 `scipy.integrate.solve_ivp` 并收紧误差容限，让能量和角动量在长时间积分下保持稳定，轨道闭合不漂移。能讲清楚“为什么显式 Euler 会让能量漂移、辛方法为什么能守住能量”，并用开普勒第三定律（周期平方正比于半长轴立方）做数量级验证，项目合格。

20.4.3 扩展方向

可以验证开普勒三大定律：第一定律，轨道是椭圆，太阳在焦点；第二定律，相同时间扫过相同面积，等价于角动量守恒；第三定律，不同半长轴的轨道周期满足 $T^2 \propto a^3$ ，扫描几条轨道画出验证图。可以加入第二颗行星做三体问题，看到不可解析、对初值敏感的混沌轨道。还可以把平方反比力故意改成别的幂次，比如 $1/r^{2.1}$ ，观察轨道为什么不再闭合、近日点如何进动。这是理解“为什么唯独平方反比力给出闭合椭圆”的绝佳数值实验。

AI 陷阱预警 · 用显式 Euler 跑长程轨道却不查守恒量

这是整个项目里最经典、也最容易被 AI 和学生一起忽略的陷阱。AI 默认会用最简单的显式 Euler，或同样不保辛的 RK4 长时间跑。轨道在前几圈看起来完美闭合，于是人就信了。可一旦积分上几十圈，能量会单调漂移、椭圆会缓慢张开成螺旋。问题的要害是：只看轨道图的人发现不了，因为前几圈真的像椭圆。唯一可靠的办法是把能量和角动量当作独立的监控量画出来，它们的相对变化超过百分之一就说明数值解已经不可信。另一个相关陷阱是步长在近日点处过大：行星越靠近太阳走得越快、力变化越剧烈，固定步长在近日点会严重失真，表现为每过一次近日点能量就跳一下。务必做收敛性检查，并优先选辛积分器或自适应步长。还要警惕 AI 把引力写成 $-GMm\vec{r}/r^2$ ，漏了一个 r ，应当是 $\vec{r}/r^3$ 因为 $\vec{r}/r$ 才是单位向量。量纲和近日点行为都会出错。

本节要点

只看轨道图是看不出来数值解漂移的。能量和角动量曲线是两条独立的时间序列，它们的相对变化一旦超过百分之一，轨道图再好看也是假的。显式 Euler 的能量会单调上涨。这是 Euler 法在哈密顿系统上的系统性缺陷，不是你的代码写错了。换辛积分器，能量才能守住。

20.5 P5 杯水冷却：牛顿冷却律与一把温度计**20.5.1 物理背景与核心问题**

一杯热水放在房间里慢慢变凉，是牛顿冷却律最朴素的实验场。牛顿冷却律说，物体降温的速率正比于它和环境的温差， $dT/dt = -k(T - T_{\text{room}})$ 。这是一个一阶 ODE，结构和 RC 放电几乎一样，只是多了一个环境温度常数项 T_{room} 。解析解是 $T(t) = T_{\text{room}} + (T_0 - T_{\text{room}})e^{-kt}$ ，温度从初值指数地趋向室温而不是趋向零。这个常数项带来的差别，正是第八章思考题留给你的钩子。这个项目的特别之处在于它真正“伸手可做”：一杯热水、一支温度计、一块表，每隔一两分钟记一个温度，就能采到一条真实的冷却曲线。核心问题：真实的杯水冷却到底服不服从牛顿冷却律的单指数形式？拟合出来的冷却常数 k 受什么因素影响，比如杯子大小、水面是否敞开、有没有盖子？

20.5.2 最低交付要求

亲手采一组数据：一杯热水，每隔固定时间用温度计记录温度，至少记到温度明显接近室温，得到一张时间对温度的数据表。用数值积分，或直接用解析解，解牛顿冷却方程，把模型曲线和实测数据点画在同一张图上。关键交付是参数拟合：把冷却常数 k ，以及可能未知的室温 T_{room} ，当作拟合参数，用第十一章的最小二乘方法从实测数据里反推出最优的 k 。画出数据、拟合曲线、残差三联图。能讨论残差里有没有系统性趋势，比如真实冷却在初期比单指数更快，因为蒸发散热在高温时更显著，并给出 k 的拟合值和它的物理合理性，项目合格。

20.5.3 扩展方向

可以做对照实验：敞口杯和加盖杯、大杯和小杯、有风扇和无风扇，各采一条曲线，比较拟合出的 k 如何随散热条件变化。把单个数据点的故事讲成一组受控变量的实验。可以检验牛顿冷却律的适用边界：当温差很大时辐射散热，正比于温度四次方之差，变得重要，单指数会偏离。能从残差里看出这个偏离就很漂亮。还可以反过来用冷却曲线估算房间的室温，把它当作拟合参数解出来，再和温度计直接测的室温对账。

AI 陷阱预警 · 把趋向室温错当趋向零，以及拟合初值乱设

RC 放电趋向零，牛顿冷却趋向室温。AI 受 RC 例子的强烈影响，常把冷却方程写成 $dT/dt = -kT$ ，漏掉了 T_{room} 。结果模型预测水会一直凉到零度甚至负温，这显然非物理。务必确认方程是 $-k(T - T_{\text{room}})$ ，并用一个极限判据兜底：时间足够长后温度必须收敛到室温而不是零。第二个陷阱在拟合环节：最小二乘对初值敏感，如果让 AI 把拟合的初始猜测 k 设成离谱的值，比如差了几个数量级，优化器可能不收敛或落进坏的局部解，拟合曲线完全贴不上数据。给一个合理的初始猜测，用前两个数据点估个粗略的 k ，并检查拟合后的残差是不是随机散布而非系统偏离。还要小心时间单位，数据按分钟记还是按秒记， k 的单位会差六十倍。量纲不一致，拟合就废了。

本节要点

牛顿冷却的方程是 $dT/dt = -k(T - T_{\text{room}})$ ，不是 $-kT$ 。长时极限下温度必须收敛到室温而不是零，这是区分“方程写对了”和“漏了室温项”的一眼看穿判据。残差里有没有系统性趋势，比 R^2 更能告诉你模型对不对。

20.6 P6 弹簧振子链：多自由度耦合与简正模式

20.6.1 物理背景与核心问题

把若干个小球用弹簧串成一条链，每个球只能沿链方向小幅振动，相邻球之间由弹簧耦合。这就是多自由度耦合振动的标准模型，也是这门课唯一真正用上线代矩阵特征值的项目。设第 i 个球的位移为 x_i ，每个球的牛顿方程都同时含有它左右邻居的位移。于是 N 个球给出 N 个互相耦合的二阶方程，可以写成矩阵形式 $\ddot{\vec{x}} = -\mathbf{A}\vec{x}$ ，其中 $\mathbf{A}$ 是一个由弹簧常数和质量决定的、对称的三对角耦合矩阵。这个看似纠缠的系统有一个极优雅的解构方式：求矩阵 $\mathbf{A}$ 的特征值和特征向量。每个特征向量对应一个简正模式，整条链以同一个频率整体振动的特定花样；对应的特征值开方就是这个模式的简正频率。任意复杂的初始扰动都能分解成这些简正模式的叠加。核心问题：把一条耦合的振子链对角化，它的简正频率和振动花样长什么样？简正频率随模式编号怎么排布？这正是固体物理里声子色散关系的雏形。

简正模式

20.6.2 最低交付要求

为一条 N 个球的弹簧链写出耦合矩阵 $\mathbf{A}$ ，用 `numpy.linalg.eigh`，对称矩阵专用，求出全部特征值和特征向量。把简正频率（特征值开方）按模式编号画出来，得到色散曲线的离散版；把前几个简正模式的特征向量画成“每个球的振幅花样”，以两端固定的链为基准，让人直观看到最低模式是所有球同向摆动、整条链像半根正弦那样鼓起，高阶模式则是相邻球反向、节点越来越多的快速振动。这是项目的线代核心，必须完整交付。在此基础上至少做一件动力学的事：选一个简正模式作为初始条件，用数值积分推进 $\ddot{\vec{x}} = -\mathbf{A}\vec{x}$ ，验证整条链确实以单一的简正频率振动而不混入别的模式。这是简正模式定义的直接检验。能讲清楚“对角化把 N 个耦合方程解耦成 N 个独立的简谐振子”这一物理图像，项目合格。

20.6.3 扩展方向

可以研究边界条件的影响：两端固定、两端自由、还是首尾相连的周期边界，简正频率谱会不同。周期边界正好对应固体里的玻恩-冯卡门条件。可以做“质量缺陷”实验：把链中某一个球的质量改大，看简正模式如何被局域化在缺陷附近。这是凝聚态里杂质态、安德森局域化的最简版本。还可以把一个非简正模式的初始扰动按特征向量基展开，验证它确实是各简正模式的线性叠加，并看不同模式因频率不同而逐渐“拍”出复杂的运动。这把线代的基变换和振动动力学完美缝合。

AI 陷阱预警 · 耦合矩阵的符号与边界写错，以及用错特征值函数

耦合矩阵 $\mathbf{A}$ 的搭建是 AI 最容易出错的地方。三对角矩阵的对角元和非对角元的符号必须自洽，恢复力指向平衡位置决定了对角元为正、近邻耦合元为负，或整体差一个符号约定。AI 经常把符号弄反，导致特征值出现负数，开方变成虚数，简正频率就成了无意义的复数。一个干净的判据是：物理上稳定的振子链，矩阵 $\mathbf{A}$ 必须半正定，所有特征值非负。如果算出负特征值，矩阵一定搭错了。边界是另一个雷区：第一个球和最后一个球只有一个邻居，它们对应的矩阵行必须特殊处理。AI 常常无脑套用中间球的三项格式，把不存在的邻居也耦合进来。还有一个工程陷阱：对称矩阵要用 `eigh` 而不是通用的 `eig`，后者不保证返回实特征值、也不保证特征向量正交，会给本该实数的简正频率带来虚假的微小虚部，干扰判断。验证判据很硬，但要分清边界：两端固定的链所有简正频率都严格大于零、没有零模；而两端自由的链则应当恰好出现一个零频模式，对应整条链一起平移不产生恢复力。算出的最低频率若和你设定的边界对不上（比如自由链却没有零模、或固定链却冒出一个零频），多半是边界行或符号搭错了。

本节要点

矩阵 $\mathbf{A}$ 必须半正定，所有特征值非负。算出负特征值，耦合矩阵一定搭错了符号或边界。两端固定的链没有零频模式，两端自由的链恰好有一个零频模式，对应整链平移。最低频率与边界条件对不上，说明边界行写错了。

20.7 P7 蒙特卡洛估算 π 与定积分：让随机数替你算几何

20.7.1 物理背景与核心问题

这个项目是第十二章蒙特卡洛思想的项目化落地。它不解微分方程，而是用大量随机点的统计行为去逼近一个确定的几何量。估算 π 的经典做法是往一个边长为 1 的正方形里均匀撒随机点，统计落在内切四分之一圆（半径为 1）里的点的比例。这个比例趋近于四分之一圆面积与正方形面积之比，也就是 $\pi/4$ ，于是 $\pi \approx 4 \times (\text{圆内点数}/\text{总点数})$ 。同样的思想可以推广到任意定积分：要算 $\int_a^b f(x) dx$ ，在区间里撒随机点取函数值的平均再乘以区间长度即可。蒙特卡洛的灵魂是它的误差规律：估计值的统计误差正比于 $1/\sqrt{N}$ ，其中 N 是采样点数，与被积函数的维度无关。这正是它在高维积分里碾压传统网格法的原因。核心问题：蒙特卡洛估计的精度如何随采样点数增长？那条 $1/\sqrt{N}$ 的收敛规律真的成立吗？

20.7.2 最低交付要求

实现蒙特卡洛估算 π ，画出估计值随采样点数 N 增大而逐渐逼近真实 π 的收敛图，横轴用对数刻度，点数从几百一直加到上百万。关键交付是误差标度验证：对每个 N 重复多次独立估计，算出估计值的标准差，即统计误差，把误差对 N 画在双对数图上，验证它确实以斜率 $-1/2$ 的直线下降，也就是 $1/\sqrt{N}$ 规律。再把方法推广去算至少一个有解析值的定积分，比如 $\int_0^1 x^2 dx = 1/3$ 或 $\int_0^\pi \sin x dx = 2$ ，和解析值对账。能讲清楚“为什么误差是 $1/\sqrt{N}$ 而不是 $1/N$ ，以及这个标度律为什么让蒙特卡洛在高维积分里不可替代”，项目合格。

20.7.3 扩展方向

可以做重要性采样：对于在某处很尖的被积函数，均匀撒点效率很低。按一个接近被积函数形状的分布去采样能大幅降低方差，用同样的点数得到更小的误差。这是蒙特卡洛进阶的核心技巧。可以算一个高维积分，比如十维单位球的体积，让传统网格法因维度灾难彻底失效、而蒙特卡洛依旧从容，亲眼见证 $1/\sqrt{N}$ 与维度无关的威力。还可以做 Buffon 投针这个更古老的 π 估算法，比较它和撒点法的效率。

AI 陷阱预警 · 随机种子不固定导致结果不可复现，与误差标度算错

蒙特卡洛的结果天然带随机性。如果不固定随机种子，每次跑出来的数都不一样，这给验证和复现带来麻烦。作为项目主理人，你应当在脚本里显式设定随机种子，比如 `np.random.seed(42)`，让结果可复现、可对账，同时清楚地区分“单次估计的随机涨落”和“系统性的偏差”。第二个更隐蔽的陷阱在误差标度的验证上：要得到统计误差随 N 的曲线，必须对每个 N 做多次独立重复来估标准差。AI 有时会偷懒只跑一次，然后用“估计值与真值之差”冒充统计误差。但单次的偏差是随机的，画出来是一团噪声而不是干净的 $-1/2$ 斜率直线，看不出标度律。正确做法是固定 N 、换不同种子重复几十次，用这批结果的标准差才是统计误差。还要小心一个低级但常见的错：估算 π 时判断点是否在圆内用的是 $x^2 + y^2 \leq 1$ ，AI 偶尔会漏掉平方或把半径搞错，导致估出来的不是 π 。

本节要点

蒙特卡洛误差以 $1/\sqrt{N}$ 收敛，与维度无关，这是它碾压网格法的根本原因。验证误差标度时必须对每个 N 独立重复几十次取标准差，单次运行里“估计值与真值之差”是随机的噪声，画不出干净的 $-1/2$ 斜率。固定随机种子，让结果可复现。

20.8 P8 自由落体录像：从手机视频到重力加速度**20.8.1 物理背景与核心问题**

这个项目把一段最普通的手机视频变成一次重力加速度的测量。核心物理简单到极致：自由落体的位移随时间满足 $y(t) = y_0 + v_0 t + \frac{1}{2}gt^2$ ，只要从视频里提取出物体在每一帧的竖直位置 and 对应时间，用第十一章的最小二乘拟合一条抛物线，二次项系数就给出 $g/2$ ，反推出重力加速度 g 。它的真正难点不在物理而在数据工程：怎么从视频里把每一帧物体的像素坐标提出来，怎么把像素坐标换算成真实的米，怎么从帧率得到每一帧的真实时间。这恰恰是这门课“让真实数据流进模型”这一能力的最佳训练。核心问题：用一段自录的下落视频，能把重力加速度测到多准？测量误差主要来自哪里？

20.8.2 最低交付要求

自录一段物体竖直下落的视频，手机慢动作模式更好，帧率高、时间分辨率细。视频里必须有一把已知长度的标尺，一根米尺或一张已知尺寸的纸，用来做像素到米的换算。用图像处理，手动逐帧点取，或用 OpenCV 做简单的颜色或亮度追踪，提取物体在每一帧的竖直像素坐标。结合帧率把帧号换算成时间，再用标尺把像素换算成米，得到一张真实的时间对位移数据表。用最小二乘把这组数据拟合成二次曲线，画出数据点、拟合曲线、残差三联图，从二次项系数解出 g 。能给出测得的 g 值、和标准值 9.8 m/s^2 的相对误差、以及对误差来源的分析，如帧率抖动、像素读取精度、标尺换算、空气阻力，项目合格。

20.8.3 扩展方向

可以同时拟合位移曲线和速度曲线，速度由位移的数值微分得到，交叉验证 g ，看哪种更稳。可以故意用不同形状的物体，纸团对小球，做对照，从残差里看出空气阻力对轻物体的影响，把 P8 和 P2 的阻力建模连起来。还可以做不确定度分析，用第十一章的方法给拟合出的 g 配一个误差棒，讨论要把 g 测准到百分之一需要多高的帧率和多长的下落距离。这把项目从“测一个数”升级成“测一个带误差棒的数”。

AI 陷阱预警 · 像素到物理单位的换算漏做或做错

这是 P8 最致命的陷阱，并且 AI 几乎不会主动替你处理。视频提取出来的坐标是像素，时间最初是帧号。如果你直接拿像素对帧号去拟合，二次项系数的单位是“像素每帧平方”，它不是 g ，差着两个换算因子：每像素多少米、每帧多少秒。AI 写追踪代码时通常只给你像素坐标就停手，把单位换算这一步留给你，而很多人会忘记做，于是拟合出一个数量级离谱的“重力加速度”还浑然不觉。务必显式地把像素乘以“米每像素”，

用视频里标尺的已知长度除以它占的像素数得到，把帧号乘以“秒每帧”，用1/帧率得到，再去拟合。一个一眼能看穿的判据是数量级检查：拟合出的 g 必须落在 9.8 m/s^2 的同一个数量级。如果你得到几百或零点几，几乎一定是换算因子漏了或用反了。另一个常见错误是 y 轴方向：图像坐标系往下为正、物理里习惯往上为正，符号搞反会让二次项系数变号。记得统一约定。

本节要点

像素到米的换算、帧号到秒的换算，缺任意一个，拟合出的 g 就会偏离 9.8 m/s^2 几个数量级。用一把已知长度的标尺同时入镜做校准，把帧率倒数作为时间步长，两项换算都做完再去拟合。图像坐标系的 y 轴方向与物理习惯相反，符号搞错会让二次项系数变号。

20.9 P9 一维随机行走：从醉汉散步到扩散定律

20.9.1 物理背景与核心问题

一个醉汉站在一条直线上，每一步以相等概率向左或向右迈一格。走很多步以后，他大概离出发点多远？这就是一维随机行走，它是布朗运动和扩散现象最纯粹的模型，把第十二章的随机采样和扩散物理缝在一起。单次行走是完全随机的，但大量行走的统计行为却服从一条铁律：行走者位置的平均值始终为零，向左向右对称；但位置的均方值，也就是离出发点距离的平方的平均，随步数线性增长， $\langle x^2 \rangle = Na^2$ ，其中 N 是步数、 a 是步长。这条均方位移正比于时间的关系，正是扩散方程的核心特征，扩散系数 D 就藏在这个正比系数里，连续极限下 $\langle x^2 \rangle = 2Dt$ 。核心问题：随机行走的均方位移真的随步数线性增长吗？从离散的随机行走能不能定量抽出扩散系数，并和扩散方程的解析解对上？

20.9.2 最低交付要求

模拟大量独立的一维随机行走，成千上万条，每条走 N 步。画出两样东西：一是若干条单独的行走轨迹，让人看到它们各不相同、毫无规律；二是统计量随步数的演化，平均位置应当在零附近涨落、均方位移应当随步数线性上升。关键交付是均方位移的线性验证：把 $\langle x^2 \rangle$ 对步数 N 画出来，应当是一条过原点的直线，从斜率定量抽出扩散系数，并和理论值 a^2 （或 $2D$ ）对账。再画出走 N 步后所有行走者最终位置的分布直方图，验证它趋近于一个高斯分布，中心极限定理的直接体现，并把高斯的方差和 $\langle x^2 \rangle = Na^2$ 对上。能讲清楚“为什么单步随机、整体却有确定的扩散规律”，以及随机行走和扩散方程的对应关系，项目合格。

20.9.3 扩展方向

可以推广到二维随机行走，看醉汉在平面上的轨迹和均方位移规律，二维下 $\langle r^2 \rangle = 2dDt$ ，维度 $d = 2$ 。可以做有偏行走，向右概率略大于向左，看均值如何随步数线性漂移、叠加在扩散涨落之上。这正好对应漂移的扩散，即对流扩散。还可以直接数值求解一维

扩散方程 $\partial_t \rho = D \partial_x^2 \rho$ ，用第八章的有限差分思想离散，把扩散方程解出的浓度分布和随机行走的位置直方图叠在一起，亲眼看到微观随机过程和宏观偏微分方程描述的是同一件事。这是统计物理里最深刻的对应之一。

AI 陷阱预警 · 把均方位移错算成位移均值的平方，与样本量不足看不出规律

随机行走最核心的统计量是均方位移 $\langle x^2 \rangle$ ，也就是“先平方每条行走的位置、再对所有行走求平均”。AI（和很多学生）极易把它和“位移均值的平方” $\langle x \rangle^2$ 搞混。对称随机行走， $\langle x \rangle$ 始终接近零，它的平方也接近零，于是画出来的是一条贴着横轴的噪声线，完全看不到线性增长，人就会误以为“均方位移不随步数增长”，得出错误结论。务必确认计算顺序是先逐条平方再平均，`np.mean(x**2)`，而不是先平均再平方，`np.mean(x)**2`。这两者物理意义天差地别。第二个陷阱是样本量：随机行走的统计规律只在大量样本下才浮现。如果只跑几十条， $\langle x^2 \rangle$ 对步数的曲线会噪声满天飞、看不出干净的直线。AI 默认的样本数往往偏小，要把行走条数加到足够多，上万条，线性规律才清晰。验证的依据是最终位置分布必须趋近高斯，并且它的方差要和 Na^2 对得上。

本节要点

$\langle x^2 \rangle$ 和 $\langle x \rangle^2$ 是两个完全不同的量。前者均方位移，随步数线性增长，是扩散的物理标志；后者是均值平方，对称行走下贴着零。写 `np.mean(x**2)` 而不是 `np.mean(x)**2`，顺序反了就看不到线性规律。样本上万条，统计才稳。

20.10 P10 光学衍射图案：从激光斑点到 sinc 平方拟合

20.10.1 物理背景与核心问题

让激光指针的光穿过一条很细的单缝，在远处的墙上会看到一排亮暗相间的条纹。中间最亮、两侧逐渐变弱，这是夫琅禾费单缝衍射，核心是光的波动性：缝上各点发出的次波在远处叠加，干涉结果给出一个有解析形式的强度分布

$$I(\theta) = I_0 \left[\frac{\sin \beta}{\beta} \right]^2, \quad \beta = \frac{\pi a \sin \theta}{\lambda},$$

其中 a 是缝宽、 λ 是波长、 θ 是衍射角。中括号里那个 $\sin \beta / \beta$ 就是大名鼎鼎的 sinc 函数，它的平方给出强度的包络：中央主极大最亮最宽，两侧次极大迅速变弱，暗纹出现在 β 为 π 整数倍处。这个项目把一张衍射照片变成一次对缝宽（或波长）的测量。核心问题：从手机拍下的衍射图案里提取出强度分布，用 sinc 平方函数去拟合，能不能反推出缝宽？拟合得有多好？

20.10.2 最低交付要求

拍一张衍射图案：激光指针照过一条细缝，或一根头发丝，头发丝衍射等效于细缝，更好操作。在墙上或屏上形成清晰的衍射条纹，用手机拍下来，记录下缝到屏的距离。用

图像处理把照片沿条纹方向取一条强度剖面线，彩色图转灰度，沿中心线读出每个像素的亮度，得到一条强度对位置的实测曲线。再把屏上位置换算成衍射角 θ ，用位置除以缝屏距离的正切关系。用最小二乘把这条曲线拟合到 sinc 平方函数，拟合参数包含缝宽 a 、中心位置、整体强度，画出数据、拟合曲线、残差三联图，从拟合出的 a 反推缝宽并和实际值或合理估计对账。能给出测得的缝宽、讨论拟合质量和误差来源，如相机的亮度非线性、背景光、条纹中心定位，项目合格。

20.10.3 扩展方向

可以做双缝衍射，强度分布是单缝包络乘以双缝干涉因子 $\cos^2$ ，拟合能同时反推缝宽和缝间距，比单缝更丰富。可以拍不同缝宽的图案，验证“缝越窄、中央主极大越宽”这个反比关系，衍射角宽度正比于 λ/a 。还可以用已知缝宽反过来测激光波长，把项目从“测缝宽”翻转成“测波长”，并和激光指针标称的波长对账。

AI 陷阱预警 · 角度的弧度与度混淆，以及拟合 sinc 平方时的归一化与平方陷阱

P10 的 sinc 平方拟合里藏着好几个 AI 高频翻车点。第一是角度单位： $\beta = \pi a \sin \theta / \lambda$ 里的 θ 必须用弧度，`numpy` 的三角函数也只认弧度。如果你从像素位置换算角度时不小心带进了度数，或者把 `np.sin` 喂了度数，整条拟合曲线的周期就全错了。第二是 sinc 函数本身的两种定义：数学里 $\text{sinc}(x) = \sin x / x$ ，但 `numpy.sinc` 用的是“归一化 sinc ” $\sin(\pi x) / (\pi x)$ ，多带了一个 π 。如果直接调 `np.sinc` 又没注意它的 π 约定，缝宽会差一个 π 因子。最稳妥的办法是自己显式写 $(\sin \beta / \beta)^2$ 并小心处理 $\beta = 0$ 处的极限，那里 sinc 等于 1 而不是除零。第三是别忘了取平方，强度是 sinc 的平方而不是 sinc 本身。AI 偶尔会拟合到一阶 sinc ，那条曲线会有负值，强度不可能为负，这是一眼能看穿的非物理信号。验证的办法是暗纹位置：拟合出的曲线，暗纹应当出现在 $a \sin \theta = \lambda$ 整数倍处。把这个解析判据和实测暗纹位置对一下，缝宽就有了独立的交叉验证。

本节要点

β 里的角度必须用弧度，喂度数给 `np.sin` 整条曲线就废了。强度是 sinc^2 不是 sinc ，取平方不能忘。暗纹应出现在 $a \sin \theta = \lambda$ 整数倍处，用它和实测位置对账是独立于拟合的交叉验证。最稳妥的写法是自己显式写 $(\sin \beta / \beta)^2$ ，不用 `np.sinc` 的归一化约定。

20.11 十个项目共享的同一套骨架

把这十个项目并排看，你会发现它们远没有表面上那么多。从数值方法看，P1（单摆）、P2（抛体）、P3（电路）、P4（行星）共享同一套“把二阶方程拆成一阶状态向量、用数值积分推进”的 ODE 骨架，区别只在右端函数 f 的物理内容不同。这正是第八到第十章反复锤的那个共同骨架。P5（冷却）是最简单的一阶版本。P7（蒙特卡洛）和 P9（随机行走）共享随机采样加统计平均加 $1/\sqrt{N}$ 误差标度这套概率骨架。P8（落体）和 P10（衍射）共享“从图像或视频提数据、做单位换算、用最小二乘拟合一个物理函数、画数据加拟合加残差三联图”这套数据工程骨架，P5 的冷却曲线拟合也属于这一类。P6（振子链）是唯

一以线代矩阵特征值为核心的项目，它独树一帜，但对角化之后每个简正模式又退回到一维简谐振子，仍和 ODE 骨架相通。认出这些共享的骨架，意味着你为一个项目写的代码、做的验证、抽象的 skill，往往能直接迁移到同骨架的另一个项目上。这正是这门课希望你带走的最值钱的东西。

本节要点

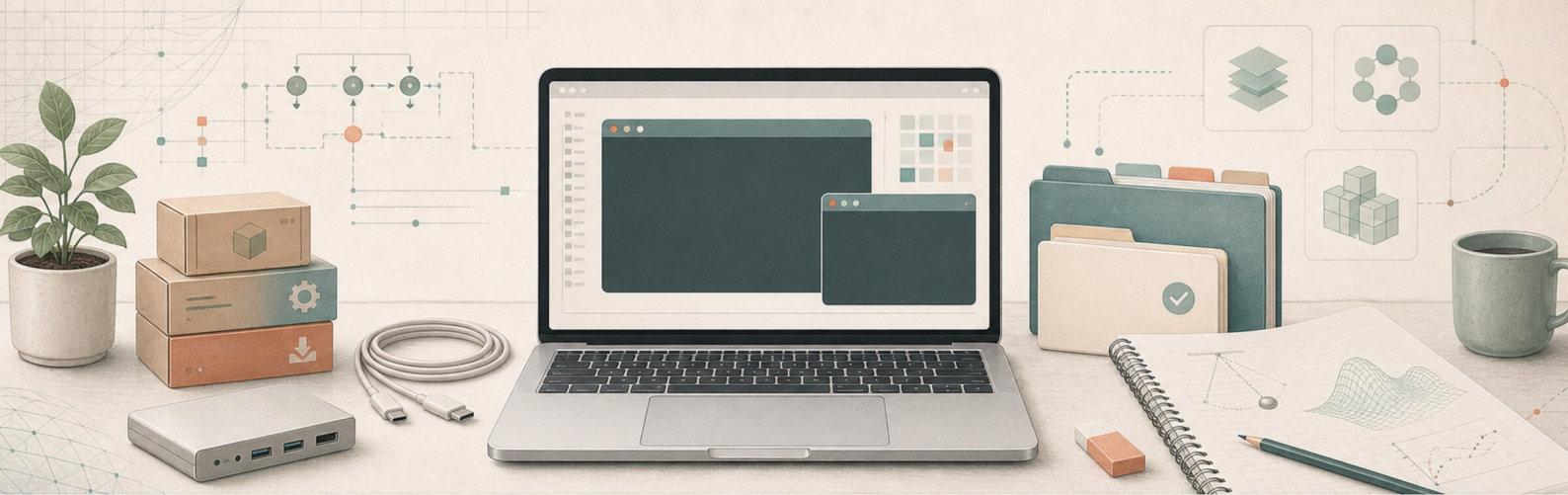
十个项目共享着四套可迁移的数值骨架：ODE 积分（P1–P4，P5 是其退化版）、随机采样加统计平均（P7 与 P9）、数据工程加最小二乘拟合（P5、P8、P10 共享）、矩阵对角化（P6 独有但简正模式退化为单振子）。一个项目的代码与验证，换个物理内容就能直接用在同骨架的另一题上，这是整章最该带走的设计眼光。

本章小结

把整门课的手艺落进十个具体的物理项目里。每个项目给了四样东西：物理背景与核心问题、最低交付要求、扩展方向、典型 AI 陷阱预警。十个项目对应选题菜单 P1 到 P10，覆盖了非线性振动、含阻力抛体、电路、行星轨道、牛顿冷却、耦合振子链、蒙特卡洛、视频测重力、随机行走与扩散、光学衍射。它们是同一套反射弧（需求文档、架构、验证、ship、skill）在不同物理情境下的反复演练，并在数值方法上共享着 ODE 积分、随机采样、数据拟合、矩阵对角化这几套可迁移的骨架。每个项目的 AI 陷阱都被钉到了具体的物理点上：P1 的小角度近似误差、P2 的二次阻力方向、P4 的能量不守恒、P8 的像素到米换算、P10 的角度单位与 sinc 平方。带着这套手艺去选你的题，把五条反射弧从头走一遍，做出一个你愿意点开给别人玩的工件。

思考题

1. 这一章末尾指出十个项目共享几套数值骨架。请你不看正文，自己重新把 P1 到 P10 按“核心数值方法”归类，看看能分成几组，并说明每一组里不同项目的差别仅仅体现在哪个具体的函数或矩阵上。哪一个项目最难归进任何一组，为什么？
2. 给你一个本章没有列出的物理现象：一根两端固定的弦被拨动后的振动（驻波）。请你把它设计成一个落在本课范围（大一普物加高数加线代）内的项目，写出它的核心方程、它最该和上面哪个已有项目共享数值骨架、它的最低交付要求里至少该有哪两张图、以及你预计 AI 在这个问题上最容易犯的一个具体错误。
3. 任选一个数据采集型项目（P1、P2、P3、P5、P8、P10 之一），为它写一份验证清单（仿照第八章的 `verify` 环境），至少列出四条能推翻结论的物理判据，并说明每一条具体要拿什么数值或参考值去对账。一份好的验证清单和一份“看起来很全”的打勾清单，差别在哪里？



A. 环境搭建与装机

这门课从第一天起就要你亲手跑代码，所以在正式上课之前，你的笔记本上得先有一套能用的工具链。这份附录是给你装机用的，不假设你以前写过任何程序，目标也很朴素：让电脑能跑 Python、能打开 Jupyter、能用一个顺手的编辑器，并且三分钟内自测出“我到底装好了没有”。如果你参加了开课前的装机 clinic，这附录是回去复习和排错的底稿；如果你缺席了，照着它从头走一遍，多数情况下也能独立搞定。

本章地图

小节	这一节解决什么	关键方法 / 产物
A.1	明确需要装哪些工具，各自做什么用	Python 解释器、Jupyter、代码编辑器三件套
A.2	Windows 和 macOS 的安装步骤与易踩坑点	conda 安装、PATH 配置、Apple Silicon 注意
A.3	用一段代码实测环境是否真的装好	hello world + numpy + matplotlib 三连测
A.4	解决装机高频故障：装库慢、中文乱码、import 报错	pip 换源、字体配置、Python 路径排查
A.5	理解虚拟环境的意义与基本操作	venv / conda 虚拟环境创建与激活

A.1 装什么：三样东西，一个都不能少

装机拆开看，真正需要的只有三样东西。第一样是 Python 本身，也就是真正执行代码的解释器。第二样是 Jupyter，一种把代码、运行结果和文字笔记混在一个网页里的交互式

环境，特别适合一边写一边看图，是这门课做实验的主场。第三样是一个代码编辑器，用来写稍长一点的脚本、管理项目文件，同时也是你和 AI 协作的工作台。三样凑齐，你就有了一套完整的物理建模工作站。

先说 Python 怎么装。对零基础的同学，最省心的做法是装 Anaconda 或它的精简版 Miniconda。它们会把 Python 解释器、常用的科学计算库（numpy、matplotlib、scipy 这些这门课要反复用到的）以及 Jupyter 一次性打包好，装完基本就能直接用，省去了逐个安装的麻烦。Anaconda 体积大、附带图形界面，适合硬盘空间充裕、喜欢点鼠标的同学；Miniconda 只装最核心的部分，干净轻量，需要的库后面用命令一条条补上，适合愿意多敲几行命令、想把环境攥在自己手里的同学。如果不想用 conda，也完全可以 from Python 官网下载官方版本，再用它自带的 pip 安装各种库。这条路更接近“原生”的 Python 体验，只是要自己多操心一点环境管理。无论走哪条路，请认准较新的稳定版本，不要装那些早已停止维护的老版本。

编辑器方面，推荐 VS Code。它免费、跨平台、插件生态成熟，装上 Python 插件后写代码、跑 Jupyter、调试都很顺。如果想要更深度的 AI 编程体验，可以试试 Cursor。它在 VS Code 的基础上把 AI 助手做成了第一等公民。国内的 Trae 也是类似定位，对中文和国内模型更友好。这三个编辑器界面和操作几乎一脉相承，挑一个顺眼的即可，不必纠结，更不必三个都装。

本节要点

装机的三件套是 Python 解释器、Jupyter、一个代码编辑器。Python 装法有三条路：Anaconda（省心但体积大）、Miniconda（轻量需手动补库）、官方版加 pip（最原生）。编辑器用 VS Code 或其 AI 加强版 Cursor / Trae 都行，界面一脉相承，挑一个顺眼的即可。

A.2 在 Windows 和 macOS 上的安装要点

具体到操作系统，两大平台的流程大同小异，但有几个容易踩的地方值得单独点一下。

在 Windows 上装 Miniconda 或 Anaconda 时，安装程序会问要不要把 conda 加进系统的 PATH 环境变量。官方默认不勾选，建议你今后都通过开始菜单里的 Anaconda Prompt（或 Anaconda Powershell Prompt）来使用 conda，而不是普通的命令提示符。对初学者来说，照默认走、用 Anaconda Prompt 最稳妥，能避开很多“命令找不到”的怪问题。装好之后，从开始菜单打开 Anaconda Prompt，看到命令行最前面有一个 (base) 字样，就说明 conda 的基础环境已经激活，可以干活了。

在 macOS 上，如果你用的是较新的 Apple 芯片（M 系列）的机器，下载安装包时要选对应 Apple Silicon (arm64) 的版本，不要误装 Intel 版，否则有些库在运行时会慢甚至出岔子。装完之后，conda 通常会自动改写你的 shell 配置，新开一个终端窗口就能看到 (base) 前缀。如果没看到，往往是配置没生效，按安装程序提示执行一下初始化命令，或者干脆关掉终端重开一个，多半就好了。macOS 第一次运行某些程序时系统可能弹出安全提示，按提示在系统设置里允许即可。

编辑器的安装在两个平台上都很简单，到 VS Code（或 Cursor、Trae）的官网下载对应系统的安装包，一路点下去就行。装好后打开它，在扩展市场里搜索并安装官方的 Python 扩展，这一步会让编辑器认识 .py 文件、能直接运行代码、还能打开 Jupyter 笔记本。

本节要点

Windows 用户用 Anaconda Prompt 代替普通命令行，macOS Apple Silicon 用户注意选 arm64 版安装包。装完后新开终端看到 (base) 前缀即表示 conda 环境就绪。编辑器安装后在扩展市场装 Python 插件即可。

A.3 装好后的三件套自测

光装完不算数，你得用一段代码来确认它真的能跑。开课前的装机环节就要求每个人跑通三件套：跑通一句 hello world、成功 import numpy、再用 matplotlib 画出一张折线图。三件事分别检验了 Python 解释器、科学计算库、绘图库是不是都到位了。下面这段脚本把三件套合在一起，你可以新建一个文件比如叫 check_setup.py，原样拷进去运行；也可以在 Jupyter 里新建一个 notebook，粘进一个单元格里执行。

Listing A.1: 装机自测三件套

```
1 # 第一件: hello world, 检验 Python 解释器本身能不能跑
2 print("hello world, Python 解释器工作正常")
3
4 # 第二件: 导入 numpy, 检验科学计算库装好了没
5 import numpy as np
6 x = np.linspace(0.0, 10.0, 100) # 生成 0 到 10 之间均匀的 100 个点
7 print("numpy 导入成功, 当前版本是", np.__version__)
8
9 # 第三件: 用 matplotlib 画一张折线图, 检验绘图库
10 import matplotlib.pyplot as plt
11 y = np.sin(x) # 拿一条正弦曲线当测试数据
12 plt.plot(x, y, "-", label="sin(x)")
13 plt.xlabel("x")
14 plt.ylabel("sin(x)")
15 plt.title("matplotlib 折线图自测")
16 plt.legend()
17 plt.tight_layout()
18 plt.show() # 弹出或内嵌显示这张图
```

跑完之后，你应该依次看到三样东西。第一件是终端里打印出 hello world，Python 解释器工作正常这一行。看不到它，说明 Python 解释器根本没被正确调用，问题出在最底层。第二件是一行 numpy 导入成功加上版本号。如果这里报出 ModuleNotFoundError: No module named 'numpy' 之类的错，说明 numpy 没装进你当前正在用的这个 Python 环境里。第三件是弹出一张正弦曲线的折线图（在编辑器里直接运行时）或者图片内嵌显示

(在 Jupyter 里运行时)。看到这条平滑起伏的曲线，绘图这一环也通了。三样齐活，装机成功；缺哪一样，问题就大致锁定在对应的那一块。

本节要点

三件套自测分别检验 Python 解释器 (hello world)、科学计算库 (import numpy)、绘图库 (matplotlib 画折线图)。缺哪一样，问题就锁定在对应模块。跑通这三件才算装机成功。

A.4 常见装机问题与对策

装机最常卡住的环节，一是装库太慢，二是图里的中文显示成乱码或方块，三是 import 莫名其妙报错。这三类问题占了求助的绝大多数，下面逐一说清楚。

装库慢几乎总是因为默认的下载源在境外，网络兜了一大圈。对策很直接，换成国内镜像。用 pip 装库时，临时指定清华大学的镜像源只要在命令后面加一个参数即可，比如安装 numpy 可以这样写：

Listing A.2: 用清华镜像源加速 pip 安装

```
1 pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple
```

换源之后下载速度通常从龟速变成几秒钟搞定。如果用 conda，也有类似的国内镜像可以配置，原理一样，都是把远在境外的仓库换成校园网能快速直连的镜像。

中文乱码是另一个高频问题。matplotlib 默认用的字体里没有中文字形，一旦在图中写中文标题或坐标轴标签，就只能画出一串小方块。解决办法很简单：画图之前，告诉 matplotlib 改用一种系统里确实带中文字形的字体。下面几行放在脚本开头即可：

Listing A.3: 让 matplotlib 正常显示中文

```
1 import matplotlib.pyplot as plt
2 # Windows 一般有黑体 SimHei; macOS 可换成 "Heiti TC" 或 "Songti SC"
3 plt.rcParams["font.sans-serif"] = ["SimHei"]
4 plt.rcParams["axes.unicode_minus"] = False # 让负号也能正常显示
```

字体名要填你系统里真实存在的那个。Windows 上常用黑体 SimHei，macOS 上可以换成 Heiti TC 或 Songti SC。设好之后，图中的中文就不再是方块了。顺带一提，那行关于负号的设置也别省，否则有时坐标轴上的负号会跟着一起变成方块。

至于 import 报错，初学者遇到的几乎都不是深奥的 bug。真正的原因不外乎两类低级但隐蔽的情况。第一类是环境没激活：你在 Anaconda Prompt 里装好了库，却又跑到另一个没激活 conda 的窗口里去运行代码。那个窗口用的根本是另一个 Python，自然找不到库。第二类是装错了 Python：很多电脑上同时存在好几个 Python（系统自带的、conda 的、官网装的），你用 A 这个 Python 装的库，却用 B 那个 Python 去跑，当然导不进来。判断的窍门是：在运行代码的同一个环境里敲 `python -c "import sys; print(sys.executable)"`，

看打印出来的路径是不是你预期的那个 Python，再回头确认你装库时用的也是同一个。把“装库”和“跑代码”统一到同一个 Python 上，这类报错就消失了。

本节要点

装库慢的解法是换国内镜像源(pip 加 `-i https://pypi.tuna.tsinghua.edu.cn/simple`)。中文乱码在脚本开头设 `plt.rcParams["font.sans-serif"]` 即可。import 报错十有八九是装库用的 Python 和跑代码用的 Python 不是同一个，用 `sys.executable` 看路径对账。

A.5 为什么要用虚拟环境

最后讲一件初学时容易忽略、但越往后越显出价值的事：虚拟环境，简单说，虚拟环境就是为某一个项目单独圈出来的一套独立的 Python 和库，和别的项目互不干扰。需要它，是因为不同项目对库版本的要求可能打架。这个项目要某个库的旧版本，那个项目偏要新版本。如果所有项目都共用同一套库，迟早会因为版本冲突互相搞坏。把每个项目装进自己的环境里，就像给每个项目准备一个独立的工具箱，谁也不动谁的，出了问题也只影响这一个项目，删掉重建即可，干净利落。

建一个虚拟环境并不麻烦。如果装的是官方 Python，用它自带的 `venv` 就行，在项目文件夹里执行下面两步，先创建再激活：

Listing A.4: 用 `venv` 创建并激活虚拟环境

```
1 # 创建一个名为 venv 的虚拟环境（在当前项目文件夹下）
2 python -m venv venv
3
4 # 激活它：Windows 用第一行，macOS 用第二行
5 venv\Scripts\activate
6 source venv/bin/activate
```

激活之后，命令行前面会多出一个 `(venv)` 的标记，提示你此刻身处独立环境。此后用 `pip` 装的库都只进这个环境。如果用的是 `conda`，做法类似，一条命令创建带指定 Python 版本的新环境，再激活：

Listing A.5: 用 `conda` 创建并激活虚拟环境

```
1 # 创建一个名为 myproj 的环境，顺便指定 Python 版本
2 conda create -n myproj python=3.11
3
4 # 激活它
5 conda activate myproj
```

这门课的项目作业不大，不用一上来就为虚拟环境焦虑。直接在 `conda` 的 `base` 环境里做也完全跑得通。但养成“一个项目一个环境”的习惯没有坏处。等将来同时维护好几

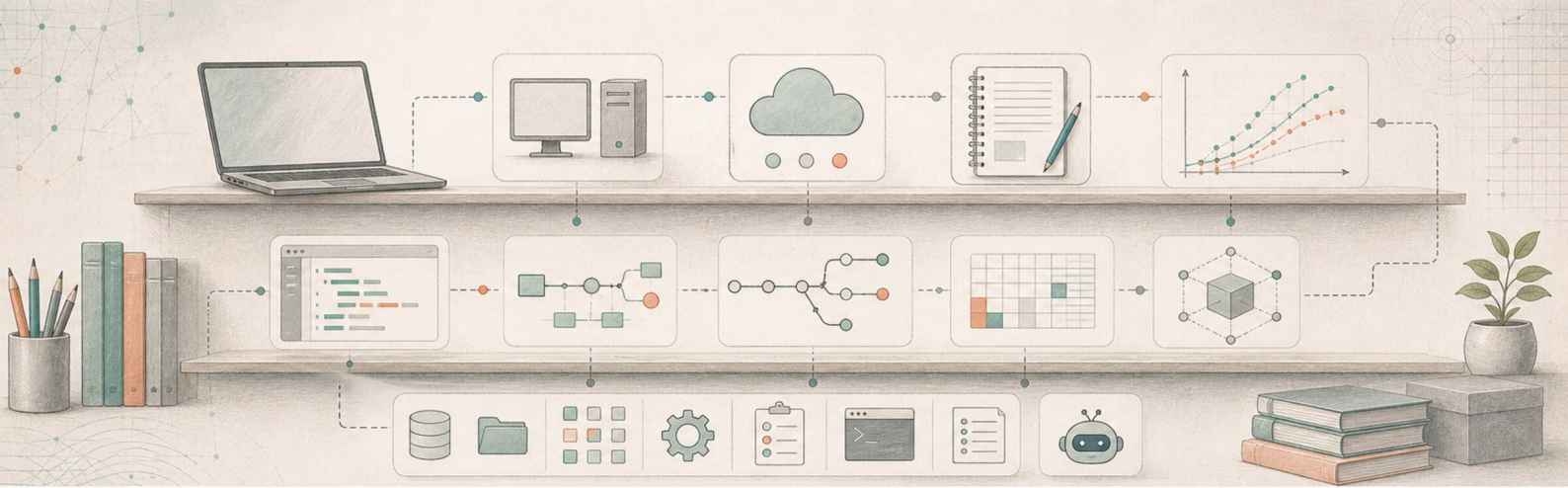
个项目、或者要把代码连同环境一起交付给别人时，你会真切体会到这套做法省下了多少麻烦。

本节要点

虚拟环境就是一个项目一套独立的 Python 和库，项目之间互不污染。Python 用户用 `python -m venv venv` 创建，conda 用户用 `conda create -n name` 创建。激活后命令行前缀会变，此后装的库只进这个环境。初学不必焦虑，但养成习惯没有坏处。

AI 陷阱预警 · AI 给的装机命令可能针对错误的系统或过时的版本

装机阶段免不了会拿报错信息去问 AI，但要警惕：AI 并不知道你坐在哪台电脑前。它很可能默认你用的是 Linux 或某个特定系统，给出一条在 Windows 或 macOS 上根本不对路的命令，比如把激活虚拟环境的写法给反了，或者用了某个你系统里没有的命令。它也可能引用了早已过时的安装方式或库版本，照着做反而装出一堆冲突。对策很简单：每次求助时，主动告诉它你的操作系统（Windows 还是 macOS、Intel 还是 Apple 芯片）和 Python 版本，并且对每一条命令保持贯穿全书的审计习惯，先看懂它在做什么，确认它针对的确实是你的系统，再敲回车。



B. 工具栈选择指南

很多同学拿到这门课，第一反应是焦虑：是不是得先翻墙？是不是得办一张境外的信用卡？是不是得花钱订阅某个听起来很厉害的 AI，才算入了门？先把这个念头放下。这门课的立场非常明确：国内学生默认就用学校补贴的 DeepSeek 加上字节跳动的 Trae，全程零自费，不必翻墙，作业从头到尾都能顺顺当当做完。这套组合不是退而求其次的廉价方案，是课程设计反复确认的默认推荐。评分这件事再说得直白一点：我们只看你怎么用 AI，不看你用的是哪一家的 AI。需求文档写得清楚，AI 写的代码审计得到位，验证清单跑得扎实，用 DeepSeek 拿到的分和用境外某个模型拿到的分没有任何区别。国内模型在这门课里是平替，不是降级。把这一条记牢，你就能省下大量本不该花在工具焦虑上的精力。

工具栈

本章地图

小节	这一节解决什么	关键方法 / 产物
B.1	按编程基础与网络条件推荐合适的工具路线	三种处境分流：零基础、正经学、无境外网
B.2	分类介绍对话式 AI、AI 编程 IDE、命令行 AI 三类工具	DeepSeek / Trae / Cursor 等工具的适用场景
B.3	境外工具的定位说明与评分无关的申明	ChatGPT、Claude 等可自选用但不加分
B.4	先用一套工具跑完全程，不反复切换	Trae + DeepSeek 默认组合，核心是练反射而非选工具

B.1 先按你的处境分流

工具的选择应该跟着你的真实处境走，而不是跟着别人嘴里哪个工具最火走。下面按三种常见处境给出建议，对号入座即可。

如果你完全没有编程基础，只想用最短的路径做出一个能跑的东西，优先考虑浏览器里即开即用的轻量工具，或者直接上字节跳动的 Trae。轻量工具不需要你在本机配环境，打开网页就能让 AI 帮你生成、运行一小段代码，适合第一周建立信心。Trae 则是一个装在本机、长得跟正经 IDE 一样的编辑器，内置了 AI，对 Python 新手相当友好。两条路都能让你在没踩过环境坑的情况下先尝到甜头，等手感起来了再往深里走。

如果你打算正经学，希望 AI 能深度参与到写代码的每一步里，推荐用 Cursor，或者装了 AI 插件的 VS Code，再配一个能力够强的模型。这类编辑器的特点是 AI 不只是旁边一个聊天框，而是能读整个项目、跨文件改代码、跟着对话一步步迭代。配上一个强模型之后，你会明显感觉到从“我让 AI 写一段”升级到了“我和 AI 一起开发一个项目”。这条路前期要花点时间熟悉编辑器，但它给你的上限最高。

如果你完全没有境外网络访问，连偶尔翻墙都做不到，不必担心。Trae 加 DeepSeek 的组合全程足够，这正是本课的默认栈。所有作业，从需求文档到最终的可交互工件，都能在这套纯国内的组合上完整走完。境外服务的任何抖动、中断、收费，都和这门课的进度无关。

本节要点

工具选择按你的真实处境走，不按别人嘴里哪个最火走。零基础的用 Trae 或浏览器轻量工具先建立信心；打算正经学的配 Cursor 或 VS Code 加强模型；完全无境外网的用 Trae 加 DeepSeek，全程足够。

B.2 各类工具简介

市面上的工具按用法分，大致是三类。下面逐类说清楚它们各自是什么、什么时候该用。

第一类是对话式 AI，也就是你打开一个网页或 App，在对话框里提问、它回你答案的那种。国内这边有 DeepSeek、通义千问、智谱 GLM、Kimi、文心一言等等，境外则有 ChatGPT、Claude、Gemini。这类工具最适合帮你把思路理清楚：让 AI 把物理问题翻译成数学骨架、解释一段代码为什么这么写、或者给需求文档提意见。它们不直接住在你的项目里，代码要自己复制粘贴回编辑器，但胜在随手可用、门槛最低。这门课里反复练的需求文档写法、审计代码的提问方式，在任何一家对话式 AI 上都通用。

第二类是 AI 编程 IDE，也就是把 AI 直接焊进代码编辑器里的工具，代表是 Trae、Cursor，以及装了 Copilot 插件的 VS Code。它和对话式 AI 最大的区别，是 AI 能看见你的整个工程，能直接在文件里动手改，不必来回搬运代码。一个项目做到中后期，文件多了、模块之间互相调用了，这类工具的优势就会非常明显。本课的默认 IDE 是 Trae，原因前面已经说过，零自费且对国内网络友好。

第三类是命令行 AI，比如 Claude Code、Codex CLI 这种直接在终端里跑的 AI 助手，属

于进阶用法。工作方式是在命令行里跟 AI 对话，AI 直接读写文件、运行命令、跑测试，自动化程度最高，但也要求你对终端和项目结构有一定熟悉。大一阶段完全可以先不碰这一类。等前两类用顺了、确实觉得手动搬运代码太慢了，再回来了解它。

需要特别点明的一点是：无论最后落在哪一类工具上，这门课教的几样规范都是通用的。需求文档是你和 AI 之间关于“要做成什么样”的约定，AGENTS.md 是写给 AI 的项目级行为约束，SKILL.md 是把反复做的动作压缩成的可复用工作流。这三样东西的价值在于它们承载的思考，不在于某个工具的文件语法。在能自动加载的工具上，它们会被工具直接读取；在没有自动加载机制的对话式 AI 上，把内容粘进系统提示词里照样能用。换句话说，你学的是一套能跨工具迁移的工作方式，不是某一个软件的快捷键。

本节要点

对话式 AI（DeepSeek、ChatGPT）适合理清思路和提问，要自己复制代码回编辑器。AI 编程 IDE（Trae、Cursor）能看见整个工程、直接在文件里改代码。命令行 AI 自动化程度最高但需要终端和项目结构的熟悉度。需求文档、AGENTS.md、SKILL.md 这三样规范的价值在于它们承载的思考，跨工具通用。

B.3 关于境外工具

ChatGPT、Claude、Cursor 这些境外工具，想用就用，悉听尊便。它们确实是很好的工具，如果你已经有稳定的访问条件，用着顺手，完全没问题。但请记住两点。第一，它们绝不是这门课的必需品，少了它们一样能把课上完、把项目做好。第二，用了它们也绝不加分。课程评分标准里没有任何一条会因为用了境外模型而给你更高的分。所以，如果为了用上境外工具需要折腾网络、办卡付费、和不稳定的连接较劲，那这笔投入对这门课来说是不划算的。把那份精力放回到怎么写好需求文档、怎么审计 AI 的输出上，回报会高得多。

本节要点

境外工具（ChatGPT、Claude、Cursor 等）可以用，不是必需，用了也绝不加分。为了用它们折腾网络、付费、对付不稳定连接，对这门课不划算。精力放回需求文档和审计上回报更高。

B.4 一句务实的建议

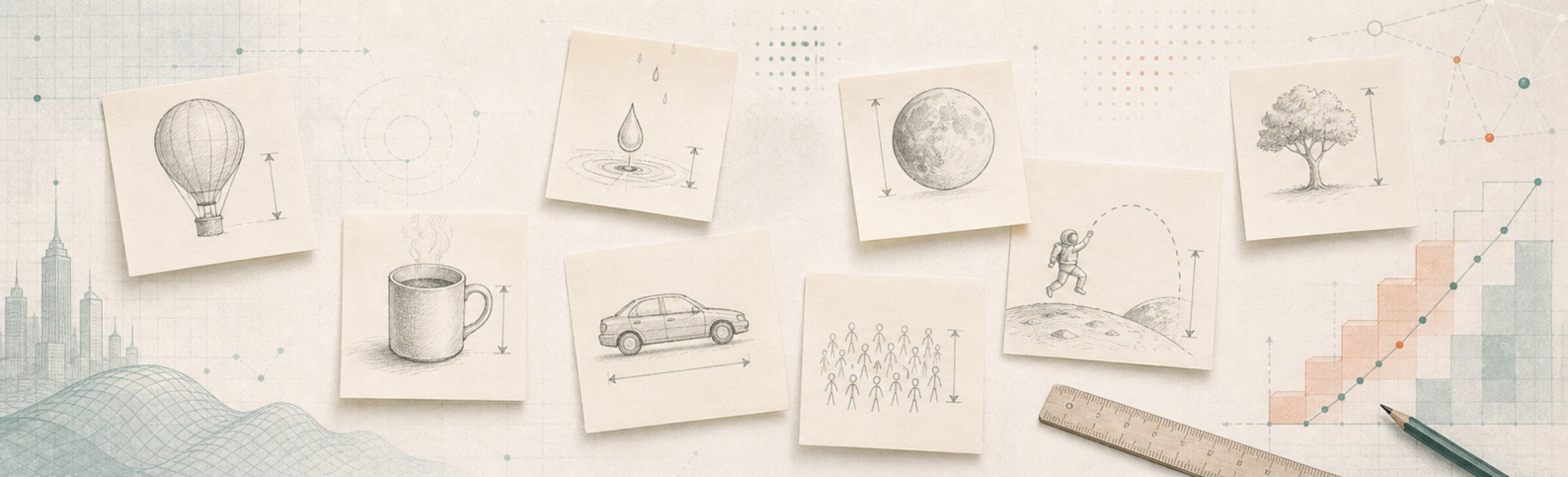
下面给一条贯穿全程的实用建议：先用一个工具把整个流程跑顺，不要在工具选择上反复横跳。新手最容易掉进的坑，是听说哪个工具好就换哪个。这周用 Trae，下周看到别人夸 Cursor 又切过去，结果每个工具都只摸了个皮毛，每次切换还得重新适应界面和习惯。真正该练的需求文档、审计、验证，反而没练扎实。正确的做法是认准一套组合，比如默认的 Trae 加 DeepSeek，先用它把一整个项目从头做到尾，把这门课要求的那套反射动作练成肌肉记忆。工具一定会换，今天的热门明年可能就过气了，但你在课里养成的那套反射动作，认骨架、写需求、审代码、跑验证，是会跟着你一辈子的。工具是船，反射

反射动作

动作才是你要带走的本事。

本节要点

认准一套组合先用它跑完全程，不要听说哪个工具好就换。真正该练的是需求文档、审计、验证，不是换工具每次重新适应界面。工具一定会换代，你在这门课养成的反射动作（认骨架、写需求、审代码、跑验证）才是跟着你一辈子的本事。



C. Fermi 估算题库

第二章里我们认识了费米（Enrico Fermi）的拆解智慧，面对一个看似无从下手、数据 **Fermi 估算** 残缺的大问题，把它拆成若干个能凭常识各报一个粗略数字的小问题，再把结果乘除起来。每一步的误差有大有小，但因为彼此独立、方向随机，相乘之后往往相互抵消，最终的数量级常常出奇地准，Fermi 估算从不追求小数点后几位，只问一件事：答案大约是 10 的几 **数量级** 次方。整个过程不开电脑、不查资料，只用一张信封背面和你脑子里的几个常识锚点。

这个附录是第二章的配套题库。那一章我们一起做了三道题：人在月球上能跳多高、咖啡冷却要多久、雨滴终端速度多大。这里再给你二十多道，按主题分成几组，专门用来把量纲与数量级的反射练成肌肉记忆。强烈建议对每一道题，先合上书自己估一遍，把拆解链条写在纸上，再去看参考答案或上网搜索。先估后查，估错了也值，因为练的不是答案本身。练的是把未知拆成已知的那条思路。

题库里挑了三道给出完整的示范估算过程，放在各组开头，演示怎么拆解、怎么取合理的数量级、怎么用量纲回头检查结果。其余的题只给一句话的问题，留给你自己动手。

本章地图

小节	这一节解决什么	关键方法 / 产物
C.1	用常识锚点估算日常生活中的数量	示范：上海钢琴调音师人数；六道自练习题
C.2	把地球与天空的宏大数字拆解成常见量	示范：地球大气总质量；六道自练习题
C.3	用数量级思维处理物理与工程问题	示范：一滴水里的分子数；七道自练习题
C.4	把生物与人体数据化为可感的量	六道自练习题，覆盖细胞数、心跳、代谢等
C.5	说清 Fermi 估算练的到底是什么本事	数量级直觉，不依赖计算机的第一道防线

C.1 第一组：日常生活

■ **Example C.1 : 示范：上海有多少架钢琴需要调音师定期照料。** 这是费米本人最爱举的经典题，我们来拆解它。第一步，上海常住人口大约 2.5×10^7 人，取 2×10^7 这个量级。第二步，多少人家里有钢琴？城市家庭平均约 3 人，于是家庭数约 7×10^6 。其中拥有钢琴的比例，凭印象大约每二十到五十家有一架，取约 1/30，得到家用钢琴约 2×10^5 架。再加上琴行、学校、琴房等机构用琴，数量同一量级，合计粗估 3×10^5 架。第三步，每架钢琴大约一年调一次音。第四步，一位调音师一年能调多少架？一次调音约 2h，一位调音师一天工作约 8h、一年工作约 200 天，于是年调琴数约 $200 \times 8/2 \approx 800$ 架。两者一除，所需调音师人数约 $3 \times 10^5/800 \approx 4 \times 10^2$ ，也就是几百位的量级。

回头做一次数量级的合理性检查：几百位调音师服务两千万人口，相当于每位调音师对应约五万人。这个比例听起来不离谱，与一座大城市里小众手艺人的稀疏程度相符。注意，全程没有一个数字是查来的，每一个都是常识锚点，而最终落在几百这个量级上，已经是一次成功的 Fermi 估算。 ■

思考题

1. 一个人一生大约喝掉多少水？
2. 一度电（也就是 $1 \text{ kW} \cdot \text{h}$ ）能给一部手机充满电多少次？
3. 上海一天大约要喝掉多少杯奶茶？
4. 一支普通铅笔从削好用到握不住，大约能画出多长的一条线？
5. 你家里所有电器待机一年，大约浪费掉多少度电？
6. 一个普通人一辈子大约睡掉多少个小时，折合多少年？

本节要点

日常生活中的 Fermi 估算题，答案都藏在你已经知道的常识锚点里：人口、家庭规模、物品使用频率。拆解链条比最终数字重要，每一步的锚点都要说得有理由。练的是拆的功夫，不是记数字的功夫。

C.2 第二组：地球与天空

■ **Example C.2：示范：地球大气层的总质量大约是多少。** 这道题看起来宏大得无从下手，但有一个漂亮的拆解：大气压强 p 本质上就是头顶整根空气柱的重量压在单位面积上。换句话说，每平方米地面上方那根直通太空的空气柱，它的重量除以 1 m^2 就等于大气压。标准大气压 $p \approx 10^5 \text{ Pa} = 10^5 \text{ N/m}^2$ ，于是每平方米头顶空气的重量约 10^5 N ，对应质量 $10^5/g \approx 10^4 \text{ kg}$ 。

接着只要乘上地球表面积。地球半径 $R \approx 6.4 \times 10^6 \text{ m}$ ，表面积 $4\pi R^2 \approx 4 \times 3 \times (6.4 \times 10^6)^2 \approx 5 \times 10^{14} \text{ m}^2$ 。两者相乘，大气总质量约 $10^4 \times 5 \times 10^{14} = 5 \times 10^{18} \text{ kg}$ ，量级是 10^{18} 到 10^{19} kg （精确值约 $5.1 \times 10^{18} \text{ kg}$ ，我们一步到位落在了正确量级）。

用量纲回头检查关键那一步： p/g 的量纲是 $\frac{ML^{-1}T^{-2}}{LT^{-2}} = ML^{-2}$ ，正是单位面积的质量，再乘面积 L^2 得到 M ，确实是质量。整条链条量纲齐次，结果可信。 ■

思考题

1. 地球绕太阳公转一圈，走过的路程大约是多少？平均速度又是多少？
2. 把全世界所有人肩并肩、头叠脚地摞起来，总高度大约是多少？能摞到月球吗？
3. 一年之中，整个地球大约被多少次闪电击中？
4. 太阳光从太阳表面出发，大约要多久才能照到你脸上？
5. 假如把地球上所有海水均匀铺在地球表面，水层大约有多厚？
6. 一架民航客机从上海飞到北京，大约烧掉多少千克航空燃油？

本节要点

地球尺度的问题看似大得无从下手，但往往有漂亮的拆解入口。大气总质量那道题的钥匙是“大气压等于头顶空气柱的重量”，这一步把整个大气问题压成了地球表面积乘一个压强，一步到位落在正确量级。用量纲回头检查每条链条的齐次性是兜底的好习惯。

C.3 第三组：物理与工程

■ **Example C.3：示范：一滴水里大约有多少个水分子。** 先定下滴水量的级。一滴水的体积约 $0.05 \text{ mL} = 5 \times 10^{-8} \text{ m}^3$ ，取量级 $5 \times 10^{-5} \text{ L}$ 更顺手。按水的密度 10^3 kg/m^3 ，一滴水的质量约 $5 \times 10^{-5} \text{ kg} = 0.05 \text{ g}$ 。

接下来用摩尔做桥梁，把宏观质量换成微观个数。水的摩尔质量约 18 g/mol ，所以一滴水约 $0.05/18 \approx 3 \times 10^{-3} \text{ mol}$ 。每摩尔含阿伏伽德罗常数 $N_A \approx 6 \times 10^{23}$ 个分子，于是一滴水里的水分子数约 $3 \times 10^{-3} \times 6 \times 10^{23} \approx 2 \times 10^{21}$ 个。

10^{21} 这个量级值得停下来体会：一滴不起眼的水里，分子数比全世界沙滩上的沙粒还多得多。回头检查整条链条的单位串接：质量 g 除以摩尔质量 g/mol 得到 mol ，再乘以 mol^{-1} 的阿伏伽德罗常数，最终化为纯个数。单位约得干干净净，结果可信。 ■

思考题

1. 此刻你所在的教室里，大约有多少个空气分子？
2. 一道闪电的功率大约是多少瓦？（提示：先估它放出的总能量和持续的时间。）
3. 把一杯室温的水烧开，理论上最少需要多少能量？折合多少度电？
4. 一节五号干电池，储存的能量大约能让一个人提着它爬多高的楼？
5. 一根普通家用电线，正常工作时里面的自由电子平均要花多久才漂移过 1 m ？
6. 一束普通激光笔射出的光，每秒大约发射多少个光子？
7. 你家空调连续制冷一小时，大约要从房间里搬走多少焦耳的热量？

本节要点

物理与工程问题里摩尔和阿伏伽德罗常数是把宏观量翻译成微观个数的通用桥梁。一滴水里的分子数（ 10^{21} 量级）比全世界沙滩上的沙粒还多，这个数字停下来体会一下比死记更有用。每道题估完都该用量纲或单位串接回头检查，单位约得干净是可信的重要信号。

C.4 第四组：生物与人体

思考题

1. 一个成年人体内大约有多少个细胞？
2. 人脑每秒大约处理多少比特的信息？（提示：可从视觉、听觉等感官的输入速率粗略。）
3. 一个人一生当中，心脏大约跳动多少次？
4. 你身体里所有细胞每天合计消耗的能量，折合大约相当于一只多少瓦的灯泡？
5. 人的一根头发一个月大约长长多少？一辈子合计长出多长的头发？
6. 一个人安静呼吸，一生大约要吸入多少千克空气？

本节要点

生物与人体数据平时不太留意，但多数有可感知的日常参照：心跳频率、睡眠时间、饮食量。利用这些参照去推不熟悉的数量，正是 Fermi 估算最核心的训练。目标永远不是精确值，是落在正确量级。

C.5 练这身本领到底是为了什么

Fermi 估算没有标准答案。判断一次估算成不成功，从来不是看你的数字和真值差多少个百分点。看的是你落在不在正确的数量级上。只要差距在一个数量级以内，这道题就算赢了。也正因如此，过程远比答案重要。一条拆解链条清晰、每个锚点都说得出理由的估算，哪怕最后差了三五倍，也远胜过一个碰巧蒙对的数字。

你真正在练的，是两样东西：一是把陌生未知拆成几个熟悉已知的反射，二是在心里随手做数量级乘除而毫不慌张的底气。这正是这门课最看重的底层能力，当 AI 替你算出一个结果、报出一个数值、给出一段看起来很专业的推导，你能不能几十秒内、不开电脑，凭数量级直觉判断它合不合理。第二章练的量纲检查挡住了形状错误的公式。这里练的数量级估算挡住的，是数值离谱的结果。两者合起来，就是作为项目主理人手里那道不依赖任何计算机的第一道防线。

本节要点

Fermi 估算没有标准答案。成功的标准是落在正确数量级上，不是和真值差几个百分点。你真正在练的，是把陌生未知拆成熟悉已知的反射，以及在脑子里随手做数量级乘除的底气。量纲检查挡住了形状错误的公式，数量级估算挡住了数值离谱的结果。两者合起来，就是你不需要开电脑的第一道防线。

AI 陷阱预警 · AI 的 Fermi 估算，常常假装精确到小数点后两位

让 AI 替你做一道 Fermi 估算时，要特别警惕它的一个典型毛病：明明是个粗略的数量级估算，它却给你一个像 4.27×10^5 这样精确到小数点后两位的数字，仿佛真的算得这么准。这是一种虚假的精确。Fermi 估算的每一个输入锚点本身就只准到一个量级，上海有几百万还是两千万人、每家钢琴比例是三十分之一还是五十分之一，全都带着两三倍的不确定度。把这些数乘起来，结果绝不可能比输入更精确。真正诚实的 Fermi 答案，只敢说大约几十万、量级在 10^5 到 10^6 之间，不敢报到小数点。所以当 AI 给你一个尾数齐全、看似精密的估算值，请把那些多余的有效数字当成警报：要么它在硬凑精度以显得可信，要么它根本没意识到自己在做的是数量级估算。你要做的，是把答案重新压回它应有的粗糙，只保留那个真正有意义的 10 的幂次。

本章小结

这个附录是第二章量纲与数量级的配套题库，给了二十多道 Fermi 估算题，按日常生活、地球与天空、物理与工程、生物与人体四组排列。前三组各放了一道完整示范：用常

识锚点逐层拆解，估出上海需要几百位钢琴调音师、地球大气总质量约 10^{18} kg、一滴水里约有 10^{21} 个水分子，并且每道都用量纲或单位串接回头检查了结果的可信度。其余的题留给你自己先估后查。这身本领追求的不是精确。它追求的是落在正确数量级上的判断力，以及把未知拆成已知的反射。它正是判断 AI 给的数值合不合理的底层能力。也别忘了那个 AI 陷阱：真正诚实的 Fermi 估算只敢报数量级，凡是精确到小数点后几位的估算值，反而要当成警报来看。



D. 项目报告与展示写作

到了这一步，你手里已经有一件能跑的东西了：一个别人点开就能玩的网页，或者一个一键启动的压缩包，这份附录要解决的问题，是这座东西旁边那几样辅助交付件该怎么写：报告、答辩 slides、还有 Demo 现场怎么讲。这些都不是这门课的主角，但它们是把你工作讲清楚的载体。写砸了，同样会让一件好工件被低估。

本章地图

小节	这一节解决什么	关键方法 / 产物
D.1	摆正工件与报告的主次关系，明确报告篇幅与定位	工件是主角，报告是说明，两到四页足矣
D.2	一份短报告的五块结构与每块的写法要点	物理问题与建模、方法与实现、结果与验证、局限、AI 使用说明
D.3	答辩 slides 的七页结构与一页一要点原则	开场钩子、工件展示、关键图、诚实说局限
D.4	Demo 现场怎么讲才真的有说服力	当场调参数让图变、本地兜底、一句话讲清物理

D.1 摆正位置：工件是主角，报告是它的说明

先记住这门课在交付件上的立场。主交付件是那可玩的工件，报告只是附在它旁边的一段说明，用来交代它背后的物理是什么、你为什么相信它给出的结果。这个主次关系决定了报告的长度：两到四页足矣。写到第五页，通常意味着你开始当成技术流水账写了。报告不是你这两周敲了多少行代码的账本。它是一个论证：你要论证你的物理建模是合理

的，你跑出来的结果是可信的。一份好报告读起来应该像一篇短小的物理论文，不是软件操作手册。如果一个完全没看过你代码的人，读完这两到四页能复述出你解决了什么物理问题、用了什么近似、为什么相信结果，那这份报告就成了。如果读完只知道你调用了哪些函数、装了哪些库，那它就失败了。

本节要点

主交付件是可玩的工件，报告只是附属说明。两到四页足矣。读完报告的人应该能复述你解决了什么物理问题、用了什么近似、为什么相信结果，而不是知道你调用了什么函数。报告是物理论证，不是技术流水账。

D.2 一份短报告里该有什么

两到四页拆开看，大致由五块构成，但要用连贯的散文写出来，不要拉一张清单逐项填空。

开头先讲物理问题与建模。说清楚你研究的是什么物理现象，然后把怎么把它落成一个可计算的模型讲出来：做了哪些假设，写下了什么方程，以及为什么允许自己这么做。这一块最考验对问题的理解，因为简化总是有代价的。你要让读者相信，砍掉的东西在你关心的范围里确实可以忽略。比如，你研究单摆周期，假设了小角度、忽略了空气阻力，那就要说清楚在你考察的摆角范围内这两个假设站得住。

接着讲方法与实现。用了什么数值方法把上面的方程解出来，是显式 Euler 还是四阶 Runge-Kutta，为什么选它而不是别的。重点始终是为为什么，不是怎么做。不需要把代码逐行搬进报告，那是软件文档干的事。你需要的是让读者明白，方法选择背后有判断，不是 AI 随手给什么就用什么。

然后是结果与验证，这是整份报告的核心。放上关键的图，写清楚五项验证清单跑下来的结论。注意，这一块的灵魂不是罗列你得到的数字。它要回答一个问题：你为什么相信这些结果是对的？因为它在解析极限下退化成了已知的公式？因为和教科书里的标准值对上了？还是因为步长减半后结果几乎不变？把这些验证的结论摆出来，你的结果才有了立足点。这一块里图远比文字重要。一张清晰标注了坐标轴、画出了数值解和解析解对照的图，胜过一整段文字描述。读者的眼睛会先落在图上，所以要把最能说明问题的那张图做好。

再往后是局限与未解决。诚实地交代没做到的事：哪个效应本想纳入但时间不够，哪个参数范围里方法会失稳，哪个结论自己其实还不太确定。这一块不会让你扣分。恰恰相反，一份清楚知道自己边界在哪的报告，比一份号称什么都搞定了的报告可信得多。

最后是 AI 使用说明段。这是报告的必含部分，采用四栏式写法：用了什么工具、让它做了什么、它哪里错了或你哪里不放心、你怎么核验并由你承担最终责任。详细写法和完整范例，在讲 AI 使用规范的那一章已经给过，这里不重复。只提醒一句：这一段不能省，也不能糊弄。它和物理分析、验证清单一样，是报告的正式组成。

AI 陷阱预警 · 让 AI 写报告，它会写成软件文档

把报告丢给 AI 写，它有几个根深蒂固的坏习惯。第一，会把报告写成代码说明书：通篇在讲“本程序一上来就导入 `numpy`，然后定义函数 `solve`，循环更新状态变量”，却一个字不提你为什么要做物理简化、结果为什么可信。这是因为 AI 看到的是你的代码，它就顺着代码讲，而报告要讲的恰恰是代码背后的物理论证。第二，喜欢堆砌技术名词。它爱往报告里塞“采用自适应步长的高精度数值积分方案”这类听起来很专业、其实没信息量的句子，把简单的事说得花里胡哨。第三，也是最危险的一点：它常常在报告结尾突然冒出一个漂亮的结论，比如“本方法精度极高、完全吻合理论”，而这个结论你从来没验证过，是它替你脑补出来的。报告是物理论证，不是软件文档。每一句结论都要能在验证清单里找到支撑。AI 替你下的、你没核过的结论，一律删掉。

本节要点

一份短报告分五块：物理问题与建模、方法与实现、结果与验证、局限与未解决、AI 使用说明。核心是结果与验证那部分，图远比文字重要，每一句结论都要能在验证清单里找到支撑。让 AI 写报告要警惕它写成代码说明书、堆砌技术名词、或在结尾冒出没验证过的结论。

D.3 答辩 slides：七页以内，一页一个要点

Demo Day 上你只有很短的时间，slides 控制在七页以内。做 slides 最常见的错误，是把报告整段整段贴上去，结果每一页都是密密麻麻的文字。你在台上照着念，台下没人看得进去。正确的做法是一页只承载一个要点，文字尽量少，让图和口头讲述成为主体。

答辩

开头第一页要在十几秒内讲清两件事：你做了什么，以及这件事为什么有意思。不要上来就讲方法细节，先给一个钩子，让听的人知道你解决的是个什么问题、为什么值得花时间听下去。中间几页是主体，用来展示工件和关键图。尽量放图、放工件的截图或现场画面，而不是代码或公式列表。结尾一页要诚实地说还有什么没解决。敢在答辩结尾主动讲清楚局限的人，给评委和同学的印象，远好过一个把一切说得天衣无缝、却在提问环节被一句话问穿的人。

本节要点

Slides 控制在七页以内，一页一个要点。开头讲清你做了什么和为什么有意思，中间放图和工件截图，结尾诚实说还没解决的问题。文字尽量少，让图和口头讲述成为主体。敢主动讲局限的，比声称一切完美的印象好得多。

D.4 Demo 怎么讲：让观众亲手把图调动

Demo 的精髓是现场打开工件，让它活起来。最有说服力的一个动作，是请观众（或你自己）当场调一个参数，让大家看着图随之变化。比如做的是阻尼振子，就当场把阻尼

系数的滑条拉大，让全场看着振荡曲线从欠阻尼变到过阻尼。这一下胜过十句话，因为它证明了这件工件是真能跑、真有物理的，不是一张截图糊弄人。

但现场总有意料之外，网络可能断，部署的网页可能起不来。所以必须准备好本地兜底：一个不依赖网络、双击或一行命令就能在你自己电脑上跑起来的版本。答辩前把兜底版本测一遍，确保投影仪接上、网断了它照样能演示。这是 Demo Day 反复强调的一点，工件起不来时有没有退路，区分了有准备的组和没准备的组。

还有一件事要练：用一句话说清楚项目的物理是什么。台下有人问起，或者开场需要一个引子，你都要能脱口而出一句不超过二十个字、抓住核心的话。“我做的是单摆在大摆角下周周期怎样偏离小角度公式”，这样一句，比绕来绕去讲半天清楚得多。能用一句话讲清楚，说明自己是真的想明白了。

■ **Example D.1：一份阻尼振子项目报告的两到四页提纲。** 假设项目是用数值方法研究阻尼谐振子，下面是一份可以照着填血肉的报告骨架。写出来连成散文即可，不要保留这种分点形式。

第一块，物理问题与建模，约半页：研究质量-弹簧-阻尼系统的运动，物理量是位移随时间的演化。写下运动方程 $m\ddot{x} + c\dot{x} + kx = 0$ ，说明假设阻尼力正比于速度、弹簧在小形变下线性，解释为什么在你考察的振幅范围内两个假设成立。

第二块，方法与实现，约半页：把二阶方程化成一阶方程组，采用四阶 Runge-Kutta 积分。说明为什么不用显式 Euler：在欠阻尼振荡里 Euler 容易引入数值耗散，把物理阻尼和数值假象混在一起。

第三块，结果与验证，约一页，是重头。放一张图，画出三种阻尼情形（欠阻尼、临界阻尼、过阻尼）的位移曲线，坐标轴标注清楚。然后写五项验证的结论：量纲一致；欠阻尼解在阻尼趋零时退化为无阻尼简谐解（解析极限对照）；振荡角频率的数值值与解析公式 $\omega_d = \sqrt{k/m - (c/2m)^2}$ 吻合到三位有效数字；步长减半后曲线几乎不变（收敛性）。明确写出你为什么相信结果。

第四块，局限与未解决，约小半页：诚实写出比如没有纳入非线性弹簧、没有考虑驱动力下的共振、过阻尼区的长时间行为没细查。

第五块，AI 使用说明，约小半页：四栏式，照前面那一章的范例写。

合起来约两到四页，图占掉相当篇幅，文字精炼，每个结论都能回溯到一次验证。 ■

本能·讲不清楚的工作，等于没做

回到 Ship something real 这条反射弧。这两周你练的，从来不只是把东西做出来，而是把它 ship 出去、讲给别人听、让别人能玩能懂。一件做得再精巧的工件，如果你站在台上讲不清它解决了什么物理问题、为什么结果可信，在这门课的尺度上，它和没做出来差别不大。科学不是私下里自己跑通就算数的事，它要经得起别人追问、要能被别人复核。所以报告、slides、Demo 这三样辅助交付件，本质上和你的代码一样重要：它们是把“我做出来了”变成“我让别人信了我做出来了”的那一步。讲不清楚，工件就传不出去；传不出去，这件工作在科学的意义上就还没真正发生。

本节要点

Demo 现场打开工件，让观众（或自己）当场调一个参数，看着图随之变化，这比十句话都有说服力。准备好本地兜底版本应对网络中断。练就能用一句话说清项目物理的本事，说明自己真的想明白了。讲不清楚的工作，在这门课的尺度上等于没做。

本章小结

这门课的主交付件是可玩的工件，报告只是它的附属说明，写两到四页就够。报告是物理论证，不是技术流水账。一份短报告依次讲物理问题与建模、方法与实现、结果与验证、局限与未解决，并以一段四栏式 AI 使用说明收尾。其中结果与验证是核心，重点是讲清你为什么相信结果，而图远比文字重要。答辩 slides 控制在七页以内，一页一个要点，开头讲清你做了什么和为什么有意思，中间展示工件和关键图，结尾诚实说还有什么没解决。Demo 要现场打开工件、让观众调一个参数看到图变，准备好网络断时的本地兜底，并能用一句话说清项目的物理。让 AI 写报告时要警惕它把报告写成软件文档、堆砌技术名词、在结尾冒出没验证过的结论。

把视野放远一点。你在这门课练的五条反射动作，Spec-first、Architecture-before-code、Trust but Verify、Ship something real、Skill-building，不会随着这两周结束而失效。等将来读研、写论文、做同行评审、回复审稿意见，工件会从 Streamlit 页面换成投稿稿件、rebuttal letter、组会报告，但这五条反射弧一字不变。今天为一件物理交互工件写报告、做 slides、讲 Demo 时学到的东西，正是将来为一篇 PRC 稿件写引言、做答辩、回应审稿人时要反复用到的同一套动作。工件在变，反射弧不变，这才是这门课真正想留在你身上的东西。



参考文献

- [HNW93] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. 2nd edition. Springer Series in Computational Mathematics. Berlin: Springer-Verlag, 1993 (cited on page 75).
- [Kar25] Andrej Karpathy. *There's a new kind of coding I call "vibe coding"*. Post on X (formerly Twitter). <https://x.com/karpathy>. 2025 (cited on page 16).
- [Pre+07] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd edition. Cambridge: Cambridge University Press, 2007 (cited on pages 104, 112).



索引

A

AGENTS.md	153
AI 审计	131
arXiv	137

D

DeepSeek	201
Demo Day	211
DOI	137

E

Euler 法	73
---------------	----

F

Fermi 估算	23, 205
----------------	---------

P

prompt engineering	55
Python	65, 195

S

ship	145
------------	-----

Skill	161
Spec-first	39
Streamlit	145

T

Trae	201
Trust but Verify	123

V

vibe coding	15
-------------------	----

?

二维运动	91
二阶常微分方程	81
交付件	145
产品需求文档	39

?

六维框架	55
几何概率	111
反射动作	15
学术诚信	169

workflows 复用 161
 工具栈 201
 常微分方程 73
 建模 31



手算对账 65
 抛体运动 91
 提示词 55
 数据拟合 101
 数量级 23, 205
 文献核验 137
 最小二乘 101
 来源链 169
 架构先行 47
 残差 101



牛顿冷却 31
 状态向量约化 81
 理想化 31
 空气阻力 91
 答辩 211
 简谐振子 81
 系统块图 47
 终端速度 91



能量守恒 81
 蒙特卡罗方法 111
 虚拟环境 195
 虚构引用 137
 装机 195
 认知所有权 169
 读代码 65



量纲分析 23

错误模式 131
 随机行走 111
 需求文档 39
 项目主理人 15
 项目实战 179
 项目报告 211
 项目案例 179
 项目级 AI 约束 153
 题库 205
 验证清单 123